UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA AERONÁUTICA Y DEL ESPACIO

TRABAJO DE FIN DE GRADO

Diseño Preliminar de un Satélite de Observación de la Tierra

AUTOR: Miguel Rosa Zazo

ESPECIALIDAD: Ciencias y Tecnologías Aeroespaciales

COTUTOR: Demetrio Zorita Gómez-Escolar

TUTOR: Ana Laverón Simavilla

Febrero de 2027

Página intencionadamente en blanco

A quienes me enseñaron a mirar más lejos,
a sostener el esfuerzo en la incertidumbre
y a no renunciar al rigor del trabajo bien hecho.

«La perseverancia no es una carrera larga;
es muchas carreras cortas, una tras otra.»

— Walter Elliot

Agradecimientos

Quiero expresar mi agradecimiento a Demetrio Zorita Gómez-Escolar, cotutor de este trabajo, por su criterio técnico, su disponibilidad y su ayuda para estructurar el problema con ambición y realismo. Agradezco también a Ana Laverón Simavilla, tutora del TFG, su seguimiento académico y sus observaciones, que han contribuido a mejorar tanto el enfoque como la presentación final del proyecto.

Extiendo este agradecimiento a mi familia y a las personas cercanas que me han acompañado durante estos meses. Su apoyo, su paciencia y su confianza han sido esenciales para sostener el esfuerzo continuado que exige un trabajo de esta naturaleza.

Por último, agradezco a la Escuela y a la formación recibida en ella haberme proporcionado las herramientas necesarias para abordar un problema multidisciplinar como este, en el que convergen óptica, mecánica orbital, subsistemas y análisis de misión.

Lista de símbolos

Símbolos ópticos y de carga útil

λ	Longitud de onda (m)
$\Delta\lambda$	Ancho de banda espectral (m)
D	Diámetro efectivo de la pupila (m o mm)
f	Distancia focal (m)
f/D	Número focal (*F-number*) (adim.)
R	Relación de oscurecimiento central de la pupila (adim.)
η	Eficiencia cuántica del detector (adim.)
τ	Transmitancia del telescopio (adim.)
GSD	Resolución espacial en tierra (*Ground Sampling Distance*) (m)
p	Tamaño de píxel del detector (m)
$N_{\mathbf{pix}}$	Número de píxeles del detector (adim.)
$t_{\mathbf{int}}$	Tiempo de integración (s)
N_e	Número de electrones generados por el detector (e^-)
$L_{\mathbf{ref}}$	Radiancia de referencia ($W\,m^{-2}\,sr^{-1}\,\mu m^{-1}$)
MTF	Función de Transferencia de Modulación (*Modulation Transfer Function*) (adim.)
SNR	Relación Señal-Ruido (*Signal-to-Noise Ratio*) (adim.)
TDI	Integración con Retardo Temporal (*Time Delay Integration*) (adim.)
$f_{\mathbf{co}}$	Frecuencia de corte óptica ($cy\,m^{-1}$)
$f_{\mathbf{Nyq}}$	Frecuencia de Nyquist del detector ($cy\,m^{-1}$)
SW	Swath (ancho de franja observada) (km)

Símbolos orbitales y de misión

h	Altura orbital (km)
a	Semieje mayor de la órbita (km)
$v_{\mathbf{orb}}$	Velocidad orbital ($\mathrm{m\,s^{-1}}$)
$T_{\mathbf{orb}}$	Periodo orbital (s)
μ	Parámetro gravitacional terrestre, $3{,}986 \times 10^{14}$ ($\mathrm{m^3\,s^{-2}}$)
R_T	Radio terrestre medio, 6 371 (km)
$N_{\mathbf{sat}}$	Número de satélites en la constelación (adim.)
$N_{\mathbf{tel}}$	Número de telescopios por satélite (adim.)
$T_{\mathbf{rev}}$	Tiempo de revisita (días)
LTAN	Hora local del nodo ascendente (*Local Time of Ascending Node*) (h)
β	Ángulo beta sol-órbita (°)
C_d	Coeficiente de resistencia aerodinámica (adim.)
ρ	Densidad atmosférica ($\mathrm{kg\,m^{-3}}$)
$H_{\mathbf{atm}}$	Altura de escala atmosférica (km)
A	Área frontal del satélite ($\mathrm{m^2}$)

Símbolos de masa y propulsión

$m_{\mathbf{seca}}$	Masa en seco del satélite (kg)
$m_{\mathbf{prop}}$	Masa de propelente (kg)
$m_{\mathbf{total}}$	Masa total del satélite (kg)
Δv	Variación de velocidad por impulso ($\mathrm{m\,s^{-1}}$)
$\Delta v_{\mathbf{tot}}$	Variación de velocidad total acumulada ($\mathrm{m\,s^{-1}}$)
I_{sp}	Impulso específico (s)
g_0	Aceleración de la gravedad estándar, 9,80665 ($\mathrm{m\,s^{-2}}$)

Símbolos del subsistema eléctrico

$P_{\mathbf{gen}}$	Potencia generada por los paneles solares (W)
$P_{\mathbf{modo}}$	Consumo total en un modo de operación (W)
$E_{\mathbf{bat}}$	Energía almacenada en las baterías (Wh)
$h_{\mathbf{P}}$	Constante de Planck, $6{,}626 \times 10^{-34}$ ($\mathrm{J\,s}$)
c	Velocidad de la luz, 3×10^{8} ($\mathrm{m\,s^{-1}}$)

Palabras clave y glosario

Palabras clave

observación de la Tierra, nanosatélite, monitorización de CO_2, SNR, MTF, órbita heliosíncrona, diseño preliminar, constelación de satélites, instrumento óptico *pushbroom*.

Glosario

GSD (*Ground Sampling Distance*) Distancia en tierra correspondiente al tamaño de un píxel del detector. Determina la resolución espacial del instrumento.

MTF (*Modulation Transfer Function*) Función de Transferencia de Modulación. Mide la capacidad del sistema óptico para reproducir el contraste de objetos en función de su frecuencia espacial.

SNR (*Signal-to-Noise Ratio*) Relación Señal-Ruido. Indicador de la calidad de la señal detectada respecto al ruido total del sistema electrónico y óptico.

SSO (*Sun-Synchronous Orbit*) Órbita heliosíncrona. Órbita polar cuyo plano precesa a la misma velocidad angular que la Tierra alrededor del Sol, manteniendo un ángulo de iluminación solar constante.

LEO (*Low Earth Orbit*) Órbita terrestre baja. Órbita con altitudes entre aproximadamente 200 y 2 000 km.

LTAN (*Local Time of Ascending Node*) Hora local del nodo ascendente. Hora solar local en el momento en que el satélite cruza el ecuador en dirección norte.

TDI (*Time Delay Integration*) Técnica de integración por retardo temporal. Permite acumular señal en detectores de barrido progresivo para mejorar el SNR.

Nanosatélite Satélite artificial de masa comprendida entre 1 y 10 kg, con frecuencia basado en arquitecturas tipo CubeSat o derivadas de ellas.

Swath (franja de observación) Ancho de la franja de terreno observada por el instrumento en una pasada del satélite.

Tiempo de revisita Intervalo de tiempo entre dos observaciones sucesivas del mismo punto geográfico.

Carga útil (*Payload*) Conjunto de instrumentos y subsistemas del satélite dedicados a la misión científica o de observación.

HPGP (*High Performance Green Propulsion*) Sistema de propulsión monopropelente de combustible verde de alto rendimiento desarrollado por ECAPS. Usa LMP-103S en lugar de hidrazina tóxica.

Índice general

1. Introducción **1**

1.1. Resumen Ejecutivo 3

2. Misiones semejantes **4**

2.1. OCO-2 (*Orbiting Carbon Observatory*) 5

2.2. GOSAT / GOSAT-2 (*Greenhouse Gases Observing Satellite*) 6

2.3. CO2M (*Carbon Dioxide Monitoring Mission*) 7

2.4. TanSat (*Chinese Carbon Dioxide Observation Satellite*) 8

2.5. MicroCarb (*Microsatellite for Carbon Observation*) 9

2.6. GHGSat 10

2.7. Hallazgos clave 11

3. Conceptos Previos **14**

3.1. Calidad de Imagen 16

3.1.1. *Aliasing* 16

3.1.2. *Ground Sampling Distance* (GSD) 17

3.1.3. Frecuencia de Nyquist(f_{Ny}) 17

3.1.4. *Modulation Transfer Function* (MTF) 18

3.1.5. *Signal To Noise Ratio* (SNR) 20

3.1.6. *Field of View* (FoV). *Swath* 23
3.2. Espectro electromagnético. Bandas de absorción 24
3.3. Detectores 27
3.3.1. Tipos de detectores 28
3.3.2. *Time Delay and Integration* (TDI) 32
3.3.3. Estrategias de Escaneo en Sistemas de Observación por Satélite . . . 32
3.3.4. Disposición de detectores 35
3.4. Óptica. Telescopios 36
3.4.1. Parámetros ópticos relevantes 37
3.4.2. Tipos de telescopio 38
3.4.3. Filtros 41
3.5. Parámetros Orbitales 43
3.5.1. Inclinación (i) 44
3.5.2. Longitud del Nodo Ascendente (Ω) 44
3.5.3. Hora de paso local (LTAN) 45
3.5.4. Argumento del Perigeo (ω) 45
3.5.5. Anomalía Verdadera (ν) 45
3.6. Perturbaciones de la órbita 46
3.6.1. Achatamiento Terrestre 46
3.6.2. Decaimiento Orbital 46
3.6.3. Radiación Solar 47
3.6.4. Problema del tercer cuerpo 47
3.6.5. Órbitas de especial interés 48
3.6.6. Consideración de las perturbaciones orbitales 50

4. Definición del instrumento **51**

4.1. GSD 53

4.2. Bandas espectrales. Ancho de banda 54

4.3. Detectores, filtros y escaneo 56

4.3.1. Elección de detectores 56

4.3.2. Método de escaneo 57

4.3.3. Elección de filtros 57

4.4. Tipo de órbita. Altura orbital. Hora de paso local 58

4.5. Cálculo de la MTF 58

4.6. Cálculo de la SNR 63

4.7. Cobertura de la región de interés: *Swath*, FoV y tiempo de revisita 68

4.8. Posibles soluciones. Cruce de los datos 79

5. Determinación de la solución óptima **81**

5.1. Estimación de la masa seca 83

5.2. Impulsos, ΔV y masa de combustible. Fin de misión. 84

5.2.1. Modelado de la resistencia aerodinámica 85

5.2.2. Cálculo del Delta-V, número de maniobras y masa de combustible . . 85

5.2.3. Fin de misión 90

5.3. Verificación de las condiciones orbitales 92

5.3.1. Condición de órbita *frozen* 92

5.3.2. Variación de altura en la zona de observación 92

5.3.3. Evolución de la excentricidad durante la misión 93

5.3.4. Condición de traza repetida (RGT) 95

5.4. Sensibilidad de la solución al GSD 96

6. Lanzadores **98**

6.1. Lanzadores Operativos 99

6.1.1. Falcon 9 99

6.1.2. Firefly Alpha 99

6.1.3. Vulcan Centaur 99

6.1.4. Electron 99

6.1.5. Ariane 6 100

6.1.6. Vega-C 100

6.2. Bases de lanzamiento y azimuts posibles 101

6.2.1. Vandenberg Space Force Base (California, EE.UU.) 101

6.2.2. Cape Canaveral Space Force Station (Florida, EE.UU.) 101

6.2.3. Centro Espacial de Guayana (Guayana Francesa) 101

6.3. Selección Final 102

7. Segmento terrestre **106**

7.1. Generación de los datos de observación 107

7.2. Estaciones terrestres candidatas 108

7.3. Simulación de los contactos con Fairbanks 109

7.4. Tasa mínima de descarga 110

7.5. Selección de la banda de enlace 110

7.5.1. Velocidad adoptada del enlace 111

8. Conclusiones **113**

Bibliografía **115**

Anexos **1**

A. Tablas de datos **2**

MTF 2

SNR 12

Análisis de Configuraciones. Posibles soluciones 22

Resultado de simulación de impulsos necesarios 23

B. Código MATLAB **24**

Código maestro. Llamada a las funciones 24

Calculo de la MTF 27

Calculo de la SNR 29

Wrapper para el cálculo del tiempo de revisita 31

Determinación de mínimos. h vs Dmin 34

Cálculo de masa seca 35

Calculo de masa total 36

Análisis de configuraciones 38

Generación de gráficas de masa 40

Análisis de contactos con la estación de Fairbanks. Presupuesto de memoria . . . 41

Días en orbita vs altitud 44

Modelo deorbitacion. Fin de misión 45

Azimut de lanzamiento 48

Simulación de puesta en orbita de transferencia 49

Cálculo de órbita RGT con J2 50

Cálculo de órbita frozen 52

Representación Gráfica de las bandas en el espectro electromagnético. 54

Simulación auxiliar del enlace en banda X 55

Índice de tablas

1.1. Resumen de las especificaciones del sistema de satélites y la misión. 3

2.1. Comparativa de misiones de observación de CO_2. 12

3.1. Comparativa de tipos de sistemas ópticos. 41

4.1. Bandas espectrales elegidas. 55

4.2. Comparación de características de detectores SWIR 56

4.3. Comparación de características de detectores para *A-Band* O_2. 56

4.4. Parámetros iniciales para el cálculo de MTF. Fuente: [17]. 59

4.5. Parámetros iniciales y fuentes de ruido para el cálculo de SNR 63

4.6. Comparativa de tiempos de revisita de misiones de satélite. 68

5.1. Instrumentos de referencia para la estimación por escalado. Fuente: [67]. . . . 83

5.2. Máximos durante ocho años en los arcos de 25°–49°N. 94

6.1. Comparativa de lanzadores espaciales para misiones de lanzamiento compartido. 100

7.1. Datos generados por satélite durante una semana. 108

7.2. Tasa mínima de descarga por satélite. 110

7.3. Bandas de comunicaciones consideradas para la descarga de datos. 110

A.1. Soluciones viables tras el filtro regional. Masa de selección sin reserva de fin de misión. 22

Índice de figuras

2.1. Representación artística del *Orbiting Carbon Observatory* (OCO)-2 de la NASA. Fuente: [1]. 5

2.2. Representación artística de GOSAT. Fuente: [3]. 6

2.3. CO2M, de la Agencia Espacial Europea. Fuente: [6]. 7

2.4. TanSat. Fuente: [8]. 8

2.5. Imagen artística de Microcarb. Fuente: [10] 9

2.6. Constelación GHGSat. Fuente: [12] 10

3.1. Ejemplo del efecto del aliasing sobre un patrón visual. Fuente: [14] 16

3.2. Efectos de distintos GSD en imágenes tomadas por satélite. Fuente: [15] . . . 17

3.3. Representación visual de la MTF. Fuente: [18]. 18

3.4. Efecto de la SNR. Fuente: [20] 20

3.5. Esquema visual del *swath* y FoV. Fuente: [22]. 23

3.6. Representación del espectro electromagnético con las longitudes de onda, frecuencias y energías asociadas. Fuente: [23]. 24

3.7. Radiación transmitida por la atmósfera y bandas de absorción. Fuente: [24]. 25

3.8. Bandas de Transmitancia del CO_2 en función del número de onda. Fuente: [26] 26

3.9. Diagrama de funcionamiento de un sensor CCD. Fuente: [29]. 28

3.10. Diagrama de funcionamiento de un sensor CMOS. Fuente: [29]. 29

3.11. Disposición del detector MCT. Fuente: [30]. 30

3.12. Disposición de capas típica del detector InGaAs. Fuente: [32]. 31

3.13. Esquema de funcionamiento del *Time Delay Integration*. Fuente: [34]. 32

3.14. Funcionamiento del escaneo *Pushbroom*. Fuente: [35]. 33

3.15. Funcionamiento del escaneo *Whiskbroom*. Fuente: [36]. 34

3.16. Funcionamiento del escaneo *Step-Stare*. Fuente: [37]. 35

3.17. Diagrama de parámetros ópticos. Fuente: [39]. 37

3.18. Telescopio Refractivo. Fuente: [40]. 38

3.19. Telescopio Cassegrain. Fuente: [41]. 39

3.20. Telescopio Korsch. Fuente: [42]. 39

3.21. Telescopio TMA. Fuente: [43]. 40

3.22. Rueda de Filtros del *James Webb Space Telescope*. Fuente: [44]. 41

3.23. Filtrado de las frecuencias. Fuente:[45]. 42

3.24. Representación gráfica de los elementos orbitales. Fuente: [46]. 43

3.25. Días de vida media en órbita vs Altura orbital (km). Fuente: Elaboración propia basada en [48]. 46

4.1. Flujo de trabajo de los scripts de MATLAB 52

4.2. Imagen satélite del cinturón industrial de Houston. $GSD \approx 8{,}5$. Fuente: Google Earth (29°42'18,0"N; 95°00'10,8"W) 53

4.3. Comparación de imágenes con distintas condiciones de GSD. Fuente: Imagen adaptada, Google Earth. 54

4.4. Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 1 . 60

4.5. Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 2 . 61

4.6. Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 3 . 62

4.7. Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 4 (STAR250) 65

4.8. Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 5 (H2RG) 66

4.9. Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 6 (SATURN VISIR) 67

4.10. Factores de limitación del *swath* en el problema de la cobertura 69

4.11. Esquemas de las configuraciones evaluadas en el problema de la cobertura. . 72

4.12. Mapas de calor resultantes del cálculo de cobertura para el Detector 1, con 2 satélites y 2 telescopios. 73

4.13. Mapas de calor resultantes del cálculo de cobertura para el Detector 2, con 2 satélites y 2 telescopios. 74

4.14. Mapas de calor resultantes del cálculo de cobertura para el Detector 3, con 2 satélites y 2 telescopios. 75

4.15. Mapas de calor resultantes del cálculo de cobertura para el Detector 1, con 3 satélites. 76

4.16. Mapas de calor resultantes del cálculo de cobertura para el Detector 2, con 3 satélites. 77

4.17. Mapas de calor resultantes del cálculo de cobertura para el Detector 3, con 3 satélites. 78

4.18. Revisita geométrica y efectiva de la solución seleccionada en las trece latitudes del filtro regional. Fuente: Elaboración propia. 79

4.19. Diámetros de pupila de las soluciones que superan los filtros ópticos y regionales. Fuente: Elaboración propia. 80

5.1. Estimación de la masa seca del satélite en función de la altura. Fuente: Elaboración propia. 84

5.2. Transferencia de Hohmann. Fuente: Elaboración propia. 86

5.3. Desglose de masa seca, masa de combustible de mantenimiento y masa total de selección. La curva gris discontinua representa las soluciones ópticas sin el filtro regional. Los marcadores de color señalan los puntos viables, que además cumplen los requisitos de cobertura y FoV. Las oscilaciones de la masa seca se deben a la selección del diámetro de pupila en una malla discreta de 1 mm y al escalado cúbico de la masa. Fuente: Elaboración propia. 87

5.4. Punto de diseño del satélite. Fuente: Elaboración propia. 88

5.5. Tiempo de deorbitación, en días, tras fin de misión. Fuente: Elaboración propia. 91

5.6. Altura sobre el elipsoide a lo largo de los arcos de adquisición ascendente y descendente entre 25° y 49°N. Fuente: Elaboración propia. 93

5.7. Excentricidad media y variación máxima dentro de cada arco durante ocho años. Fuente: Elaboración propia. 95

5.8. Análisis auxiliar a 690 km: sensibilidad de masa y diámetro de pupila frente al GSD. Fuente: Elaboración propia. 97

6.1. Programa de lanzamiento compartido de SpaceX Fuente: [84]. 102

6.2. Falcon 9 de SpaceX despegando. Fuente: [76]. 102

6.3. Firefly Alpha en la plataforma de lanzamiento de Vandenberg SFB. Fuente: Firefly Aerospace. 103

6.4. Azimut de lanzamiento desde Vandenberg SFB. Fuente: Elaboración propia. 104

7.1. Antena de la estación de Fairbanks, Alaska. Fuente: [89]. 109

7.2. Evolución de la memoria a bordo durante una semana. 111

A.1. Mapas de calor resultantes del cálculo de MTF: Banda 0,76 µm; Detector 1 . 3

A.2. Mapas de calor resultantes del cálculo de MTF: Banda 0,76 µm; Detector 2 . 4

A.3. Mapas de calor resultantes del cálculo de MTF: Banda 0,76 µm; Detector 3 . 5

A.4. Mapas de calor resultantes del cálculo de MTF: Banda 1,61 µm; Detector 1 . 6

A.5. Mapas de calor resultantes del cálculo de MTF: Banda 1,61 µm; Detector 2 . 7

A.6. Mapas de calor resultantes del cálculo de MTF: Banda 1,61 µm; Detector 3 . 8

A.7. Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 1 . 9

A.8. Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 2 . 10

A.9. Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 3 . 11

A.10. Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 4 . 13

A.11. Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 5 . 14

A.12. Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 6 . 15

A.13.Mapas de calor resultantes del cálculo de SNR: Banda 1,61 µm; Detector 1 . 16

A.14.Mapas de calor resultantes del cálculo de SNR: Banda 1,61 µm; Detector 2 . 17

A.15.Mapas de calor resultantes del cálculo de SNR: Banda 1,61 µm; Detector 3 . 18

A.16.Mapas de calor resultantes del cálculo de SNR: Banda 2,01 µm; Detector 1 . 19

A.17.Mapas de calor resultantes del cálculo de SNR: Banda 2,01 µm; Detector 2 . 20

A.18.Mapas de calor resultantes del cálculo de SNR: Banda 2,01 µm; Detector 3 . 21

Capítulo 1

Introducción

El cambio climático es uno de los grandes desafíos del siglo XXI, y el dióxido de carbono (CO_2) uno de los principales gases de efecto invernadero responsables del calentamiento global. Monitorizar con precisión las fuentes antropogénicas de CO_2 es esencial para diseñar estrategias de mitigación y cumplir los acuerdos internacionales sobre el clima.

En este contexto, el presente Trabajo de Fin de Grado aborda el diseño preliminar y el estudio de viabilidad de un sistema de satélites destinado a la observación y monitorización precisa de las emisiones de CO_2 provenientes de fuentes humanas, en las principales ciudades de Estados Unidos. El proyecto responde a una necesidad específica planteada por un usuario final, cuyo enunciado se cita a continuación:

> *El usuario es el llamado U.S. Observatory Against Global Change, que quiere monitorizar con precisión las fuentes de producción de Dióxido de Carbono CO_2 antropogénico en las principales ciudades de los estados de Estados Unidos. Quiere un mapa completo de EE. UU. una vez cada semana, teniendo en cuenta que la cobertura de nubes sobre EE. UU. es un día cubierto por cada cinco descubiertos. Para lograr mapas de calidad adecuada necesita una MTF de 0,25 (25 % de devolución de contraste) a la frecuencia de Nyquist, y una SNR de 400 a una radiancia de* $100\,\mathrm{W/m^2\,sr\,\mu m}$*. Deriva de estas necesidades requisitos de GSD, swath y bandas espectrales, y a partir de ellos y de las prestaciones que pide el usuario, diseña una misión satelital para cumplirlas. El tiempo de misión es de 8 años.*

El proceso de diseño seguirá una metodología iterativa, comenzando con un **análisis de misiones similares y del estado del arte** en observación terrestre. Posteriormente, se aborda el diseño de la **carga útil**, definiendo los parámetros ópticos y los detectores más adecuados para la misión. Se evalúan diferentes configuraciones de telescopios y se comprueba su rendimiento en términos de MTF, SNR y tiempo de revisita.

El estudio incluirá también una segunda fase en la cual se realizará el **análisis de misión**:

selección de órbita, cobertura, estimación de masa seca, combustible de mantenimiento y masa total del vehículo espacial.

Finalmente, se evaluarán distintas opciones en el mercado de **lanzadores** que puedan poner nuestra plataforma de observación en la órbita escogida. Este análisis incluirá tanto lanzadores existentes como emergentes, considerando la capacidad de carga a la órbita deseada, la fiabilidad histórica y el coste aproximado de lanzamiento. Se incluirá también una comprobación del **segmento de Tierra** necesario para descargar los datos de observación y operar la misión.

Para los cálculos requeridos se ha desarrollado un conjunto de programas en MATLAB, disponible en el Anexo B. Los datos resultantes de la ejecución de este paquete se presentan en el Anexo A.

1.1. Resumen Ejecutivo

Se muestra, en primer lugar, un resumen de los parámetros más relevantes escogidos como solución al problema propuesto, para comodidad del lector:

Tabla 1.1: Resumen de las especificaciones del sistema de satélites y la misión.

Detector	
Detector	2 × Teledyne H2RG
Bandas espectrales	0,76; 1,61; 2,01 µm
Ancho de banda	20 nm
GSD	80 m
SNR	738 (en 0,76 μm)
MTF @ Nyquist	0,253 (en 2,01 μm)
Telescopio	2 × Refractivos
Diámetro de apertura	38 mm
Distancia focal	15,53 cm
Relación focal	F/4,09
Field of View	20°

Plataforma Satelital	
Dimensiones	0,348 × 0,348 × 0,348 m
Volumen	0,0423 m^3
Masa del instrumento	0,835 kg
Masa de combustible	0,196 kg
Masa seca	3,342 kg
Masa total	3,538 kg
Potencia del instrumento	0,591 W
Isp	220 s
Delta-V	113,392 m/s
Maniobras de mant.	2 + deorbitado final

Órbita y Misión	
Tipo de órbita	SSO *frozen*
LTAN	06:00 (*Dawn-Dusk*)
Altitud	690 km
Inclinación	98,15°
Período orbital	98,68 min*
Ancho de barrido (*Swath*)	240 km
Revisita efectiva	6,642 días
Número de satélites	2 ($\Delta\nu = 180°$)

Lanzamiento, Comunicaciones y Datos	
Lanzador	Falcon 9
Base de lanzamiento	Vandenberg SFB
Azimut de lanzamiento	190°
Estación de Tierra	Fairbanks, Alaska
Frecuencia y antena	Banda S
Datos generados	31,5 GB/semana
Velocidad mínima	5,024 Mbps

*Período nodal con J_2. La revisita efectiva incluye la corrección media de nubosidad del factor 6/5.

En los siguientes capítulos se detalla el proceso de obtención de dichos resultados, el marco teórico, la metodología y la justificación de las decisiones tomadas durante la elaboración de este trabajo.

Capítulo 2

Misiones semejantes

Contenido

2.1. OCO-2 (*Orbiting Carbon Observatory*) 5
2.2. GOSAT / GOSAT-2 (*Greenhouse Gases Observing Satellite*) . . 6
2.3. CO2M (*Carbon Dioxide Monitoring Mission*) 7
2.4. TanSat (*Chinese Carbon Dioxide Observation Satellite*) 8
2.5. MicroCarb (*Microsatellite for Carbon Observation*) 9
2.6. GHGSat 10
2.7. Hallazgos clave 11

En este apartado, se pretende revisar y comparar las características principales de misiones previas que han abordado retos similares en términos de resolución espacial, cobertura, requisitos espectrales y diseño de la misión. Por último, se presenta una tabla comparativa con las características de cada misión que son interesantes para inspirar el presente trabajo.

2.1. OCO-2 (*Orbiting Carbon Observatory*)

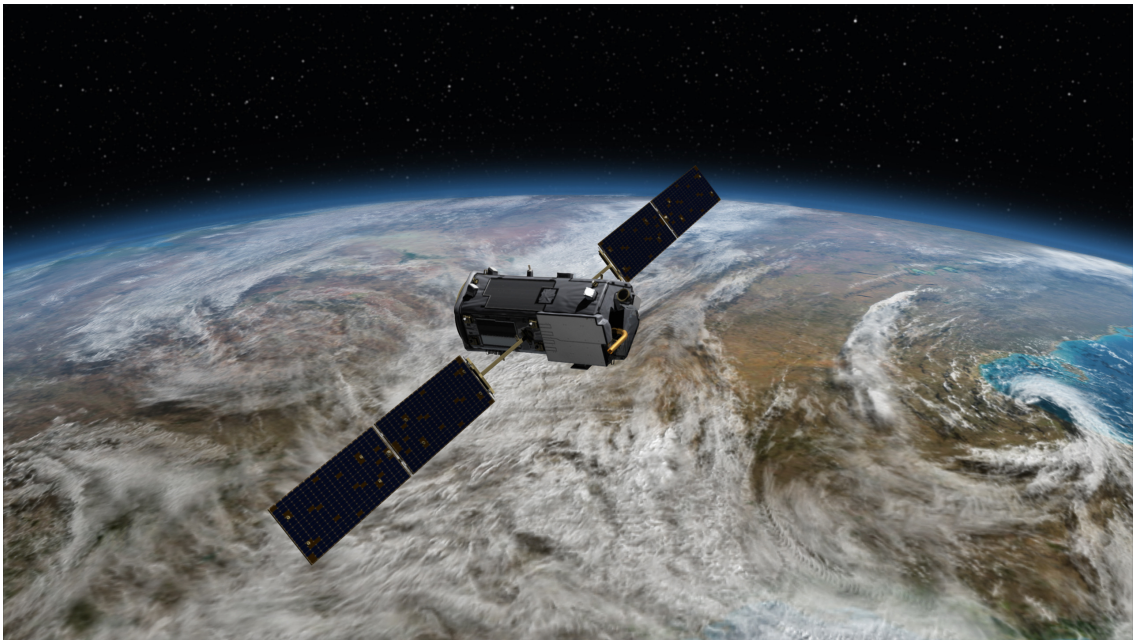


Figura 2.1: Representación artística del *Orbiting Carbon Observatory* (OCO)-2 de la NASA. Fuente: [1].

OCO-2 representa una de las misiones más notables dedicadas exclusivamente a la monitorización de CO_2 atmosférico. Lanzado en julio de 2014 por la NASA, este satélite se diseñó para proporcionar mediciones precisas de la concentración de CO_2 a escala global, ofreciendo datos cruciales para la comprensión del ciclo del carbono terrestre.

El satélite emplea tres espectrómetros de rejilla que operan en bandas espectrales del infrarrojo cercano: 0,76 µm (banda O_2 A), 1,61 µm y 2,06 µm (bandas de absorción de CO_2). Los detectores utilizados son matrices HgCdTe (HAWAII-1RG) fabricadas por Teledyne (con silicio en la banda O_2), mantenidas a -120 °C mediante un sistema criogénico para minimizar el ruido térmico.

El telescopio presenta un diseño Cassegrain con una apertura de 11 cm de diámetro. La resolución espacial alcanza 1,3×2,3 km con un ancho de barrido de 10 km. El satélite orbita a 705 km de altitud en una órbita heliosíncrona con inclinación de 98,2°, proporcionando un período de revisita de 16 días. La masa total del sistema es de 449 kg [2].

2.2. GOSAT / GOSAT-2 (*Greenhouse Gases Observing Satellite*)

Figura 2.2: Representación artística de GOSAT.
Fuente: [3].

El sistema GOSAT, desarrollado por JAXA, utiliza el espectrómetro de transformada de Fourier TANSO-FTS, GOSAT-1 contaba con las siguientes bandas: Banda 1 (0,758-0,775 µm) centrada en la banda de absorción del oxígeno A, Banda 2 (1,56-1,72 µm) optimizada para la detección de CO_2 y CH_4, Banda 3 (1,92-2,08 µm) diseñada para CO_2 y H_2O, y Banda 4 (5,56-14,3 µm) que abarca el infrarrojo térmico para CO_2 y CH_4. Los detectores empleados incluyen tecnología de silicio (Si) para la banda 1, InGaAs para las bandas 2 y 3, y PC-MCT (*Mercury Cadmium Telluride*) para la banda 4 en el infrarrojo térmico. El telescopio presenta una apertura de 68 mm de diámetro.

La resolución espacial nominal es de 10,5 km en nadir. GOSAT-2, la versión mejorada lanzada en octubre de 2018, opera a 613 km de altitud en órbita heliosíncrona con inclinación de 98,1°, ofreciendo un período de revisita de 6 días. La masa del sistema alcanza aproximadamente 1700 kg (masa seca), con una masa total inferior a 2000 kg.

GOSAT-2 incorpora mejoras notables respecto a su predecesor, incluyendo cinco bandas espectrales (en lugar de cuatro) con rangos optimizados, capacidad para medir monóxido de carbono (CO) adicional al CO_2 y CH_4 y un mecanismo de apuntamiento inteligente para evitar nubes [4] [5].

2.3. CO2M (*Carbon Dioxide Monitoring Mission*)

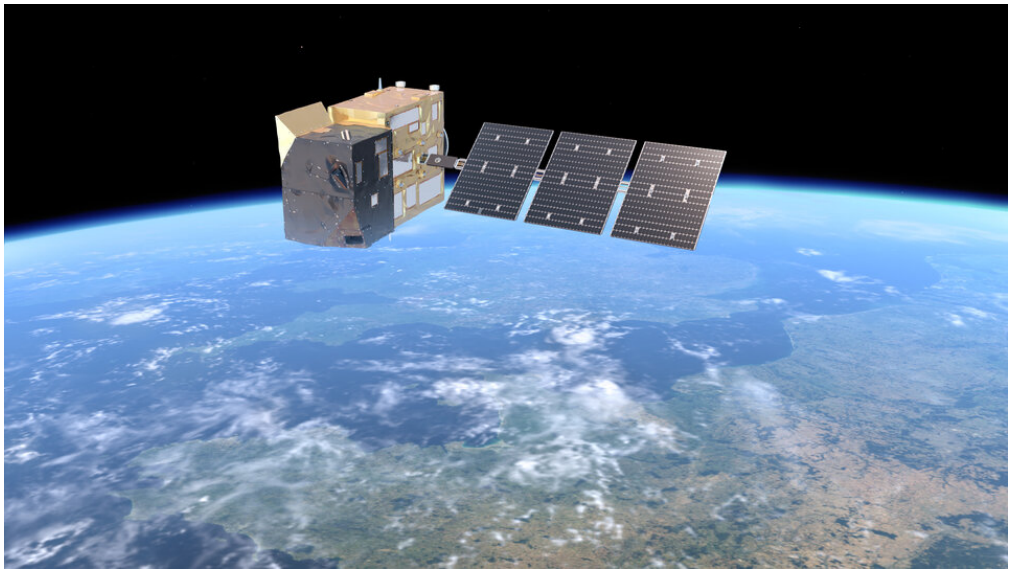


Figura 2.3: CO2M, de la Agencia Espacial Europea.
Fuente: [6].

CO2M representa una iniciativa pionera de la Agencia Espacial Europea (ESA) dentro del programa Copernicus, diseñada específicamente para monitorizar las emisiones antropogénicas de CO_2 a escala global. Programada para lanzarse a partir de 2025, esta misión constará de una constelación de tres satélites (CO2M-A, CO2M-B y CO2M-C) que trabajarán en conjunto para proporcionar una cobertura temporal muy superior a la de misiones anteriores.

El instrumento principal, denominado CO2I (Integrated CO_2 and NO_2 Imaging Spectrometer), es un espectrómetro de alta precisión que opera en múltiples bandas espectrales: 405-490 nm (visible), 747-773 nm (infrarrojo cercano) y dos rangos de infrarrojo de onda corta (1,59-1,67 µm y 1,98-2,08 µm). Los detectores utilizados son de tecnología MCT (*Mercury Cadmium Telluride*). Con una resolución espacial de 4 km^2, el sistema está optimizado para identificar fuentes puntuales como centrales eléctricas y complejos industriales.

Una característica distintiva de CO2M es su sistema integrado de corrección atmosférica, que combina el instrumento CLIM (imagen de nubes en tres bandas) y el MAP (polarímetro multiangular) para filtrar datos afectados por cubierta nubosa. La constelación completa logrará una revisita global cada 3,5 días, útil para la monitorización efectiva de áreas urbanas dinámicas, con una masa aproximada de 900 kg por satélite [6] [7].

2.4. TanSat (*Chinese Carbon Dioxide Observation Satellite*)

Figura 2.4: TanSat.
Fuente: [8].

TanSat, lanzado en diciembre de 2016 por la Academia China de Ciencias, se convirtió en el primer satélite chino dedicado específicamente a la monitorización de CO_2. Su objetivo principal es medir la concentración de dióxido de carbono en la atmósfera con alta precisión y contribuir a la comprensión del ciclo global del carbono.

TanSat incorpora dos instrumentos principales: un espectrómetro de CO_2 de alta resolución (CarbonSpec) que opera en las bandas de 0,76 µm (banda de oxígeno A), 1,61 µm y 2,06 µm (bandas de CO_2), y un polarímetro de nube y aerosol (CAPI) que observa en siete bandas espectrales desde el ultravioleta hasta el infrarrojo cercano. Los detectores utilizados son matrices de fotodiodos de InGaAs (Indio, Galio, Arsénico) para las bandas de CO_2.

El telescopio utiliza un diseño óptico tipo Offner que proporciona una resolución espacial de 2 km $\times$ 2 km con un ancho de barrido de 20 km. Este satélite orbita a una altitud de aproximadamente 700 km en una órbita heliosíncrona con inclinación de 98,2°, similar a la de OCO-2. Su masa total es de aproximadamente 620 kg [9].

2.5. MicroCarb (*Microsatellite for Carbon Observation*)

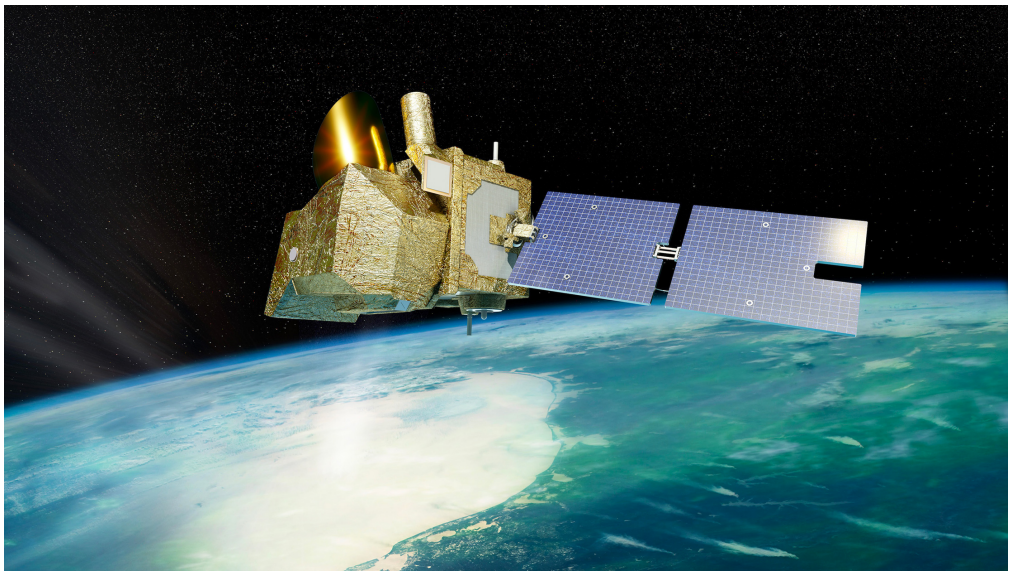


Figura 2.5: Imagen artística de Microcarb.
Fuente: [10]

MicroCarb es una misión desarrollada por la agencia espacial francesa CNES, con lanzamiento previsto en 2025. Representa un enfoque innovador al utilizar una plataforma de microsatélite para realizar mediciones precisas de CO_2 atmosférico, demostrando que misiones efectivas pueden implementarse con presupuestos y masas reducidas.

El instrumento principal de MicroCarb es un espectrómetro de rejilla compacto que opera en cuatro bandas espectrales: 0,76 µm (O_2), 1,27 µm (O_2), 1,6 µm (CO_2) y 2,01 µm (CO_2). Esta configuración permite mediciones precisas de concentración de CO_2 con una resolución espacial aproximada de 4,5 km × 9 km. Permite además alcanzar una resolución máxima de 2 km × 2 km en modo de escaneo urbano.

MicroCarb operará en una órbita heliosíncrona a aproximadamente 650 km de altitud con un período de revisita de 25 días. Lo que destaca de esta misión es su masa total de solo 180 kg, apreciablemente menor que otras misiones similares, demostrando los avances en la miniaturización de tecnologías espaciales [11].

2.6. GHGSat

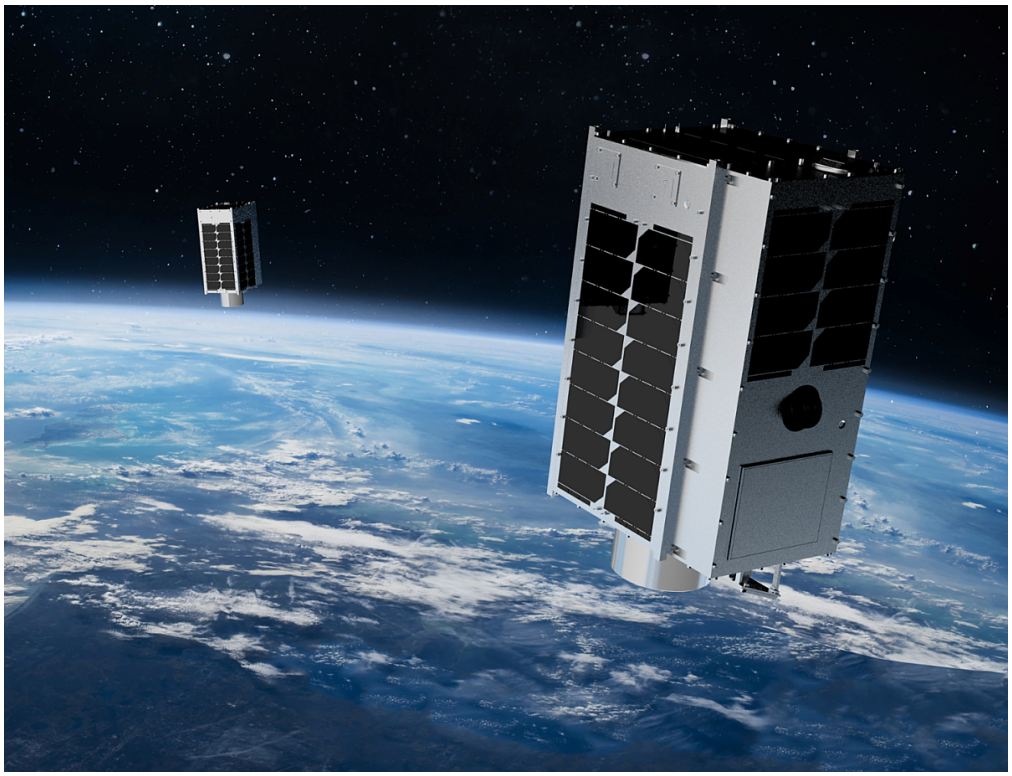


Figura 2.6: Constelación GHGSat.
Fuente: [12]

La constelación GHGSat representa un enfoque comercial a la monitorización de gases de efecto invernadero. Esta constelación se dedica a la detección de emisiones de metano y CO_2 a nivel de instalaciones individuales, ofreciendo una resolución espacial excepcionalmente alta para aplicaciones específicas de monitorización de fuentes puntuales.

Cada satélite GHGSat está equipado con un espectrómetro de imagen compacto denominado WAF-P (Wide-Angle Fabry-Perot) que opera en bandas del infrarrojo cercano específicas para la detección de metano y CO_2. Los detectores utilizados son de tecnología InGaAs, optimizados para la detección en el rango espectral de 1600-1700 nm. El telescopio emplea un diseño reflectivo con una apertura de aproximadamente 5 cm.

La tecnología patentada permite una resolución espacial de aproximadamente 25 metros, lo que permite la identificación precisa de fuentes de emisión. Los satélites GHGSat operan en órbitas heliosíncronas a altitudes entre 500-550 km con una inclinación de 97,5°. Cada unidad tiene una masa de aproximadamente 15-20 kg, ejemplificando el potencial de los nanosatélites para aplicaciones de observación terrestre especializadas [13].

2.7. Hallazgos clave

La tabla siguiente resume las características principales de las misiones analizadas:

Tabla 2.1: Comparativa de misiones de observación de CO_2.

Parámetro	**OCO-2**	**GOSAT2**	**CO2M**	**TanSat**	**MicroCarb**	**GHGSat**
Detector	HgCdTe/Si	Si/InGaAs/MCT	MCT	InGaAs	MCT	InGaAs
Diámetro pupila (cm)	11	10	15	12	8	5
GSD (km)	1,3×2,3	10	2	2	4,5	0,025
Swath (km)	10	10	250	20	250	12
Altura orbital (km)	705	613	735	700	650	500
Tipo órbita	SSO 98,2°	SSO 98,1°	SSO 97,7°	SSO 97,4°	SSO 98,0°	SSO 97,6°
Constelación	No	No	Sí (3)	No	No	Sí (10+)
Tipo telescopio	Cassegrain	FTS	Cassegrain	Offner	Off-axis	Reflective
Duración (años)	2 (+10)	5	7,5	5	3	5
Revisita (días)	16	6	3,5	16	25	5
Masa (kg)	450	1800	900	600	180	15
Paneles solares (m^2)	8	24	12	10	4	1,8
Cobertura	Global	Global	Global	Global	Global	Puntual

Del análisis comparativo de estas misiones se pueden extraer los siguientes hallazgos clave, que serán útiles para inspirar la misión encomendada:

- Las misiones analizadas muestran diferentes aproximaciones a la monitorización de CO_2, desde satélites individuales de gran tamaño (GOSAT-2) hasta constelaciones de microsatélites (GHGSat), pasando por soluciones intermedias como MicroCarb.
- Se utilizan diferentes tecnologías de detección según los requerimientos específicos de cada misión, desde fotodiodos de silicio (OCO-2) hasta detectores de InGaAs (TanSat, GHGSat) y MCT (GOSAT-2, CO2M).
- Las constelaciones como CO2M y GHGSat logran períodos de revisita mucho menores (3,5 y 5 días respectivamente) que los satélites individuales, mejorando la capacidad de monitorización temporal.
- La selección universal de órbitas heliosíncronas en todas las misiones analizadas demuestra su importancia crítica para la observación terrestre. Esta configuración orbital garantiza condiciones de iluminación consistentes al pasar sobre cualquier punto de la superficie terrestre a la misma hora solar local, lo que permite comparar datos entre pasadas de forma fiable. Además, la órbita heliosíncrona facilita la cobertura global sistemática con períodos de revisita predecibles, optimizando tanto la adquisición de datos como la planificación de misiones de monitorización ambiental.

Capítulo 3

Conceptos Previos

Contenido

3.1. Calidad de Imagen 16

3.1.1. *Aliasing* 16

3.1.2. *Ground Sampling Distance* (GSD) 17

3.1.3. Frecuencia de Nyquist(f_{Ny}) 17

3.1.4. *Modulation Transfer Function* (MTF) 18

3.1.5. *Signal To Noise Ratio* (SNR) 20

3.1.6. *Field of View* (FoV). *Swath* 23

3.2. Espectro electromagnético. Bandas de absorción 24

3.3. Detectores 27

3.3.1. Tipos de detectores 28

3.3.2. *Time Delay and Integration* (TDI) 32

3.3.3. Estrategias de Escaneo en Sistemas de Observación por Satélite . . 32

3.3.4. Disposición de detectores 35

3.4. Óptica. Telescopios 36

3.4.1. Parámetros ópticos relevantes 37

3.4.2. Tipos de telescopio 38

3.4.3. Filtros 41

3.5. Parámetros Orbitales 43

3.5.1. Inclinación (i) 44

3.5.2. Longitud del Nodo Ascendente (Ω) 44

3.5.3. Hora de paso local (LTAN) 45

3.5.4. Argumento del Perigeo (ω) 45

3.5.5. Anomalía Verdadera (ν) 45

3.6. Perturbaciones de la órbita 46
3.6.1. Achatamiento Terrestre 46
3.6.2. Decaimiento Orbital 46
3.6.3. Radiación Solar 47
3.6.4. Problema del tercer cuerpo 47
3.6.5. Órbitas de especial interés 48
3.6.6. Consideración de las perturbaciones orbitales 50

Este apartado sienta las bases teóricas necesarias para definir la carga útil y buscar la solución de diseño.

3.1. Calidad de Imagen

3.1.1. *Aliasing*

Figura 3.1: Ejemplo del efecto del aliasing sobre un patrón visual.
Fuente: [14]

El *aliasing* es un fenómeno que ocurre cuando una señal con componentes de alta frecuencia es muestreada a una tasa insuficiente, lo que provoca que las frecuencias originales se mezclen y aparezcan como frecuencias más bajas en el espectro reconstruido. En términos de imágenes de satélite, el *aliasing* puede causar artefactos visuales o distorsiones en los datos, degradando la calidad de la imagen capturada. Este problema está directamente relacionado con la frecuencia de Nyquist, que establece que la tasa de muestreo debe ser al menos el doble de la frecuencia espacial máxima presente en la escena para evitar *aliasing*.

3.1.2. *Ground Sampling Distance* (GSD)

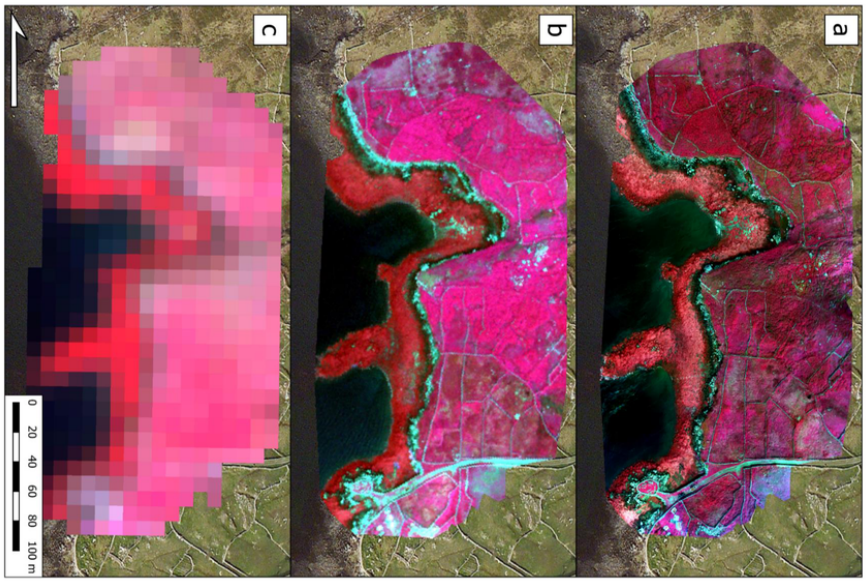


Figura 3.2: Efectos de distintos GSD en imágenes tomadas por satélite.
Fuente: [15]

La **Ground Sampling Distance (GSD)** es la distancia en terreno que corresponde a un solo píxel en la imagen capturada por el satélite, determinando la resolución espacial del sistema. Se calcula mediante la relación geométrica entre el tamaño del píxel del detector (p), la altura orbital (h) y la distancia focal del telescopio (f):

$$\text{GSD} = \frac{p \cdot h}{f} \tag{3.1.1}$$

3.1.3. Frecuencia de Nyquist(f_{Ny})

La **frecuencia de Nyquist** es un concepto fundamental en la teoría de muestreo que define la máxima frecuencia espacial que un sistema de imagen puede capturar sin distorsión por *aliasing*. En el contexto de un satélite de observación terrestre, esta frecuencia está directamente ligada a la resolución espacial (GSD) y determina la capacidad del sistema para preservar detalles finos en la escena observada.

La frecuencia de Nyquist (f_{Nyquist}) se calcula como:

$$f_{\text{Nyquist}} = \frac{1}{2 \cdot p} \tag{3.1.2}$$

donde el tamaño del píxel actúa como el intervalo de muestreo espacial.

3.1.4. *Modulation Transfer Function* (MTF)

La Función de Transferencia de Modulación (MTF) es una herramienta matemática fundamental para evaluar la calidad óptica de un sistema de observación terrestre. La MTF mide la capacidad del sistema óptico para reproducir el contraste de la imagen en función de la frecuencia espacial [16] [17].

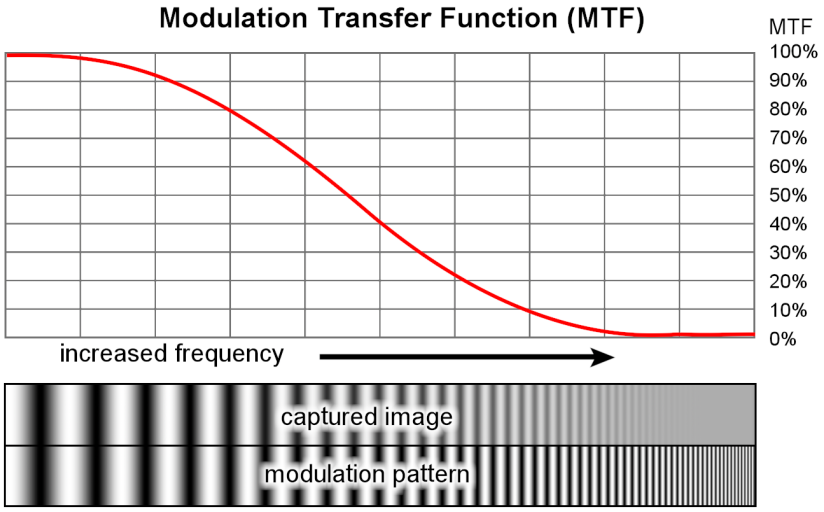


Figura 3.3: Representación visual de la MTF.
Fuente: [18].

En términos simples, la MTF determina cómo un sistema óptico transfiere el contraste desde el objeto hasta la imagen final. Cuando se observa un patrón de franjas blancas y negras (alto contraste), la MTF indica qué porcentaje de ese contraste se preserva en la imagen final. Al aumentar la frecuencia espacial (franjas más densas), el contraste disminuye.

La MTF total del sistema es el producto de las MTF de cada uno de los subsistemas ópticos y electrónicos relevantes. Los principales contribuyentes son:

- **MTF de Difracción:** Limitación física impuesta por el diámetro de la apertura del telescopio y la longitud de onda de observación. Para una apertura circular sin obstrucción, la MTF de difracción se calcula mediante:

$$MTF_{\text{difracción}}(f_x) = \frac{2}{\pi}\left[\arccos\left(\frac{f_x}{f_{co}}\right) - \frac{f_x}{f_{co}}\sqrt{1-\left(\frac{f_x}{f_{co}}\right)^2}\right] \tag{3.1.3}$$

  donde $f_{co} = D/\lambda$ es la frecuencia de corte, D el diámetro de la apertura y λ la longitud de onda.

Para un telescopio con obstrucción central[1], como es el caso de los telescopios Cassegrain y Korsch, su MTF de difracción vendrá definida, en función del diámetro de obstrucción d_{obs}. Para el presente trabajo se establecerá $R_{obs} = 0,2$ como valor inicial.[19]

$$\mathrm{OTF}_{\mathrm{diff}} = \frac{A + B + C}{1 - R^2} \tag{3.1.4}$$

$$R = \frac{d_{\mathrm{obs}}}{D_o} \tag{3.1.5}$$

$$X = \frac{f_x}{f_{\mathrm{oco}}}, \quad Y = \frac{X}{R}, \quad \alpha = \cos^{-1}\left(\frac{1 + R^2 - 4X^2}{2R}\right) \tag{3.1.6}$$

$$A = \begin{cases} \frac{2}{\pi}\left[\cos^{-1}(X) - X\sqrt{1 - X^2}\right], & \text{si } 0 \leq X \leq 1 \\ 0, & \text{en otro caso} \end{cases} \tag{3.1.7}$$

$$B = \begin{cases} \frac{2R^2}{\pi}\left[\cos^{-1}(Y) - Y\sqrt{1 - Y^2}\right], & \text{si } 0 \leq Y \leq 1 \\ 0, & \text{en otro caso} \end{cases} \tag{3.1.8}$$

$$C = \begin{cases} -2R^2, & \text{si } 0 < X \leq \frac{1-R}{2} \\ \frac{2R}{\pi}\sin\alpha + \frac{1+R^2}{\pi}\alpha - \frac{2(1-R^2)}{\pi}\tan^{-1}\left[\left(\frac{1+R}{1-R}\right)\tan\left(\frac{\alpha}{2}\right)\right] - 2R^2, & \text{si } \frac{1-R}{2} < X < \frac{1+R}{2} \\ 0, & \text{si } X \geq \frac{1+R}{2} \end{cases} \tag{3.1.9}$$

Para calcular el requerimiento de MTF, se impone $f_x = f_{Ny} * f$.

- **MTF de Aberraciones:** Degradación adicional debida a imperfecciones ópticas (aberraciones esféricas, cromáticas, etc.). Un valor típico para sistemas bien diseñados es $MTF_{\mathrm{aberraciones}} \approx 0{,}95$.
- **MTF de Fabricación:** Defectos en la fabricación de los espejos/lentes, típicamente $MTF_{\mathrm{fabricación}} \approx 0{,}98$.
- **MTF de Alineamiento:** Errores en el alineamiento de los elementos ópticos. El valor depende del tipo de telescopio.
- **MTF por Vibraciones y Termoelasticidad:** Cambios entre el alineamiento en tierra y en órbita (vibraciones del lanzamiento y efectos térmicos), típicamente $\approx 0{,}99$ y $\approx 0{,}95$ respectivamente.
- **MTF del Detector:** Es un parámetro definido por el diseño y tecnología del detector, normalmente proporcionado por el fabricante.

[1] obstrucción central que bloquea parte de la luz (normalmente un espejo) que entra al sistema óptico, siendo d_{obs} la medida de dicha obstrucción

Por tanto, la MTF del sistema será:

$$MTF_{\text{sistema}} = MTF_{\text{difracción}} \times MTF_{\text{aberraciones}} \times MTF_{\text{fabricación}} \times MTF_{\text{alineamiento}} \times MTF_{\text{vibraciones}} \times MTF_{\text{termoelástico}} \times MTF_{\text{detector}} \tag{3.1.10}$$

3.1.5. *Signal To Noise Ratio* (SNR)

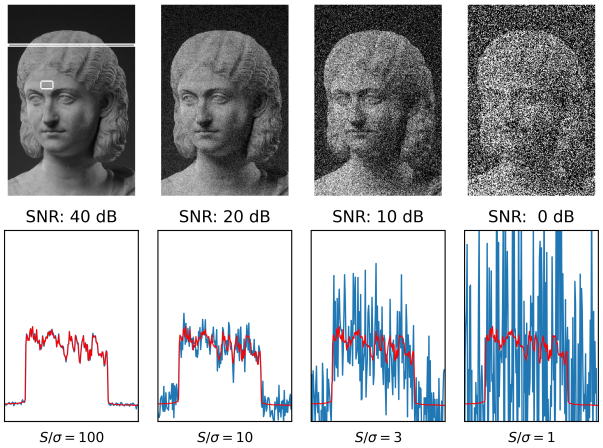


Figura 3.4: Efecto de la SNR.
Fuente: [20]

La *Signal-to-Noise Ratio* (SNR, o relación señal-ruido) es un indicador clave que indica la calidad radiométrica de un sistema de observación óptica. Representa la relación entre la señal útil (fotones detectados provenientes de la escena) y el ruido total (suma de todas las fuentes de ruido del sistema) en el detector. Un SNR alto significa que la señal es claramente distinguible del ruido, lo que permite obtener imágenes precisas y fiables, mientras queun SNR bajo implica que la señal está enmascarada por el ruido, dificultando la detección de detalles y la calidad de los datos obtenidos[16][21].

Matemáticamente se expresa como:

$$SNR = \frac{\text{Señal}}{\text{Ruido total}} \tag{3.1.11}$$

donde la señal y el ruido se expresan en número de electrones generados (e^-) en el detector durante el tiempo de integración.

Cálculo de la Señal

Se considera una radiancia de referencia L_{ref} expresada en $\mathrm{W\,m^{-2}\,sr^{-1}\,\mu m^{-1}}$, centrada en una longitud de onda λ_c con un ancho de banda espectral $\Delta\lambda$. La radiancia incidente se transmite a través del sistema óptico con una eficiencia total condicionada por la transmisión óptica del telescopio τ y la eficiencia cuántica del detector η.

La irradiancia E en el plano focal se estima mediante:

$$E = \frac{\pi \cdot \tau \cdot \Delta\lambda \cdot L_{\text{ref}}}{1 + 4\,T\#^2} \tag{3.1.12}$$

donde $F\# = \frac{f}{D}$ es la relación focal del sistema, con f la distancia focal y D el diámetro de la apertura.[2].

La cantidad de electrones generados por píxel durante el tiempo de integración se calcula como:

$$N_e = \frac{E \cdot A_p \cdot \eta \cdot \lambda_c \cdot TDI \cdot t_{\text{int}}}{h \cdot c} \tag{3.1.13}$$

donde:

- $A_p = d_x \cdot d_y$ es el área de un píxel en el detector.
- TDI es el número de etapas del sistema Time Delay Integration.[3].
- $t_{\text{int}} = \frac{\text{GSD}}{v_{\text{orb}}}$ es el tiempo de integración basado en la velocidad del satélite sobre el suelo.
- h es la constante de Planck.
- c es la velocidad de la luz.

El tiempo de integración t_{int} representa la duración durante la cual la imagen de un punto en la superficie terrestre permanece proyectada sobre un mismo píxel del detector mientras el satélite avanza en su órbita. Este tiempo depende directamente de la *Ground Sampling Distance* (GSD) y de la velocidad del satélite sobre el suelo v_{orb}. Se calcula como:

$$t_{\text{int}} = \frac{\text{GSD}}{v_{\text{orb}}} \tag{3.1.14}$$

[2]Para los sistemas con obstrucción central se utilizará la relación focal equivalente $T\#$, definida más adelante en la ecuación (3.4.1)

[3]Dicho concepto se explicará más adelante en 3.3.2

La velocidad orbital del satélite a una altitud H se estima aplicando la tercera ley de Kepler:

$$v_{\text{orb}} = \sqrt{\frac{\mu}{R_T + H}} \tag{3.1.15}$$

donde $\mu = 3{,}986 \times 10^{14}\,\text{m}^3/\text{s}^2$ es el parámetro gravitacional estándar de la Tierra y $R_T = 6371\,\text{km}$ es el radio medio terrestre. Este tiempo de integración se utiliza directamente en el cálculo de N_e, y representa el tiempo efectivo durante el cual se acumulan fotones en cada píxel del detector. Se emplea la velocidad orbital, ligeramente superior a la velocidad de la traza sobre el suelo, lo que proporciona un tiempo de integración conservador.

Cálculo del Ruido Total

El ruido total es la raíz cuadrada de la suma de las varianzas de todas las fuentes de ruido independientes del sistema, expresadas en número de electrones:

$$\text{Ruido total} = \sqrt{\sigma^2_{\text{fotónico}} + \sigma^2_{\text{stray}} + \sigma^2_{\text{dark}} + \sigma^2_{\text{lectura}} + \sigma^2_{\text{amp}} + \sigma^2_{\text{vídeo}} + \sigma^2_{\text{jitter}} + \sigma^2_{\text{EMC}} + \sigma^2_{\text{cuant}}} \tag{3.1.16}$$

En la implementación numérica se sustituye σ_{stray} por N_{nonlin}.

Las principales fuentes de ruido son:

- **Ruido fotónico (shot noise):** Raíz cuadrada del número de electrones generados por la señal, debido a la naturaleza estadística de la llegada de fotones ($\sqrt{N_{e^-}}$).
- **Ruido de corriente de oscuridad:** Asociado a electrones generados térmicamente en el detector, incluso en ausencia de luz.
- **Ruido de lectura:** Introducido por la electrónica de lectura del detector.
- **Otros ruidos electrónicos:** Como preamplificadores, cadena de vídeo, jitter, EMC, etc.
- **Ruido de cuantificación:** Introducido al digitalizar la señal.

Expresión Final de la SNR

Realizando el cociente entre señal y ruido, resulta la ecuación:

$$SNR = \frac{N_{e^-}}{\sqrt{N_{e^-} + \sigma^2_{\text{stray}} + \sigma^2_{\text{dark}} + \sigma^2_{\text{lectura}} + \sigma^2_{\text{amp}} + \sigma^2_{\text{vídeo}} + \sigma^2_{\text{jitter}} + \sigma^2_{\text{EMC}} + \sigma^2_{\text{cuant}}}} \tag{3.1.17}$$

3.1.6. *Field of View* (FoV). *Swath*

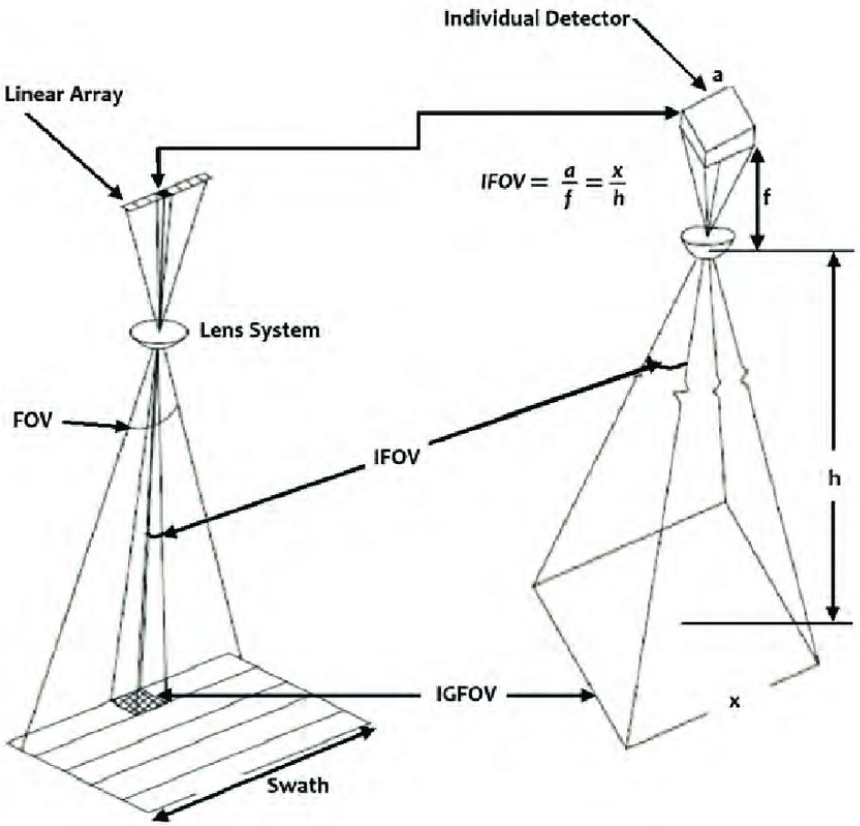


Figura 3.5: Esquema visual del *swath* y FoV.
Fuente: [22].

El ***swath*** se define como la anchura de la franja de terreno que el satélite puede observar en una pasada. Se mide en kilómetros y depende tanto de la óptica del instrumento como de la altura orbital. Un *swath* mayor permite cubrir más superficie terrestre en menos tiempo, lo que reduce el tiempo de revisita necesario para obtener mapas completos del área de interés. Se relaciona con el GSD mediante la ecuación:

$$Swath = GSD \cdot N_{pix} \tag{3.1.18}$$

Siendo N_{pix} el número de píxeles situados en línea perpendicular al sentido de movimiento del satélite.

***Field of View* (FoV)** es el ángulo total que abarca el sistema óptico del instrumento, es decir, el ángulo desde el cual la óptica puede captar luz y formar imagen sobre el detector. El FoV se mide en grados o milirradianes y determina, junto con la altura orbital, el swath en tierra.

Ambos valores se relacionan de la siguiente manera:

$$Swath = 2h \cdot \tan\left(\frac{FoV}{2}\right) \tag{3.1.19}$$

Siendo h la altura orbital.

En este estudio se supone que solo se observará al **nadir**, es decir, el instrumento siempre apunta directamente hacia abajo, al punto situado justo debajo del satélite en la superficie terrestre (*sub satellite point*). Esto significa que no se realizarán observaciones oblicuas (*off-nadir*), lo que simplifica el diseño óptico y la planificación de misión. Asimismo, el punto subsatélite es el punto de máxima resolución y mínima distorsión geométrica en la imagen captada, con lo que se maximiza la calidad geométrica y radiométrica de la imagen y se simplifica la calibración y el procesamiento de datos. Además, el cálculo del GSD y del MTF es directo, ya que no hay degradación por ángulo de observación. Tampoco se considerará la deformación producida por la esfericidad de la Tierra, considerando la superficie a capturar como plana, ni las deformaciones producidas en los extremos del array debido a su proyección.

3.2. Espectro electromagnético. Bandas de absorción

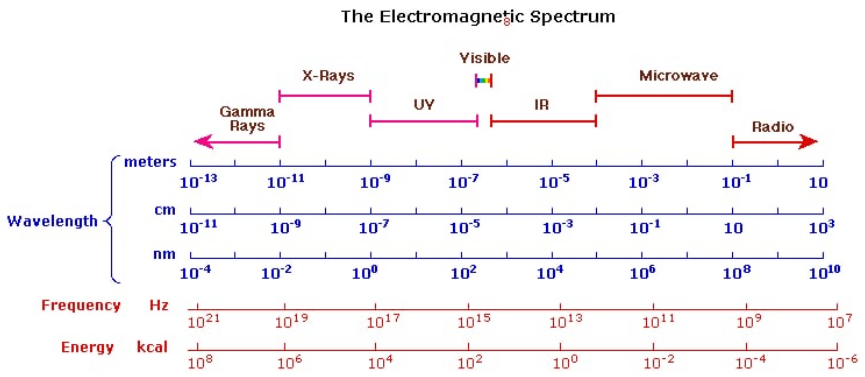


Figura 3.6: Representación del espectro electromagnético con las longitudes de onda, frecuencias y energías asociadas.
Fuente: [23].

El espectro electromagnético abarca todas las formas de radiación, desde los rayos gamma (longitudes de onda ultracortas) hasta las ondas de radio (longitudes de onda largas). Cada

elemento químico absorbe o emite radiación en frecuencias específicas, actuando como una «huella digital» única. Estas interacciones se miden mediante espectrómetros, que identifican los patrones de absorción para determinar la composición química de un objeto o atmósfera. La detección por satélite aprovecha estas propiedades para analizar gases en la Tierra, seleccionando bandas espectrales donde la señal del gas objetivo es clara y diferenciable de interferencias.

Ventanas de observación y transmitancia atmosférica

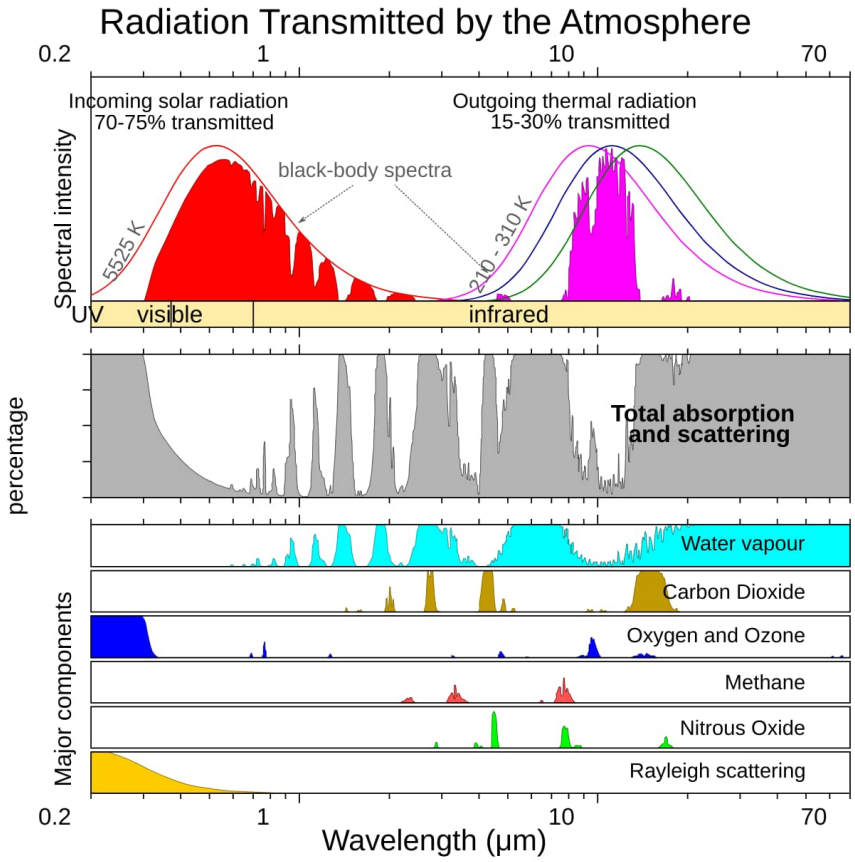


Figura 3.7: Radiación transmitida por la atmósfera y bandas de absorción.
Fuente: [24].

La atmósfera terrestre no es transparente a todas las longitudes de onda. Gases como el CO_2, el vapor de agua y el ozono absorben fuertemente en regiones específicas, creando «bandas de absorción» que bloquean la radiación. Por ejemplo, el vapor de agua absorbe en el infrarrojo medio (6–8 μm) y el ozono en el ultravioleta (200–300 nm). Sin embargo, existen «ventanas atmosféricas» donde la radiación atraviesa con menor interferencia, como la región de 8–12 μm (infrarrojo térmico). Estas ventanas son críticas para la teledetección, ya que permiten a los satélites capturar la radiación emitida o reflejada por la superficie terrestre [25].

Bandas de absorción del CO_2

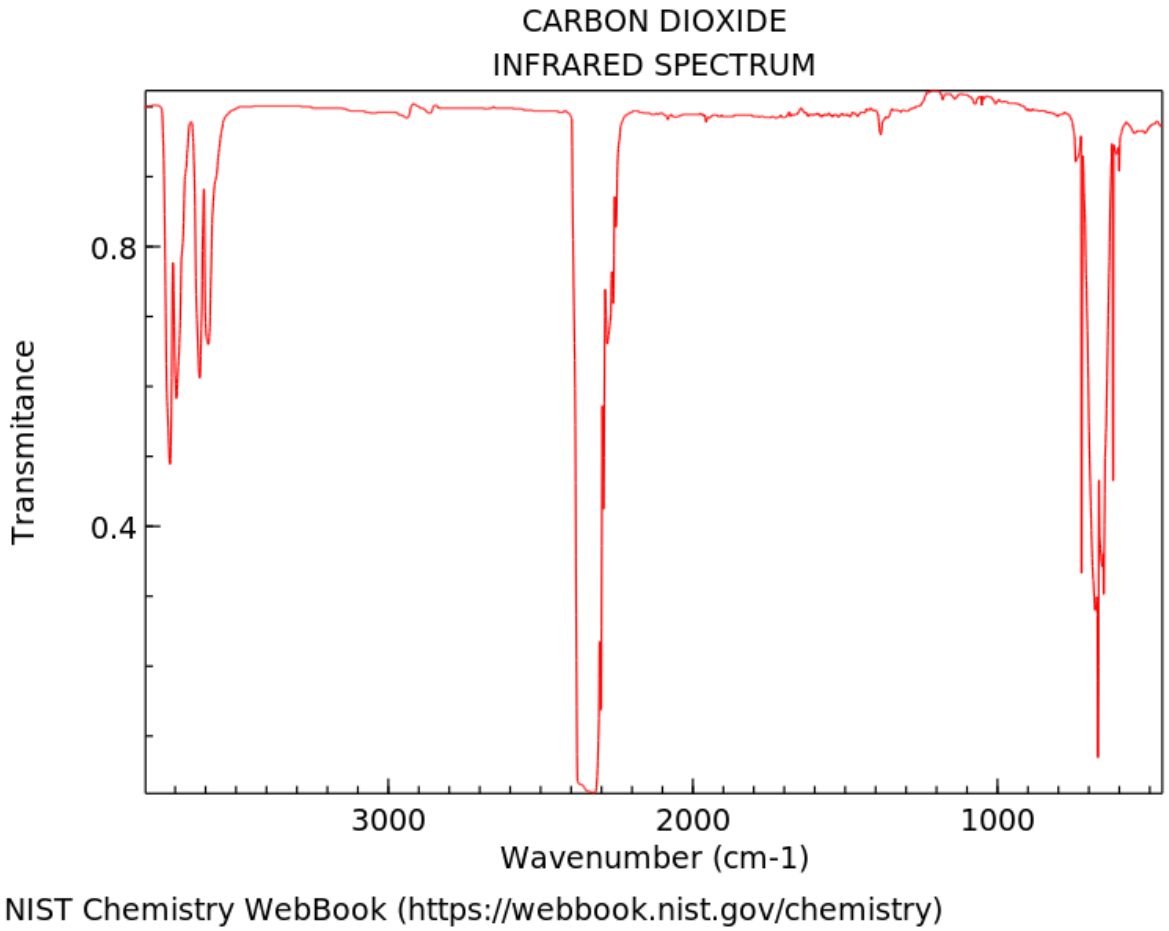


Figura 3.8: Bandas de Transmitancia del CO_2 en función del número de onda. Fuente: [26]

El CO_2 presenta bandas de absorción características en diferentes regiones espectrales. Las bandas en torno a 1,6 μm y 2,0 μm en el infrarrojo cercano son utilizadas por satélites como OCO-2 y GOSAT para medir la concentración columnar de CO_2 (XCO_2) mediante análisis de reflectancia solar. Estas bandas permiten la detección de CO_2 atmosférico aprovechando la absorción en la radiación solar reflejada por la superficie terrestre [2].

La banda de 14–16 μm, aunque presenta fuerte absorción de CO_2, resulta inadecuada para mediciones de concentración desde plataformas espaciales. Esta banda mide principalmente temperatura atmosférica en lugar de concentraciones de CO_2, ya que la opacidad atmosférica impide la penetración hasta niveles superficiales donde ocurren las emisiones antropogénicas. Los algoritmos de recuperación requieren perfiles de temperatura extremadamente precisos, introduciendo incertidumbres que complican la separación entre efectos térmicos y variaciones de concentración. La fuerte absorción del CO_2 en 15 μm genera opacidad atmosférica elevada que limita la radiación detectada a niveles de la alta atmósfera, reduciendo la sensibilidad a

las concentraciones cerca de la superficie terrestre [27].

3.3. Detectores

Los detectores de imágenes utilizados en satélites de observación son dispositivos electrónicos que convierten la radiación electromagnética en señales eléctricas. Este proceso se basa en el efecto fotoeléctrico, mediante el cual los fotones incidentes generan pares electrón-hueco en materiales semiconductores. Los detectores modernos se componen de matrices de píxeles, capaces de acumular carga eléctrica proporcional a la intensidad de la radiación recibida. Su estructura básica incluye una capa fotosensible de silicio u otro material semiconductor, junto con componentes electrónicos para la gestión, amplificación y lectura de estas cargas.

Para operar en el espacio, estos detectores deben superar condiciones particularmente exigentes: radiación cósmica, variaciones extremas de temperatura y ausencia de mantenimiento físico. Además, se requiere alta sensibilidad en las longitudes de onda de interés, bajo nivel de ruido para detectar señales débiles y un consumo energético reducido, dado que las plataformas de satélite disponen de recursos limitados. Parámetros como la resolución espacial, temporal y radiométrica son determinantes en la calidad de los datos adquiridos y su utilidad en distintas aplicaciones científicas y operativas [28].

3.3.1. Tipos de detectores

Sensores CCD (Charge-Coupled Device)

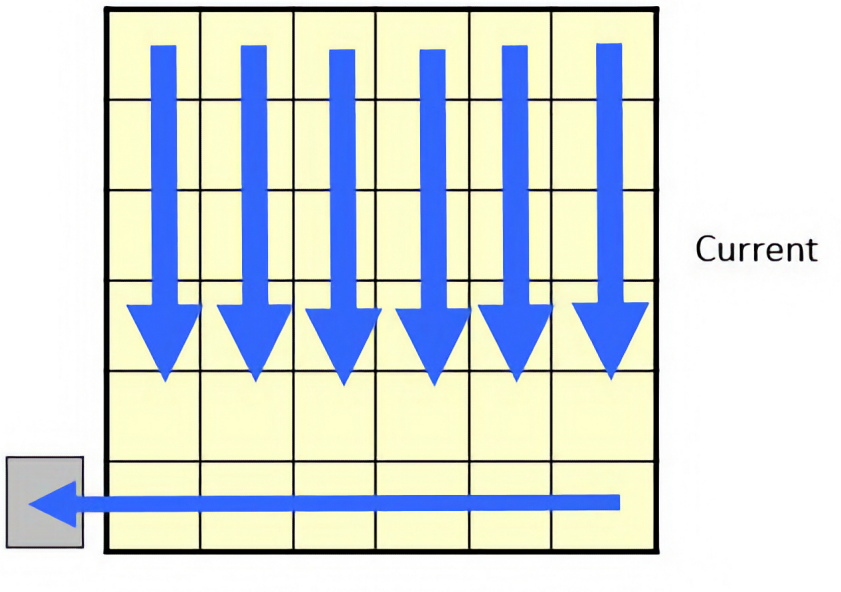


Figura 3.9: Diagrama de funcionamiento de un sensor CCD.
Fuente: [29].

Los sensores CCD (Dispositivo de Carga Acoplada) han sido durante años el estándar en misiones de observación espacial, gracias a su excelente calidad de imagen y rendimiento en condiciones de baja iluminación. Funcionan acumulando carga en pozos de potencial dentro de un sustrato semiconductor, transfiriendo luego estas cargas secuencialmente a un nodo de salida para su lectura.

Destacan por su amplio rango dinámico, que puede duplicar el de los CMOS, y su bajo ruido de lectura, ideal para observaciones nocturnas o en bandas espectrales de baja radiancia. Presentan una notable uniformidad entre píxeles, reduciendo el ruido de patrón fijo y facilitando la calibración radiométrica. Además, al tener menos electrónica integrada en cada píxel, ofrecen un mayor factor de relleno, aprovechando mejor el área fotosensible.

Como aspectos menos favorables, presentan un consumo energético elevado, lo que puede ser un inconveniente en plataformas con recursos limitados. La dependencia de electrónica externa incrementa volumen y masa del sistema. También pueden sufrir el efecto *blooming*, donde un píxel saturado afecta a los adyacentes, y su velocidad de lectura, relativamente baja, restringe su capacidad para captar escenas dinámicas. Asimismo, su sensibilidad a la radiación espacial puede degradar su rendimiento en misiones prolongadas [28].

Sensores CMOS (Complementary Metal-Oxide Semiconductor)

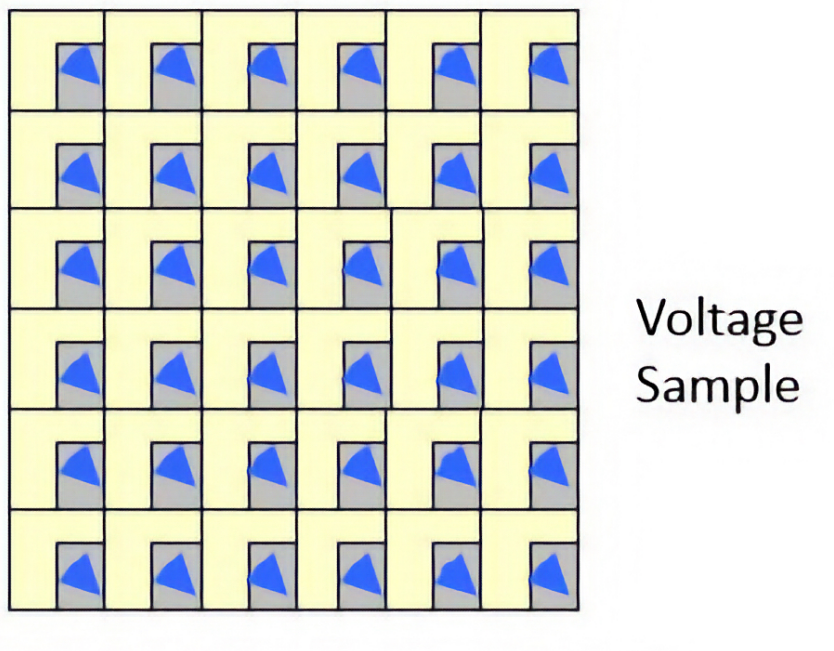


Figura 3.10: Diagrama de funcionamiento de un sensor CMOS.
Fuente: [29].

Los sensores CMOS han ganado protagonismo en aplicaciones espaciales en los últimos años, debido a sus ventajas operativas en entornos con restricciones de recursos. A diferencia de los CCD, los CMOS integran circuitos de amplificación y procesamiento en cada píxel, permitiendo lecturas directas y selectivas.

Entre sus ventajas sobresalen su bajo consumo energético, típicamente entre una décima y una centésima parte del de un CCD comparable. Permiten lecturas parciales a alta velocidad, facilitando modos operativos flexibles, y su integración de funciones adicionales en el chip simplifica los sistemas, reduciendo su masa. También presentan una mayor tolerancia a la radiación, lo que los hace atractivos para misiones prolongadas.

Como desafíos, los primeros sensores CMOS mostraban mayores niveles de ruido de lectura y ruido de patrón fijo, aunque los desarrollos recientes han mitigado estas diferencias. El menor factor de relleno, debido a la electrónica en cada píxel, se compensa actualmente con microlentes que concentran la luz. Otro aspecto a considerar es el fenómeno *rolling shutter*, que puede distorsionar imágenes con objetos en rápido movimiento, aunque las versiones con *global shutter* resuelven este inconveniente. Finalmente, su sensibilidad en condiciones de baja iluminación, anteriormente inferior a la de los CCD, ha mejorado considerablemente con las tecnologías retroiluminadas [28].

Sensores HgCdTe

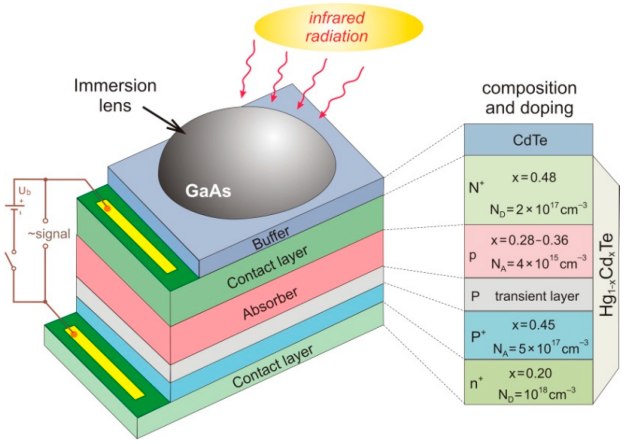


Figura 3.11: Disposición del detector MCT.
Fuente: [30].

Los sensores de telururo de mercurio y cadmio (abreviado como MCT o HgCdTe) son reconocidos por su versatilidad espectral, con un rango ajustable entre 1 y 30 µm mediante variaciones en la composición de la aleación. Esta adaptabilidad, unida a una eficiencia cuántica excepcional y tiempos de respuesta en nanosegundos, los posiciona como estándar en aplicaciones críticas como imágenes térmicas militares, astronomía infrarroja y detección de gases. Sus ventajas incluyen la capacidad de operar en modos fotoconductivo y fotovoltaico, alta detectividad incluso a temperaturas cercanas a ambiente y la posibilidad de crear estructuras de banda prohibida graduada para optimizar la absorción de fotones. Sin embargo, enfrentan desafíos como la toxicidad intrínseca del mercurio, que complica su fabricación y disposición final, y la sensibilidad a defectos cristalográficos durante el crecimiento epitaxial, lo que eleva los costos y reduce el rendimiento en dispositivos de gran área. Además, requieren sistemas de enfriamiento avanzados (criogénicos o mediante efecto Peltier) para minimizar el ruido térmico en aplicaciones de alta precisión, incrementando la complejidad del sistema global. A pesar de esto, siguen siendo insustituibles en aplicaciones que exigen cobertura espectral amplia y gran resolución térmica [31].

Sensores InGaAs

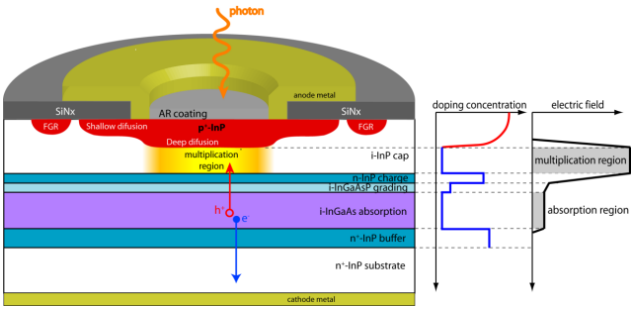


Figura 3.12: Disposición de capas típica del detector InGaAs.
Fuente: [32].

Los sensores de arseniuro de galio e indio (InGaAs) destacan por su capacidad para detectar radiación en el infrarrojo cercano (NIR), con un rango espectral típico entre 0,9 y 1,7 µm, extendible hasta 2,55 µm en versiones especializadas. Su estructura híbrida combina fotodiodos de InGaAs con circuitos CMOS, aprovechando una alta eficiencia cuántica y baja corriente oscura, lo que los hace ideales para aplicaciones que requieren precisión en condiciones de baja luminosidad. Entre sus ventajas destacan la sensibilidad extendida en el NIR, la capacidad de operar con requisitos de enfriamiento moderados y la ausencia de materiales tóxicos como el mercurio o el cadmio, cumpliendo con regulaciones ambientales. No obstante, presentan limitaciones como un ruido de corriente oscura significativo en entornos de alta temperatura, costos elevados debido a los procesos de fabricación complejos y fragilidad mecánica que exige manipulación cuidadosa. Además, su rendimiento en longitudes de onda superiores a 2 µm tradicionalmente ha sido inferior al de otras tecnologías, aunque avances recientes están mitigando esta brecha [33].

3.3.2. *Time Delay and Integration* (TDI)

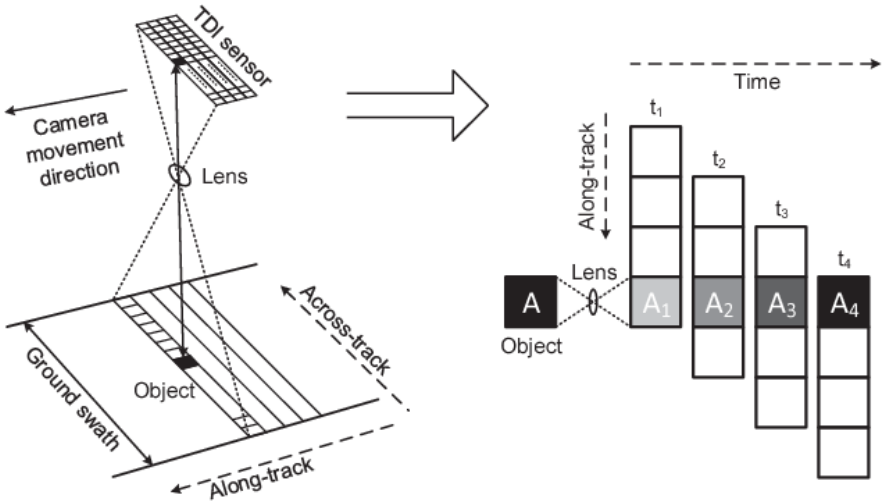


Figura 3.13: Esquema de funcionamiento del *Time Delay Integration*.
Fuente: [34].

El TDI permite aumentar el tiempo efectivo de exposición sin desenfoque por movimiento, mejorando la relación señal-ruido en condiciones de baja iluminación o cuando se requieren tiempos de integración cortos debido a la velocidad de desplazamiento[16].

Consiste en una serie de líneas de píxeles que capturan sucesivamente la misma escena, desplazando la carga generada de una línea a otra en sincronía con el movimiento de la imagen sobre el detector. Esto permite acumular la carga correspondiente a un mismo punto a lo largo de varias etapas, aumentando efectivamente el tiempo de exposición.

Esta técnica exige una sincronización precisa entre la velocidad de transferencia de carga y la velocidad de desplazamiento de la imagen proyectada. Cualquier desajuste afectaría la resolución espacial, provocando desenfoque. Al multiplicar el tiempo de exposición por el número de etapas, permite mejorar notablemente la sensibilidad sin sacrificar resolución espacial.

3.3.3. Estrategias de Escaneo en Sistemas de Observación por Satélite

Para adquirir imágenes de la superficie terrestre desde una órbita, es necesario definir una estrategia de escaneo que permita cubrir progresivamente la zona de interés. Estas estrategias determinan cómo se captura la información espacial y temporalmente, considerando tanto la configuración del detector como el movimiento relativo entre el satélite y la superficie terrestre. A continuación, se describen las principales técnicas empleadas en sistemas de observación orbital, junto con sus ventajas, limitaciones y ámbitos de aplicación característicos[16].

Tecnología Pushbroom (Barrido Lineal)

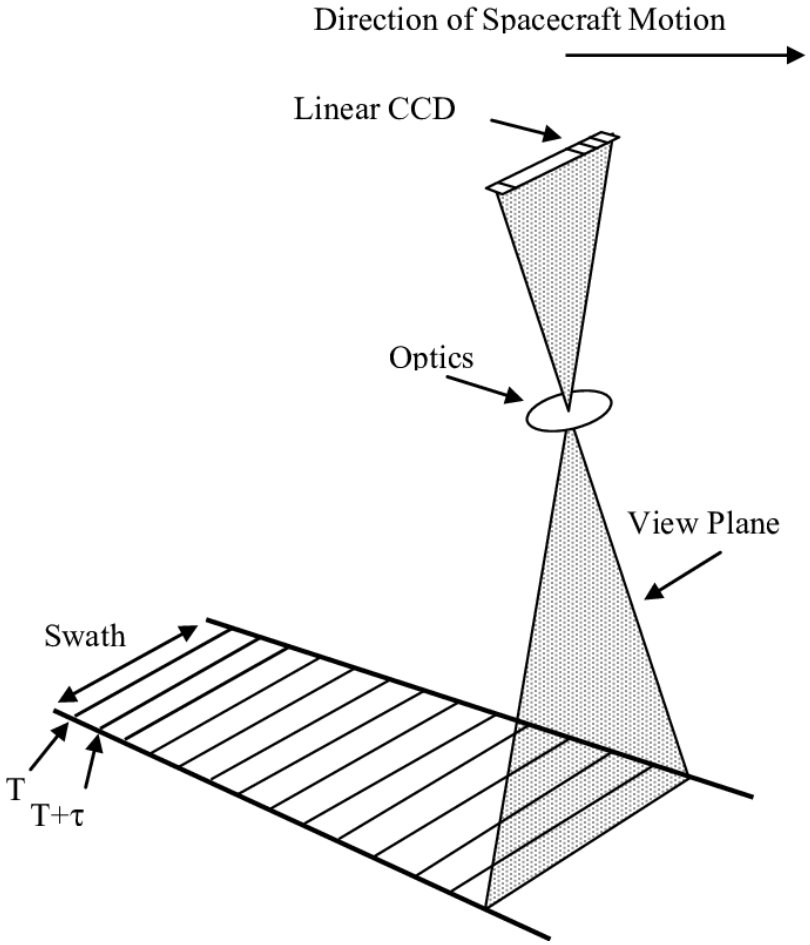


Figura 3.14: Funcionamiento del escaneo *Pushbroom*.
Fuente: [35].

La tecnología Pushbroom es una de las más utilizadas actualmente en observación terrestre desde satélites. Utiliza una línea de detectores dispuestos perpendicularmente a la dirección de vuelo. Esta línea captura simultáneamente una franja del terreno, mientras que el movimiento orbital permite generar la imagen completa.

Sus ventajas incluyen una mayor relación señal-ruido gracias al tiempo de observación más prolongado sobre cada punto y la ausencia de componentes mecánicos móviles, lo que aumenta su fiabilidad. Como desafío, presenta variaciones en la sensibilidad entre detectores individuales que pueden provocar franjas si no se calibra adecuadamente. Además, suele ofrecer una resolución espacial ligeramente inferior a los escáneres Whiskbroom para un mismo ancho de barrido.

Tecnología Whiskbroom (Barrido Transversal)

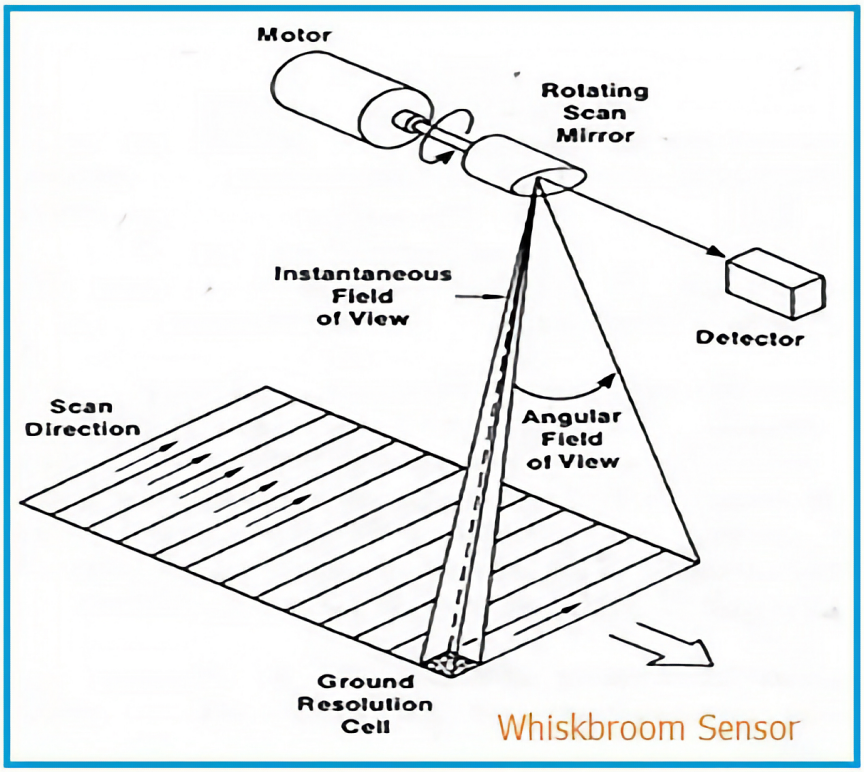


Figura 3.15: Funcionamiento del escaneo *Whiskbroom*.
Fuente: [36].

Los escáneres Whiskbroom funcionan mediante un espejo oscilante que barre perpendicularmente a la trayectoria del satélite, dirigiendo la luz hacia uno o pocos detectores.

Su principal ventaja es la alta resolución espacial alcanzable, ya que permite concentrar la observación en pequeñas áreas mediante un número reducido de detectores, facilitando la uniformidad radiométrica. Como contrapartida, incorporan componentes mecánicos móviles que aumentan el peso y la complejidad, y su menor tiempo de observación por píxel limita su sensibilidad en condiciones de baja iluminación. Además, su naturaleza secuencial restringe la velocidad máxima de adquisición.

Enfoque Step-Stare (Configuración de Imagen Estática)

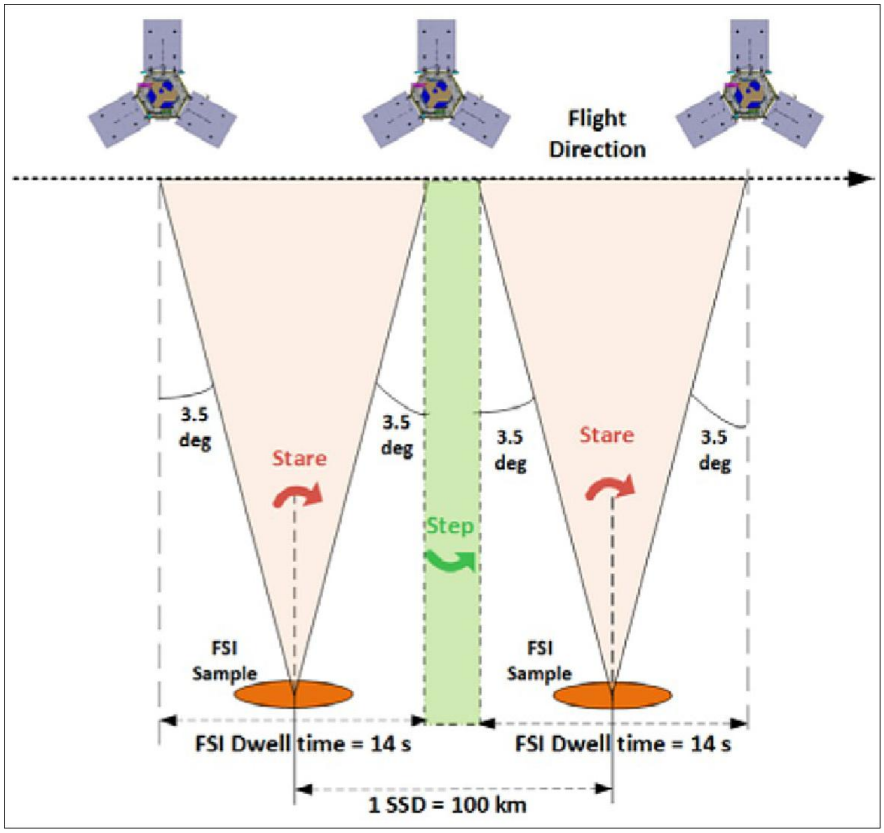


Figura 3.16: Funcionamiento del escaneo *Step-Stare*.
Fuente: [37].

El enfoque Step-Stare captura imágenes bidimensionales completas de forma discreta, similar a una cámara fotográfica. Entre exposiciones, el sistema óptico se reorienta hacia la siguiente área de interés.

Esta configuración maximiza la eficiencia en la recolección de fotones al dedicar todo el tiempo de exposición a cada imagen, logrando excelente relación señal-ruido. Permite también apuntamiento flexible, ideal para monitorización de emergencias o eventos dinámicos. Como aspectos a considerar, requiere compensar el movimiento de la plataforma para evitar desenfoque y presenta menor eficiencia en la cobertura continua de grandes áreas, debido al tiempo adicional necesario para reposicionar el sistema entre capturas.

3.3.4. Disposición de detectores

En el diseño del plano focal para la carga útil óptica, la disposición de los detectores es un aspecto clave que afecta tanto la calidad de imagen como la viabilidad técnica y económica

del satélite. Existen varias estrategias para el *layout* de detectores, entre las que destacan [38]:

- **Divisores de campo (Divoli o *Field Splitters*)**: Esta solución emplea elementos ópticos para dividir el campo de visión en regiones independientes, cada una dirigida a un detector específico. Permite optimizar la captación de diferentes bandas espectrales o maximizar el área cubierta por cada detector, facilitando la especialización de cada canal detector para una banda concreta o para mejorar la redundancia y la relación señal/ruido (SNR) en segmentos críticos del espectro.
- **Divisores de haz (*Beam Splitters*)**: Utilizados para separar la luz en distintas longitudes de onda y redirigirlas a detectores diferentes. Esta técnica es útil cuando se requieren bandas espectrales muy diferenciadas, aunque introduce pérdidas ópticas y puede complicar el alineamiento y la calibración radiométrica.
- **Solapamiento parcial de detectores**: En algunos diseños, se solapan parcialmente las áreas sensibles de los detectores para mejorar la corregistración entre bandas o aumentar la redundancia. Sin embargo, esto incrementa la complejidad del procesamiento y puede penalizar la eficiencia espacial y el peso del sistema.

Para dimensionar la misión se establece un límite máximo de **tres detectores**, con el fin de simplificar la disposición de los mismos [16].

3.4. Óptica. Telescopios

La óptica de un satélite de observación es el conjunto de elementos (lentes, espejos, filtros, etc.) que recogen la radiación electromagnética proveniente de la Tierra y la enfocan sobre el detector. El telescopio es el corazón de este sistema óptico: su función es formar una imagen nítida y de alta calidad sobre el plano focal, maximizando la resolución espacial y la eficiencia de captación de luz, minimizando aberraciones y pérdidas de contraste. La elección del tipo de telescopio depende de los requisitos de la misión (resolución, campo de visión, peso, facilidad de alineamiento y fabricación, etc.). Los diseños más usados en teledetección espacial son los telescopios refractivos y los reflectores de dos o tres espejos (Cassegrain, Korsch, TMA) [16].

3.4.1. Parámetros ópticos relevantes

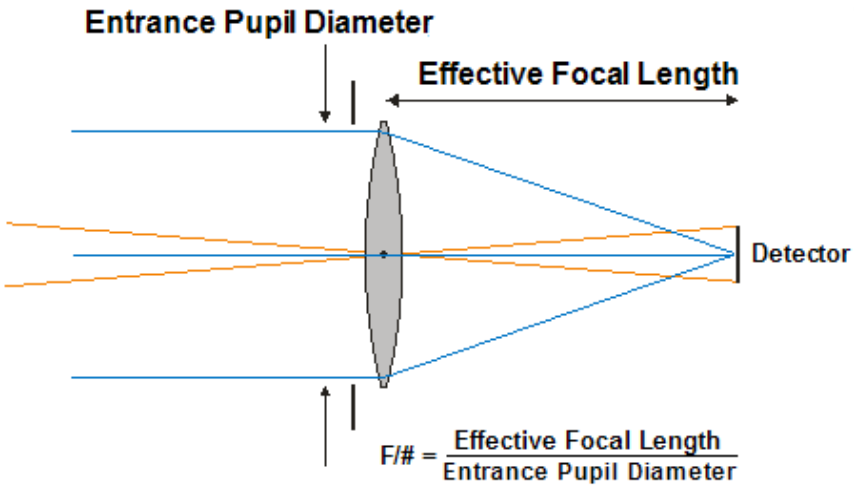


Figura 3.17: Diagrama de parámetros ópticos.
Fuente: [39].

Diámetro de pupila (D)

El diámetro de pupila es el tamaño de la apertura circular por la que entra la luz al sistema óptico, también conocido como apertura del telescopio. Este parámetro es crucial porque determina la cantidad de luz que puede captar el sistema (afectando a la SNR) y el límite de difracción, es decir, la máxima resolución teórica que puede alcanzar el instrumento. Un mayor diámetro de pupila permite captar más luz y obtener una mejor resolución espacial, pero también incrementa el peso y el tamaño del instrumento, lo que es crítico en satélites donde se busca minimizar la masa para reducir costes de lanzamiento.

Distancia focal (f)

La distancia focal es la separación entre el centro óptico del sistema (o el espejo principal, en sistemas reflectores) y el plano focal donde se forma la imagen. Este parámetro define el aumento del sistema: a mayor distancia focal, mayor es la imagen proyectada sobre el detector para un mismo objeto observado en la Tierra. En instrumentos de observación, la distancia focal se elige para cumplir con el GSD requerido, ya que está relacionada con el tamaño del píxel del detector y la resolución en superficie.

Relación focal ($F\#$)

La relación focal se define como el cociente entre la distancia focal y el diámetro de pupila ($F\# = \frac{f}{D}$). Para los sistemas con obstrucción central, se define la relación focal equivalente $T\#$, que viene dada por la relación:

$$T\# = \frac{f}{\sqrt{D^2(1 - R_{obs}^2)}} \tag{3.4.1}$$

Siendo R_{obs} la misma relación de obstrucción en la ecuación (3.1.5). Es un parámetro clave ya que un número $F\#$ bajo (óptica «rápida») implica una apertura grande respecto a la distancia focal, lo que permite captar más luz y mejora la SNR, pero puede aumentar las aberraciones ópticas y reducir la profundidad de campo. Un número $F\#$ alto (óptica «lenta») reduce la cantidad de luz captada, pero suele simplificar el diseño y controlar mejor las aberraciones. La relación focal afecta directamente a la MTF, la SNR y la saturación del detector.

3.4.2. Tipos de telescopio

Refractivo

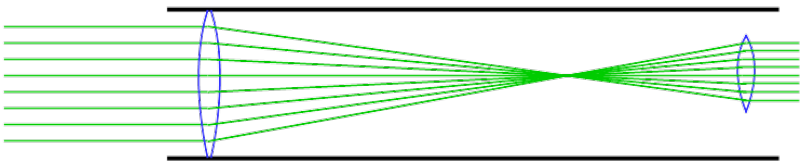


Figura 3.18: Telescopio Refractivo.
Fuente: [40].

El telescopio refractivo utiliza lentes para enfocar la luz sobre el detector. Su principal ventaja es la ausencia de obstrucción, ya que toda la apertura está disponible para captar luz. Esto se traduce en una MTF de difracción más alta, especialmente a frecuencias espaciales elevadas, y en una penalización por alineamiento relativamente baja ($MTF_{alineamiento} \approx 0,90$), pues el sistema es sencillo de montar y ajustar. Sin embargo, los telescopios refractivos presentan aberraciones cromáticas, lo que limita su uso en aplicaciones multiespectrales o en longitudes de onda fuera del visible. Además, para grandes aperturas (entre 80 y 100 mm), las lentes se vuelven pesadas y difíciles de fabricar, lo que desincentiva su uso en satélites de observación de alta resolución, o de órbitas superiores a LEO (*Low Earth Orbit*).

Cassegrain

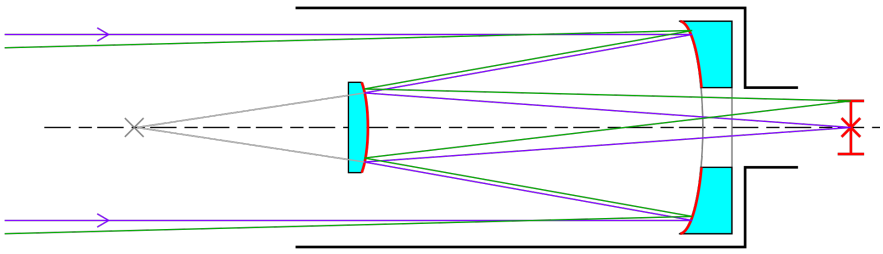


Figura 3.19: Telescopio Cassegrain.
Fuente: [41].

El diseño Cassegrain es un reflector de dos espejos, con un primario parabólico y un secundario hiperbólico. La luz se refleja primero en el espejo primario, luego en el secundario y finalmente pasa a través de un orificio en el primario hacia el detector. Este sistema introduce una obstrucción central debida al secundario, lo que reduce la MTF de difracción a frecuencias altas, pero permite una configuración compacta y una buena corrección de aberraciones cromáticas. El alineamiento es moderadamente sencillo, con una penalización típica de $MTF_{alineamiento} \approx 0,85$. El campo de visión es limitado, típicamente en torno a 3°.

Korsch

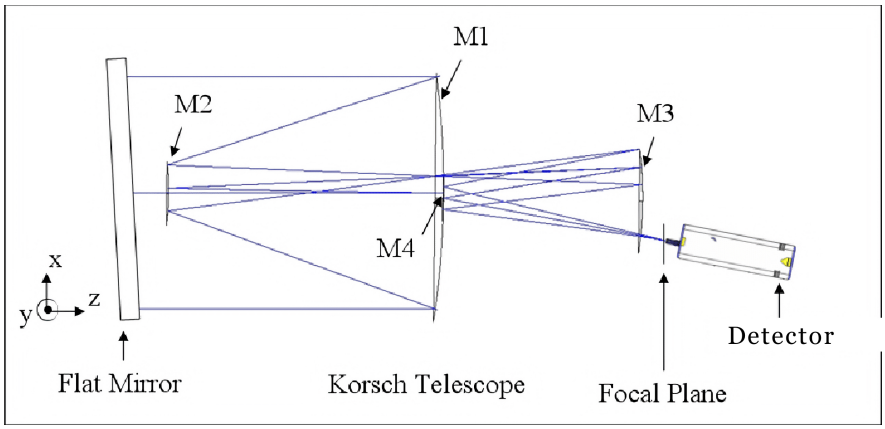


Figura 3.20: Telescopio Korsch.
Fuente: [42].

El telescopio Korsch es un reflector de tres espejos, también con obstrucción central. Su diseño está optimizado para controlar la luz parásita (straylight). Permite focales largas y una buena calidad de imagen en todo el campo, aunque el alineamiento es más complejo y la penalización en la MTF por este motivo es mayor ($MTF_{alineamiento} \approx 0,80$). La presencia de la

obstrucción central sigue penalizando la MTF de difracción, aunque el control de aberraciones es excelente. Posee un FoV similar al Cassegrain, de 3°.

Three-mirror anastigmat (TMA)

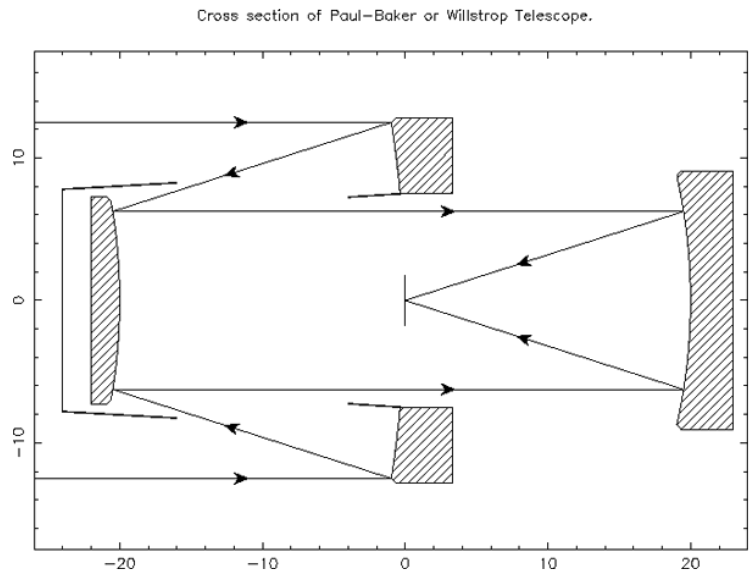


Figura 3.21: Telescopio TMA.
Fuente: [43].

El telescopio de tres espejos anastigmático es un reflector de tres espejos sin obstrucción central, ya que el haz óptico se desvía fuera del eje óptico principal (diseño off-axis). Esto permite un campo de visión más amplio, del orden de 8°, y una calidad de imagen homogénea en todo el campo, sin penalización por la presencia de elementos que bloqueen la apertura. Sin embargo, el alineamiento de un TMA es extremadamente exigente, con una penalización en la MTF de alineamiento significativa ($MTF_{alineamiento} \approx 0,70$). Además, la fabricación y verificación de estos sistemas es más compleja y costosa.

Tabla comparativa de Telescopios

Tabla 3.1: Comparativa de tipos de sistemas ópticos.

Tipo	**Obstrucción** R_{obs}	FoV_{lim}	**MTF alineamiento**	**Transmisión óptica** (τ)
Refractivo*	No	10º	0,90	0,8
Cassegrain	Sí (0,2)	3º	0,85	0,7
Korsch	Sí (0,2)	3º	0,80	0,65
TMA	No	8º	0,70	0,6

*Limitado a 80 mm de diámetro máximo válido para el presente trabajo.

3.4.3. Filtros

Figura 3.22: Rueda de Filtros del *James Webb Space Telescope*.
Fuente: [44].

La selección de filtros ópticos en el detector es fundamental para definir las bandas espectrales de interés, optimizar la relación señal-ruido (SNR), garantizando la calidad radiométrica y espacial de la imagen.

Los filtros ópticos seleccionan las longitudes de onda que llegan al detector, permitiendo medir únicamente las bandas espectrales relevantes para el objetivo de la misión.

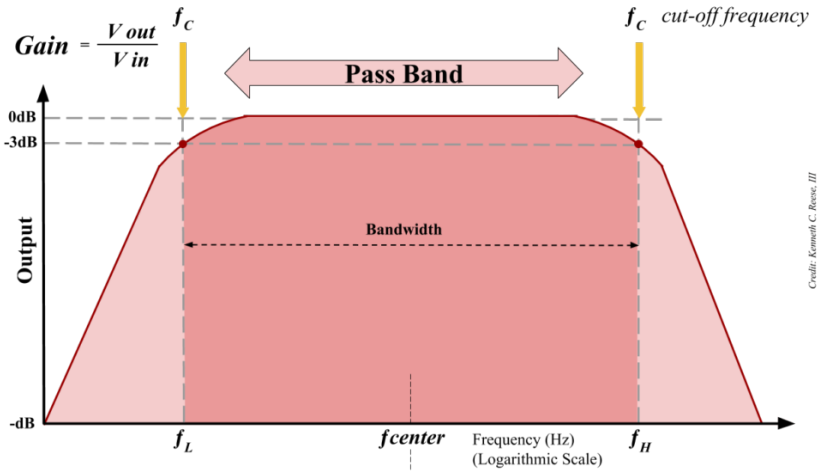


Figura 3.23: Filtrado de las frecuencias.
Fuente:[45].

La anchura de banda debe ser lo suficientemente estrecha para aislar las líneas de absorción de interés, pero lo suficientemente ancha para mantener una SNR adecuada. Los filtros deben tener alta transmitancia en la banda de interés para maximizar la señal y baja transmitancia fuera de banda para evitar contaminación espectral. Asimismo, deben ser capaces de bloquear al máximo el resto de las bandas fuera de nuestro interés para reducir el ruido.

Tipos de Filtros y Configuración

- **Rueda de filtros**: Un solo detector y una rueda que posiciona secuencialmente diferentes filtros delante del detector. Es una solución asequible pero introduce partes móviles y puede limitar la velocidad de adquisición.
- **Módulos de filtro independientes**: Cada detector tiene su propio filtro fijo. Permite mayor robustez y elimina partes móviles, pero puede aumentar el tamaño del plano focal y dificultar la corregistración espacial.
- **Filtros tipo *microstrip***: Filtros miniaturizados directamente sobre el detector o muy próximos a él. Permiten una disposición compacta y robusta, adecuada para configuraciones multiespectrales con alta corregistración.

3.5. Parámetros Orbitales

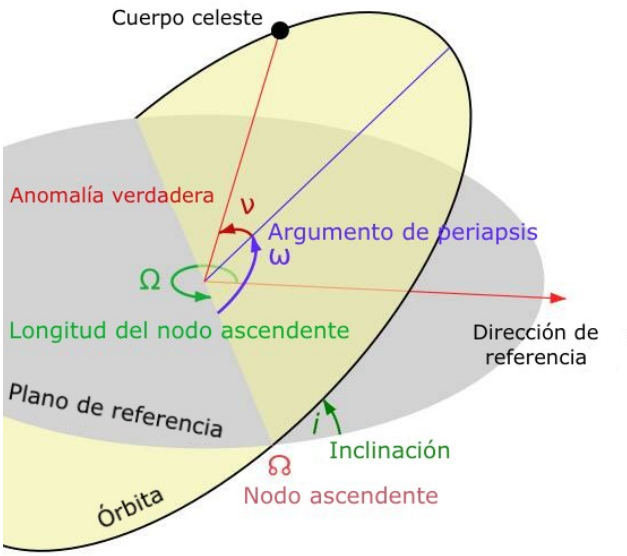


Figura 3.24: Representación gráfica de los elementos orbitales.
Fuente: [46].

A continuación se definen los principales parámetros que definen la órbita del cuerpo de interés, así como las perturbaciones asociadas de mayor relevancia en la mecánica orbital para el presente problema [47].

Semieje mayor (a)

El semieje mayor determina el tamaño básico de la órbita elíptica, representando la distancia desde el centro de la elipse hasta su extremo más alejado sobre el eje principal.

Excentricidad (e)

La excentricidad define la forma de la órbita, como la desviación del cuerpo central respecto al centro geométrico de la elipse. Matemáticamente:

$$e = \sqrt{1 - \frac{b^2}{a^2}} \tag{3.5.1}$$

donde b es el semieje menor. La excentricidad determina varios tipos de órbitas:

- $e = 0$: Órbita perfectamente circular
- $0 < e < 1$: Órbita elíptica (como la mostrada en las imágenes)
- $e = 1$: Trayectoria parabólica
- $e > 1$: Trayectoria hiperbólica

En el análisis preliminar, el tamaño de una órbita terrestre puede expresarse convenientemente mediante la altura orbital, definida como $h = a - R_T$, siendo R_T el radio de la Tierra. La excentricidad se conserva como parámetro independiente para distinguir entre una aproximación circular y una órbita ligeramente elíptica.

3.5.1. Inclinación (i)

La inclinación representa el ángulo entre el plano orbital y el plano de referencia (generalmente el ecuador terrestre). En la primera imagen se muestra como el ángulo formado entre estos dos planos. Para órbitas terrestres, se clasifica de la siguiente manera:

- $i = 0°$: Órbita ecuatorial
- $0° < i < 90°$: Órbita prógrada
- $i = 90°$: Órbita polar
- $90° < i < 180°$: Órbita retrógrada

Este parámetro es crucial para determinar qué regiones de la Tierra sobrevolará el satélite.

3.5.2. Longitud del Nodo Ascendente (Ω)

La longitud del nodo ascendente define la orientación del plano orbital en el espacio tridimensional. Como se visualiza en la primera imagen, es el ángulo medido desde la dirección de referencia hasta el nodo ascendente (punto donde la órbita cruza el plano de referencia en dirección norte). Este elemento es particularmente importante para órbitas heliosíncronas, donde precesa aproximadamente 1° diario para mantener una hora solar local prácticamente constante, compensando el movimiento de traslación de la Tierra alrededor del Sol.

3.5.3. Hora de paso local (LTAN)

La hora de paso local en el nodo ascendente (*Local Time of Ascending Node*, o LTAN) es el instante del día, medido en tiempo solar local, en que el satélite cruza el meridiano de su nodo ascendente. Es fundamental para misiones de observación de la Tierra, ya que determina las condiciones de iluminación solar en cada pasada.

Se calcula relacionando la longitud ascendente Ω con la posición del Sol en longitud eclíptica $\lambda_\odot(t)$, según:

$$T_{\mathrm{LST}} = \frac{\Omega - \lambda_\odot(t)}{15^\circ} \quad [\text{horas}],$$

donde:

- Ω es la longitud del nodo ascendente (°),
- $\lambda_\odot(t)$ es la longitud eclíptica del Sol en el instante t (°),
- el factor 15°/h convierte grados en horas solares locales.

En la práctica se suele referir el LTAN a valores redondeados (por ejemplo, 10:30 h o 13:30 h) para garantizar escenas con sombras y contraste adecuados. En órbitas heliosíncronas, la precesión de Ω ($\approx 1^{\mathrm{o}}$/día) se ajusta de modo que el LTAN permanezca prácticamente constante durante toda la misión.

3.5.4. Argumento del Perigeo (ω)

El argumento del perigeo describe la orientación de la elipse dentro del plano orbital. En las figuras se muestra como el ángulo entre el nodo ascendente y el punto de máximo acercamiento al cuerpo central (perigeo). Este parámetro determina dónde se producirán las distancias mínima y máxima de la órbita. Para satélites científicos en órbitas excéntricas, este valor se elige cuidadosamente para sobrevolar determinadas regiones a la altura mínima.

El barrido inicial se aproxima mediante órbitas circulares. La solución final se comprueba como órbita *frozen* de pequeña excentricidad, en la que el argumento del perigeo sí determina la geometría de observación.

3.5.5. Anomalía Verdadera (ν)

La anomalía verdadera es el único elemento que varía con el tiempo para una órbita no perturbada. Representa la posición instantánea del cuerpo orbitante, medida como el ángulo desde el perigeo hasta la posición actual del satélite, en la dirección del movimiento.

3.6. Perturbaciones de la órbita

3.6.1. Achatamiento Terrestre

La Tierra no es perfectamente esférica, presenta un abultamiento en el ecuador debido a su rotación. Este achatamiento genera un campo gravitacional asimétrico que introduce una perturbación en las órbitas de los satélites. Este efecto provoca dos movimientos importantes en los elementos orbitales:

1. **Regresión del nodo ascendente** (Ω): El nodo ascendente, el punto donde la órbita cruza el ecuador de sur a norte, experimenta un movimiento de precesión regresiva.
2. **Precesión del argumento del perigeo** (ω): El argumento del perigeo también se ve afectado, lo cual es relevante para órbitas con cierta excentricidad.

Estas perturbaciones dependen de la altitud, la inclinación y otros parámetros orbitales. En el próximo apartado (3.6.5) se detalla cómo acoplar dichas variables para definir órbitas heliosíncronas.

3.6.2. Decaimiento Orbital

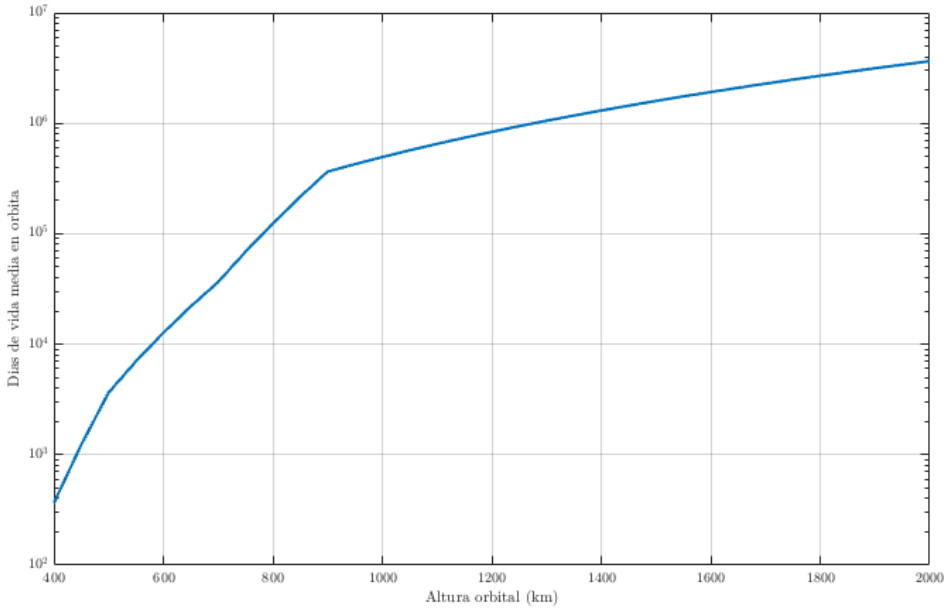


Figura 3.25: Días de vida media en órbita vs Altura orbital (km).
Fuente: Elaboración propia basada en [48].

Los satélites en órbita baja terrestre (LEO), típicamente por debajo de los 1000 km de altitud, están expuestos a una atmósfera tenue pero no nula. A esas alturas, aún existen partículas atmosféricas que ejercen una fuerza de resistencia aerodinámica sobre el satélite a medida que este se desplaza a velocidades orbitales. Esta resistencia aerodinámica provoca una pérdida continua de energía orbital, que hace que el satélite descienda gradualmente a órbitas más bajas. Este fenómeno se conoce como **decaimiento orbital**.

Con el tiempo, un satélite que no cuenta con sistemas de propulsión para mantener su altitud experimentará una disminución continua de su altura orbital, y reingresará finalmente en la atmósfera densa, donde se desintegrará por el calor generado por la fricción. Este mecanismo actúa también como forma pasiva de mitigación de desechos espaciales, lo cual conviene tener en consideración una vez llegue el fin de vida del satélite.

3.6.3. Radiación Solar

La radiación solar está compuesta por partículas (fotones) que viajan a la velocidad de la luz. Estos fotones, aunque no tienen masa, sí tienen energía y cantidad de movimiento.

Cuando los fotones de la radiación solar golpean un objeto, como un satélite, se produce una transferencia de momento del fotón al objeto. Esta pequeña fuerza es la presión de radiación.

Aunque la presión es pequeña, para satélites con grandes superficies expuestas (como los que tienen paneles solares grandes o velas solares), puede producirse una desviación progresiva de la órbita. Puede cambiar el argumento del perigeo, haciendo que la órbita se desplace gradualmente.

3.6.4. Problema del tercer cuerpo

El problema del tercer cuerpo se refiere a la dinámica gravitacional que surge cuando tres objetos celestes (en nuestro caso, el sistema Tierra-Luna- satélite, o Tierra-Sol- satélite) interactúan entre sí mediante la fuerza de gravedad. A diferencia del problema de dos cuerpos, que tiene una solución matemática exacta y predecible, la introducción de un tercer cuerpo introduce un nivel de complejidad significativo.

Cuando solo dos cuerpos interactúan gravitacionalmente, sus movimientos forman órbitas estables y predecibles. Sin embargo, cuando se introduce un tercer cuerpo, este genera perturbaciones en las órbitas que serían estables en el caso de dos cuerpos.

El tratamiento analítico de este problema es extraordinariamente difícil porque la perturbación depende explícitamente del tiempo. Las ecuaciones diferenciales que describen este sistema no tienen una solución cerrada o analítica exacta, lo que ha llevado al desarrollo de métodos numéricos para aproximar las soluciones.

El problema de los tres cuerpos es intrínsecamente caótico, lo que significa que es extremadamente sensible a las condiciones iniciales. Pequeñas variaciones en la posición o velocidad inicial de cualquiera de los cuerpos pueden resultar en trayectorias completamente diferentes a largo plazo. Esto imposibilita predicciones precisas sin conocer con exactitud absoluta las condiciones iniciales.

3.6.5. Órbitas de especial interés

Órbitas heliosíncronas

Las órbitas heliosíncronas ajustan la precesión del nodo ascendente para conservar una hora solar local casi constante. Esta propiedad interesa en observación óptica pasiva porque el instrumento mide radiación solar reflejada; mantener el LTAN reduce cambios de iluminación entre pasadas.

El achatamiento terrestre, modelado mediante J_2, permite imponer esa precesión. Para una órbita casi circular, se escoge la inclinación de modo que la regresión nodal iguale la velocidad angular aparente del Sol respecto a la Tierra ($\approx 0{,}9856°$/día):

$$\cos i = -\frac{2r^{7/2}\,\omega_{\text{sol}}}{3J_2R_T^2\sqrt{\mu}} \tag{3.6.1}$$

donde:

- i es la inclinación orbital,
- r es el radio orbital (radio de la Tierra más la altitud del satélite),
- $\omega_{\text{sol}} \approx 1{,}991 \times 10^{-7}$ rad/s es la velocidad angular aparente del Sol respecto a la Tierra,
- $J_2 \approx 1{,}08263 \times 10^{-3}$ es el coeficiente zonal que representa el achatamiento terrestre,
- $R_T = R_\oplus \approx 6378{,}137$ km es el radio ecuatorial de la Tierra,
- $\mu \approx 3{,}986004418 \times 10^{14}$ m^3/s^2 es el parámetro gravitacional estándar de la Tierra.

Órbitas de Traza Repetida

Una órbita de traza repetida (RGT) repite la proyección del satélite sobre la superficie terrestre tras k_{day2rep} rotaciones terrestres y k_{orb2rep} órbitas. La condición RGT exige que la

frecuencia angular nodal media coincida con la velocidad angular de la traza respecto a la Tierra:

$$n_k(a) + \dot{M}_1(a,i,e) + \dot{\omega}(a,i,e) = \left[\omega_\oplus - \dot{\Omega}(a,i,e)\right] \frac{k_{\text{orb2rep}}}{k_{\text{day2rep}}} \tag{3.6.2}$$

donde $n_k(a) = \sqrt{\mu/a^3}$ es el movimiento medio de la órbita kepleriana y $\omega_\oplus$ es la velocidad angular de rotación terrestre respecto al sistema inercial. En esta condición se emplea el periodo sideral:

$$T_\oplus = \frac{2\pi}{\omega_\oplus} = 86164{,}09 \text{ s}, \qquad \omega_\oplus = 7{,}292115373 \times 10^{-5} \text{ rad/s}. \tag{3.6.3}$$

Las cantidades $\dot{\Omega}$, $\dot{\omega}$ y $\dot{M}_1$ son las tasas seculares asociadas a J_2. Las expresiones generales de estas tasas son [49]:

$$\dot{\Omega}_{J_2} = -\frac{3J_2 n R_\oplus^2}{2p^2} \cos i, \tag{3.6.4}$$

$$\dot{\omega}_{J_2} = \frac{3J_2 n R_\oplus^2}{4p^2} \left(4 - 5\sin^2 i\right), \tag{3.6.5}$$

$$\dot{M}_{1,J_2} = \frac{3J_2 n \sqrt{1-e^2} R_\oplus^2}{4p^2} \left(2 - 3\sin^2 i\right), \tag{3.6.6}$$

con $n = n_k(a)$, $p = a(1-e^2)$ y $a = R_\oplus + h$. Para el caso circular, $e = 0$ y $p = a$; por tanto, el período nodal se obtiene como caso particular de la expresión general:

$$T_{\text{nodal}} = \frac{2\pi}{n_k(a) + \dot{M}_{1,J_2}(a,i,0) + \dot{\omega}_{J_2}(a,i,0)} = \frac{T_k}{1 + \frac{3}{4} J_2 \left(\frac{R_\oplus}{a}\right)^2 (6 - 8\sin^2 i)}, \qquad T_k = 2\pi\sqrt{\frac{a^3}{\mu}} \tag{3.6.7}$$

La condición heliosíncrona (3.6.1) determina $i = i_{\text{SSO}}(h)$. Al sustituir esta inclinación en (3.6.2), el modelo resuelve una ecuación para cada pareja de enteros $(k_{\text{orb2rep}}, k_{\text{day2rep}})$ [49].

Órbitas *frozen*

Una órbita *frozen* se obtiene al escoger la excentricidad y la orientación de la órbita para cancelar, en la aproximación de primer orden, los términos seculares dominantes asociados

a J_3 en la evolución de e y ω. Esta condición reduce las variaciones seculares de ambos parámetros, pero no las elimina por completo. Para $e \ll 1$ y la orientación analizada en este trabajo, $\omega = 90°$, la excentricidad se aproxima mediante [49]:

$$e_{frozen} \approx -\frac{J_3}{2J_2}\,\frac{R_T}{a}\,\sin i, \qquad e \ll 1, \quad \omega = 90° \tag{3.6.8}$$

donde $J_3 \approx -2{,}5327 \times 10^{-6}$ es el armónico zonal impar que rompe la simetría norte-sur del campo gravitatorio. Aunque $\omega = 270°$ puede aparecer como orientación alternativa en formulaciones generales, esta orientación no se considera en el presente análisis.

El interés de esta condición para una misión de observación de la Tierra reside en que limita la deriva secular de la geometría orbital. Por tanto, el perfil de altura y la geometría de observación se repiten con menor variación entre órbitas, aunque permanecen variaciones residuales debidas a los términos no cancelados y a las perturbaciones que no se incluyen en esta aproximación.

La excursión entre apogeo y perigeo es $\Delta h = 2ae_{frozen}$. Su valor corresponde a la órbita completa. El beneficio para la observación se evalúa mediante la variación de altura dentro de los arcos que recorren la región de interés. La banda de mantenimiento se aplica al semieje mayor y se analiza por separado [49].

3.6.6. Consideración de las perturbaciones orbitales

Para el presente trabajo no se modelan la presión de radiación solar ni el problema del tercer cuerpo. En una órbita casi circular de 690 km, la aceleración gravitatoria terrestre vale aproximadamente 8,0 m/s^2. La perturbación lunar queda del orden de 10^{-6} m/s^2, y la solar en torno a 10^{-7} m/s^2. La presión de radiación solar, para un satélite compacto con relación área/masa cercana a 0,04 m^2/kg, queda alrededor de 10^{-7} m/s^2. Estas contribuciones son pequeñas frente al término central y no gobiernan la selección preliminar de altura [49].

Sí se incluyen dos efectos porque condicionan directamente la misión: J_2, que fija la inclinación heliosíncrona y el LTAN, y la resistencia aerodinámica, que determina el decaimiento orbital y la masa de combustible.

Capítulo 4

Definición del instrumento

Contenido

4.1. GSD 53
4.2. Bandas espectrales. Ancho de banda 54
4.3. Detectores, filtros y escaneo 56
4.3.1. Elección de detectores 56
4.3.2. Método de escaneo 57
4.3.3. Elección de filtros 57
4.4. Tipo de órbita. Altura orbital. Hora de paso local 58
4.5. Cálculo de la MTF 58
4.6. Cálculo de la SNR 63
4.7. Cobertura de la región de interés: *Swath*, FoV y tiempo de revisita 68
4.8. Posibles soluciones. Cruce de los datos 79

Definidos los fundamentos teóricos, se determinan las variables que cumplen los requisitos impuestos.

Se ha creado para el trabajo un paquete de códigos disponible en el anexo B que, a partir de los datos de entrada iniciales, calcula los requerimientos y exporta las soluciones óptimas listas a interpretar por el usuario. El código maestro, que aloja dichos parámetros de entrada y las llamadas a las funciones, alberga el siguiente flujo de trabajo:

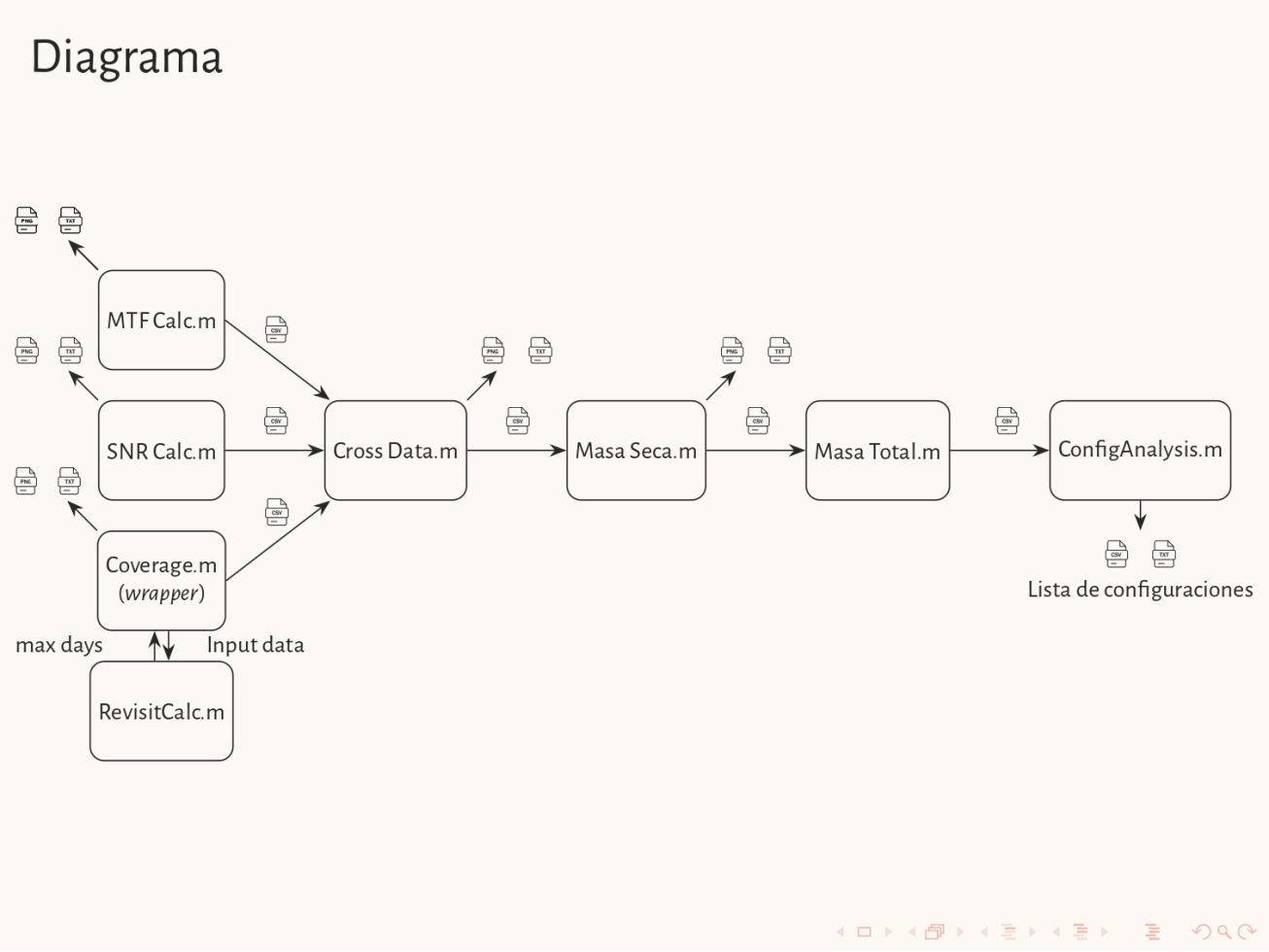


Figura 4.1: Flujo de trabajo de los scripts de MATLAB

4.1. GSD

Se comenzará por definir el GSD, ya que es un parámetro que implícitamente viene dado por las condiciones del cliente. Según el requerimiento, es necesario poder diferenciar fuentes puntuales de CO2 en las ciudades estadounidenses.

Se parte de una imagen de satélite del cinturón industrial de Houston como referencia para la selección del GSD. Con el software *open source* GIMP se pixela esta imagen para apoyar visualmente la elección del GSD. Dicha foto tiene 1673 píxeles de ancho, y mediante el software Google Earth, proporciona el dato del ancho de la imagen en la realidad, correspondiente a 14251 m. Por tanto, el GSD de esta captura será:

$$\frac{14251\ m}{1673\ pix} \approx 8{,}518 \tag{4.1.1}$$

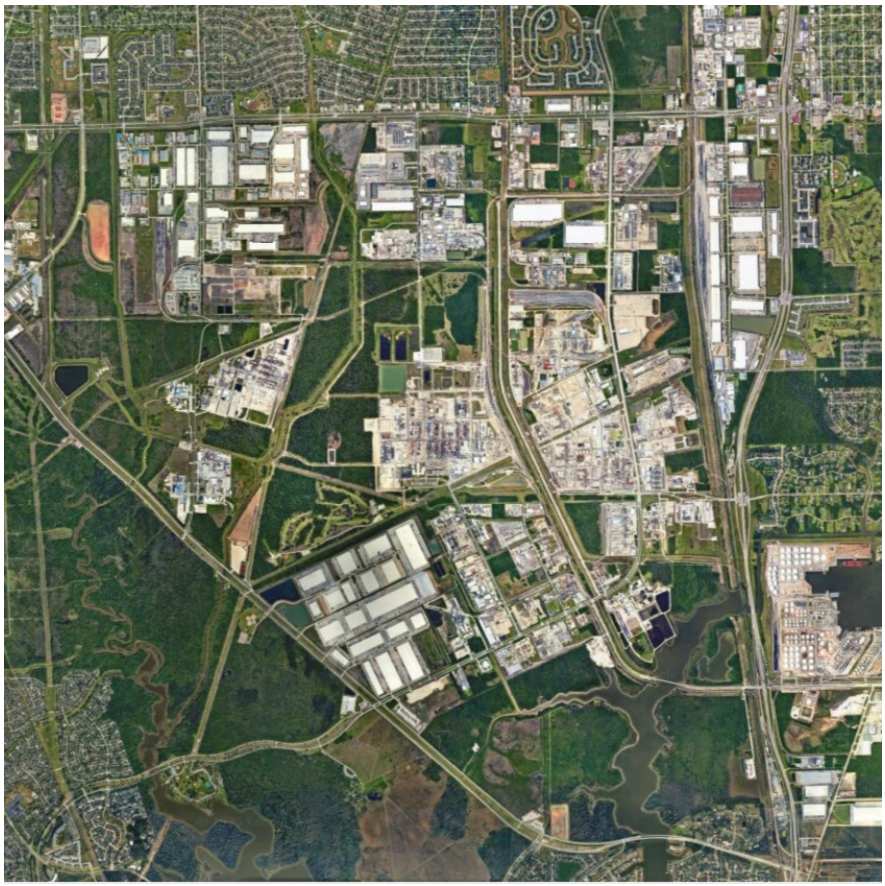


Figura 4.2: Imagen satélite del cinturón industrial de Houston. $GSD \approx 8{,}5$.
Fuente: Google Earth (29°42'18,0"N; 95°00'10,8"W)

Esta resolución resulta excesiva para las especificaciones del proyecto, ya que permite observar las instalaciones industriales con gran detalle. Se puede relajar este parámetro para

reducir el diámetro de pupila, la masa del instrumento y la masa de combustible. Un análisis visual con diferentes valores de GSD muestra que una resolución de **80 metros** permite diferenciar edificios industriales entre sí y distinguirlos de las zonas residenciales. Por ello se adopta como caso base.

(a) GSD 30 (b) GSD 50 (c) GSD 80

(d) GSD 100 (e) GSD 150 (f) GSD 200

Figura 4.3: Comparación de imágenes con distintas condiciones de GSD.
Fuente: Imagen adaptada, Google Earth.

El capítulo 5 incluye un análisis de sensibilidad con $GSD = 60, 80$ y 100 m para comprobar cómo cambia esta elección el dimensionado final.

4.2. Bandas espectrales. Ancho de banda

La selección de bandas espectrales se fundamenta en las características de absorción del CO_2 y los requisitos de calibración atmosférica necesarios para garantizar la precisión de las mediciones.

Banda O_2 A-Band (0,76 µm)

La banda de absorción del oxígeno atmosférico a 0,76 µm se emplea para la calibración de la columna de aire. El oxígeno presenta una concentración atmosférica estable y conocida (20,95 %), lo que permite determinar con precisión la masa de aire atravesada por la radiación solar. Esta referencia permite normalizar las mediciones de CO_2 y corregir variaciones debidas a diferencias en el camino óptico atmosférico.

Bandas CO_2 SWIR (1,61 y 2,01 µm)

Las bandas de 1610 nm y 2010 nm corresponden a las principales regiones de absorción del CO_2 en el infrarrojo de onda corta, como se vio en el capítulo 3. La utilización de dos bandas independientes permite implementar algoritmos de validación cruzada y mejora el cálculo de estimaciones de concentración.

Ancho de banda

En cuanto al ancho de banda espectral, se ha seleccionado un valor de 20 nm para todas las bandas. Esta elección representa un equilibrio entre la resolución espectral y la relación señal-ruido, permitiendo aislar adecuadamente las características de absorción objetivo sin comprometer la sensibilidad del sistema. Un ancho de banda más estrecho proporcionaría mayor especificidad espectral, pero reduciría la cantidad de fotones detectados y, por tanto, la SNR. Por el contrario, un ancho de banda excesivamente amplio dificultaría la discriminación de las líneas de absorción. El valor adoptado se encuentra dentro de los estándares habituales en instrumentación óptica para aplicaciones de teledetección y garantiza el cumplimiento de los requisitos de precisión y robustez en la recuperación de concentraciones de CO_2 y parámetros atmosféricos [50].

Tabla 4.1: Bandas espectrales elegidas.

Banda	λ **(µm)**	$\Delta\lambda$ **(nm)**	**Propósito principal**
O_2 A-Band	0,76	20	Calibración columna aire
CO_2 SWIR 1	1,61	20	Absorción principal de CO_2
CO_2 SWIR 2	2,01	20	Absorción secundaria de CO_2

4.3. Detectores, filtros y escaneo

4.3.1. Elección de detectores

Definidas las bandas de interés, se busca el detector capaz de cubrir dichos rangos de detección para la misión. Para ello, se elabora una lista de detectores disponibles en el mercado de los cuales se escogerán más adelante, el más apropiado según sus especificaciones. Para las bandas 1,61 µm y 2,01 µm se evaluarán los siguientes detectores:

Tabla 4.2: Comparación de características de detectores SWIR

	Detector 1 [51]	**Detector 2** [52]	**Detector 3** [53]
Nombre	LYNRED CAPYORK	**Teledyne H2RG**	Saturn VISIR
Tamaño píxel	15 × 15 µm	18 × 18 µm	30 × 30 µm
Array	1200 × 12	2048 × 2048	1000 × 256
Canales[*]	4	4	4 ó 8
TDI	Si (4 etapas)	No	No
Rango espectral	0,8-5 µm	0,4-2,5 µm	0,3-2,5 µm
Eficiencia cuántica	85 %	70 %	70 %
MTF @ Nyquist	0,45	0,5	0,48

[*]Evaluar los canales de salida permite evaluar cuántas bandas independientes es capaz de capturar un detector.

Se aprecia que los detectores 2 y 3 tienen un rango válido para evaluarse también en la banda 0,76 µm, mientras que el Detector CAPYORK será incapaz de recepcionar datos en esta banda. Por ello, para la banda O_2 se evaluarán, además de los detectores 2 y 3, un detector adicional que complemente al LYNRED CAPYORK. Resulta el CMOS STAR250 una opción a evaluar:

Tabla 4.3: Comparación de características de detectores para *A-Band* O_2.

	Detector 4 [54]	**Detector 5**[52]	**Detector 6** [53]
Nombre	STAR250	**Teledyne H2RG**	SATURN VISIR
Tamaño píxel	25 × 25 µm	18 × 18 µm	30 × 30 µm
Array	512 × 512	2048 × 2048	1000 × 256
Canales	4	4	4 ó 8
TDI	No	No	No
Rango espectral	0,4–1,1 µm	0,4–2,5 µm	0,3 –2,5 µm
Eficiencia cuántica	35 %	70 %	50 %[*]
MTF @ Nyquist	0,36	0,5	0,45

[*]Según especificaciones del fabricante; desciende un 20 % para rango visible.

Más adelante se determinará el número de detectores a utilizar, así como su disposición, ya que se debe trabajar con un *swath* mínimo para asegurar cobertura.

4.3.2. Método de escaneo

El escaneo ***pushbroom*** se prefiere frente a alternativas como *whiskbroom* o *step and stare* por su fiabilidad, simplicidad mecánica y optimización de la relación señal-ruido. A diferencia del *whiskbroom*, que utiliza espejos oscilantes para barrer transversalmente la franja de observación, el *pushbroom* emplea un array lineal de detectores fijos que captura toda la franja simultáneamente. Esto elimina componentes mecánicos, reduce el riesgo de desgaste y simplifica el diseño.

Además, el *pushbroom* ofrece un mayor tiempo de integración por píxel, lo que incrementa la SNR al permitir que los detectores acumulen más fotones. En comparación, el *step and stare* limita la cobertura continua y exige maniobras del satélite complejas, añadiendo dificultad a las operaciones de mantenimiento en órbita, incrementando el peso para el control de actitud y limitando el tiempo de revisita.

4.3.3. Elección de filtros

Se escogen filtros ***microstrip*** integrados en el mismo detector. La principal ventaja de los filtros microstrip frente a alternativas como la rueda de filtros reside en su baja masa y fiabilidad mecánica, ya que carecen completamente de partes móviles que pudieran fallar durante los 8 años de misión. Esta característica es crucial considerando el entorno espacial con sus ciclos térmicos extremos y vibraciones durante el lanzamiento. Además, permiten la adquisición simultánea de todas las bandas espectrales necesarias, lo que es fundamental para asegurar la correcta corregistración espacial de los datos.

Desde el punto de vista óptico, los filtros microstrip pueden optimizarse individualmente para cada banda espectral, permitiendo un ajuste fino del ancho de banda.

La implementación práctica implicaría fabricar filtros de interferencia multicapa específicamente diseñados para cada banda y detector, con materiales optimizados según el rango espectral (dieléctricos para visible/NIR, y compuestos especiales para SWIR). Estos se montarían directamente sobre cada array de detectores, lo que también minimiza el tamaño del instrumento al evitar mecanismos adicionales [55].

4.4. Tipo de órbita. Altura orbital. Hora de paso local

Como se ha expuesto en el capítulo 2, las misiones ópticas de observación terrestre con requerimientos comparables adoptan, de manera unánime, **órbitas heliosíncronas**. Esta elección responde a la necesidad de garantizar condiciones de iluminación solar consistentes, ya que el instrumento óptico pasivo empleado depende de la radiación solar reflejada por la superficie terrestre.

Una limitación inherente a estos sistemas es su incapacidad para operar eficazmente en condiciones de penumbra. Por tanto, es necesario que la trayectoria orbital proporcione una iluminación solar directa y estable en cada pasada del satélite. La órbita heliosíncrona cumple precisamente este requisito, ya que su precesión mantiene el ángulo entre el plano orbital y el Sol constante a lo largo del año, asegurando que cada observación sobre una misma región se realice a una hora solar local fija. Esta estabilidad favorece la coherencia radiométrica de las imágenes y mejora la comparabilidad temporal de los datos adquiridos. Por ello se calcularán soluciones de órbita **heliosíncrona entre los 350 km y los 1400 km de altitud**, con una discretización de 10 km de paso [56].

Para la presente misión se ha seleccionado una **hora de paso ascendente local (LTAN) de 06:00**, lo que proporciona una configuración *Dawn-Dusk*. En este tipo de órbita, el satélite cruza el ecuador terrestre cerca de la línea crepuscular, donde la transición entre el día y la noche es continua. Esta disposición asegura que el satélite permanezca próximo al límite entre la zona iluminada y la zona en sombra de la Tierra, evitando así la entrada en el cono de sombra terrestre y garantizando la exposición solar durante toda la órbita. Como resultado, se obtienen condiciones de iluminación homogéneas tanto en las pasadas ascendentes como en las descendentes, maximizando las pasadas útiles [57].

4.5. Cálculo de la MTF

Con la base teórica definida en el apartado 3.1.4, así como las especificaciones de los detectores y telescopios a evaluar, se procede a calcular en función de la altura orbital y el diámetro de pupila, mediante un script de MATLAB, los valores de MTF totales de la instrumentación óptica. La discretización del diámetro de pupila será de un paso de 1 mm.

Los valores que se emplearán para el cálculo son:

Tabla 4.4: Parámetros iniciales para el cálculo de MTF.
Fuente: [17].

Parámetro	**Valor**
MTF aberraciones	0,95
MTF fabricación	0,98
MTF vibraciones	0,99
MTF termoelástico	0,95
MTF Margen	10 %

Dado que el proyecto se encuentra en una etapa temprana de diseño, se establece un margen del **10 %** en el cálculo con el fin de cubrir posibles desviaciones asociadas a incertidumbres en los parámetros de entrada, tolerancias de fabricación, o variaciones operacionales. Este margen permite mantener una estimación conservadora sin comprometer la viabilidad preliminar del sistema.

Mediante un análisis dimensional, puede demostrarse que la función de transferencia de modulación (MTF, por sus siglas en inglés) es inversamente proporcional a la longitud de onda λ. Al aumentar λ, disminuye la frecuencia de corte f_{co} y, en consecuencia, la capacidad del sistema para transmitir detalles de alta frecuencia espacial se ve reducida.

Por lo tanto, la banda más restrictiva corresponde a la mayor longitud de onda considerada. En el presente caso, para $\lambda = 2{,}01\,\mu$m, se obtiene la frecuencia de corte más baja y, por ende, la MTF más limitada, con lo que solo se presentan los datos de dicha banda. El resto de los datos se pueden encontrar en el anexo A. Así, junto con los valores de MTF de los detectores en la tabla 4.2 y los valores de MTF de alineamiento de los telescopios de la tabla 3.1, se obtienen los siguientes resultados:

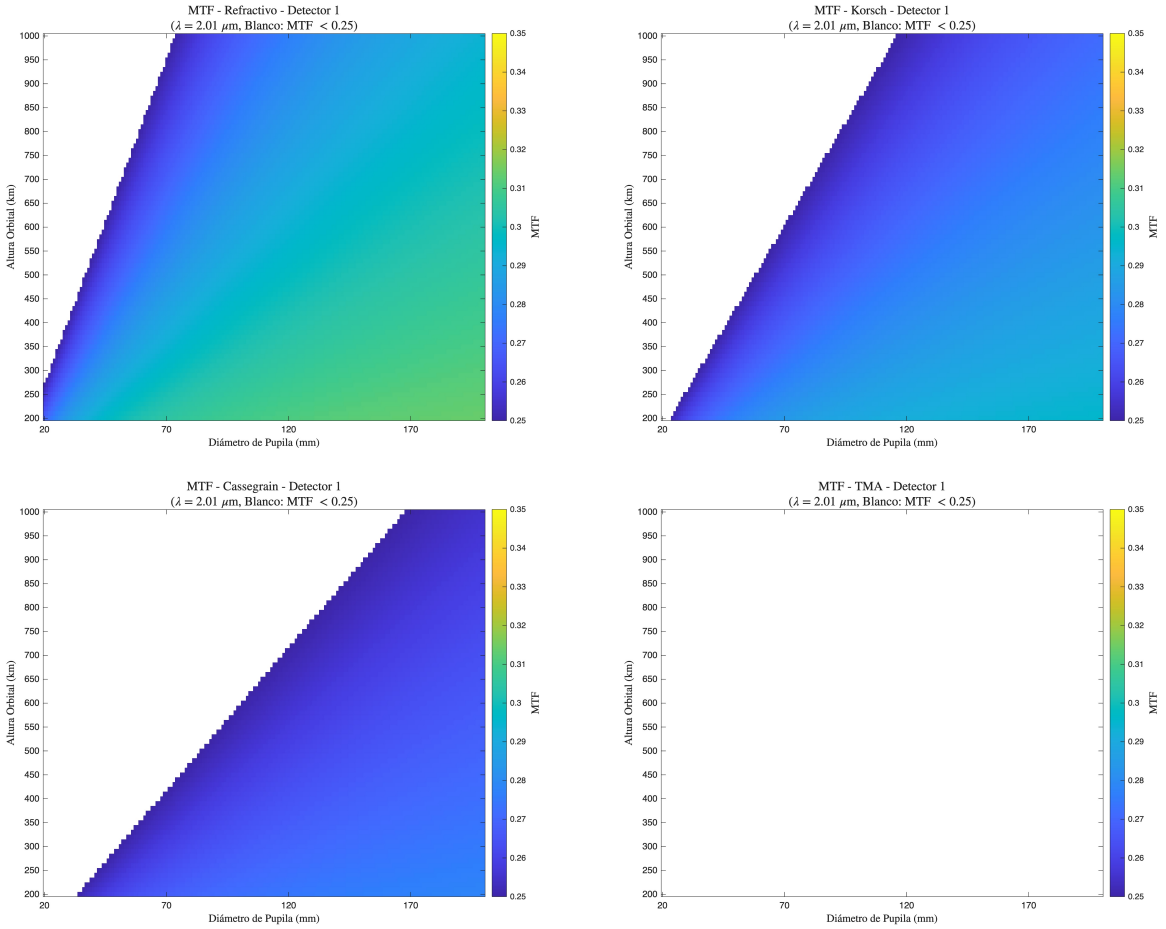


Figura 4.4: Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 1

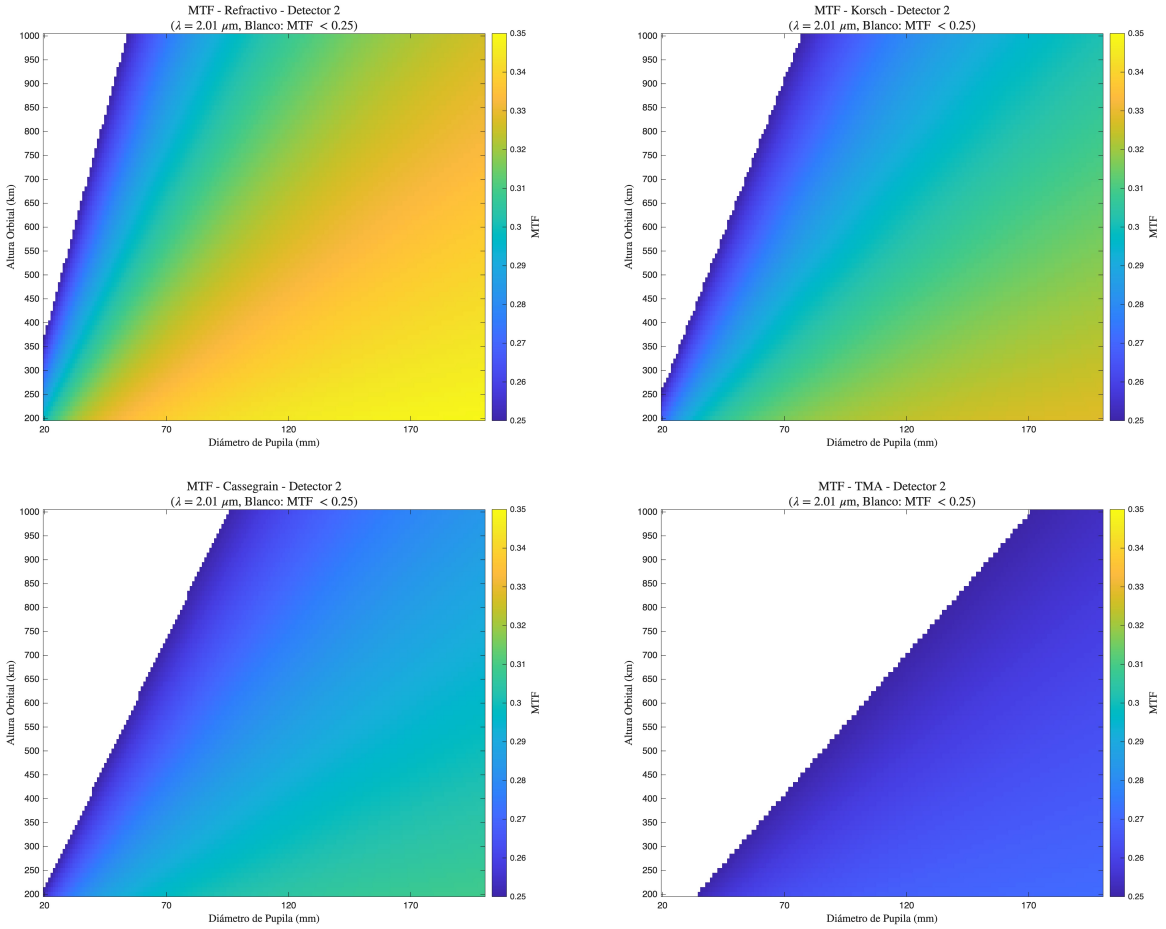


Figura 4.5: Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 2

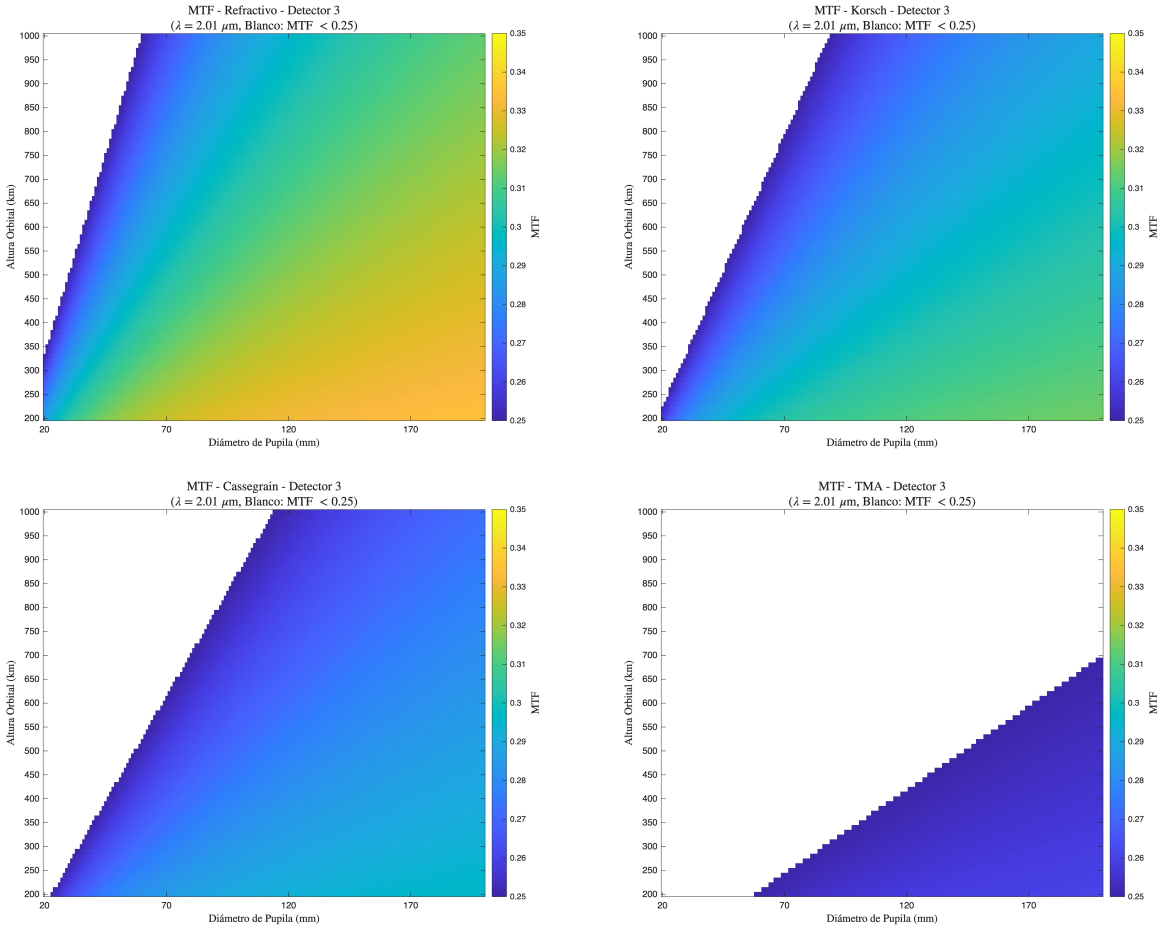


Figura 4.6: Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 3

Estos gráficos permiten extraer las siguientes consideraciones:

- Como se aprecia, es un requisito que impone limitaciones considerables al diseño. No obstante, todavía aloja un espacio de diseño muy amplio, pues la única combinación no viable resulta en el detector 1 combinado con el telescopio TMA.
- El rendimiento más bajo, aunque con soluciones viables, es el detector 1, pues su MTF es la menor.
- De igual manera, es esperable, debido al alineamiento, que el telescopio con mejor rendimiento sea el refractivo, mientras el de menor rendimiento resulte el TMA.

4.6. Cálculo de la SNR

El cálculo de la relación señal-ruido (SNR) se ha realizado empleando las ecuaciones de la sección 3.1.5. Para cada combinación de diámetro de pupila y altura orbital, y considerando los diferentes detectores y tipos de telescopio, se han generado mapas de calor que representan los valores de SNR obtenidos.

En este análisis se han utilizado los parámetros expuestos de detectores (tablas 4.3) y de telescopios(tabla 3.1), así como los parámetros iniciales especificados en la tabla 4.5, que incluyen tanto las características instrumentales como los valores de ruido en condiciones de peor caso. Este enfoque permite identificar, para cada configuración, las regiones del espacio de diseño que cumplen con los requisitos mínimos de SNR establecidos para la misión.

Tabla 4.5: Parámetros iniciales y fuentes de ruido para el cálculo de SNR

Parámetros iniciales	
TDI	1*
Radiancia de referencia	100 $W/m^2 \cdot sr \cdot \mu m$
Ruidos[58]	
N_{dark}	50 e^-
N_{read}	100 e^-
N_{preamp}	5 e^-
N_{video}	10 e^-
N_{jitter}	5 e^-
N_{emc}	5 e^-
N_{quant}	2 e^-
N_{nonlin}	2 e^-

*Un TDI=1 equivale a no implementado.

De igual manera que en el cálculo de la MTF total, se añade al **SNR un margen del 10 %** en calidad de reserva de diseño. En las etapas iniciales, los parámetros contribuyentes

experimentan variaciones que pueden degradar el rendimiento del sistema. Este margen de seguridad permite absorber dichas desviaciones y garantizar el cumplimiento de los requisitos establecidos.

De nuevo, solo se presenta la banda más restrictiva. En este caso, será la de menor longitud de onda: la banda 0,76 μm, ya que el número de electrones generados es proporcional a la longitud de onda (para una misma radiancia espectral) y, por tanto, resulta menor cuanto menor es λ. El resto se encuentran de nuevo en el anexo A. A continuación se presentan las gráficas correspondientes, que muestran la variación de la SNR en función del diámetro de pupila y la altura orbital para las distintas opciones de detectores y telescopios evaluados:

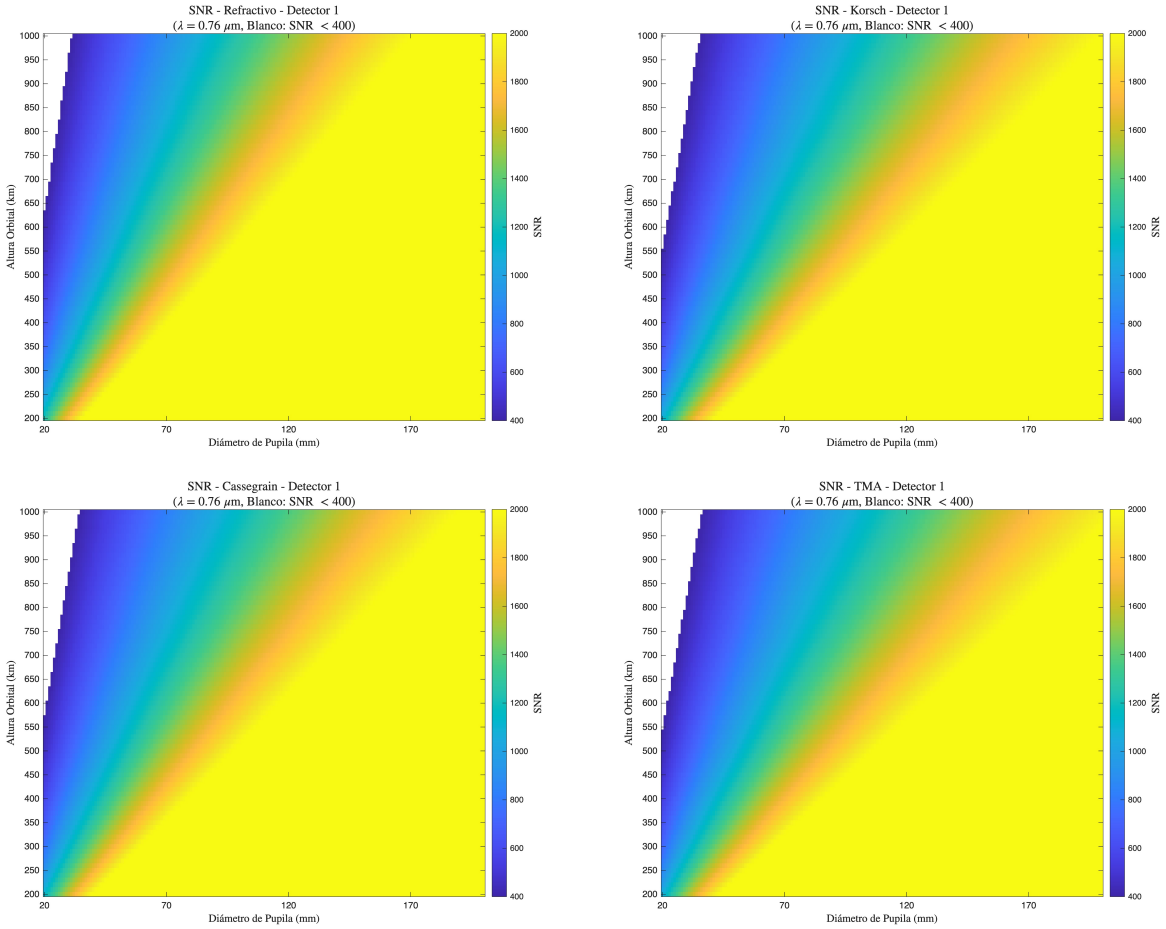


Figura 4.7: Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 4 (STAR250)

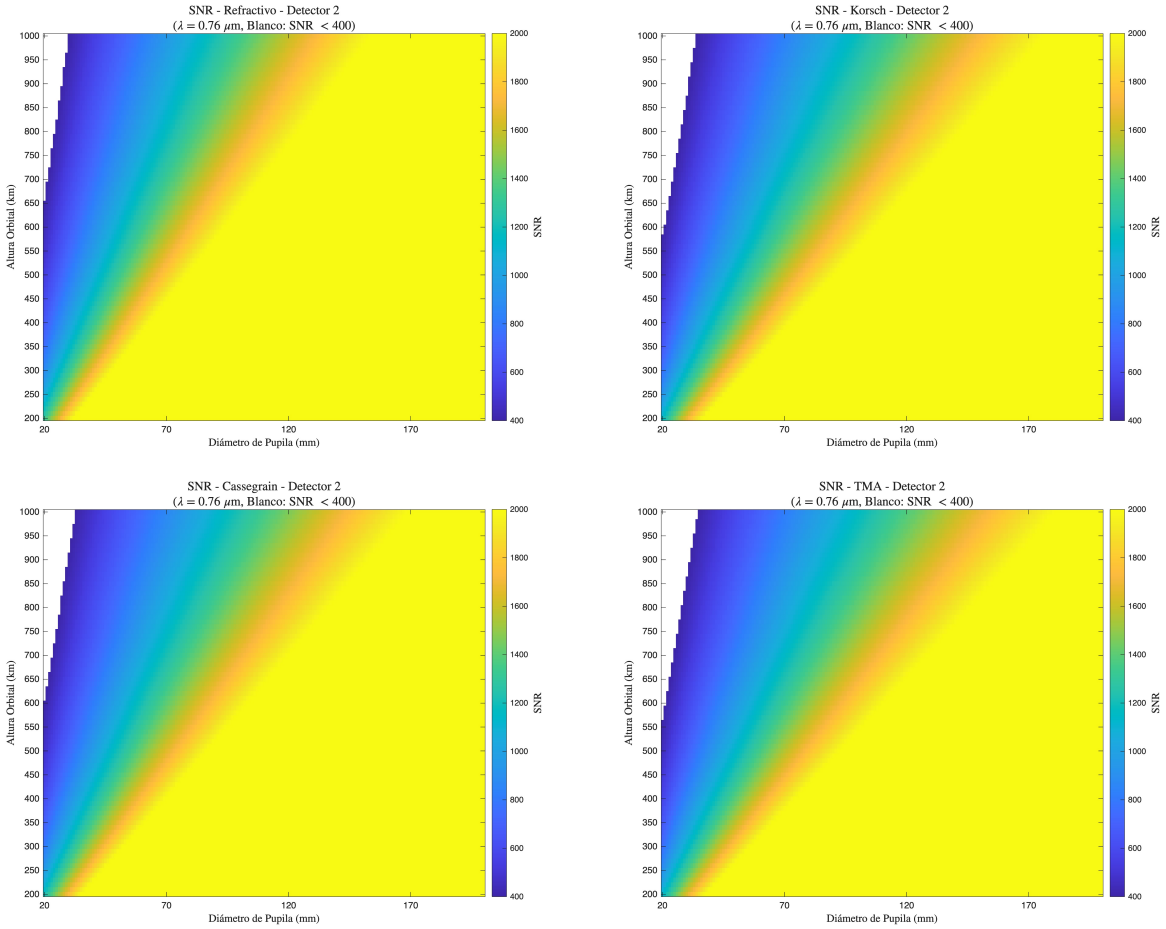


Figura 4.8: Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 5 (H2RG)

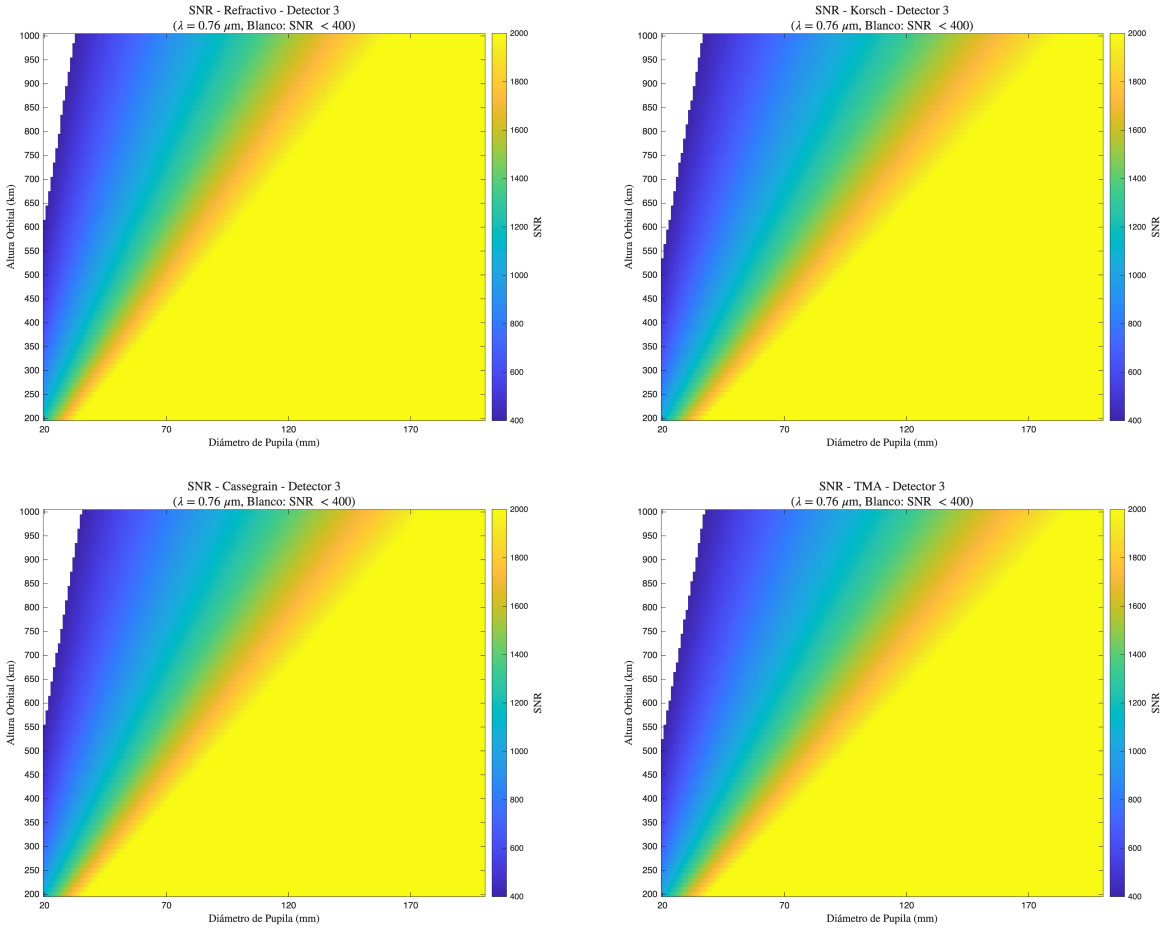


Figura 4.9: Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 6 (SATURN VISIR)

De la interpretación de estos gráficos se extraen las siguientes observaciones:

- Es el requisito menos restrictivo: con píxeles grandes, alta eficiencia cuántica y longitudes de onda por encima del visible, se alcanzan valores de SNR elevados con facilidad.
- Se puede descartar la incorporación de TDI, ya que con una integración simple se captura una relación señal-ruido que cumple holgadamente con el requisito de la misión para todos los casos.

4.7. Cobertura de la región de interés: *Swath*, FoV y tiempo de revisita

Otro de los requerimientos impuestos por el cliente es el obtener un mapa cada **7 días** en la región, o lo que es lo mismo, un tiempo de revisita de **7 días** o menor. Si se comparan con otras misiones de observación similares, se puede observar que es un tiempo exigente que requiere habitualmente una constelación por lo que será de los parámetros más limitantes para encontrar la solución óptima:

Tabla 4.6: Comparativa de tiempos de revisita de misiones de satélite.

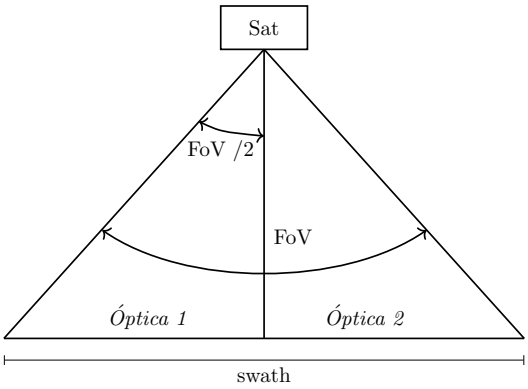
	N.º Satélites	*Swath* (km)	Tiempo revisita* (días)
Landsat 8 [59]	1	185	16
Landsat Next [60]	3	164	6
Sentinel-2 [61]	2	290	5

*Estos valores son más favorables que los calculados en este trabajo, pues no se consideran los factores limitantes descritos en esta página. Por ello, no serán directamente comparables a los resultados obtenidos, aunque permiten hacerse una idea aproximada de la solución.

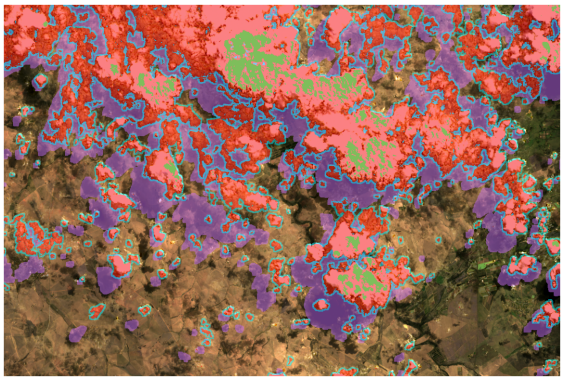
Además, se tendrán en cuenta 2 factores adicionales para el cálculo:

- **Solapamiento del *swath***: Para asegurar la cobertura completa de la región entre pasadas, se establece un factor de solapamiento del *swath* mínimo del **5 %** que compensa posibles desviaciones en la proyección del swath y pérdidas de altura del satélite antes de la reinyección orbital.
- **Cobertura de nubes**: A diferencia de otros sistemas de observación, como el SAR (Radar de Apertura Sintética, o *Synthetic Aperture Radar*, [62]), los detectores SWIR pasivos no son capaces de capturar la señal de la superficie terrestre a través de los cúmulos nubosos. El enunciado del cliente establece un día cubierto por cada cinco despejados. Por ello, el modelo adopta una fracción media de pasadas no utilizables

$f_{\text{nube}} = 1/6$ y una fracción media útil $1 - f_{\text{nube}} = 5/6$. Este valor procede del enunciado y no representa una simulación meteorológica píxel a píxel.



(a) Solapamiento del *swath*.
Fuente: Elaboración propia



(b) Datos no válidos debido a cobertura de nubes de la misión Landsat, en rojo.
Fuente: [63]

Figura 4.10: Factores de limitación del *swath* en el problema de la cobertura

Para el cálculo de la cobertura en la región de interés, se utiliza un software *open-source* desarrollado en el artículo *A Semi-Analytical Method for Calculating Revisit Time for Satellite Constellations with Discontinuous Coverage* [64]. Para aplicar dicho código al presente trabajo, se ha desarrollado una función de interfaz (*wrapper*) que actúa como adaptador entre los parámetros específicos del presente estudio y las funciones originales del software mencionado.

El barrido inicial utiliza $\phi_{\text{inf}} = 25°\text{N}$ como prefiltro. El mayor radio de ese paralelo no garantiza que sea la latitud con peor revisita, pues también interviene la separación de las trazas. Antes de seleccionar la solución se comprueba el conjunto:

$$\Phi = \{25°, 27°, \ldots, 49°\}\text{N}. \tag{4.7.1}$$

En cada latitud se calcula, mediante trigonometría esférica sobre un elipsoide terrestre, el semiancho angular en longitud que cubre el *swath* del sensor en cada paso del satélite. La traza subsatelital se propaga después con movimiento kepleriano circular corregido por la deriva secular del nodo debida al J2 y por la rotación terrestre. El modelo obtiene los instantes sucesivos en que la traza observa esa latitud dentro del campo de visión del instrumento. La métrica resultante es el intervalo máximo entre observaciones consecutivas sobre la franja de referencia dentro de la ventana simulada.

El sensor observa exclusivamente al **nadir**, con barrido *pushbroom* centrado en la traza subsatelital: no existe apuntamiento transversal ni maniobra de *roll*. Por tanto, el ángulo central de observación fuera de nadir es nulo; el parámetro ψ usado más abajo no representa

apuntamiento, sino el semiángulo geométrico desde el nadir hasta el borde del *swath*. En consecuencia, el *swath* queda determinado únicamente por la altura orbital y el campo de visión (FoV) del telescopio, o bien por el número de detectores dispuestos en línea, lo que primero se alcance:

$$W_{\mathrm{FoV}} = 2\,h\tan\!\left(\frac{\mathrm{FoV}}{2}\right) \tag{4.7.2}$$

$$W_{\mathrm{det}} = N_{\mathrm{det}} \cdot N_{\mathrm{pix}} \cdot GSD \tag{4.7.3}$$

$$W_{\mathrm{ef}} = W\,(1 - f_{\mathrm{ov}}), \qquad \psi = \arctan\!\left(\frac{W_{\mathrm{ef}}}{2h}\right) \tag{4.7.4}$$

$$T_{\mathrm{rev,geom}} = \max_{\phi\in\Phi,\,\lambda,\,k} \left(t_{k+1} - t_k\right)_{\phi,\lambda} \tag{4.7.5}$$

donde h es la altura orbital, FoV el campo de visión total del telescopio (duplicado si se embarcan dos telescopios), N_{det} el número de detectores alineados (hasta 3), N_{pix} los píxeles por detector, GSD la distancia de muestreo en tierra, W el *swath* adquirido, limitado por $\min(W_{\mathrm{FoV}}, W_{\mathrm{det}})$, f_{ov} la fracción de solapamiento (5 %) y ψ el semiángulo geométrico desde el nadir hasta el borde del *swath*. En el adaptador implementado para este trabajo, ψ se calcula a partir de W_{ef} y se introduce como entrada angular en el método semianalítico empleado. La ecuación (4.7.5) expresa la métrica geométrica como el mayor intervalo entre observaciones consecutivas sobre las latitudes comprobadas, tomando la malla de longitudes del método.

Primero se calcula esta métrica geométrica. Después se aplica la penalización media por nubosidad indicada en el enunciado. El requisito de 7 días se aplica al tiempo efectivo:

$$T_{\mathrm{rev,eff}} = \frac{T_{\mathrm{rev,geom}}}{1 - 1/6} = 1{,}2\,T_{\mathrm{rev,geom}} \tag{4.7.6}$$

La corrección representa la fracción media de días despejados del enunciado; no simula nubes correlacionadas en el espacio o en el tiempo ni garantiza un mapa semanal sin nubes. Este valor efectivo es el que se guarda en los ficheros de cobertura, se representa en los mapas de calor y se compara directamente con el requisito de **7 días**. El valor geométrico equivalente sería $T_{\mathrm{rev,geom}} \leq 7(5/6) = 5{,}83$ días.

El método se apoya en las siguientes hipótesis: propagación kepleriana circular con corrección secular J2 en el nodo, Tierra modelada como elipsoide para el radio local de la latitud de cálculo y observación estrictamente al nadir. El adaptador desarrollado en este proyecto (*wrapper*) parametriza la altitud, el número de satélites y telescopios, el ancho de barrido y

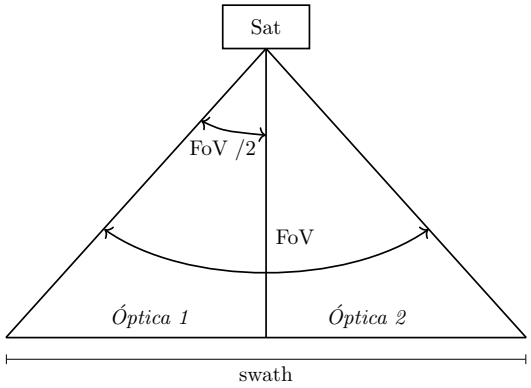
el solapamiento, encadenando las ecuaciones (4.7.2)–(4.7.5) con las funciones originales del software para automatizar el barrido paramétrico completo sobre el espacio de diseño.

Cada configuración evaluada (combinación de número de satélites, telescopios y detector, que fija el *swath* disponible) se simula de forma independiente. El *wrapper* obtiene la inclinación heliosíncrona correspondiente a cada altura, calcula el tiempo de revisita efectivo y lo escribe en un archivo `coverage_*.csv`. Estos archivos proporcionan el prefiltro a 25° N. Para las soluciones candidatas, el análisis de configuraciones vuelve a evaluar las trece latitudes y conserva anchos de barrido que cumplen el límite efectivo en todas ellas. La discretización de altura y *swath* se realiza en pasos de 10 km. Se evalúan distintas configuraciones:

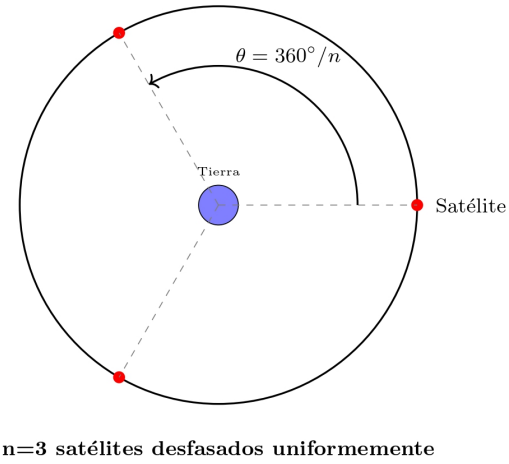
- Configuración (i): 1 único satélite con 1 telescopio embarcado.
- Config. (ii): 1 satélite con 2 telescopios embarcados. Los telescopios serán dispuestos según la Figura 4.11a, duplicando el ancho de barrido efectivo.
- Config. (iii): Constelación de 2 satélites ($i : 2/1/0$[1]).
- Config. (iv): Constelación de **2 satélites**, con **2 telescopios** embarcados en cada uno. ($i : 2/1/0$).
- Config. (v): Constelación de 3 satélites ($i : 3/1/0$).

En todas las constelaciones, los satélites se dispondrán desfasados en anomalía verdadera de manera que $\Delta\nu = 360°/n$ siendo n = N.º satélites, dentro de la misma órbita (véase Figura 4.11b). Esta separación uniforme reparte los satélites a lo largo de la órbita y reduce el intervalo temporal entre pasadas sucesivas sobre la región a medida que la Tierra rota. Se mantiene un único plano SSO para que todos compartan la misma inclinación, RAAN, LTAN y geometría de iluminación.

[1]Escrito en notación de Walker [65]



(a) Disposición de dos telescopios embarcados en un satélite.
Fuente: Elaboración propia



(b) Desfase de la constelación de satélites en anomalía verdadera.
Fuente: Elaboración propia.

Figura 4.11: Esquemas de las configuraciones evaluadas en el problema de la cobertura.

Se consideran dos limitaciones al *swath*:

- El FoV de cada telescopio, definido en la tabla 3.1, que limita el *swath* en función de la altura, mediante la relación (3.1.19).
- El array del detector, mediante la ecuación (3.1.18). Para ampliarlo y dar más flexibilidad a esta restricción, se considera el uso de hasta 3 detectores en línea.

Además, solo se considerará el territorio EE. UU. continental, representado mediante las latitudes de la ecuación (4.7.1). Ya que el interés del cliente reside en la elaboración del mapa de las principales ciudades, se pueden excluir, por simplicidad de cálculo, otros territorios del país como Hawaii, Puerto Rico y otros territorios de ultramar.

Las figuras siguientes muestran el prefiltro de cobertura a 25° N para las configuraciones evaluadas. En cada mapa, el eje horizontal representa el *swath* nominal, el eje vertical la altura orbital y el color el tiempo de revisita efectivo, incluyendo la penalización media por nubosidad. El cálculo emplea $W_{\text{ef}} = 0{,}95W$. Las celdas coloreadas cumplen el límite de 7 días; las blancas superan ese límite o incumplen una restricción física. La línea roja marca el límite de FoV y las líneas negras los límites de uno, dos y tres detectores dispuestos en línea. El filtro regional posterior comprueba las trece latitudes antes de seleccionar la solución.

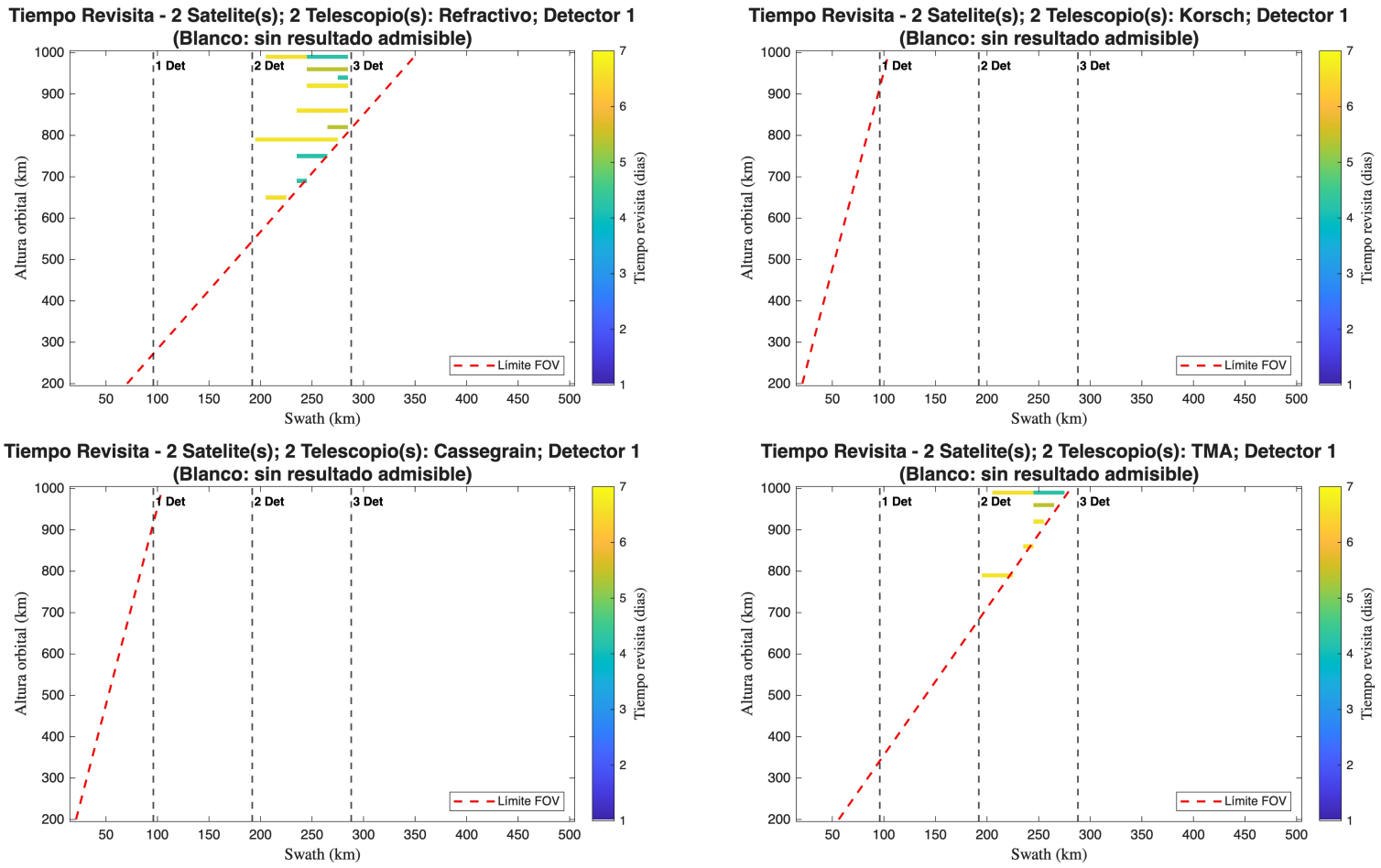


Figura 4.12: Mapas de calor resultantes del cálculo de cobertura para el Detector 1, con 2 satélites y 2 telescopios.

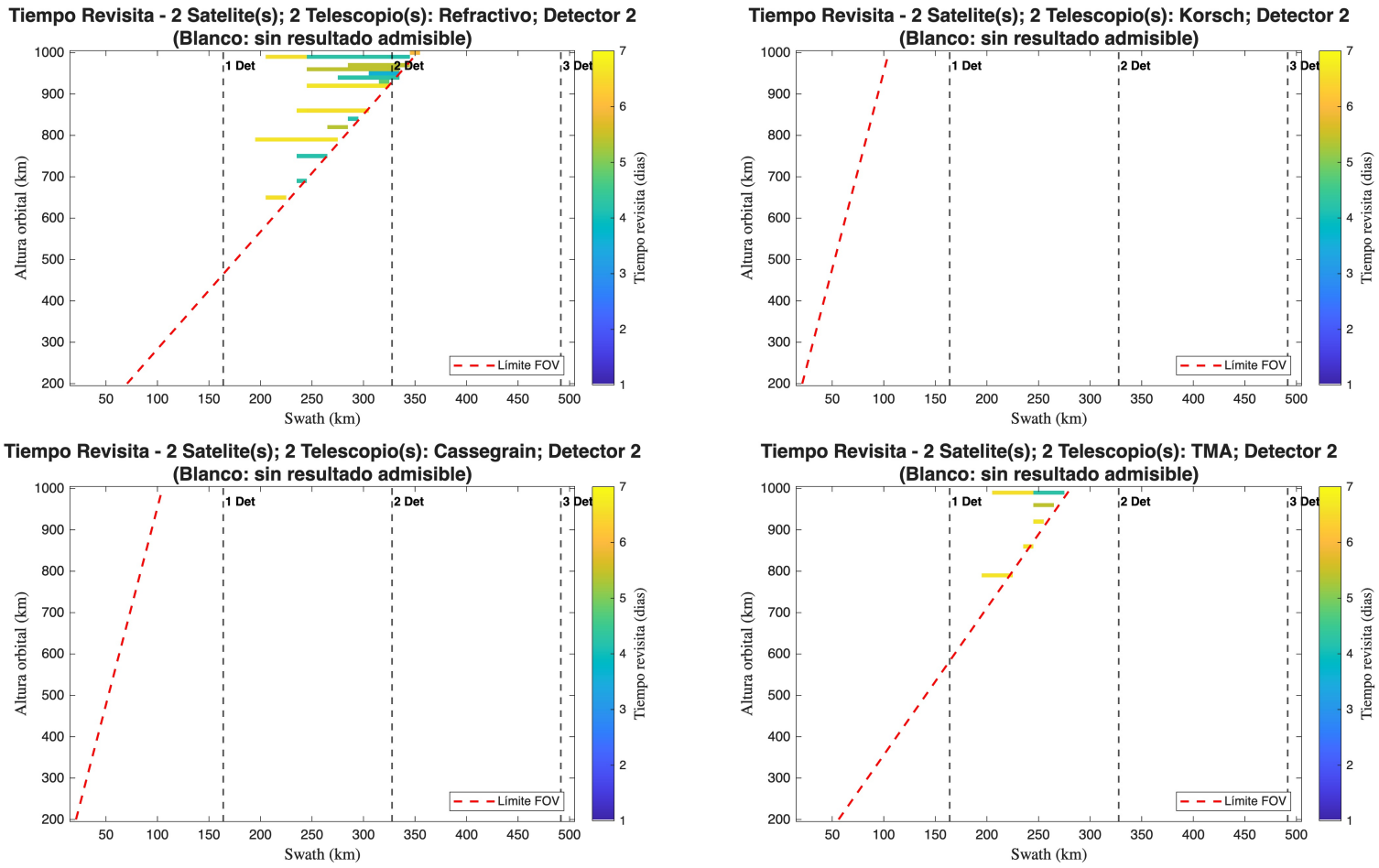


Figura 4.13: Mapas de calor resultantes del cálculo de cobertura para el Detector 2, con 2 satélites y 2 telescopios.

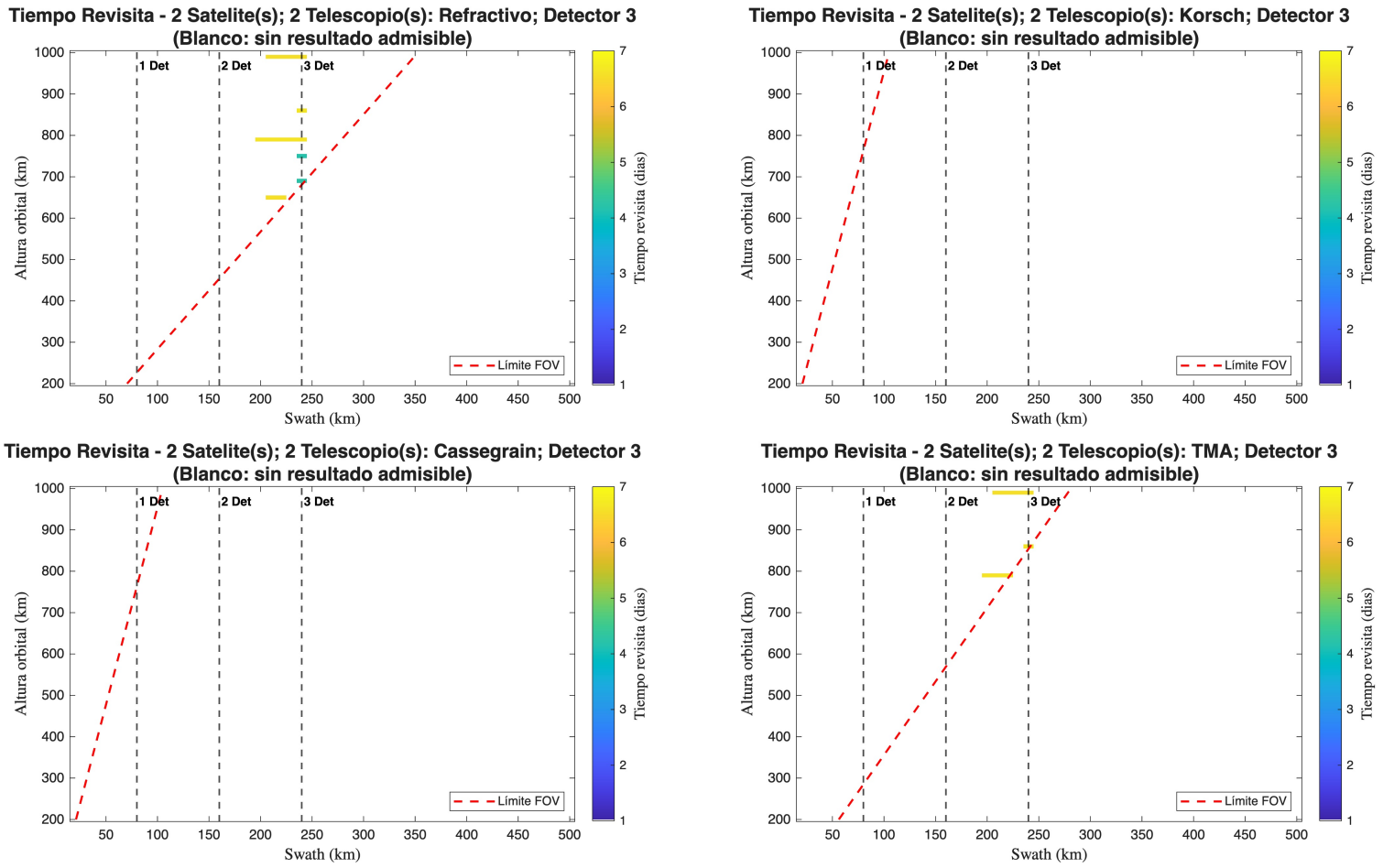


Figura 4.14: Mapas de calor resultantes del cálculo de cobertura para el Detector 3, con 2 satélites y 2 telescopios.

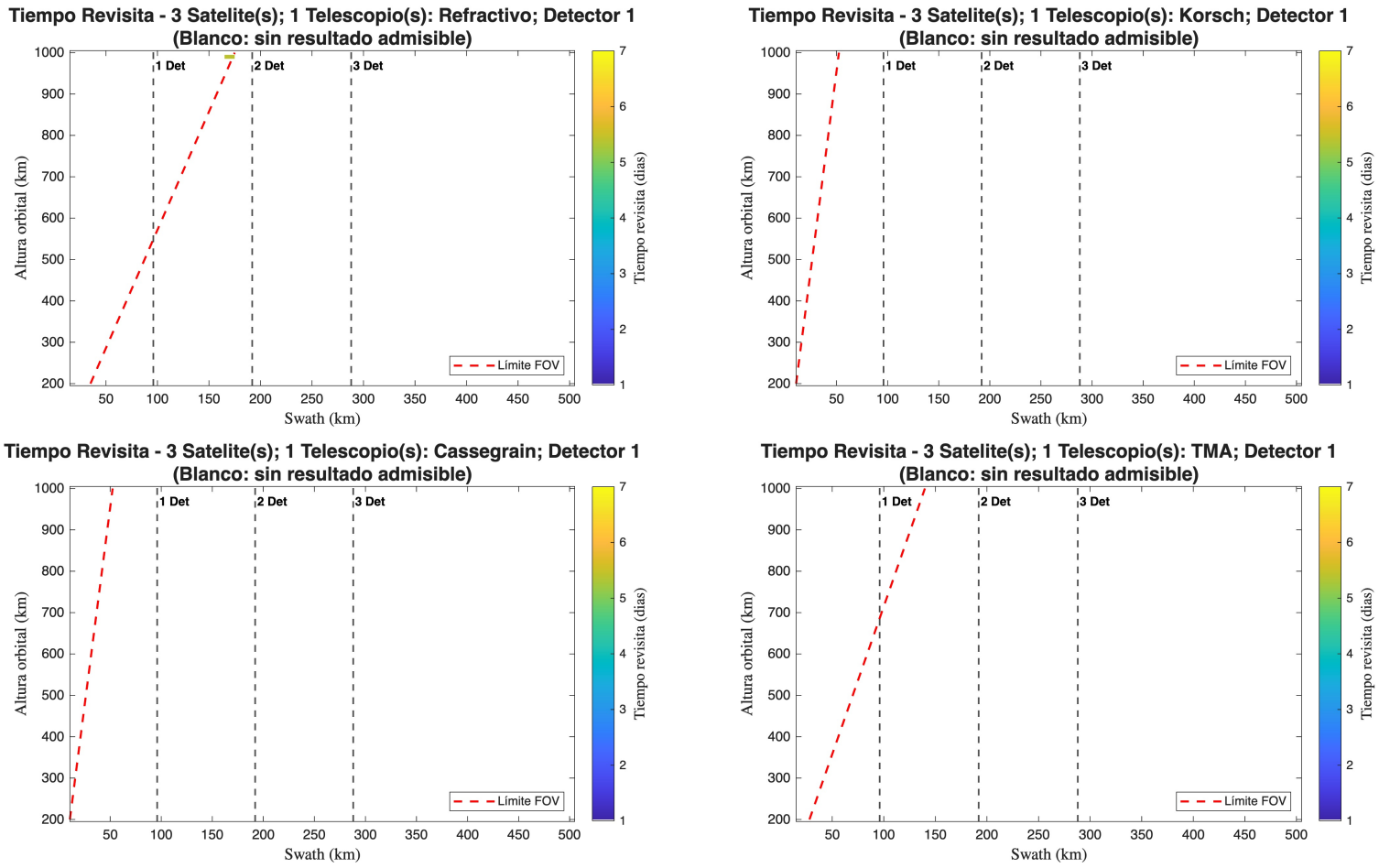


Figura 4.15: Mapas de calor resultantes del cálculo de cobertura para el Detector 1, con 3 satélites.

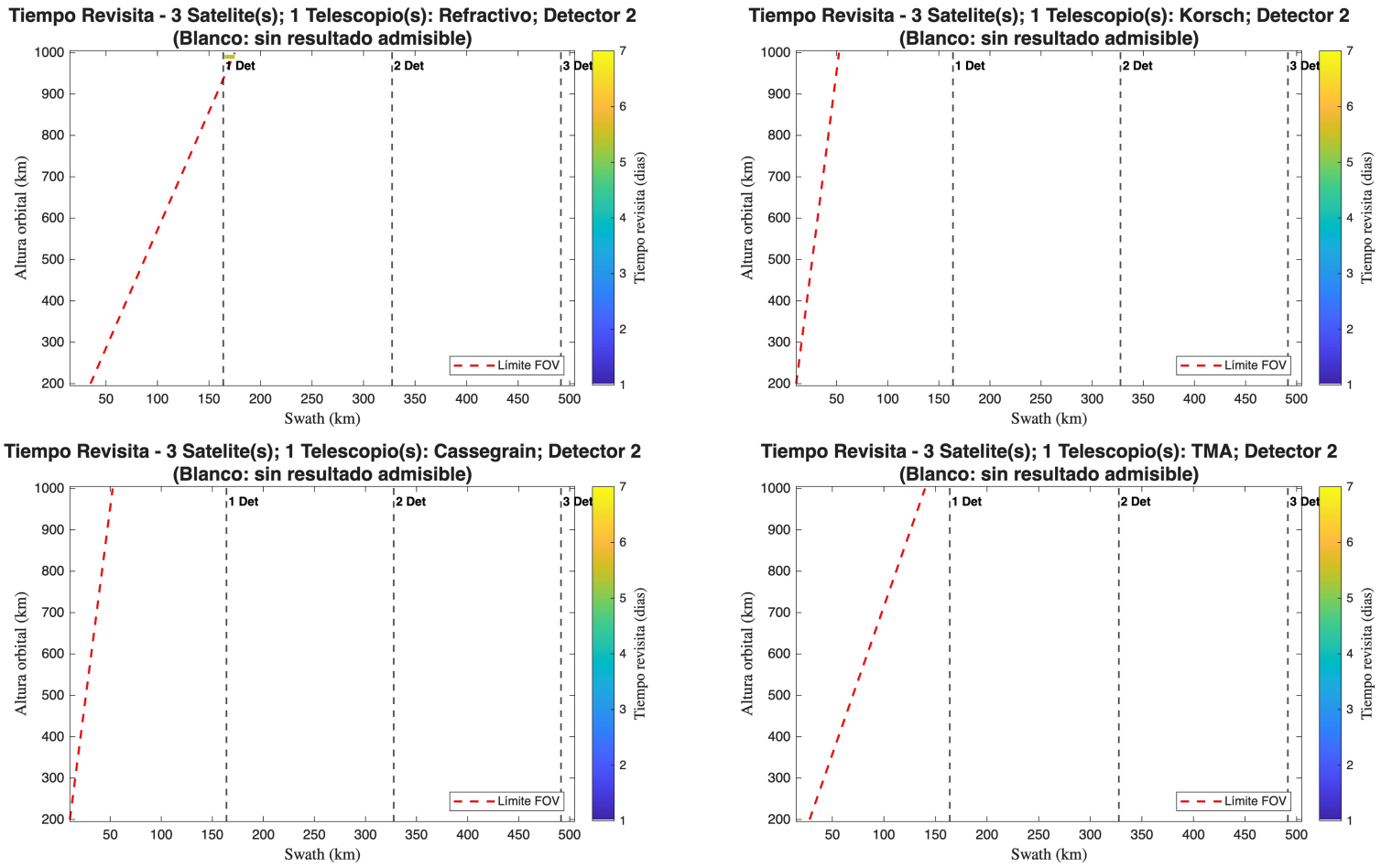


Figura 4.16: Mapas de calor resultantes del cálculo de cobertura para el Detector 2, con 3 satélites.

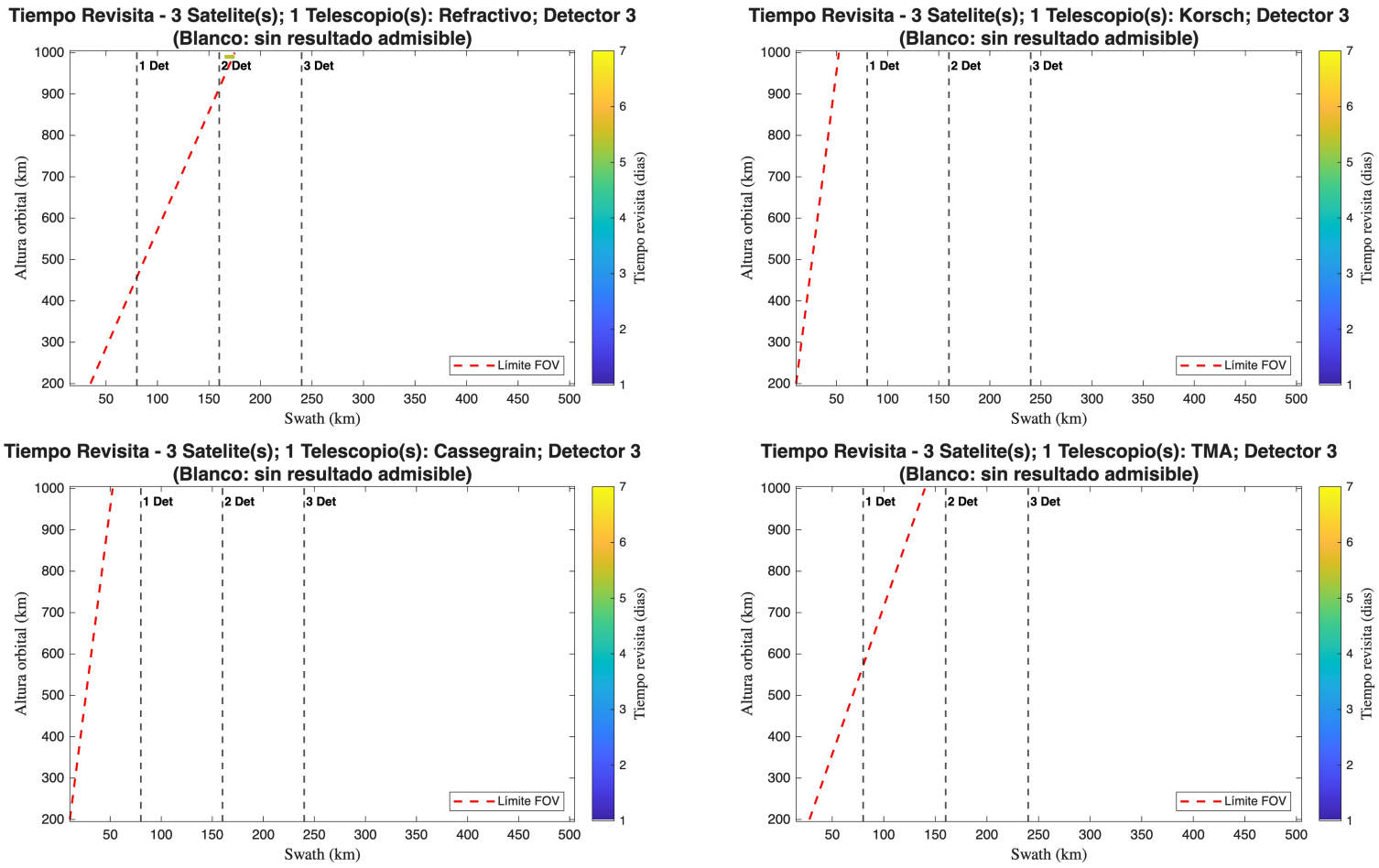


Figura 4.17: Mapas de calor resultantes del cálculo de cobertura para el Detector 3, con 3 satélites.

La figura 4.18 muestra la comprobación regional de la solución finalmente seleccionada. El peor valor no se obtiene a 25° N, sino a 41° N. Para $h = 690$ km, $W = 240$ km y $W_{\text{ef}} = 228$ km, la revisita máxima es de 5,535 días geométricos y 6,642 días efectivos.

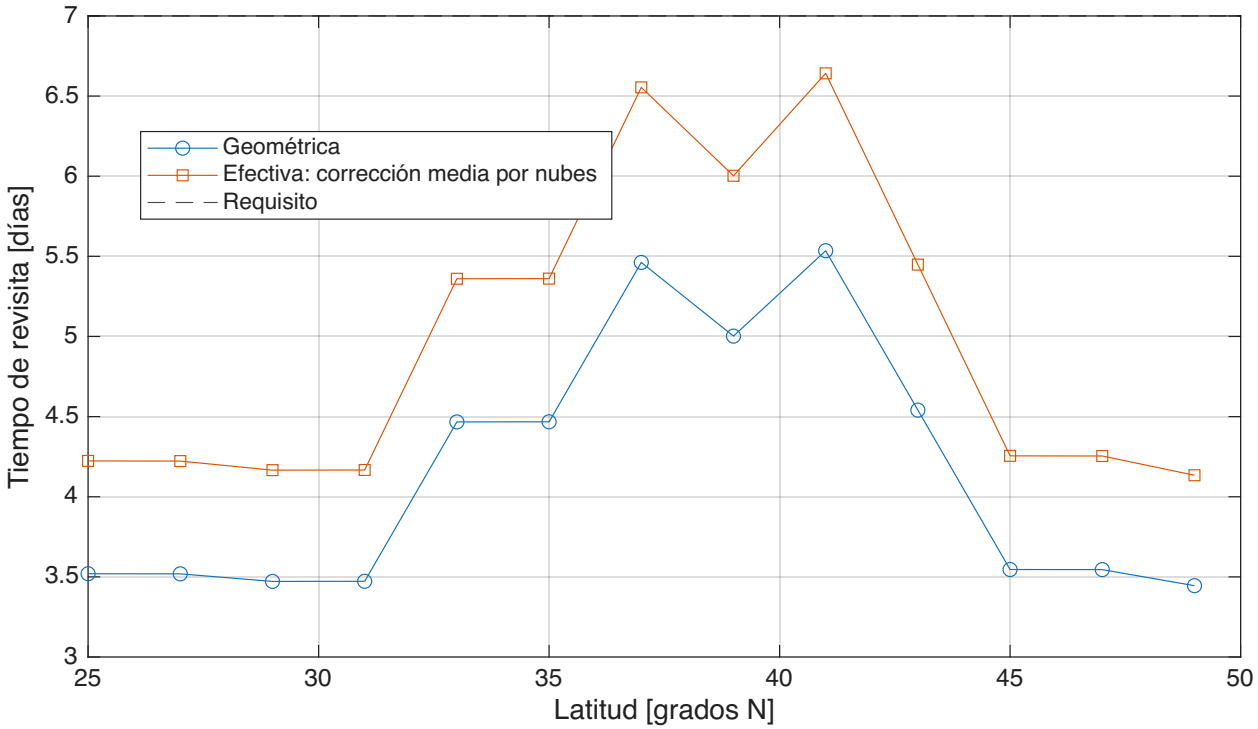


Figura 4.18: Revisita geométrica y efectiva de la solución seleccionada en las trece latitudes del filtro regional. Fuente: Elaboración propia.

La comprobación utiliza una malla de latitudes, no una integración continua sobre todo el territorio. El requisito semanal se satisface en esas muestras con la penalización media de nubosidad adoptada. El efecto de condiciones meteorológicas persistentes requiere un análisis adicional.

4.8. Posibles soluciones. Cruce de los datos

Para identificar las soluciones viables que satisfacen los requisitos del cliente, se ha desarrollado un *script* adicional que integra los resultados de MTF, SNR y cobertura, filtrando las combinaciones que satisfacen simultáneamente los requisitos de MTF, SNR y tiempo de revisita. Tras comprobar las trece latitudes quedan 20 soluciones del conjunto discreto explorado. Se comparan sus masas secas y de mantenimiento en el capítulo siguiente; no se selecciona la altura por ser la primera celda que supera únicamente el filtro óptico.

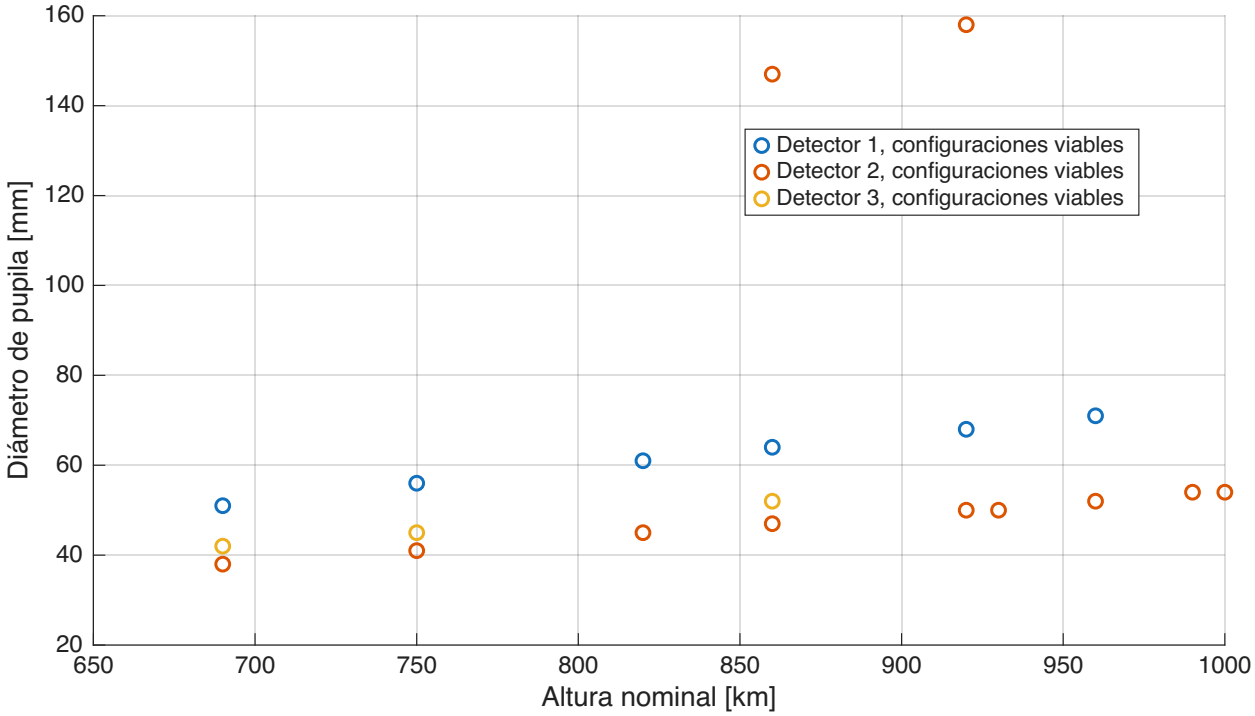


Figura 4.19: Diámetros de pupila de las soluciones que superan los filtros ópticos y regionales. Fuente: Elaboración propia.

La solución de menor masa de selección utiliza **telescopios refractivos** y el **detector Teledyne H2RG**, con **dos telescopios y dos detectores por satélite** y una constelación de **dos satélites**. La altura definitiva se fija en el capítulo siguiente, al incorporar el combustible de mantenimiento.

Capítulo 5

Determinación de la solución óptima

Contenido

5.1. Estimación de la masa seca 83

5.2. Impulsos, ΔV y masa de combustible. Fin de misión. 84

5.2.1. Modelado de la resistencia aerodinámica 85

5.2.2. Cálculo del Delta-V, número de maniobras y masa de combustible 85

5.2.3. Fin de misión 90

5.3. Verificación de las condiciones orbitales 92

5.3.1. Condición de órbita *frozen* 92

5.3.2. Variación de altura en la zona de observación 92

5.3.3. Evolución de la excentricidad durante la misión 93

5.3.4. Condición de traza repetida (RGT) 95

5.4. Sensibilidad de la solución al GSD 96

Para evaluar la mejor solución y determinar la altura orbital óptima, se realiza una simulación que estima la masa total de las posibles soluciones de satélite. El cálculo separa la masa seca de la masa de combustible para no mezclar salidas de naturaleza distinta. El procedimiento es el siguiente:

1. Se estima la masa seca del satélite a partir del diámetro de pupila seleccionado y de instrumentos de referencia. De este mismo escalado se obtienen la geometría y la superficie del instrumento.

2. Se calcula la masa de combustible con un modelo de decaimiento orbital que devuelve el número de maniobras, el ΔV acumulado y el propulsante necesario para mantener la órbita durante 8 años.

3. Se suma la masa seca y la masa de combustible de mantenimiento para obtener la masa total en cada altura viable. La solución óptima será el mínimo de esa curva dentro de las restricciones de cobertura, SNR y MTF.

4. Una vez fijada la solución, se calcula la reserva adicional de combustible para deorbitado activo al final de misión.

5.1. Estimación de la masa seca

El libro *System Mission Analysis and Design* (1999, 3.ª ed.) [66], en su capítulo 9.5 , presenta un método para estimar la masa del instrumento, así como del satélite, en función de otros instrumentos con diámetros de pupilas y masas conocidos. Dichas ecuaciones son:

$$R = \frac{A_i}{A_o}; \quad L_i = R \cdot L_o; \quad S_i = L_i^2; \quad V_i = L_i^3$$
$$W_i = K \cdot R^3 \cdot W_o \tag{5.1.1}$$

- El subíndice i indica el parámetro de la misión actual, mientras que el subíndice o será el parámetro de la misión de referencia.
- A, L, S, V, W definen, respectivamente, el diámetro, longitud, superficie, volumen y masa del instrumento.
- Se define el factor K como 1 cuando $R \geq 0,5$. En caso contrario se establece $K = 2$.

Usando datos de otras misiones proporcionadas por el tutor (THEMATIC MAPPER Y SEOSAT), se realiza una estimación por escalado: las dimensiones lineales se escalan con la razón de apertura y la masa del instrumento se aproxima con una ley cúbica, proporcional al volumen resultante, antes de promediar ambas referencias.

Tabla 5.1: Instrumentos de referencia para la estimación por escalado.
Fuente: [67].

Parámetro	Thematic Mapper	SEOSAT Primary Payload
Dimensiones	2 m × 1 m × 1 m	1 m × 1 m × 1 m
Masa	240 kg	100 kg
Pupila	0,4 m	0,25 m

Suponiendo además que la masa del instrumento es un 25 % de la masa del satélite, que un telescopio adicional aumenta la masa del instrumento en un 50 % [67], y utilizando como entrada la relación h *vs* D_{min} extraída previamente, se obtienen los siguientes resultados:

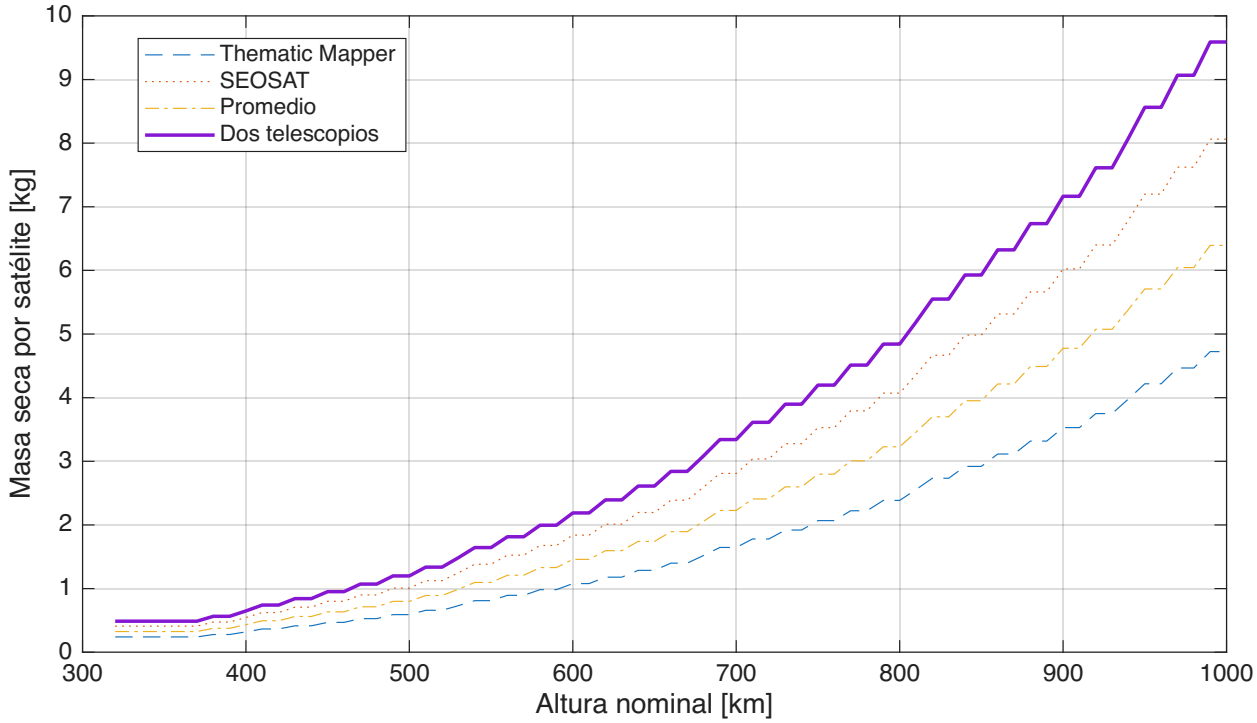


Figura 5.1: Estimación de la masa seca del satélite en función de la altura.
Fuente: Elaboración propia.

Aparecen, en la figura 5.1, las estimaciones de masa seca realizadas con los puntos extraídos de la relación h *vs* D_{min}, mediante THEMATIC MAPPER y mediante SEOSAT, junto con su promedio. La relación propuesta (en amarillo) aumentará un 50 % sobre este promedio al tener un telescopio adicional.

En la sección 5.2.2 se incorpora la masa de combustible a esta estimación. La solución óptima es el mínimo de la masa total (seca más combustible) entre los puntos que cumplen cobertura, SNR y MTF. La geometría de iluminación queda fijada por el LTAN de la órbita SSO. Al distribuir los satélites en un único plano, se conserva la arquitectura de dos satélites que resulta del filtro de viabilidad y de la comparación de masa.

5.2. Impulsos, ΔV y masa de combustible. Fin de misión.

Para calcular la masa de combustible necesaria para mantener la misión durante los 8 años requeridos, se debe realizar un modelo de resistencia aerodinámica.

5.2.1. Modelado de la resistencia aerodinámica

La variación temporal del semieje mayor $a = R_T + h$ debida a la resistencia aerodinámica se modela mediante

$$\frac{\mathrm{d}a}{\mathrm{d}t} = -\frac{C_d\, A\, \rho(h)\, \sqrt{\mu\, a}}{m}, \tag{5.2.1}$$

donde C_d es el coeficiente de resistencia aerodinámica, A el área frontal efectiva, m la masa del satélite y μ el parámetro gravitacional terrestre. Se toma $C_d = 2,5$ [68]. Para no subestimar el área expuesta, el cálculo usa $A = 1,2A_{\mathrm{ref}}$, donde A_{ref} procede del dimensionado geométrico del satélite. La densidad $\rho(h)$ se aproxima con un modelo exponencial por capas, basado en valores tabulados de atmósfera estándar para órbitas bajas [49], [69]:

$$\rho(h) = \rho_0\, \exp\!\left[-\frac{h - h_{\mathrm{base}}}{H}\right], \tag{5.2.2}$$

siendo ρ_0 la densidad tabulada en la base de cada capa, h_{base} la altura de referencia y H su altura de escala.

5.2.2. Cálculo del Delta-V, número de maniobras y masa de combustible

La cobertura y el dimensionado del instrumento se calculan a la altura nominal h_{nom}. El mantenimiento orbital utiliza una banda alrededor de esa altura:

$$h_{\mathrm{mín}} = 0{,}98 h_{\mathrm{nom}}, \qquad h_{\mathrm{máx}} = 1{,}02 h_{\mathrm{nom}}. \tag{5.2.3}$$

Cada ciclo comienza en $h_{\mathrm{máx}}$. El satélite desciende hasta $h_{\mathrm{mín}}$ y ejecuta una maniobra que lo devuelve a $h_{\mathrm{máx}}$.

El código calcula el tiempo de cada ciclo a partir de la ley de decaimiento de la ecuación (5.2.1). Divide el intervalo entre $h_{\mathrm{mín}}$ y $h_{\mathrm{máx}}$ en los tramos definidos por la tabla atmosférica y evalúa la integral en cada uno de ellos. El tiempo de ciclo resulta

$$T_{\mathrm{ciclo}} = \int_{h_{\mathrm{mín}}}^{h_{\mathrm{máx}}} \frac{m\, \mathrm{d}h}{C_d A \rho(h) \sqrt{\mu (R_\oplus + h)}}. \tag{5.2.4}$$

La integración proporciona el tiempo transcurrido hasta alcanzar $h_{\mathrm{mín}}$. Después de cada maniobra, el mismo ciclo comienza de nuevo en $h_{\mathrm{máx}}$ y se repite durante los ocho años de misión. El incremento de velocidad se obtiene mediante una transferencia de Hohmann entre ambas órbitas [47]: $r_1 = R_\oplus + h_{\mathrm{mín}}$ y $r_2 = R_\oplus + h_{\mathrm{máx}}$. Aquí, r_1 es el radio de la órbita al final del decaimiento y r_2 el radio de la órbita de reinicio. El incremento de velocidad por maniobra es

$$\Delta v = \left(\sqrt{\frac{\mu}{r_1}}\left[\sqrt{2r_2/(r_1+r_2)} - 1\right]\right) + \left(\sqrt{\frac{\mu}{r_2}}\left[1 - \sqrt{2r_1/(r_1+r_2)}\right]\right). \tag{5.2.5}$$

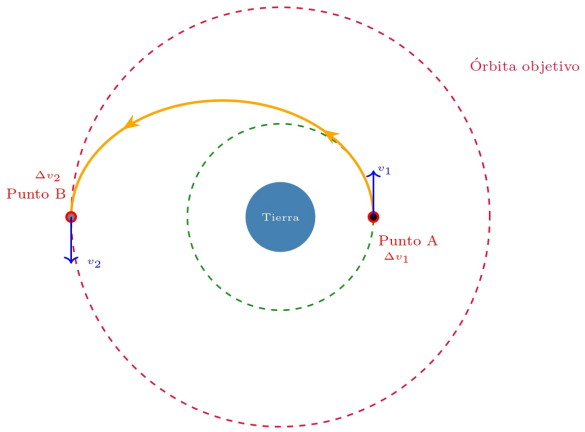


Figura 5.2: Transferencia de Hohmann.
Fuente: Elaboración propia.

El número de maniobras se obtiene a partir del tiempo de ciclo:

$$N = \left\lfloor \frac{T_{\text{misión}} - \text{eps}(T_{\text{misión}})}{T_{\text{ciclo}}} \right\rfloor, \qquad \Delta v_{\text{tot}} = N \Delta v_{\text{ciclo}}, \tag{5.2.6}$$

donde $T_{\text{misión}} = 8 \cdot 365{,}25 \cdot 24 \cdot 3600$ s. Cada maniobra se ejecuta al alcanzar $h_{\text{mín}}$.

La masa de combustible se calcula con la ecuación de Tsiolkovski [47]:

$$m_{\text{prop},0} = m_{\text{seca}} \left(\exp\left[\frac{\Delta v_{\text{tot}}}{I_{sp}\, g_0} \right] - 1 \right), \tag{5.2.7}$$

donde se emplea un impulso específico de $I_{sp} = 220$ s, valor típico para propulsión química, y $g_0 = 9{,}80665\ \text{m}\,\text{s}^{-2}$. Además, se añade un margen del **10 %** para contingencias: $m_{\text{prop, mant}} = 1{,}1\, m_{\text{prop},0}$, que se suma a la masa seca para obtener la masa de selección: $m_{\text{sel}} = m_{\text{seca}} + m_{\text{prop, mant}}$. La reserva de fin de misión se incorpora después, cuando ya se ha seleccionado la órbita.

El programa aplica este cálculo a cada altura y a cada configuración que supera los filtros de viabilidad.

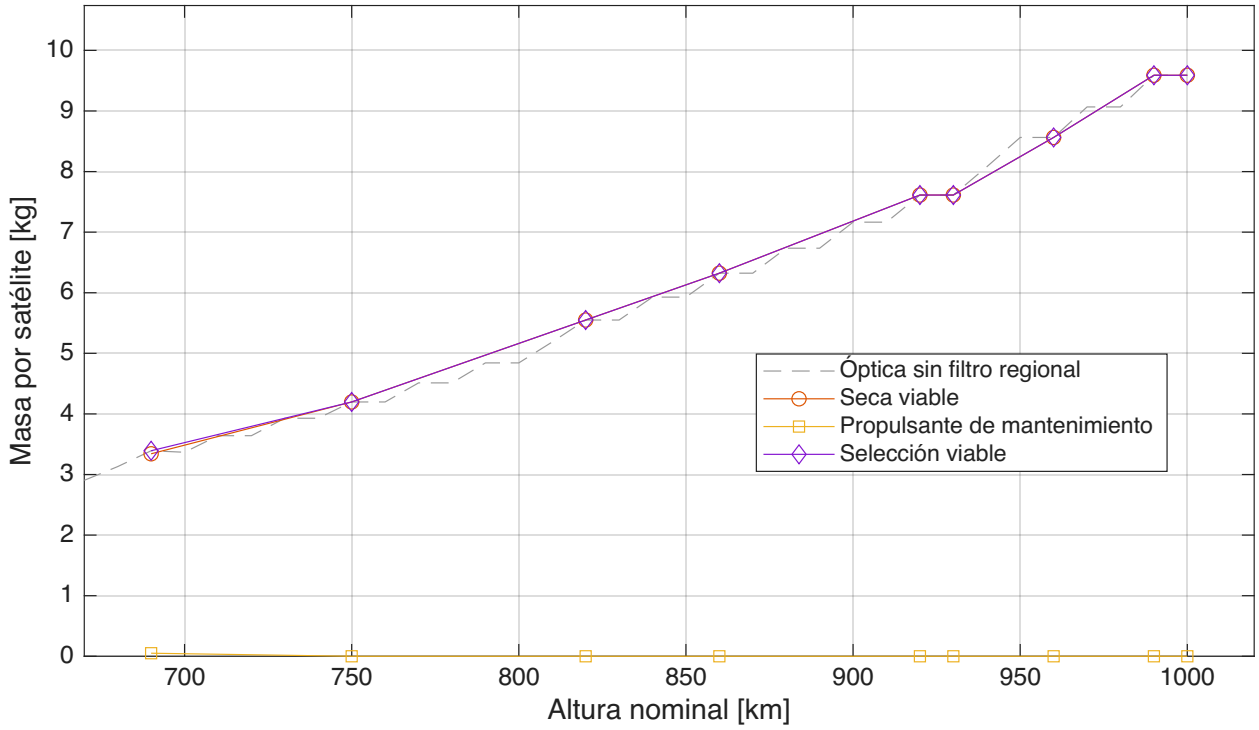
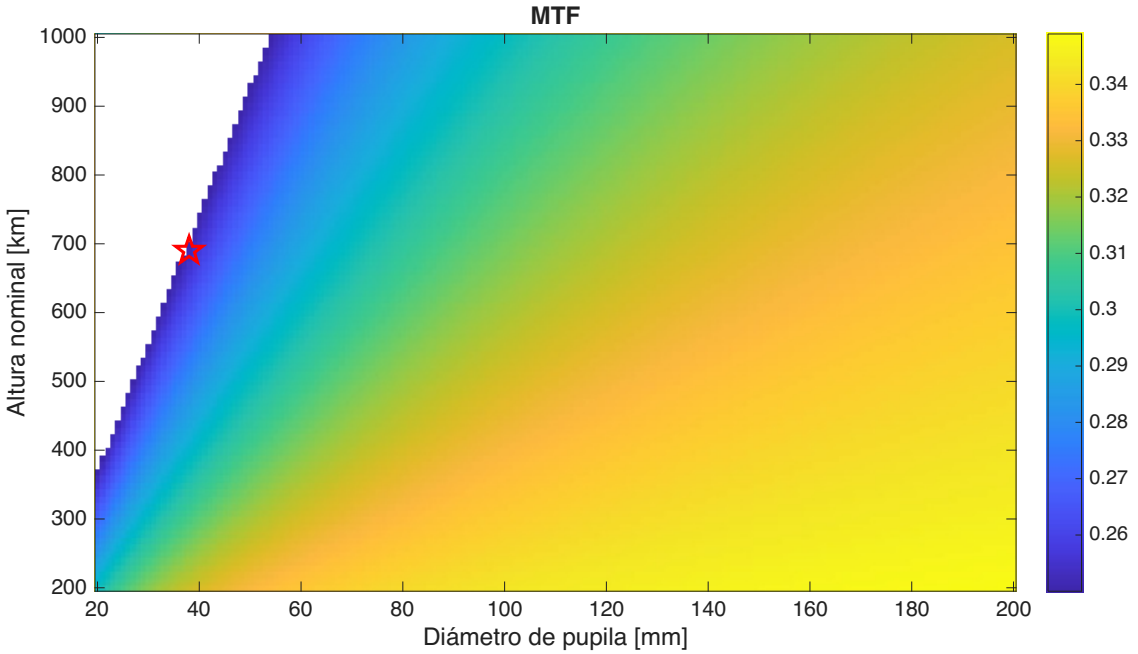


Figura 5.3: Desglose de masa seca, masa de combustible de mantenimiento y masa total de selección. La curva gris discontinua representa las soluciones ópticas sin el filtro regional. Los marcadores de color señalan los puntos viables, que además cumplen los requisitos de cobertura y FoV. Las oscilaciones de la masa seca se deben a la selección del diámetro de pupila en una malla discreta de 1 mm y al escalado cúbico de la masa.
Fuente: Elaboración propia.

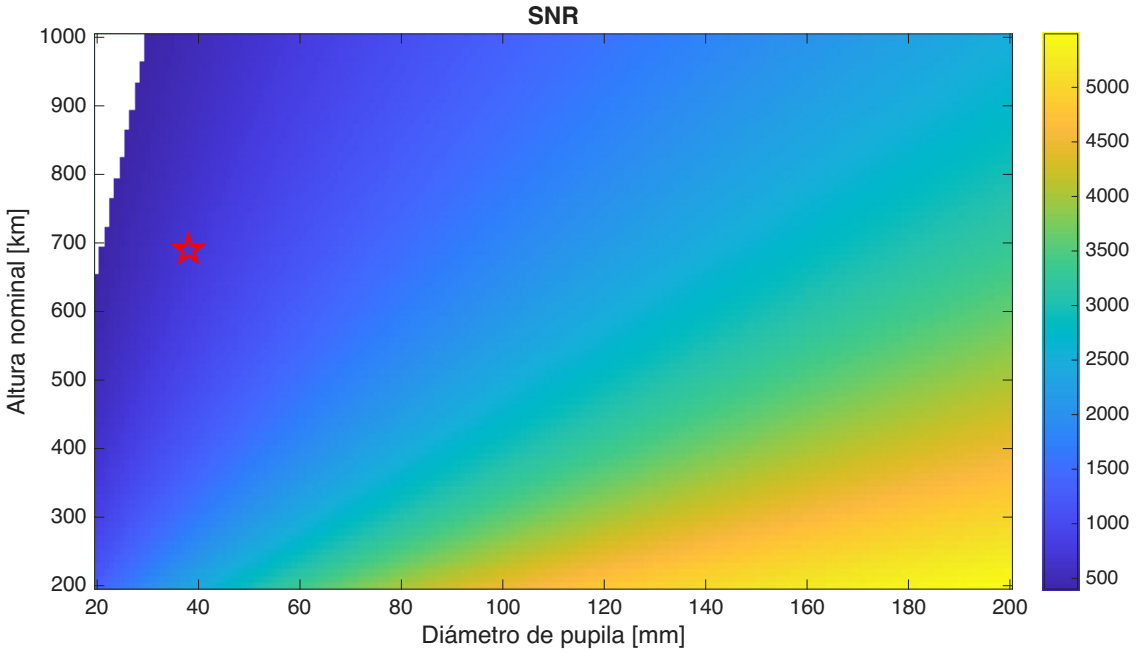
El mínimo de masa de selección entre las soluciones viables del barrido corresponde a **690 km** y a un diámetro de pupila de **38 mm**. Por satélite se obtienen $m_{\text{seca}} = 3{,}342$ kg y $m_{\text{prop,mant}} = 0{,}0503$ kg, cuya suma es $m_{\text{sel}} = 3{,}392$ kg. La constelación suma 6,784 kg antes de añadir la reserva de fin de misión. Esta selección minimiza masa seca más propulsante de mantenimiento; la reserva de deorbitado se calcula después y no se ha utilizado para ordenar todas las alternativas.

El ΔV_{mant} acumulado en ocho años es **29,325 m/s**, con **dos maniobras de mantenimiento**. Cada transferencia de Hohmann requiere dos encendidos; el número de maniobras no equivale al número de impulsos individuales.

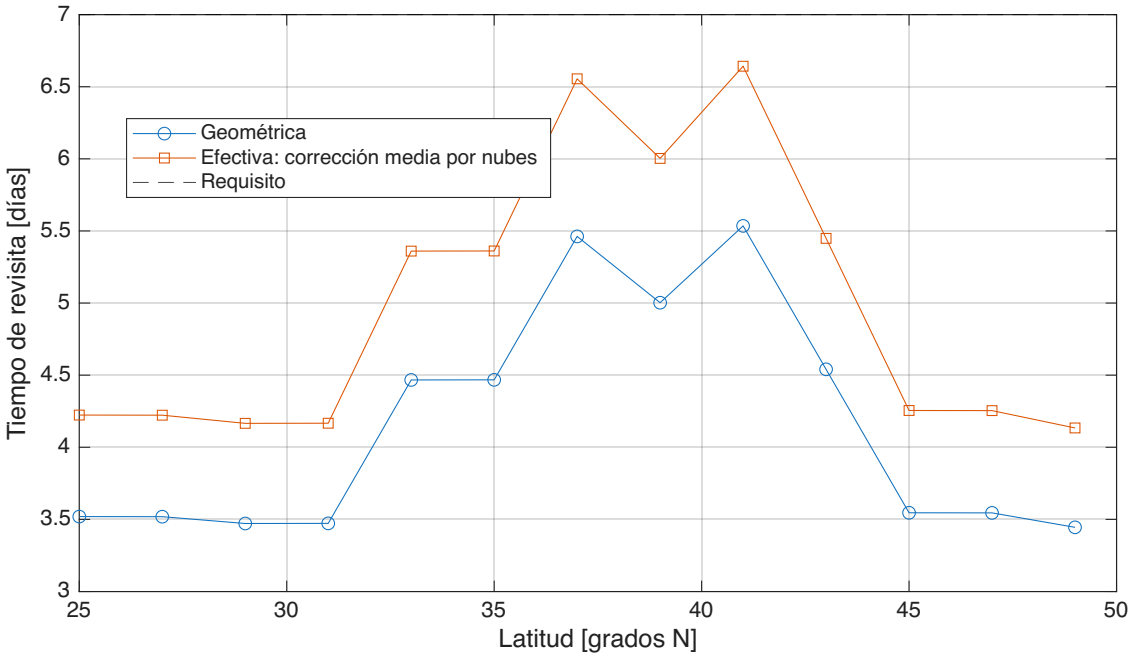


(a) MTF en banda $2,01\ \mu m$. Blanco: valores inferiores a 0,25.

Figura 5.4: Punto de diseño del satélite.
Fuente: Elaboración propia.



(b) SNR en banda $0,76\ \mu m$. Blanco: valores inferiores a 400.



(c) Revisita geométrica y efectiva en las latitudes comprobadas

El *swath* adquirido es $W = 240$ km, frente a los 243,331 km disponibles por FoV combinado en el modelo. El solapamiento del 5 % se aplica al ancho adquirido, de modo que $W_{\text{ef}} = 228$ km. Cada detector H2RG dispone de 2048 píxeles por línea, pero se utilizan 1500 por telescopio:

$$N_{\text{pix,activo}} = \frac{240\,000}{2 \cdot 80} = 1500.$$

Los dos telescopios aportan, por tanto, 3000 píxeles por línea combinada. La distancia focal es 155,25 mm, la relación focal es $f/4{,}09$, la MTF en 2,01 μm es 0,253 y la SNR en 0,76 μm es 738. La revisita efectiva máxima en las trece latitudes comprobadas es 6,642 días a 41°N, frente a 5,535 días geométricos.

En el anexo A se recogen las 20 soluciones viables del barrido y sus masas de selección. La solución presentada es el mínimo de ese conjunto discreto dentro de las hipótesis adoptadas.

5.2.3. Fin de misión

La reserva de fin de misión se estima para reducir la permanencia en órbita de los satélites tras el cese de operaciones [70]. Se adopta como objetivo de diseño un tiempo de reentrada inferior a **365 días** desde el fin de misión.

La siguiente figura se obtiene con el mismo modelo exponencial de resistencia aerodinámica usado para el mantenimiento orbital, aplicado desde la altura del semieje mayor al final de operaciones, de 699,945 km. Se emplean el área frontal efectiva del dimensionado geométrico

y $C_d = 2{,}5$. Se comparan dos casos: decaimiento natural sin maniobra y decaimiento con un impulso retrógrado al final de la misión.[1]

El cálculo de la maniobra activa se plantea como una transferencia tangencial en el apogeo de la órbita final. Para un perigeo candidato r_p, el semieje mayor de la elipse de deorbitado es:

$$a_d = \frac{r_a + r_p}{2}, \tag{5.2.8}$$

donde $r_a = R_\oplus + h_{\text{final}}$. El impulso retrógrado requerido en el apogeo es la diferencia entre la velocidad circular final y la velocidad en el apogeo de la elipse:

$$\Delta V_{\text{deorbit}} = \sqrt{\frac{\mu}{r_a}} - \sqrt{\mu\left(\frac{2}{r_a} - \frac{1}{a_d}\right)}. \tag{5.2.9}$$

Después se integra de nuevo la ecuación de decaimiento aerodinámico, ecuación (5.2.1), desde el semieje mayor de transferencia hasta $h = 100$ km, tomada como altura de reentrada. El código busca por bisección el perigeo más alto que todavía cumple $t_{\text{reentrada}} \leq 365$ días, de esta forma, se obtiene el impulso mínimo dentro del modelo.

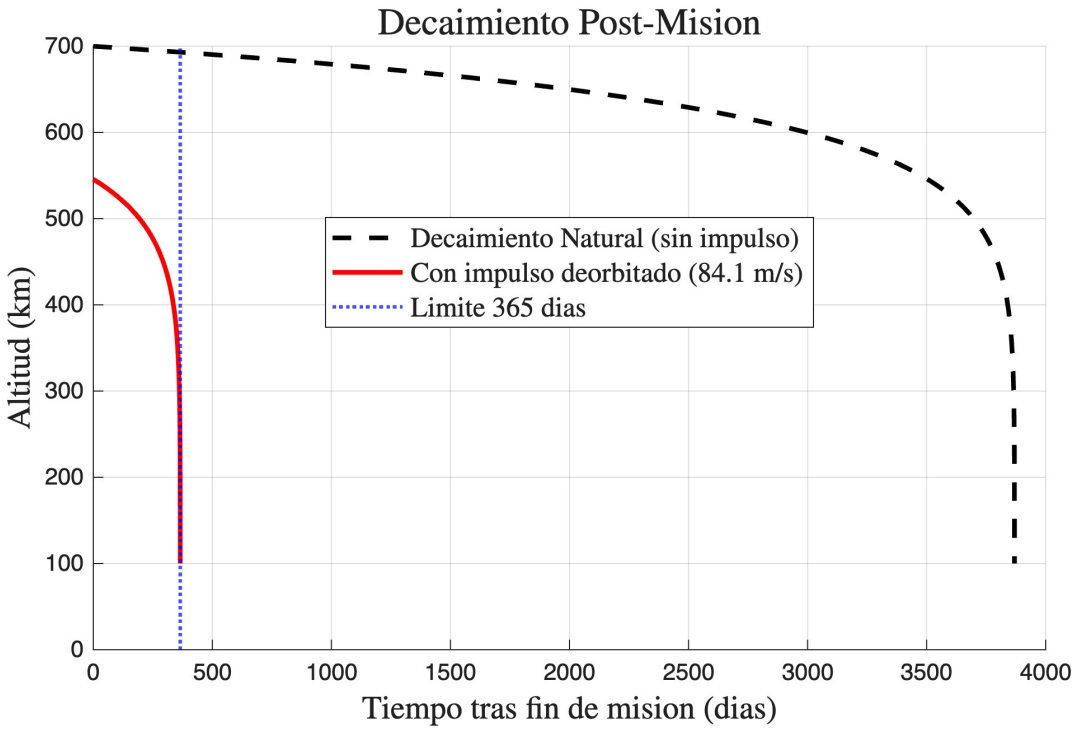


Figura 5.5: Tiempo de deorbitación, en días, tras fin de misión.
Fuente: Elaboración propia.

La simulación obtiene un tiempo de decaimiento natural de **3869 días**, superior al objetivo de **365 días**. La maniobra reduce el perigeo a **391,434 km**, con $\Delta V_{\text{deorbit}} = 84{,}067$ m/s y una

[1]Sin incluir variabilidad solar extrema, actitud no nominal ni cambios de área proyectada durante el giro descontrolado (*tumbling*).

reserva de **0,1461 kg** por satélite, incluido el margen del 10 %. Sumada al mantenimiento, la masa total de propulsante es **0,1964 kg**; la masa final es **3,538 kg** por satélite y **7,076 kg** para la constelación.

El modelo de resistencia conserva una aproximación circular equivalente: propaga el semieje mayor de la transferencia, sin integrar la resistencia a lo largo de la órbita excéntrica. El perigeo y la reserva calculados son estimaciones preliminares que requieren comprobación con un modelo de reentrada de mayor fidelidad.

5.3. Verificación de las condiciones orbitales

5.3.1. Condición de órbita *frozen*

Para la altura nominal de 690 km, la ecuación (3.6.1) proporciona $i_{\mathrm{SSO}} = 98{,}1473°$. Aplicando la ecuación (3.6.8), con $a = 7068{,}137$ km y $\omega = 90°$, resulta:

$$e_{\mathrm{frozen}} = \frac{2{,}5327 \times 10^{-6}}{2 \cdot 1{,}082629989 \times 10^{-3}} \cdot \frac{6378{,}137}{7068{,}137} \cdot \sin(98{,}1473°) \approx 1{,}04486 \times 10^{-3}. \tag{5.3.1}$$

La excursión entre apogeo y perigeo, $2ae$, describe la órbita completa. Para evaluar la adquisición se calcula, en cambio, la variación de altura sobre la región observada. La banda de mantenimiento actúa sobre $a - R_{\oplus}$, entre 676,2 y 703,8 km, y no sobre la altura instantánea de cada punto de la órbita.

$$\begin{aligned} h_{\mathrm{mín}} &= 0{,}98 h_{\mathrm{nom}} = 676{,}2 \text{ km}, \\ h_{\mathrm{máx}} &= 1{,}02 h_{\mathrm{nom}} = 703{,}8 \text{ km}, \\ \Delta h_{\mathrm{mant,full}} &= h_{\mathrm{máx}} - h_{\mathrm{mín}} = 27{,}6 \text{ km}. \end{aligned} \tag{5.3.2}$$

5.3.2. Variación de altura en la zona de observación

Se consideran los arcos ascendente y descendente que recorren el intervalo completo de latitudes 25°–49°N. Cada arco contiene 241 muestras de latitud; se transforma la latitud geodésica a geocéntrica para obtener el argumento de latitud orbital. Al no recortar por longitud, el cálculo representa una envolvente por latitudes de la región y no una pasada geográfica concreta sobre EE. UU.

En cada punto se obtiene el radio orbital y la altura sobre el elipsoide:

$$r(\nu) = \frac{a(1 - e^2)}{1 + e\cos\nu}, \qquad h(\nu, \varphi) = r(\nu) - R_{\mathrm{el}}(\varphi). \tag{5.3.3}$$

La variación dentro de cada arco j, ascendente o descendente, se define como:

$$\Delta h_j(t) = \max_{\nu \in j} h(\nu, t) - \min_{\nu \in j} h(\nu, t). \tag{5.3.4}$$

La envolvente conjunta se calcula tomando el máximo y el mínimo sobre ambos arcos en un mismo instante. Puede reunir extremos de pasos distintos y, por tanto, no equivale a una variación dentro de una sola pasada.

En la órbita *frozen*, la altura sobre la región queda entre 690,651 y 696,519 km. La variación en cada sentido de paso es 5,868 km y la máxima desviación respecto a 690 km es 6,519 km. El barrido paramétrico de cobertura utiliza la altura nominal; esta comprobación cuantifica la variación de geometría que ese barrido no resuelve.

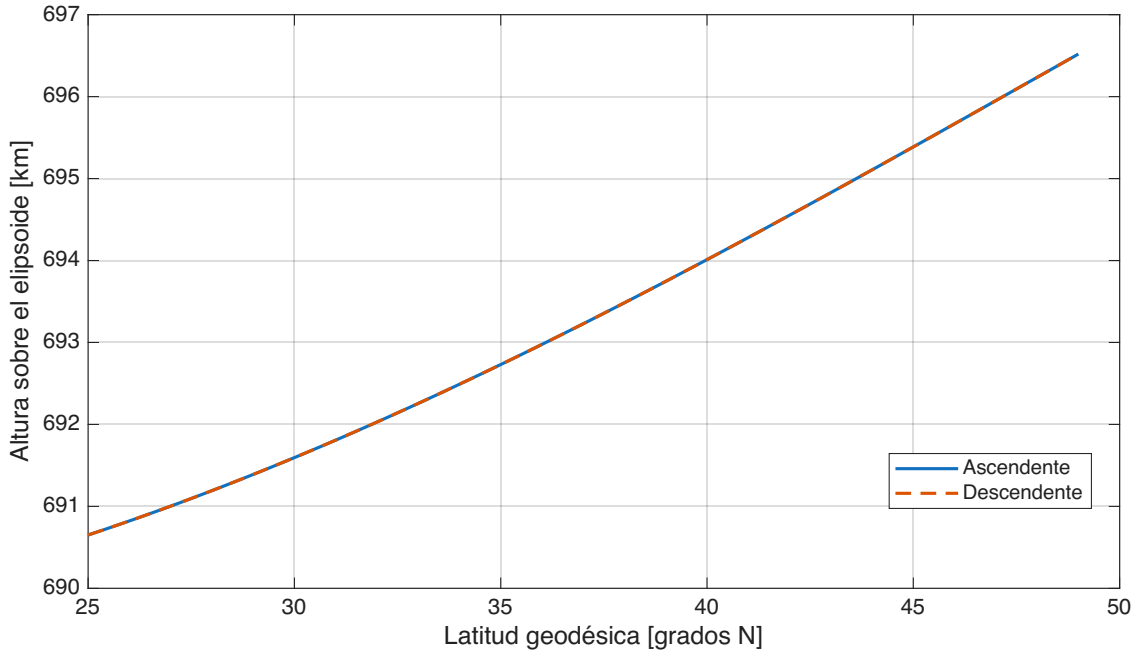


Figura 5.6: Altura sobre el elipsoide a lo largo de los arcos de adquisición ascendente y descendente entre 25° y 49°N. Fuente: Elaboración propia.

5.3.3. Evolución de la excentricidad durante la misión

Para completar la comparación entre una órbita *frozen* y una órbita inicialmente circular, se propaga durante los ocho años la evolución de su vector de excentricidad media. Se utiliza el modelo analítico de primer orden del problema J_2–J_3 [71]. Este modelo describe la rotación del vector alrededor de la excentricidad *frozen* y permite comparar la variación secular sin introducir un propagador orbital de alta fidelidad.

El vector se define como $\boldsymbol{e} = (e\cos\omega, e\sin\omega)$. Su centro es:

$$\boldsymbol{e}_{\mathrm{f}} = \left(0, -\frac{J_3}{2J_2}\frac{R_\oplus}{p}\sin i\right), \qquad p = a(1-e^2), \tag{5.3.5}$$

En el cálculo del centro se toma $p \simeq a$, coherente con la aproximación de pequeña excentricidad. Su evolución se calcula mediante una rotación del vector inicial alrededor de dicho centro. La velocidad angular de esta rotación se aproxima por:

$$\dot{\alpha} = -3nJ_2\left(\frac{R_\oplus}{p}\right)^2\left(\frac{5}{4}\sin^2 i - 1\right), \qquad n = \sqrt{\frac{\mu}{a^3}}. \tag{5.3.6}$$

La órbita *frozen* se inicializa en el centro del círculo, con $e(0) = e_{\mathrm{f}}$ y $\omega(0) = 90°$. La órbita circular se inicializa con el vector nulo. Para cada día de la misión se evalúa de nuevo la altura sobre los arcos de adquisición y se registra su desviación respecto a la altura nominal.

Tabla 5.2: Máximos durante ocho años en los arcos de 25°–49°N.

Caso	$e_{\mathrm{máx}}$	Variación por arco [km]	Envolvente conjunta [km]	máx $\lvert h - 690\rvert$ [km]
Frozen	$1{,}04486 \times 10^{-3}$	5,868	5,868	6,519
Inicialmente circular	$2{,}08971 \times 10^{-3}$	8,980	17,602	13,904

La órbita *frozen* mantiene constante la excentricidad media en este modelo de primer orden. En la órbita inicialmente circular, la excentricidad oscila entre cero y aproximadamente el doble de la excentricidad *frozen*. La mayor variación dentro de un arco es de 8,980 km; los 17,602 km corresponden a la envolvente conjunta de ambos sentidos de paso.

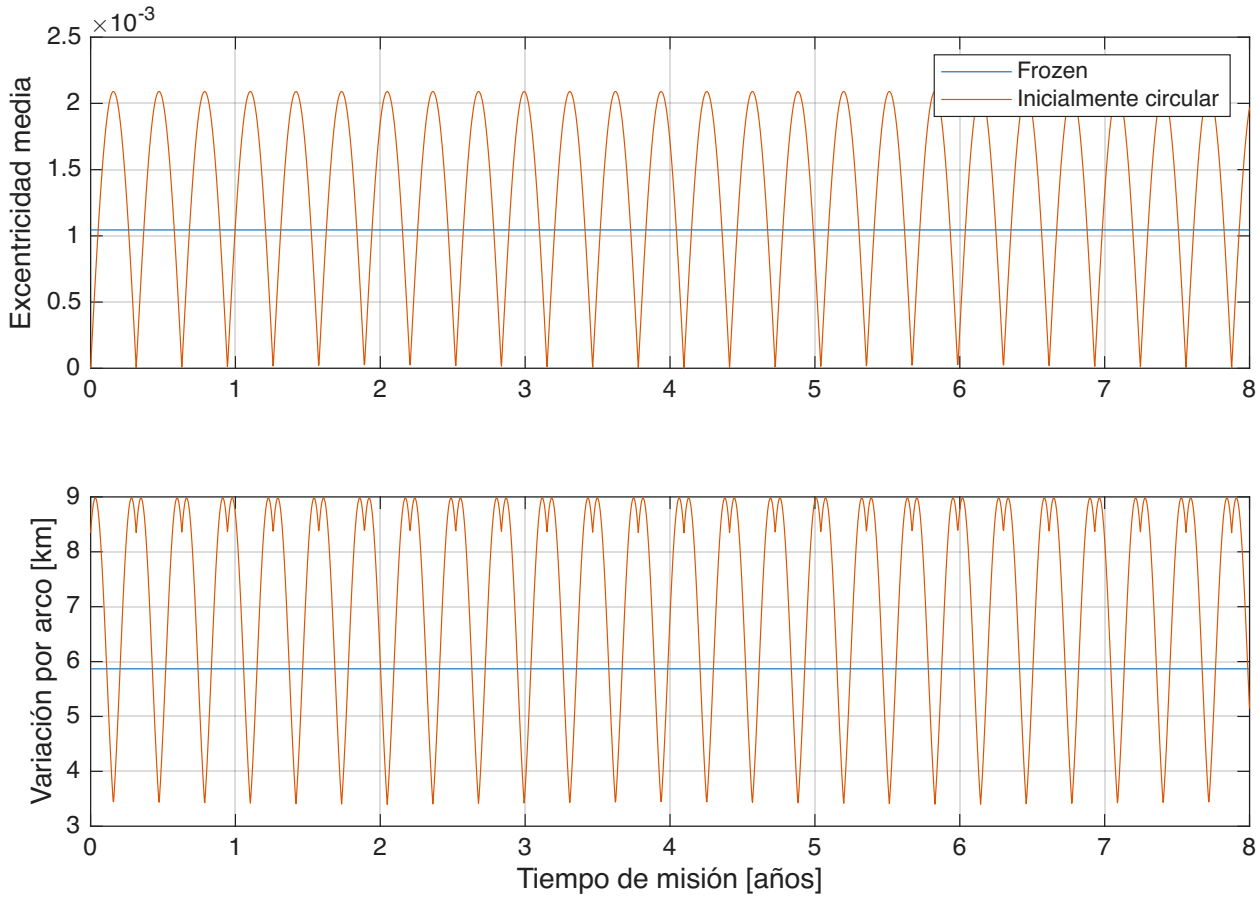


Figura 5.7: Excentricidad media y variación máxima dentro de cada arco durante ocho años. Fuente: Elaboración propia.

Se adopta la órbita *frozen* por la mayor estabilidad de la geometría de observación. Esta comparación mantiene fijo el semieje mayor y no incluye perturbaciones de orden superior. El mantenimiento por resistencia aerodinámica se calcula por separado mediante la banda de $\pm 2\,\%$ de $a - R_{\oplus}$; no se decide comparando esa banda con la excursión entre apogeo y perigeo.

5.3.4. Condición de traza repetida (RGT)

Con estos valores se evalúa la condición RGT de la ecuación (3.6.2). Para ello se usa la velocidad de rotación terrestre $\omega_{\oplus}$ y el periodo sideral de la ecuación (3.6.3) [49].

El programa evalúa parejas enteras con $k_{\text{day2rep}} \leq 30$ y busca las raíces de:

$$F(h) = \omega_{\text{nodal}}(h) - \left[\omega_{\oplus} - \dot{\Omega}(h)\right] \frac{k_{\text{orb2rep}}}{k_{\text{day2rep}}}. \tag{5.3.7}$$

Se aplica bisección entre 200 y 1000 km hasta alcanzar $|F(h)| < 10^{-10}$ o 100 iteraciones. La

raíz más próxima a la altura nominal de 690 km corresponde a:

$$h_{\mathrm{RGT}} = 689{,}83078 \text{ km}, \qquad \frac{k_{\mathrm{orb2rep}}}{k_{\mathrm{day2rep}}} = \frac{394}{27}. \tag{5.3.8}$$

En esa raíz se obtiene $i_{\mathrm{SSO}} = 98{,}14658°$ y $T_{\mathrm{nodal}} = 5920{,}813$ s. La diferencia con la altura nominal es de aproximadamente **169 m**. El cierre corresponde a 394 revoluciones nodales y 27 ciclos de rotación terrestre relativos al nodo, aproximadamente 27 días solares en esta órbita SSO. No equivale a 27 rotaciones siderales, puesto que la condición incluye $\dot{\Omega}$.

Esta raíz indica una opción cercana de ajuste orbital. Los presupuestos de cobertura, masa y datos de este trabajo conservan la altura nominal de 690 km; no se presentan como resultados recalculados a la altura exacta RGT.

5.4. Sensibilidad de la solución al GSD

Para evaluar la sensibilidad de la solución al requisito de GSD, se realiza un análisis auxiliar a altura fija de 690 km, variando el GSD requerido y representando el diámetro de pupila y la masa total resultantes frente al GSD. Se mantienen dos satélites, dos telescopios refractivos por satélite y detectores H2RG, con un barrido requerido de 240 km. Para cada GSD se recalcula el diámetro mínimo, en pasos de 1 mm, que cumple los requisitos de MTF y SNR, junto con la masa seca y el combustible de mantenimiento.

Al exigir un GSD más fino, el diámetro de pupila crece con rapidez y la masa total lo sigue, ya que la masa seca del instrumento escala con el cubo del diámetro. Relajar el requisito de GSD, por el contrario, reduce sensiblemente la masa de la constelación. Este análisis muestra la tendencia del escalado óptico de la arquitectura adoptada; no constituye una nueva optimización regional para cada GSD. Las masas representadas incluyen mantenimiento, sin la reserva de desorbitado.

Para GSD de 60, 80 y 100 m se obtienen diámetros de 50, 38 y 30 mm y masas de constelación de 15,339, 6,784 y 3,338 kg, respectivamente. El punto de 80 m reproduce la solución seleccionada.

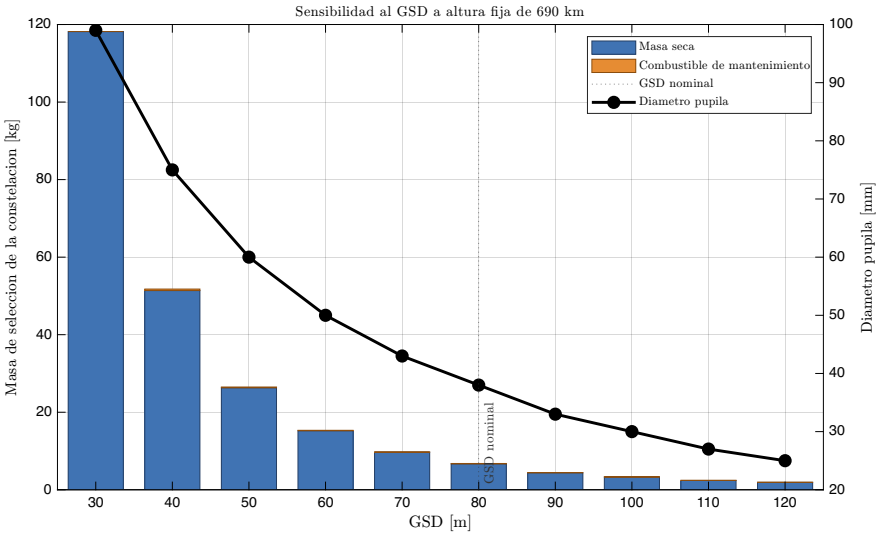


Figura 5.8: Análisis auxiliar a 690 km: sensibilidad de masa y diámetro de pupila frente al GSD.
Fuente: Elaboración propia.

Capítulo 6

Lanzadores

Contenido

6.1. Lanzadores Operativos 99
6.1.1. Falcon 9 99
6.1.2. Firefly Alpha 99
6.1.3. Vulcan Centaur 99
6.1.4. Electron 99
6.1.5. Ariane 6 100
6.1.6. Vega-C 100
6.2. Bases de lanzamiento y azimuts posibles 101
6.2.1. Vandenberg Space Force Base (California, EE.UU.) 101
6.2.2. Cape Canaveral Space Force Station (Florida, EE.UU.) 101
6.2.3. Centro Espacial de Guayana (Guayana Francesa) 101
6.3. Selección Final 102

La selección del vehículo de lanzamiento para misiones de pequeños satélites requiere priorizar la viabilidad económica mediante servicios de lanzamiento compartido. Esta modalidad permite distribuir los costes de lanzamiento entre múltiples operadores, reduciendo los gastos para cargas útiles pequeñas [72].

Los lanzamientos compartidos han reducido la barrera de entrada para satélites de masa reducida. Utilizan adaptadores especializados y dispensadores para integrar múltiples cargas útiles en la etapa superior del cohete [73]. El coste fijo del vehículo se reparte entre las cargas útiles, lo que se traduce en un coste de lanzamiento por kilogramo muy inferior al de un lanzamiento dedicado.

6.1. Lanzadores Operativos

6.1.1. Falcon 9

Falcon 9 de SpaceX domina el mercado de lanzamiento compartido con su programa SmallSat Rideshare (programa de lanzamiento compartido para satélites pequeños). Con 468 lanzamientos hasta abril de 2025 y una fiabilidad del 99,35 %, su rendimiento supera ampliamente al de sus competidores [74]. El programa *Transporter* ofrece lanzamientos dedicados a órbitas heliosíncronas con un coste de 6 500 USD por kilogramo [72], [75]. Su arquitectura de 9 motores Merlin proporciona redundancia, mientras que su etapa superior (LOX/RP-1, con capacidad de reencendido) permite ajustes orbitales precisos para inserción en órbitas de hasta 700 km [76].

6.1.2. Firefly Alpha

El Firefly Alpha es un lanzador de dos etapas desarrollado por Firefly Aerospace para el mercado de satélites pequeños. Con una capacidad de 630 kg a órbitas heliosíncronas de 500 km y 1 030 kg a LEO, ocupa un nicho intermedio en el mercado. Firefly representa una empresa emergente y disruptiva dentro del acceso dedicado para cargas ligeras. El coste de lanzamiento oscila entre 15–19 millones USD, resultando en aproximadamente 24 000–30 000 USD por kilogramo. Opera desde Vandenberg Space Force Base y ofrece opciones limitadas de lanzamiento compartido [77].

6.1.3. Vulcan Centaur

El Vulcan Centaur es el sucesor estadounidense del Atlas V y Delta IV Heavy. Aunque proyecta una fiabilidad del 100 % basada en pruebas iniciales, carece de historial operativo extenso en 2025 [78]. Su capacidad de 14 400 kg a SSO y coste aproximado de 110 millones USD lo posicionan fuera del segmento económico para satélites pequeños. La ausencia de servicios de lanzamiento compartido establecidos limita su competitividad.

6.1.4. Electron

El Electron se especializa en cargas útiles ligeras con lanzamientos rápidos desde Nueva Zelanda. Su coste de 7,5 millones USD por lanzamiento resulta en aproximadamente 25 000 USD por kilogramo [79]. Con capacidad máxima de 300 kg a SSO, ofrece servicios de lanzamiento compartido limitados y flexibilidad operativa elevada con varias ventanas de lanzamiento [76]. No se adopta como opción principal porque obliga a desplazar la campaña

a Mahia y no mejora el coste por kilogramo frente a Falcon 9 ni la compatibilidad logística de Vandenberg.

6.1.5. Ariane 6

El Ariane 6 representa la autonomía estratégica europea en acceso al espacio. Sus variantes Ariane 62 y Ariane 64 ofrecen capacidades de 6 500 kg y 15 000 kg a SSO respectivamente [73]. Los servicios de lanzamiento compartido permiten costes aproximados de 11 164,55 USD por kilogramo [79]. Su etapa superior con capacidad de reinicio múltiple optimiza la inserción orbital para múltiples cargas útiles [73].

6.1.6. Vega-C

El Vega-C es el lanzador ligero más avanzado de Europa, optimizado para el segmento de satélites pequeños y medianos. Con capacidad de 2 300 kg a órbitas de 700 km, utiliza el adaptador SSMS (Small Spacecraft Mission Service) para facilitar misiones de lanzamiento compartido [80]. Los costes aproximados de 20 000 USD por kilogramo reflejan su especialización en el mercado europeo [73].

Tabla 6.1: Comparativa de lanzadores espaciales para misiones de lanzamiento compartido.

Lanzador	**Fiabilidad [a] (2025)**	**Coste por kg (USD)**	**Capacidad a LEO/SSO**	**Lanzamiento compartido**	**Ubicaciones de lanzamiento**
Falcon 9	99,35 %	6 500	17 400 kg	Sí	Vandenberg SFB, Cabo Cañaveral
Firefly Alpha	57,1 %[c]	26 984	630 kg	Sí	Vandenberg SFB
Vulcan Centaur	100 %	4 044	14 400 kg	No	Vandenberg SFB, Cabo Cañaveral
Electron	95 %	25 000	300 kg	Sí	Mahia (Nueva Zelanda)
Ariane 6	Activo	5 500	Hasta 15 000 kg	Sí	Centro Espacial de Guayana
Vega-C	90,9 %[b]	19 565	2 300 kg	Sí	Centro Espacial de Guayana

[a] Considerando solo éxito total de misión

[b] Fiabilidad del programa Vega, no solo el lanzador actual

[c] Fiabilidad estimada a marzo de 2026 (4 éxitos en 7 lanzamientos).

Los costes por kg son estimaciones aproximadas a partir de fuentes públicas; varían según la configuración y el año.

6.2. Bases de lanzamiento y azimuts posibles

6.2.1. Vandenberg Space Force Base (California, EE.UU.)

Vandenberg SFB constituye la instalación óptima para órbitas heliosíncronas desde territorio estadounidense [81]. Su ubicación a 34,4° N permite azimuts de lanzamiento entre 147° y 201°, facilitando inclinaciones orbitales de 56° a 104° [81]. Las trayectorias despejadas sobre el océano Pacífico minimizan riesgos operativos, mientras que su infraestructura avanzada optimiza las operaciones tanto del Falcon 9 como del Firefly Alpha [82].

6.2.2. Cape Canaveral Space Force Station (Florida, EE.UU.)

Cabo Cañaveral opera con azimuts restringidos entre 35° y 120° debido a limitaciones de sobrevuelo terrestre. Estas restricciones limitan las inclinaciones orbitales alcanzables a 28,5°–59°, inadecuadas para órbitas heliosíncronas [81]. La infraestructura avanzada y soporte técnico compensan parcialmente estas limitaciones geográficas [83].

6.2.3. Centro Espacial de Guayana (Guayana Francesa)

El Centro Espacial de Guayana ofrece azimuts de lanzamiento entre -11° y 90°, permitiendo inclinaciones de 5,2° a 100,5° [81]. Su proximidad ecuatorial (5,2° N) proporciona ventajas energéticas significativas, aunque requiere adaptaciones de trayectoria para órbitas heliosíncronas [76]. Las operaciones Ariane 6 y Vega-C se benefician de trayectorias despejadas sobre el Atlántico [80].

6.3. Selección Final

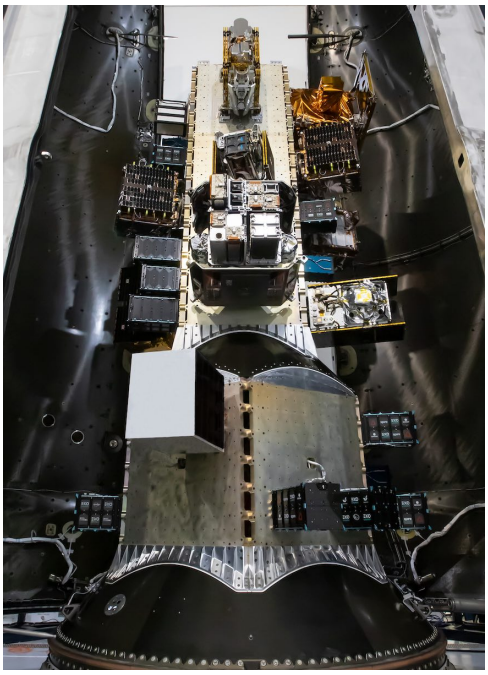


Figura 6.1: Programa de lanzamiento compartido de SpaceX
Fuente: [84].

Figura 6.2: Falcon 9 de SpaceX despegando.
Fuente: [76].

La decisión final recae en el **Falcon 9** como lanzador principal por precio publicado, disponibilidad de misiones *Transporter* a SSO, historial operativo y compatibilidad con dispensadores para cargas pequeñas. Su etapa superior (LOX/RP-1) ofrece una ventana de reencendido de hasta 6 horas, permitiendo ajustes finos de la órbita y el despliegue secuencial de ambos satélites. El programa de lanzamiento compartido permite situar el coste de lanzamiento en torno a los 6 500 USD/kg [72], [75].

La base de lanzamiento de **Vandenberg SFB**, que es una base muy utilizada para lanzamientos a órbitas polares o heliosíncronas, permitirá lanzar dicho vehículo a la órbita requerida, y ya está adaptada para el lanzador elegido. Esto evita sobrecostes al existir ya en el lugar la infraestructura, logística, cadena de suministro y personal cualificado para el lanzamiento del Falcon 9.

Como lanzador de respaldo se selecciona el **Firefly Alpha**. Opera desde Vandenberg SFB, compartiendo base de lanzamiento con el Falcon 9 principal. Es un lanzador estadounidense y un proveedor emergente con servicios de lanzamiento compartido.

Figura 6.3: Firefly Alpha en la plataforma de lanzamiento de Vandenberg SFB.
Fuente: Firefly Aerospace.

Azimut de lanzamiento

Para validar este lanzamiento se puede calcular el azimut inicial de lanzamiento y comprobar que es viable desde Vandenberg. Se usa la convención de azimut Az medido en el plano local desde el norte geográfico, positivo hacia el este; i es la inclinación orbital y θ la latitud de la base. Mediante la ecuación:

$$\cos(i) = \sin(Az) \cdot \cos(\theta) \tag{6.3.1}$$

Siendo $i = 98,15°$ la inclinación de interés, $\theta = 34,7°$, la latitud a la que se sitúa la base de lanzamiento de Vandenberg SFB, se obtiene un azimut $Az \approx 190°$. Este valor es

compatible con los rangos disponibles de la base de lanzamiento [85] y aplica por igual al Falcon 9 y al Firefly Alpha, puesto que ambos parten de Vandenberg hacia la misma órbita SSO y comparten, por tanto, la misma inclinación.

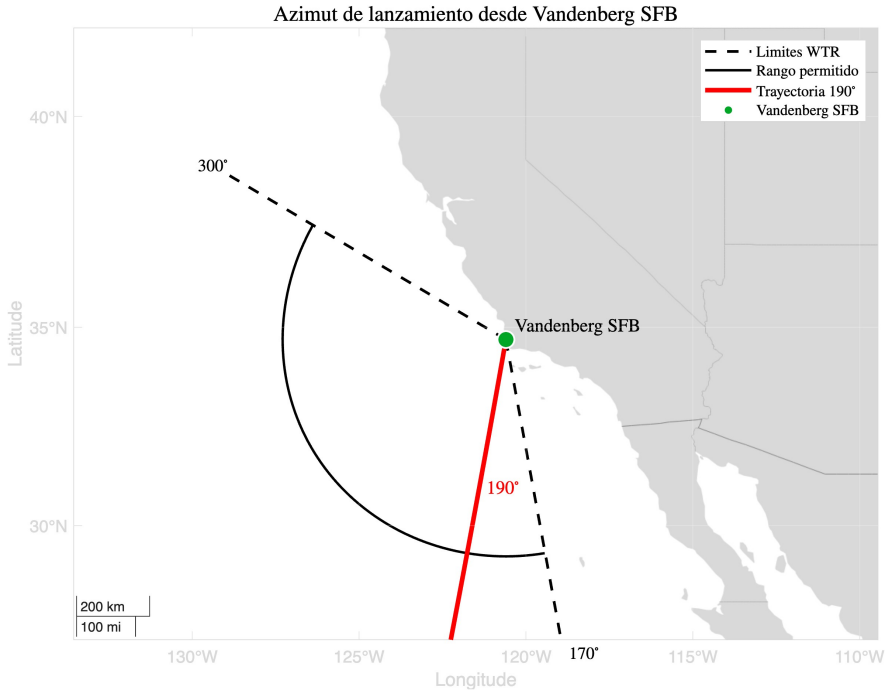


Figura 6.4: Azimut de lanzamiento desde Vandenberg SFB.
Fuente: Elaboración propia.

Despliegue de dos satélites en órbita

Ambos satélites comparten el mismo plano orbital SSO y los mismos parámetros orbitales, y solo se diferencian en su anomalía, con un desfase de 180° sobre la órbita. Para lograr esta configuración, la etapa superior del lanzador ejecuta una estrategia de fase en tres maniobras:

1. Inserción orbital y primer despliegue. El lanzador coloca la etapa superior y los satélites en la órbita objetivo, que en este caso es una SSO casi circular a 690 km de altitud. Inmediatamente tras alcanzar la órbita, se procede al despliegue del primer satélite, que comienza su operación en la trayectoria planificada.

2. Maniobra de fase: tras el primer despliegue, la etapa superior ejecuta una maniobra para elevar temporalmente el apogeo de su órbita. Al aumentar el apogeo aumenta también el semieje mayor y, por tanto, el periodo orbital de la etapa respecto al del satélite ya desplegado. Esta diferencia de periodo hace que la etapa acumule, en cada revolución, un desfase angular

$$\Delta\varphi = 2\pi \frac{T_{\text{fase}} - T_{\text{circular}}}{T_{\text{circular}}} \tag{6.3.2}$$

respecto al primer satélite, hasta alcanzar la separación de 180° buscada.

3. Recircularización y segundo despliegue: una vez alcanzada la separación angular de 180°, la etapa superior realiza una nueva maniobra para regresar a la órbita circular de operación de 690 km. Esta recircularización iguala el periodo del segundo satélite con el del primero y evita que el desfase siga creciendo durante la operación nominal.

La secuencia descrita aproxima la órbita de operación como circular. La inserción en la pequeña excentricidad *frozen*, el LTAN y la capacidad de la etapa o de un vehículo de transferencia para completar el faseo requieren confirmación del proveedor. La ventana de reencendido del lanzador no acredita por sí sola la capacidad de mantener la etapa operativa durante toda la maniobra.

Capítulo 7

Segmento terrestre

Contenido

7.1. Generación de los datos de observación 107

7.2. Estaciones terrestres candidatas 108

7.3. Simulación de los contactos con Fairbanks 109

7.4. Tasa mínima de descarga 110

7.5. Selección de la banda de enlace 110

7.5.1. Velocidad adoptada del enlace 111

Para asegurar el cumplimiento de los objetivos de la misión, resulta imprescindible disponer de una estación de comunicaciones capaz de recibir los datos generados por la constelación. El segmento Tierra comprende el conjunto de infraestructuras y recursos ubicados en la superficie terrestre dedicados a la gestión, control y operación de los sistemas espaciales. Su función no se limita únicamente a la recepción de información, sino que abarca también tareas de telemetría, seguimiento, control, envío de comandos y mantenimiento orbital. En este apartado se presenta un análisis conciso del segmento terrestre, con el propósito de validar la viabilidad técnica de la misión propuesta.

7.1. Generación de los datos de observación

Se parte de la configuración seleccionada en los capítulos anteriores con la región de interés definida mediante la caja geográfica $25° \leq \varphi \leq 49°$ y $-125° \leq \lambda \leq -66°$, que representa el territorio continental de EE. UU. en el modelo de cobertura.

El sensor *pushbroom* genera una línea de imagen mientras avanza sobre la traza. Para cada satélite i, el programa propaga la órbita durante una semana con un paso temporal de 10 s y suma la longitud de los segmentos de traza contenidos en la región de interés. La longitud resultante se denomina L_i.

El programa utiliza un escenario de *Aerospace Toolbox* en MATLAB con el propagador SGP4 para propagar las órbitas durante siete días desde el 28 de abril de 2024. La función obtiene la posición geográfica de cada satélite con un paso de 10 s. A continuación, el programa analiza cada par de posiciones consecutivas, recorta el segmento a la caja geográfica definida y calcula la distancia entre los extremos recortados mediante la fórmula de Haversine:

$$d_{12} = 2R_{\oplus} \arcsin\left(\sqrt{\sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}\right), \tag{7.1.1}$$

donde φ_1 y φ_2 son las latitudes de los extremos del segmento, λ_1 y λ_2 sus longitudes, todas expresadas en radianes, y $R_{\oplus} = 6371$ km es el radio medio de la Tierra. La distancia d_{12} se calcula sobre la esfera terrestre. La suma de las distancias proporciona L_i para cada satélite.

El número de líneas y de píxeles por línea se calcula como:

$$N_{\text{líneas},i} = \frac{L_i}{\text{GSD}}, \qquad N_{\text{píxeles/línea}} = \frac{W}{\text{GSD}}, \tag{7.1.2}$$

expresando las longitudes en las mismas unidades, con $W = 240$ km. Se utilizan 1500 píxeles por telescopio, 3000 por línea combinada, y tres bandas espectrales. El volumen de datos brutos de cada satélite es

$$D_i = N_{\text{líneas},i} N_{\text{píxeles/línea}} N_{\text{bandas}} b_{\text{píxel}}, \tag{7.1.3}$$

donde $b_{\text{píxel}} = 12$ bits por banda. La expresión se convierte a gigabytes dividiendo entre 8×10^9. Cada segmento de traza se contabiliza una vez, por lo que el volumen obtenido corresponde a los datos brutos generados durante la semana. No se descuenta el sexto de imágenes con nubes, puesto que también generan datos brutos. Los datos de los dos telescopios se agrupan en un único flujo, ya que ambos forman el barrido combinado.

La simulación obtiene los siguientes volúmenes semanales:

Tabla 7.1: Datos generados por satélite durante una semana.

Satélite	Datos brutos [GB]
1	16,133
2	15,391
Total	**31,524**

7.2. Estaciones terrestres candidatas

Se consideran cuatro alternativas para la estación de Tierra:

1. **Estación de Fairbanks, Alaska**: ubicada en Gilmore Valley, es una instalación de comunicaciones por satélite situada a alta latitud y orientada, entre otras funciones, a la adquisición de datos de satélites de observación terrestre [86].

2. **Estación de Wallops, Virginia**: constituye otra instalación de NOAA para el seguimiento y la adquisición de datos de satélite. Está localizada en Wallops Island, a $37,94°$N y $75,47°$W, y proporciona capacidades especializadas para satélites de observación terrestre [87].

3. **Red de estaciones europeas**: EUMETSAT coordina una red de estaciones terrestres europeas desde su centro de control en Darmstadt, Alemania. Entre las instalaciones consideradas se encuentran Fucino, Italia, Cheia, Rumanía, y Svalbard, Noruega. Esta última resulta especialmente interesante para satélites de órbita polar [88].

4. **Construcción de una estación propia**: permitiría adaptar la antena y el procesamiento a las necesidades de la misión, pero requeriría una inversión inicial y el desarrollo de una infraestructura específica.

Figura 7.1: Antena de la estación de Fairbanks, Alaska.
Fuente: [89].

Fairbanks se preselecciona para el análisis detallado, debido a su latitud elevada, favorable para maximizar los contactos de satélites en órbita polar. Además, se encuentra en territorio nacional de EE.UU. y dispone de antenas en bandas S, X y Ka, además de soporte y recursos para la transmisión de datos científicos y funciones de telemetría, seguimiento y telecomando [89] [86]. La simulación permite estimar los accesos geométricos a esa localización. Las instalaciones NOAA de Gilmore Valley y las antenas ASF son instalaciones distintas; sus capacidades no se intercambian en el modelo.

7.3. Simulación de los contactos con Fairbanks

El programa utiliza SGP4 y propaga los dos satélites durante una semana con el escenario creado en *Aerospace Toolbox* para calcular el tiempo de contacto. La estación se modela en Fairbanks, con coordenadas $64,84°$N y $-147,712°$, y una elevación mínima de $10°$. El cálculo obtiene los intervalos de acceso de cada satélite por separado.

El tiempo de contacto semanal es de 25 690 s para el satélite 1 y de 25 640 s para el satélite 2.

7.4. Tasa mínima de descarga

Para cada satélite, el programa calcula la tasa mínima mediante

$$R_{\text{mín},i} = \frac{D_i}{T_{\text{contacto},i}}. \tag{7.4.1}$$

En esta expresión se utiliza D_i en bits y $T_{\text{contacto},i}$ en segundos. El requisito del enlace es el mayor de los dos valores:

$$R_{\text{mín}} = \max\left(R_{\text{mín},1}, R_{\text{mín},2}\right). \tag{7.4.2}$$

Tabla 7.2: Tasa mínima de descarga por satélite.

Satélite	Datos [GB]	Contacto [s]	$R_{\text{mín}}$ [Mbps]
1	16,133	25 690	5,024
2	15,391	25 640	4,802
Requisito del enlace			**5,024**

Por tanto, el enlace debe proporcionar al menos $5,024$ Mbps durante los contactos útiles del satélite más restrictivo. Esta tasa constituye el requisito mínimo de diseño y se calcula antes de seleccionar la banda de comunicaciones.

7.5. Selección de la banda de enlace

La tabla compara las bandas de radiofrecuencia habituales en satélites pequeños y misiones de observación de la Tierra [90], [91], [92].

Tabla 7.3: Bandas de comunicaciones consideradas para la descarga de datos.

Banda	Frecuencia orientativa	Tasa típica orientativa
UHF	0,3–1 GHz	0,01–1,5 Mbps
S	2–4 GHz	1–14 Mbps
X	8–12 GHz	10–560 Mbps
Ka	27–40 GHz	100–3500 Mbps

Se selecciona la banda S como referencia para un enlace de 10 Mbps, superior al requisito de 5,024 Mbps. La guía de usuarios de la red NEN documenta para la antena ASF AS1, en Fairbanks, telemetría en banda S de hasta 10 Mbps con FM/PCM [93, tabla 7-2]. Esta cifra corresponde a un modo de una instalación concreta; no es un máximo universal de la banda S ni acredita una tasa contratada para la estación NOAA modelada. La disponibilidad del servicio y el presupuesto de enlace quedan pendientes de la selección del equipo y del operador.

7.5.1. Velocidad adoptada del enlace

Se adopta una tasa nominal de 10 Mbps como hipótesis de dimensionado preliminar. Esta tasa se fija después de calcular la tasa mínima:

$$M_R = \frac{R_{\text{adoptada}}}{R_{\text{mín}}} = \frac{10}{5,024} \approx 1,99. \tag{7.5.1}$$

La tasa adoptada supera en un 99,0 % la mínima del balance científico semanal. Ese cociente no sustituye al margen radioeléctrico ni incluye pérdidas de protocolo, planificación de accesos o disponibilidad de la estación.

Gestión de la memoria de a bordo

Por último, se modela el estado de la memoria de cada satélite a lo largo del tiempo, considerando tanto la generación continua de datos como la descarga durante los accesos, verificando que la memoria no se sature. En caso de que algún satélite agote su capacidad de almacenamiento, se considera que la configuración no es viable. Mediante el escenario de *Aerospace Toolbox* creado previamente, y utilizando la tasa adoptada de 10 Mbps durante los accesos:

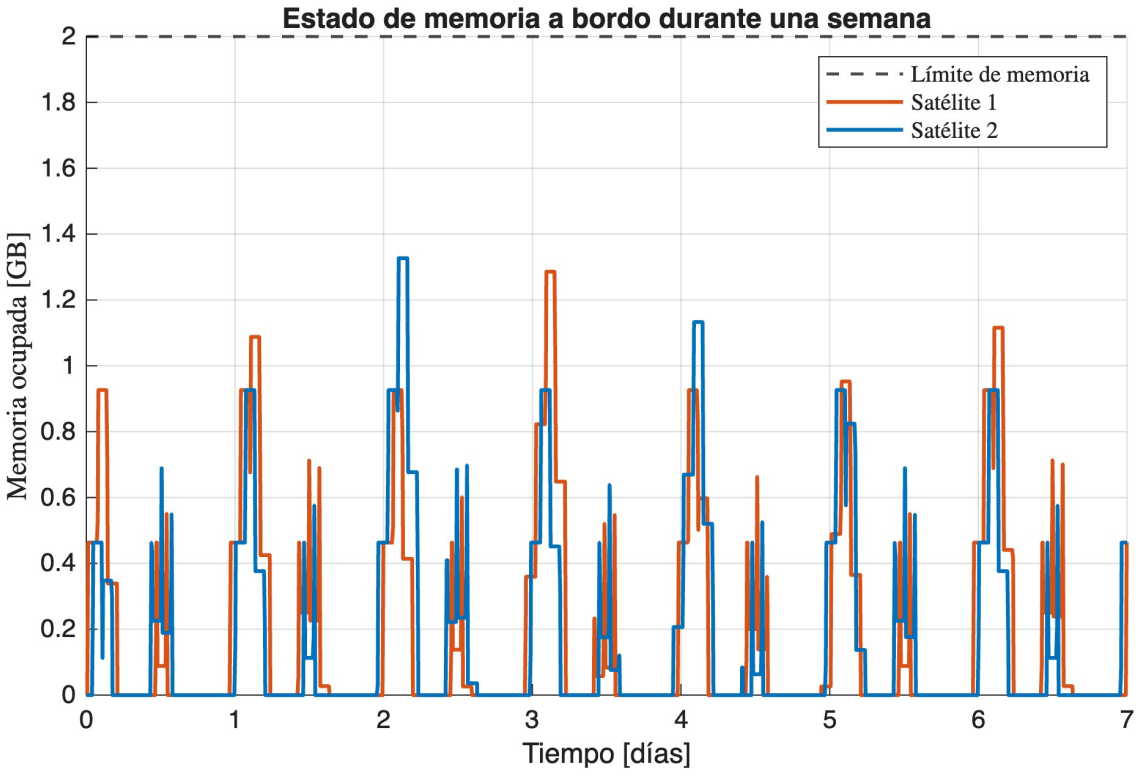


Figura 7.2: Evolución de la memoria a bordo durante una semana.

La ocupación máxima es de 1,286 GB en el satélite 1 y 1,327 GB en el satélite 2. Una

memoria de 2 GB permite almacenar los datos en la semana simulada con descarga a 10 Mbps. La reserva frente a interrupciones más largas debe comprobarse con un calendario de contactos y fallos de servicio.

Como comprobación de sensibilidad al propagador, el modelo kepleriano de dos cuerpos obtiene 15,950 y 15,524 GB, con tasas mínimas de 4,913 y 4,799 Mbps, respectivamente. Los valores adoptados en las tablas y en la memoria corresponden a SGP4 con las mismas condiciones iniciales. Esta comparación no representa una validación frente a efemérides observadas.

Capítulo 8

Conclusiones

El culmen de este Trabajo de Fin de Grado ha supuesto la integración de los conocimientos, así como competencias adquiridas a lo largo de la trayectoria académica. Durante el desarrollo del proyecto he alcanzado una comprensión profunda de las tendencias fundamentales entre los parámetros clave del sistema. El análisis iterativo de la influencia de las variables me ha permitido identificar relaciones críticas. Esta capacidad de interpretar y anticipar el comportamiento de los sistemas ante cambios en los parámetros de diseño constituye una de las competencias más valiosas adquiridas durante el trabajo.

El carácter multidisciplinar del proyecto se ha reflejado en la necesidad de integrar conocimientos de óptica, mecánica orbital, desarrollo de software, etc. La comprobación de los cálculos mediante campañas de simulación y empleando software profesional ha permitido contrastar los resultados obtenidos dentro de las hipótesis de los modelos utilizados.

Tras meses de trabajo se ha conseguido una solución que minimiza la masa seca y el combustible de mantenimiento dentro del conjunto discreto estudiado, que cumple con los requisitos especificados bajo las hipótesis adoptadas. Uno de los logros más significativos ha sido el desarrollo de un código propio, modular, capaz de facilitar la iteración ágil sobre los parámetros de diseño. Esta herramienta ha permitido explorar de manera eficiente el espacio de soluciones, con la agilidad necesaria para cambiar parámetros y probar diferentes configuraciones, lo que ha optimizado el proceso de toma de decisiones.

La solución seleccionada consta de dos satélites en una órbita SSO *frozen* a 690 km, con dos telescopios refractivos de 38 mm de pupila y dos detectores H2RG por satélite. El barrido adquirido de 240 km permite obtener una revisita máxima de 6,642 días efectivos en las trece latitudes comprobadas, incluyendo la corrección media por nubosidad del enunciado. La masa total, con las reservas de mantenimiento y deorbitado, es de 3,538 kg por satélite. La semana simulada con SGP4 requiere una tasa mínima de descarga de 5,024 Mbps; con 10 Mbps, la memoria máxima ocupada es de 1,327 GB por satélite. Estos resultados quedan sujetos al alcance de la malla de cobertura, al escalado preliminar de masa y potencia, a la aproximación circular equivalente del deorbitado y a la disponibilidad del servicio de enlace

y despliegue.

El trabajo realizado pone de manifiesto la necesidad de adoptar metodologías iterativas en el diseño de sistemas complejos. La experiencia adquirida en la gestión de incertidumbres y en la interpretación crítica de los resultados constituye una base sólida para afrontar retos profesionales en el ámbito de la ingeniería aeroespacial.

Bibliografía

[1] NASA/JPL-Caltech, *Orbiting Carbon Observatory-2 (Artist's Concept)*, https://photojournal.jpl.nasa.gov/catalog/PIA18610, Ilustracion artistica del satelite OCO-2 en orbita terrestre. Credito de la imagen: NASA/JPL-Caltech., jul. de 2014.

[2] European Space Agency, *OCO-2 (Orbiting Carbon Observatory-2) - eoPortal*, https://www.eoportal.org/satellite-missions/oco-2, Última actualizacion: 15 de octubre de 2024, 2024.

[3] European Space Agency, *GOSAT Series - ESA Earth Online*, Consultado: 2025, 2022. dirección: https://earth.esa.int/eogateway/missions/gos-1

[4] Japan Aerospace Exploration Agency, *GOSAT Project*, Consultado: 2025, 2021. dirección: https://earth.jaxa.jp/en/research/projects/gosat/index.html

[5] M. M. Gogoi et al., «The Greenhouse gas Observations of Biospheric and Local Emissions from the Upper sky (GOBLEU): a mission overview, instrument description, and results from the first flight,» *Carbon Balance and Management*, vol. 19, n.º 1, págs. 1-25, 2024. DOI: 10.1186/s13021-024-00273-1

[6] European Space Agency, *CO2M - SentiWiki - Copernicus*, Consultado: 2025, 2025. dirección: https://sentiwiki.copernicus.eu/web/co2m

[7] Y. Meijer et al., «Progress on the development of the Copernicus CO2M mission,» en *Proc. SPIE*, SPIE, vol. 13192, 2024, pág. 3 033 794. DOI: 10.1117/12.3033794

[8] Xinhua News Agency, *China Focus: TanSat satellite detects, reports human-caused carbon dioxide emissions*, Consultado: 2025, 2022. dirección: https://english.news.cn/20221025/a8fecde2ddf747aa924562944fe656cf/c.html

[9] Earth Observation Portal, *TanSat - eoPortal*, Consultado: 2025, 2024. dirección: https://www.eoportal.org/satellite-missions/tansat

[10] Airbus, *Airbus delivers MicroCarb instrument, the first European mission to measure CO2 globally*, Consultado: 2025, 2022. dirección: https://www.airbus.com/en/newsroom/stories/2022-12-airbus-delivers-microcarb-instrument-the-first-european-mission-to-measure

[11] Centre National d'etudes Spatiales, *MicroCarb - CNES*, Consultado: 2025, 2025. dirección: https://cnes.fr/en/projects/microcarb

[12] Earth Observation Portal, *GHGSat Constellation - eoPortal*, Consultado: 2025, 2024. dirección: https://www.eoportal.org/satellite-missions/ghgsat-con

[13] Satellite Today, *GHGSat Methane Data to Be Available in NASA's Commercial Smallsat Data Program*, Consultado: 2025, 2024. dirección: https://www.satellitetoday.com/imagery-and-sensing/2024/07/31/ghgsat-methane-data-to-be-available-in-nasas-commercial-smallsat-data-program/

[14] Kit de Actores. «En que afecta el efecto Moire a los actores?» Guia sobre el efecto moire en actuacion y cinematografia, visitado 17 de jun. de 2025. dirección: https://kitdeactores.com/en-que-afecta-el-efecto-moire-a-los-actores/

[15] T. Rossiter, *Comparison of the multispectral ground sampling distance (GSD) from each of the three platforms*, Figure 4.2 in PhD thesis: Developing an integrated approach to seaweed resource assessment, UAV = 5 cm/pixel, aircraft = 60 cm/pixel, Satellite = 10 m/pixel, 2020. dirección: https://www.researchgate.net/figure/Comparison-of-the-multispectral-ground-sampling-distance-GSD-from-each-of-the-three_fig15_363319656

[16] *Design Workshop of an Optical Earth Observation Instrument*, Material didectico para el diseño de instrumentos opticos de observacion terrestre, Universidad, 2023.

[17] D. Zorita Gomez-Escolar, «MTF (Modulation Transfer Function),» inf. téc., 2023, Documento tecnico sobre MTF en instrumentos opticos espaciales.

[18] Edmund Optics. «Introduction to Modulation Transfer Function,» visitado 16 de jun. de 2025. dirección: https://www.edmundoptics.es/knowledge-center/application-notes/optics/introduction-to-modulation-transfer-function/

[19] R. F. H. Foadi y A. K. Ahmed, «Designing Cassegrain Telescope System with Best Obscuration Ratio of Secondary Mirror,» *Iraqi Journal of Science*, 2023. dirección: https://api.semanticscholar.org/CorpusID:267425188

[20] Wikipedia contributors, *SNR image demonstration*, Image file from Wikipedia article on Signal-to-noise ratio, Demonstration of visual effects of different SNR levels in imaging, 2025. visitado 17 de jun. de 2025. dirección: https://en.wikipedia.org/wiki/Signal-to-noise_ratio#/media/File:SNR_image_demonstration.png

[21] D. Zorita Gomez-Escolar, «SNR (Relacion Señal Ruido),» inf. téc., 2023, Documento tecnico sobre celculo de SNR en instrumentos opticos espaciales.

[22] Remote Sensing Educational Resource. «Swath and Field of View Diagram for Remote Sensing Systems.» Diagram showing relationship between linear array, lens system, individual detector, IFOV, FOV, IGFOV, and swath parameters.

[23] S. Durant. «Electromagnetic spectrum - WikiLectures.» El contenido hace referencia a conferencias de 2011, visitado 16 de jun. de 2025. dirección: https://www.wikilectures.eu/w/Electromagnetic_spectrum

[24] Atmospheric Physics Reference. «Radiation Transmitted by the Atmosphere.»

[25] A. C. Wilber, D. P. Kratz y S. K. Gupta, «Surface Emissivity Maps for Use in Retrievals of Longwave Radiation Satellite,» NASA Langley Research Center, Hampton, Virginia, inf. téc. NASA/TP-1999-209362, ago. de 1999. dirección: https://ntrs.nasa.gov/api/citations/19990100634/downloads/19990100634.pdf

[26] NIST Chemistry WebBook. «Carbon Dioxide Infrared Spectrum.» NIST Quantitative Infrared Database - CO2 transmittance spectrum, visitado 17 de jun. de 2025. dirección: https://webbook.nist.gov/chemistry/

[27] Unknown, «Absorption coefficient of carbon dioxide across atmospheric troposphere layer,» *PMC Scientific Reports*, oct. de 2018, Systematic study of CO2 absorption bands at 15, 4.3, 2.7, and 2 um. DOI: 10.1038/s41598-018-33756-8

[28] T. Kuroda, *Essential Principles of Image Sensors*, 1st. CRC Press, 2014, pág. 304, Traducido al inglés desde la versión japonesa original. DOI: 10.1201/b17411

[29] CCD Technology Reference. «CCD Operation and Charge Transfer Mechanism.» Cross-sectional diagram showing CCD pixel structure with three-phase electrode system (I1, i2, I3) and charge transfer process.

[30] MDPI AG, «Coatings, Volume 11, Issue 5, Article 611,» inglés, *Coatings*, vol. 11, n.º 5, pág. 611, mayo de 2021, This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license, ISSN: 2079-6412. DOI: 10.3390/coatings11050611 visitado 17 de jun. de 2025. dirección: https://www.mdpi.com/2079-6412/11/5/611

[31] P. Capper y J. Garland, *Mercury Cadmium Telluride: Growth, Properties and Applications*, 1st. John Wiley & Sons, sep. de 2010, pág. 696, ISBN: 978-0-470-66946-4. DOI: 10.1002/9780470669464

[32] Every Photon Counts Research Group. «InGaAs/InP SPAD.» Single Photon Avalanche Diodes for near-infrared applications (1.0-1.7 um), visitado 17 de jun. de 2025. dirección: http://www.everyphotoncounts.com/device-ingaas.php

[33] M. Henini, *Handbook of Infrared Detection Technologies*. Elsevier, dic. de 2002, pág. 696, ISBN: 978-1-85617-388-9.

[34] SPIE Conference Proceedings, «High speed TDI embedded CCD in CMOS sensor,» en *Conference Proceedings of SPIE*, TDI prototype sensor system-on-chip with 512x1024 pixel array, vol. 10562, sep. de 2017. dirección: https://www.spiedigitallibrary.org/conference-proceedings-of-spie/10562/105622P/High-speed-TDI-embedded-CCD-in-CMOS-sensor/10.1117/12.2296149.full

[35] Remote Sensing Technology Reference. «Pushbroom Sensor Linear CCD Array Configuration.» Diagram showing pushbroom scanner with linear CCD array, optics, view plane, and swath coverage during spacecraft motion.

[36] Remote Sensing Educational Resource. «Whiskbroom Sensor Diagram.» Diagram showing whiskbroom scanner operation with rotating scan mirror, instantaneous field of view, and ground resolution cell.

[37] Aerospace Engineering Reference. «Step-Stare Imaging System Configuration.» Diagram showing step-stare operation with 3.5 degree FOV, FSI sample areas, and 14-second dwell times over 100 km distance.

[38] H. Kaplan, *Optical Systems Engineering*, 1st. McGraw-Hill Education, 2011, pág. 464, Capítulo 6: Detectors and Focal Plane Arrays, ISBN: 978-0-07-175440-8.

[39] Optical Engineering Resource. «Optical System Entrance Pupil Diagram.» Diagram showing entrance pupil diameter, effective focal length, and f-number relationship in optical systems.

[40] Optical Systems Reference. «Refractor vs Reflector Telescope Optical Diagram.» Comparative diagram showing light path differences between refracting and reflecting telescope designs.

[41] Classical Optics Reference. «Cassegrain Telescope Optical Configuration.» Ray diagram showing Cassegrain telescope with primary parabolic mirror, secondary hyperbolic mirror, and light path convergence.

[42] Advanced Optical Systems Reference. «Korsch Telescope Optical System Configuration.» Diagram showing Korsch telescope with flat mirror, four-mirror system (M1-M4), focal plane, and detector arrangement.

[43] Optical Engineering Reference. «Three Mirror Anastigmat (TMA) Telescope Cross Section.» Cross-sectional diagram showing TMA telescope configuration with primary, secondary, and tertiary mirrors.

[44] Space Telescope Science Institute. «MIRI Filters and Dispersers.» Documentacion oficial del instrumento MIRI del JWST, cuya rueda de filtros se muestra en la imagen. La rueda aloja los filtros para imagen, los diafragmas para coronografia y el prisma para espectroscopia de baja resolucion., visitado 7 de jun. de 2025. dirección: `https://jwst-docs.stsci.edu/jwst-mid-infrared-instrument/miri-instrumentation/miri-filters-and-dispersers`

[45] K. C. Reeve III. «Understanding Active Band Pass Filter Circuits with Formulas,» visitado 7 de jun. de 2025. dirección: `https://electronics-formulas.com/understanding-active-band-pass-filter-circuits-with-formulas/`

[46] Wikipedia, la enciclopedia libre. «Elementos orbitales,» visitado 16 de jun. de 2025. dirección: `https://es.wikipedia.org/wiki/Elementos_orbitales`

[47] H. D. Curtis, *Orbital Mechanics for Engineering Students*, 4.ª ed. Amsterdam: Elsevier, 2020, Incluye capitulos detallados sobre transferencia de Hohmann y la ecuacion de Tsiolkovsky, ISBN: 978-0-08-102133-0.

[48] Space Academy, *Satellite Orbital Lifetimes*, Accedido el 2 de mayo de 2025, 2017. dirección: `https://www.spaceacademy.net.au/watch/debris/orblife.htm`

[49] D. A. Vallado y W. D. McClain, *Fundamentals of Astrodynamics and Applications* (Space Technology Library 21), 4.ª ed. Hawthorne, California: Microcosm Press, 2013, Texto de referencia estandar para mecanica orbital y astrodinamica, con tratamiento detallado del arrastre atmosferico y sus efectos en elementos orbitales, ISBN: 978-1-881883-18-0.

[50] D. A. Skoog, F. J. Holler y S. R. Crouch, *Principles of Instrumental Analysis*, 7th. Cengage Learning, 2017, cap. 13, págs. 339-365, Chapter on Optical Spectroscopy: Bandwidth considerations and signal-to-noise ratio.

[51] LYNRED, *CAPYORK 1200x12: High Performance Infrared Detector for Earth Observation Applications*, https://lynred.com/products/capyork-1200x12, Ficha tecnica del detector IR CAPYORK de 1200x12 pixeles. Diseñado para satelites de observacion terrestre., 2022. dirección: https://lynred.com/products/capyork-1200x12

[52] Teledyne Imaging Sensors, «HAWAII-2RG™ Visible and Infrared Focal Plane Array,» Teledyne Imaging Sensors, Datasheet / Technical Report, 2022, Publicado por Teledyne Imaging Sensors; 2048x2048 pixeles, 18 um, HgCdTe IR y tecnologia HyViSI visible, compatible con el ASIC SIDECAR™. visitado 13 de jun. de 2025. dirección: https://www.teledyneimaging.com/en/aerospace-and-defense/products/sensors-overview/infrared-hgcdte-mct/hawaii-2rg/

[53] Sofradir-EC, *Saturn VISIR - Very Large, High Performance IR Detectors for Visible and Short Wave Infrared Imaging 1000x256 HgCdTe SWIR (30um Pitch)*, 09-11 rev10, Product datasheet, Sofradir-EC, Fairfield, NJ, USA, 2011.

[54] Cypress Semiconductor Corporation, *CYIS1SM0250-AA STAR250 - 250K Pixel Radiation Hard CMOS Image Sensor*, 38-05713, Product datasheet, Cypress Semiconductor Corporation, 2009.

[55] H. A. Macleod, *Thin-Film Optical Filters* (Series in Optics and Optoelectronics), 4.ª ed. Boca Raton, FL: CRC Press, 2010, pág. 632, ISBN: 978-1-4200-7302-7.

[56] NASA Ames Research Center, «Slot Architecture for Separating Satellites in Sun-Synchronous Orbits,» en *AIAA SPACE 2011 Conference & Exposition*, American Institute of Aeronautics y Astronautics, sep. de 2011. dirección: https://aviationsystems.arc.nasa.gov/publications/2011/AIAA-2011-7184.pdf

[57] R. J. Boain, «A-B-Cs of Sun-Synchronous Orbit Mission Design,» *Jet Propulsion Laboratory, NASA*, 2004. visitado 9 de jun. de 2025. dirección: https://ntrs.nasa.gov/citations/20210001902

[58] O. Gravrand et al., «Development and characterisation of MCT detectors for space astrophysics at CEA,» en *Proc. SPIE 10563, Image Sensing Technologies: Materials, Devices, Systems, and Applications IV*, vol. 10563, SPIE, nov. de 2017, 105630T. DOI: 10.1117/12.2304252

[59] J. Li y D. P. Roy, «Global Revisit Interval Analysis of Landsat-8 -9 and Sentinel-2A,» *Remote Sensing*, vol. 12, n.º 22, pág. 3699, 2020, Análisis detallado de los intervalos de revisita para Landsat-8/9 y Sentinel-2A/2B. DOI: 10.3390/rs12223699 dirección: https://pmc.ncbi.nlm.nih.gov/articles/PMC7699383/

[60] C. S. Neigh, «Landsat Next Mission Status and Overview,» en *Joint Satellite Workshop 2023*, Presentación oficial sobre el estado y características de la misión Landsat Next, NASA Goddard Space Flight Center, 2023. dirección: https://cce-datasharing.gsfc.nasa.gov/files/conference_presentations/jsw2023/BDEC_presentations/DAY2/1450_Neigh.pdf

[61] Copernicus Programme. «S2 Mission - SentiWiki.» Documentación oficial de la misión Sentinel-2 con especificaciones técnicas, European Space Agency, visitado 17 de jun. de 2025. dirección: https://sentiwiki.copernicus.eu/web/s2-mission

[62] Y. Lou, «Synthetic Aperture Radar for STV Overview,» Presentacion en el Radar Breakout Workshop, STV Incubation Study, sep. de 2020. dirección: https://science.nasa.gov/wp-content/uploads/2023/05/STVRadarBreakout.pdf

[63] C. O'Sullivan. «Removing Clouds from Landsat Satellite Images with Python.» Publicado en TDS Archive, Medium, visitado 5 de jun. de 2025. dirección: https://medium.com/data-science/removing-clouds-from-landsat-satellite-images-with-python-246e73494bc

[64] N. Crisp, S. Livadiotti y P. C. Roberts, «A Semi-Analytical Method for Calculating Revisit Time for Satellite Constellations with Discontinuous Coverage,» English, Cornell University, WorkingPaper, jul. de 2018. dirección: https://research.manchester.ac.uk/en/publications/a-semi-analytical-method-for-calculating-revisit-time-for-satelli

[65] J. G. Walker, «A Phase-Based Approach to Satellite Constellation Analysis,» Defense Technical Information Center, inf. téc. ADA243548, 1991, Accedido: 17 de junio de 2025. dirección: https://apps.dtic.mil/sti/tr/pdf/ADA243548.pdf

[66] J. R. Wertz y W. J. Larson, eds., *Space Mission Analysis and Design* (Space Technology Library), 3rd. El Segundo, CA y Dordrecht, The Netherlands: Microcosm Press y Kluwer Academic Publishers, 1999, ISBN: 978-1881883104.

[67] D. Zorita Gomez-Escolar, «Dimensionamiento del instrumento - Volumen y peso,» inf. téc., 2023, Documento tecnico sobre escalacion de dimensiones de instrumentos espaciales.

[68] G. E. Cook, «Satellite drag coefficients,» *Planetary and Space Science*, vol. 13, n.º 10, págs. 929-946, 1965. DOI: 10.1016/0032-0633(65)90150-9

[69] «U.S. Standard Atmosphere, 1976,» National Oceanic and Atmospheric Administration, National Aeronautics and Space Administration, U.S. Air Force, Technical Memorandum NOAA-S/T-76-1562, NASA-TM-X-74335, oct. de 1976, Accession Number: 77N16482. dirección: https://ntrs.nasa.gov/citations/19770009539

[70] NASA, «Deorbit Systems,» NASA Small Satellite Technology State of the Art Institute, Technical Report, 2025, cap. 13. visitado 16 de jun. de 2025. dirección: https://www.nasa.gov/smallsat-institute/sst-soa/deorbit-systems/

[71] K. Kiedron y R. Cook, «Frozen orbits in the $J_2 + J_3$ problem,» en *Proceedings of the AAS/AIAA Astrodynamics Conference*, 1992, págs. 1273-1289. dirección: https://ntrs.nasa.gov/citations/19920060700

[72] SpaceX. «SmallSat Rideshare Program,» visitado 14 de jun. de 2025. dirección: https://www.spacex.com/rideshare/

[73] E. S. Agency. «Vega return to flight proves new rideshare service,» visitado 14 de jun. de 2025. dirección: https://www.esa.int/Enabling_Support/Space_Transportation/Vega/Vega_return_to_flight_proves_new_rideshare_service

[74] S. Today. «5 Years of SpaceX Rideshare Missions: The Spoils of Monopoly,» visitado 14 de jun. de 2025. dirección: https://interactive.satellitetoday.com/via/september-2024/5-years-of-spacex-rideshare-missions-the-spoils-of-monopoly

[75] P. Space. «SpaceX Increases Rideshare Prices,» visitado 14 de jun. de 2025. dirección: https://payloadspace.com/spacex-increases-rideshare-prices/

[76] Wikipedia. «Falcon 9,» visitado 14 de jun. de 2025. dirección: https://en.wikipedia.org/wiki/Falcon_9

[77] Firefly Aerospace. «Alpha Launch Vehicle,» visitado 15 de jun. de 2025. dirección: https://fireflyspace.com/alpha/

[78] S. Scholar, «Launch Vehicle High-Energy Performance Dataset,» 2023. visitado 14 de jun. de 2025. dirección: https://www.semanticscholar.org/paper/606321dd750ed69e19fc445c1e24c013e4d1ce25

[79] S. Insider. «How Much Does It Cost to Launch a Rocket?» Visitado 14 de jun. de 2025. dirección: https://spaceinsider.tech/2023/08/16/how-much-does-it-cost-to-launch-a-rocket/

[80] Arianespace. «Vega C: the lightweight, high-performance launcher by Arianespace,» visitado 14 de jun. de 2025. dirección: https://www.arianespace.com/vega-c/

[81] SatObs.org. «What Are Some Of The Ground Facilities In Use Today?» Visitado 14 de jun. de 2025. dirección: http://www.satobs.org/faq/Chapter-09.txt

[82] MDPI, «Seasonal Variations in Lunar-Assisted GEO Transfer Capability for Southward Launch,» *Aerospace*, vol. 11, n.º 4, pág. 321, 2024. dirección: https://www.mdpi.com/2226-4310/11/4/321

[83] S. S. Exchange. «Why does a launch due south not translate into a polar orbit from Vandenberg?» Visitado 14 de jun. de 2025. dirección: https://space.stackexchange.com/questions/58565/why-does-a-launch-due-south-180-azimuth-not-translate-into-a-polar-orbit-from

[84] S. Now. «Live Coverage: SpaceX to Launch 90 Payloads on Transporter-9 Falcon 9 Mission from Vandenberg,» visitado 14 de jun. de 2025. dirección: https://spaceflightnow.com/2023/11/11/live-coverage-spacex-to-launch-90-payloads-on-transporter-9-falcon-9-mission-from-vandenberg/

[85] A. Karabeyoglu, *Lecture 7: Launch Trajectories, AA 284a Advanced Rocket Propulsion*, Also affiliated with Mechanical Engineering, KOC University, oct. de 2019.

[86] NOAA Office of Satellite and Product Operations, *Fairbanks Command and Data Acquisition Station*, Accessed: 2025, 2023. dirección: https://www.ospo.noaa.gov/assets/pdf/Fairbanks_Ground_Station.pdf

[87] NOAA Satellite and Information Service, *NOAA Wallops CDA Station GOES Data Collection System*, Technical Working Group Presentation, 2019. dirección: https://noaasis.noaa.gov/pdf/docs/TWG2019/0930%5C%20Wallops%5C%20Update.pdf

[88] EUMETSAT, *Ground Segment Infrastructure*, Last updated: 28 February 2025, 2025. dirección: https://www.eumetsat.int/ground-segment

[89] Alaska Satellite Facility (ASF), *ASF–NASA Near Space Network (NSN) Satellite Tracking Stations*, Web page, Consultado el 14 de junio de 2025, 2025. dirección: https://asf.alaska.edu/asf-satellite-tracking-stations/#ASF-NASA-NSN

[90] NASA Small Spacecraft Technology Institute. «Ground Data Systems and Mission Operations.» Comparación de bandas UHF, S, X y Ka, capacidades de enlace y requisitos de apuntamiento para SmallSats, visitado 12 de sep. de 2026. dirección: https://www.nasa.gov/smallsat-institute/sst-soa/ground-data-systems-and-mission-operations/

[91] European Space Agency. «MetOp Operations.» Ejemplos de tasas de 4096 bps en S-band y 70 Mbps en X-band, visitado 12 de sep. de 2026. dirección: https://www.esa.int/Applications/Observing_the_Earth/Meteorological_missions/MetOp/Operations

[92] European Space Agency. «Sentinel-2 Operations.» Enlace S-band de TM/TC y enlace X-band de datos de carga útil a 560 Mbps, visitado 12 de sep. de 2026. dirección: https://www.esa.int/Enabling_Support/Operations/Sentinel-2_operations

[93] *Near Earth Network (NEN) Users' Guide*, Revision 4, 453-NENUG, NASA Goddard Space Flight Center, 14 de mar. de 2019. visitado 4 de nov. de 2026. dirección: https://explorers.larc.nasa.gov/2023ESE/pdf_files/NENUG%20Rev4.pdf

[94] *International Conference on Space Optics — ICSO 2022*, (3-7 de oct. de 2022), vol. 12777, Toulouse, France: SPIE, 2022.

[95] O. Yadid-Pecht, «Geometrical modulation transfer function for different pixel active area shapes,» *Optical Engineering*, vol. 39, n.º 4, págs. 859-865, 2000. DOI: 10.1117/1.602462 dirección: https://www.spiedigitallibrary.org/journals/optical-engineering/volume-39/issue-4/0000/Geometrical-modulation-transfer-function-for-different-pixel-active-area-shapes/10.1117/1.602462.full

[96] N. H. Crisp, S. Livadiotti y P. C. E. Roberts, *RevisitTime: MATLAB toolkit for calculating revisit time metrics*, ver. 1.0, Accessed: 2025-06-03, 29 de oct. de 2020. dirección: https://github.com/nhcrisp/RevisitTime

[97] G. C. Holst, *Electro-Optical Imaging System Performance*, 4th. Winter Park, Florida USA y Bellingham, WA: JCD Publishing y SPIE Press, 2006, vol. PM-160, ISBN: 0-9707749-2-3.

[98] SENER Ingenieria y Sistemas S.A., «SEOSAT Primary Payload Instrument - Signal-to-Noise Ratio,» SENER Ingenieria y Sistemas S.A., Getxo, inf. téc. SEOS-TN-SEN-PP-065, 2023, Revision 0, Date: 09-10-23, págs. 18-38.

[99] Teledyne Imaging Sensors, «H2RG™ Visible & Infrared Focal Plane Array,» Teledyne Imaging Sensors, Camarillo, CA, Technical Brochure TSI-0855, 2012, Cleared for Public Release by the DoD's Office of Security Review, Case 12-S-1868. dirección: https://panic.iaa.es/sites/default/files/H2RG_Brochure_rev6_v2_2_OSR.pdf

[100] R. Blank, S. Anglin, J. W. Beletic et al., «H2RG Focal Plane Array and Camera Performance Update,» en *Proceedings of SPIE*, Performance data for H2RG arrays at various operating temperatures, SPIE, vol. 8453, Amsterdam, Netherlands, 2012.

[101] B. Fieque, P. Chorier, A. Lamoure y O. Offranc, «Status of Space Activity and Science Detectors Development at SOFRADIR,» en *International Conference on Space Optics (ICSO 2018)*, Comprehensive overview of SOFRADIR detector portfolio including Saturn series, SPIE, vol. 11180, Chania, Greece, oct. de 2018, pág. 1 118 074. DOI: 10.1117/12.2536140

[102] Teledyne e2v, *CHROMA-D18 SWIR HgCdTe Infrared Detector*, ver. v1/DM/IS, Product datasheet, Teledyne e2v Space Imaging, sep. de 2023.

[103] Vision Systems Design. «Sofradir signs infrared detector contract.» Accedido el 13 de junio de 2025. Sofradir recibio un contrato multimillonario para proveer matrices de plano focal IR de alta resolucion para la constelacion militar MUSIS/CSO, reemplazando los satelites Helios II, Vision Systems Design. dirección: https://www.vision-systems.com/cameras-accessories/article/16747004/sofradir-signs-infrared-detector-contract

[104] Infineon Technologies / Cypress Semiconductor, «STAR-250: 250 K-pixel Radiation-Hard CMOS Image Sensor,» Infineon / Cypress, Datasheet / Technical Report, 2009. visitado 13 de jun. de 2025. dirección: https://www.mouser.es/ProductDetail/Infineon-Cypress/STAR-250?qs=eg8mrW3YyI7j4uRE32sOuA%3D%3D

[105] C. P. S. The CubeSat Program, «CubeSat Design Specification (1U–12U), Revision 14.1,» California Polytechnic State University, San Luis Obispo, inf. téc. CP-CDS-R14.1, feb. de 2022, Public Domain. dirección: https://www.nasa.gov/wp-content/uploads/2018/01/cubesatdesignspecificationrev14_12022-02-09.pdf

[106] Aerospace Propulsion Products B.V. «Hydrazine Thrusters.» Accessed: 2025-06-10, Airbus Safran Launchers. dirección: https://www.space-propulsion.com/brochures/hydrazine-thrusters/hydrazine-thrusters.pdf

[107] EnduroSat. «Transmisor X-Band.» Transmisor X-Band para CubeSats y SmallSats, velocidad mexima de 150 Mbps, frecuencia 8.025–8.4 GHz, compatible con DVB-S2. Consultado el 10 de junio de 2025. dirección: https://www.endurosat.com/products/x-band-transmitter/

[108] «GMAT-1.» Ficha de producto para el magnetorquer GMAT-1, Michigan Aerospace Manufacturers Association, visitado 16 de jun. de 2025. dirección: https://www.satcatalog.com/component/gmat-1/

[109] «Nanosats Frequency.» Diagrama de las bandas de frecuencia utilizadas por nanosatelites. La fecha se infiere del nombre del archivo., Nanosats Database, visitado 16 de jun. de 2025. dirección: https://www.nanosats.eu/img/fig/Nanosats_frequency_black_2025-04-30.png

[110] J. Guzmen, L. Fernendez, D. alvarez, R. Martin y M. Garcia, «Design and manufacture of CubeSat-type nanosatellite thermal control subsystem for educational and research purposes,» *Scientific Reports*, vol. 15, n.º 1, pág. 12 345, 2025. DOI: 10.1038/s41598-025-86688-3 dirección: https://www.nature.com/articles/s41598-025-86688-3

[111] ECAPS AB. «ECAPS 100 mN HPGP Thruster,» satsearch, visitado 15 de jun. de 2025. dirección: https://satsearch.co/products/ecaps-100-m-n-hpgp-thruster

[112] NASA. «3.0 Power - NASA SmallSat Institute.» While Rosetta's solar cells achieved around 20 % efficiency, the latest GaAs triple-junction cells reach around 30 %. dirección: https://www.nasa.gov/smallsat-institute/sst-soa/power-subsystems/

[113] E. Kulu. «Nanosats Database.» Database tracking CubeSats, pico- and nanosatellites since 2014. Over 4248 entries encompass 2714 launched nanosatellites as of 2024, visitado 16 de jun. de 2025. dirección: https://www.nanosats.eu

[114] HIANSA, *Panel Cub 3GR-Slimsun - Ficha Tecnica*, Lamina solar flexible de solo 3 kg/m² de peso. Potencia nominal 165 W, ancho panel solar flexible 358 mm, espesor panel solar flexible 2.00 mm, HIANSA, oct. de 2022. dirección: https://www.hiansa.com/wp-content/uploads/2022/10/HIANSA-Panel-Cub-3GR-Slimsun.pdf

[115] Sunket Power. «Ultra-detailed Lithium Battery Knowledge.» Densidad energetica de baterias de litio: 200-260 Wh/kg comparado con 50-70 Wh/kg para baterias de plomo-acido. dirección: https://sunketpower.com/es/ultra-detailed-lithium-battery-knowledge/

[116] ISISPACE. «ISISPACE On-Board Computer (IOBC).» Flight-proven, high-performance processing unit based around 400 MHz ARM9 processor. Mass: 94g mainboard only, 100g with EM daughter board. Power consumption: 400mW average, Innovative Solutions In Space B.V., visitado 16 de jun. de 2025. dirección: https://www.isispace.nl/product/isispace-iobc/

[117] A. Tsinas y C. Welch, «Efficient Satellite Structural Design Optimised for Volume Production,» en *15th Annual USU Conference on Small Satellites*, The prospect of new materials and technology can offer reductions in the overall structural mass of satellites in the region of 15-20 % which in turn can lead to significant overall mass savings and reduced launched costs, Utah State University, 2001, SSC01-IV-5.

[118] Space Mission Engineering and Design, *Spacecraft Subsystems I—Propulsion*, For tanks of hydrazine, the relation mass of the tank respect to the combustible is approximately 0.05 kg per kilogram of propellant, Space Technology Library, 2002, cap. 18.

[119] NASA. «7.0 Thermal Control - NASA SmallSat Institute.» Multi-layer insulation (MLI) and thermal control coatings for spacecraft thermal management. dirección: https://www.nasa.gov/smallsat-institute/sst-soa/thermal-control/

[120] P. Lionnet. «Space Launch Services Prices,» visitado 14 de jun. de 2025. dirección: https://www.linkedin.com/pulse/space-launch-services-prices-pierre-lionnet-bvhbe

[121] Firefly Aerospace. «Payload User's Guide - Alpha Launch Vehicle,» visitado 15 de jun. de 2025. dirección: https://fireflyspace.com/wp-content/uploads/2025/02/Alpha-PUG-5.1_2.pdf

[122] Space.com. «Firefly Aerospace's Alpha rocket fails during launch, falls into the sea near Antarctica,» visitado 15 de jun. de 2025. dirección: https://www.space.com/space-exploration/launches-spacecraft/firefly-aerospaces-alpha-rocket-fails-during-6th-ever-launch-falls-into-the-sea-near-antarctica

[123] Flying Magazine. «Firefly Alpha Rocket Grounded After In-Flight Anomaly,» visitado 15 de jun. de 2025. dirección: https://www.flyingmag.com/firefly-alpha-rocket-grounded-after-in-flight-anomaly/

[124] NewSpace Index. «Firefly - Launcher - NewSpace Index,» visitado 15 de jun. de 2025. dirección: https://www.newspace.im/launchers/firefly

[125] NASASpaceflight. «Firefly's Alpha successfully launches VICTUS NOX rapid response mission,» visitado 15 de jun. de 2025. dirección: https://www.nasaspaceflight.com/2023/09/firefly-victus-nox/

[126] Firefly Aerospace. «Alpha FLTA005 Noise of Summer,» visitado 15 de jun. de 2025. dirección: https://fireflyspace.com/missions/noise-of-summer/

[127] DTIC, «Exploiting Northern Latitudes For Efficient Space Launch,» Defense Technical Information Center, 2024. visitado 14 de jun. de 2025. dirección: https://apps.dtic.mil/sti/tr/pdf/AD1003726.pdf

[128] International Telecommunication Union, «Specific attenuation model for rain for use in prediction methods,» International Telecommunication Union, Geneva, Switzerland, Recommendation ITU-R P.838-3, 2005, Proporciona los coeficientes k y α para el cálculo de la atenuación específica por lluvia en función de la frecuencia y polarización.

[129] European Space Agency. «Calling all radio amateurs – be the first to hear satellite EIRSAT-1 from orbit.» Ejemplo de enlace UHF a 437,1 MHz y 9,6 kbps, visitado 12 de sep. de 2026. dirección: https://www.esa.int/Education/Educational_Satellites/Calling_all_radio_amateurs

[130] European Space Agency. «Help the SSETI Express team.» Ejemplo de enlace UHF a 9,6 kbps y enlace de misión a 38,4 kbps, visitado 12 de sep. de 2026. dirección: https://www.esa.int/Education/SSETI_Express/Help_the_SSETI_Express_team

[131] «A Study on the Effects of Rain Attenuation for an X-Band Satellite,» *Progress in Electromagnetics Research*, 2012, Estudio sobre atenuacion por lluvia en enlaces satelitales de banda X. visitado 14 de jun. de 2025. dirección: https://www.jpier.org/issues/volume.html?paper=12083108

[132] Climate-Data.org. «Clima Fairbanks: Temperatura, Climograma y Tabla climatica.» Datos historicos del clima de Fairbanks con tablas climaticas, visitado 14 de jun. de 2025. dirección: https://es.climate-data.org/america-del-norte/estados-unidos-de-america/alaska/fairbanks-1403/

[133] S. Altunc, «X-band CubeSat Communication System Demonstration,» en *29th Annual AIAA/USU Conference on Small Satellites*, Analysis of elevation angle effects on X-band communication performance, NASA, 2016. visitado 14 de jun. de 2025. dirección: https://s3vi.ndc.nasa.gov/ssri-kb/static/resources/20160001365.pdf

[134] Benha University, *Satellite Coordinates and Look Angles - System Model*, Analysis of signal deterioration with elevation angles and atmospheric losses. visitado 14 de jun. de 2025. dirección: https://www.bu.edu.eg/portal/uploads/Engineering,%20Shoubra/Electrical%20Engineering/2460/crs-13718/Files/Adv.Com.Sys_Lect_05%20Satellite%20Coordinates%20and%20Look%20Angles.pdf

[135] J.-M. Muylaert et al., «QB50: An International Network of 50 CubeSats in Lower Thermosphere,» *Acta Astronautica*, vol. 132, págs. 258-264, 2017, Red de nanosatélites 2U en órbita SSO con magnetopares y barras de gradiente gravitatorio para estabilización nadir. DOI: 10.1016/j.actaastro.2016.10.050

[136] Wikipedia. «Geography of the United States.» CIA World Factbook: 9,826,675 km² (3,794,100 sq mi) total area, visitado 14 de jun. de 2025. dirección: https://en.wikipedia.org/wiki/Geography_of_the_United_States

[137] «XLink-X – X-Band Transceiver SDR.» Transceptor SDR en banda X para CubeSats; hasta 200 Mbps, masa 200 g, consumo máx. 16 W, IQ Spacecom. dirección: https://www.cubesatshop.com/product/iq-spacecom-x-link-x/

[138] «UHF Transceiver II.» Transceptor UHF para CubeSats; 437 MHz, hasta 9,6 kbps, masa 85 g, consumo RX = 0,9 W, TX = 1 W, EnduroSat, visitado 31 de mayo de 2026. dirección: https://www.endurosat.com/cubesat-store/cubesat-communication-modules/uhf-transceiver-ii/

[139] «SSOC-D60 – Digital Sun Sensor.» Sensor solar digital MEMS para nanosatélites; masa ≈ 5 g, precisión $\pm 0{,}5°$, consumo $< 0{,}1$ W, Solar MEMS Technologies, visitado 31 de mayo de 2026. dirección: https://www.solar-mems.com/products/sun-sensors/ssoc-d60/

Anexos

Disponibilidad de Código y Datos

Todo el código fuente utilizado para el procesamiento de datos, los *outputs* en .txt y .csv, así como las figuras presentadas en este documento se encuentra disponible públicamente en el siguiente repositorio de GitHub:

`https://github.com/miguelrzazo/PreliminaryDesignEO`

El repositorio incluye los *scripts* de análisis, los conjuntos de datos originales y las instrucciones necesarias para reproducir íntegramente los resultados aquí expuestos con el fin de mantener la transparencia.

Apéndice A

Tablas de datos

MTF

Banda $0,76\ \mu m$

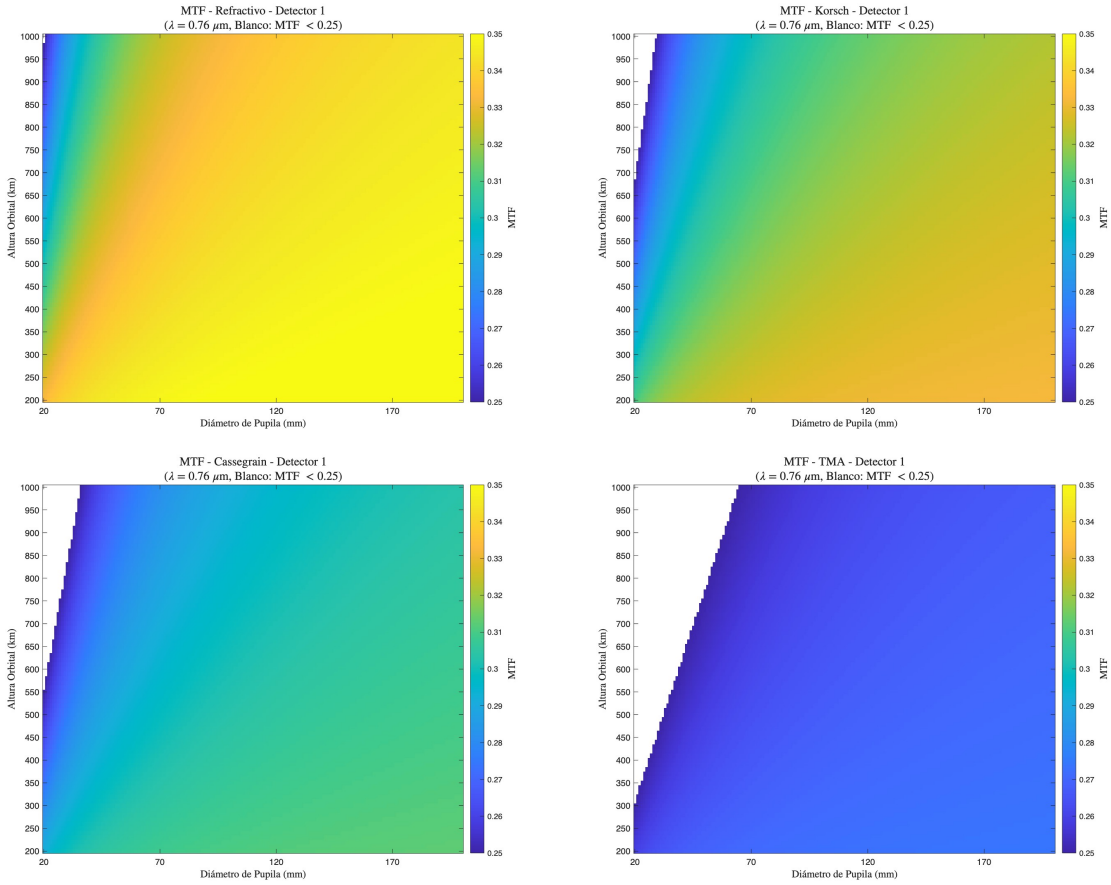


Figura A.1: Mapas de calor resultantes del cálculo de MTF: Banda 0,76 µm; Detector 1

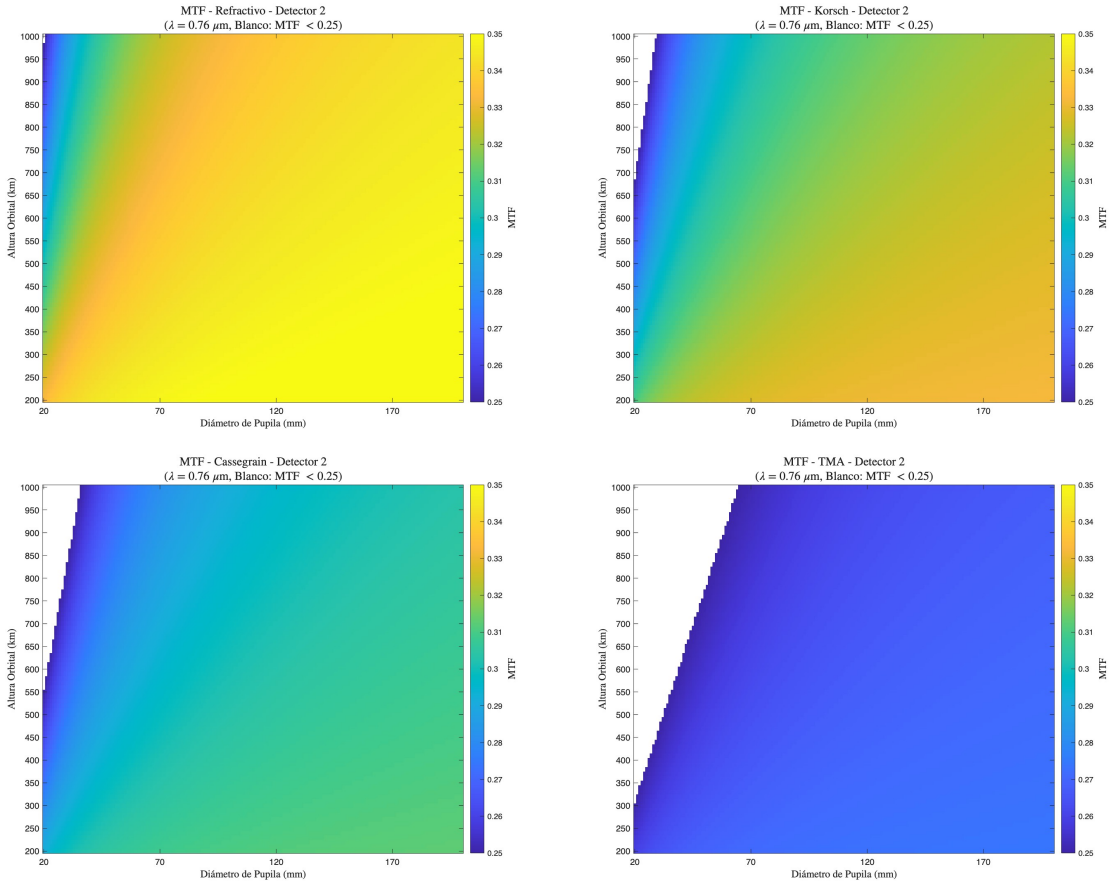


Figura A.2: Mapas de calor resultantes del cálculo de MTF: Banda 0,76 µm; Detector 2

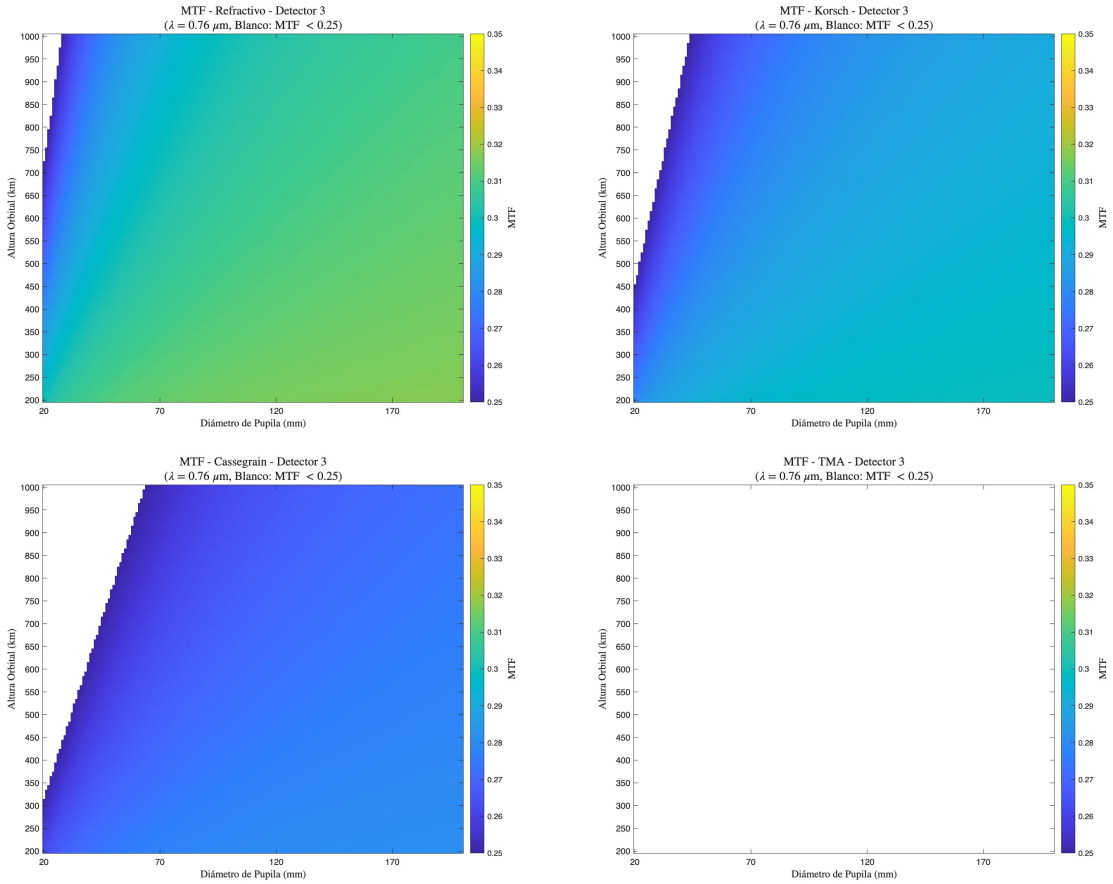


Figura A.3: Mapas de calor resultantes del cálculo de MTF: Banda 0,76 µm; Detector 3

Banda $1,61\ \mu m$

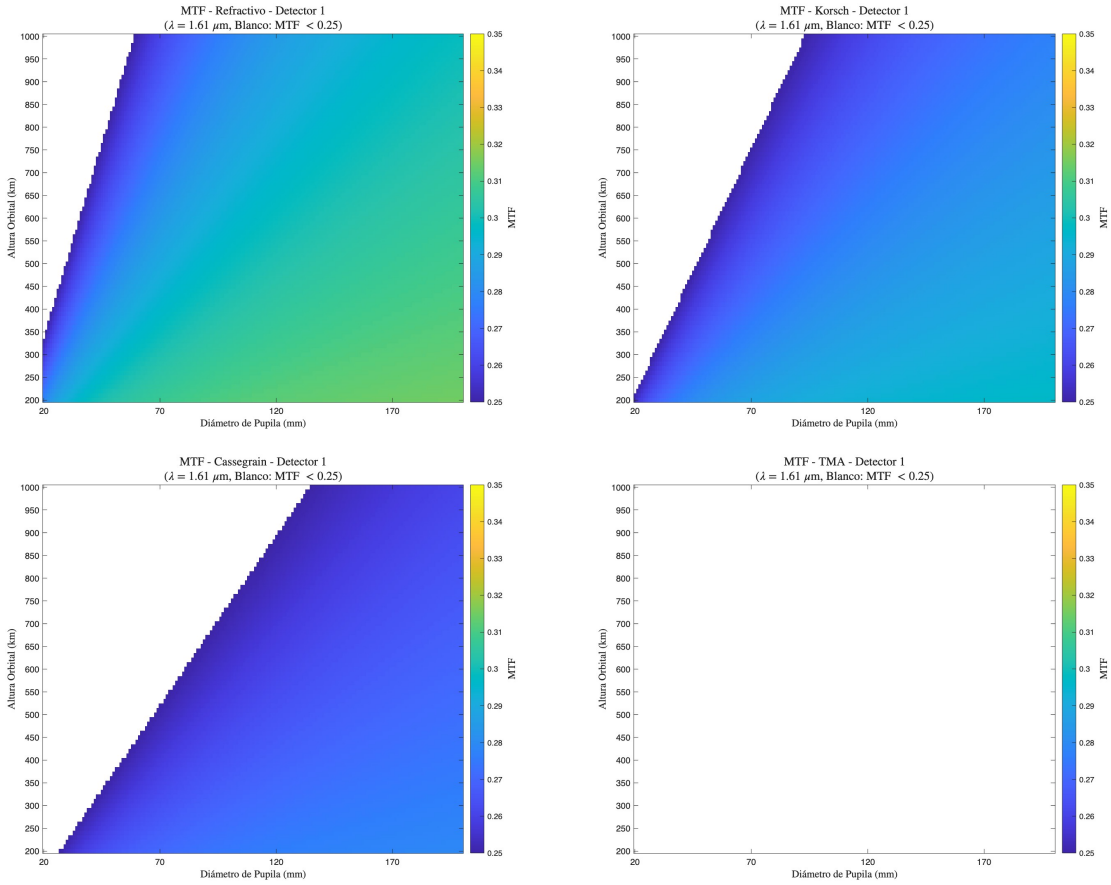


Figura A.4: Mapas de calor resultantes del cálculo de MTF: Banda 1,61 µm; Detector 1

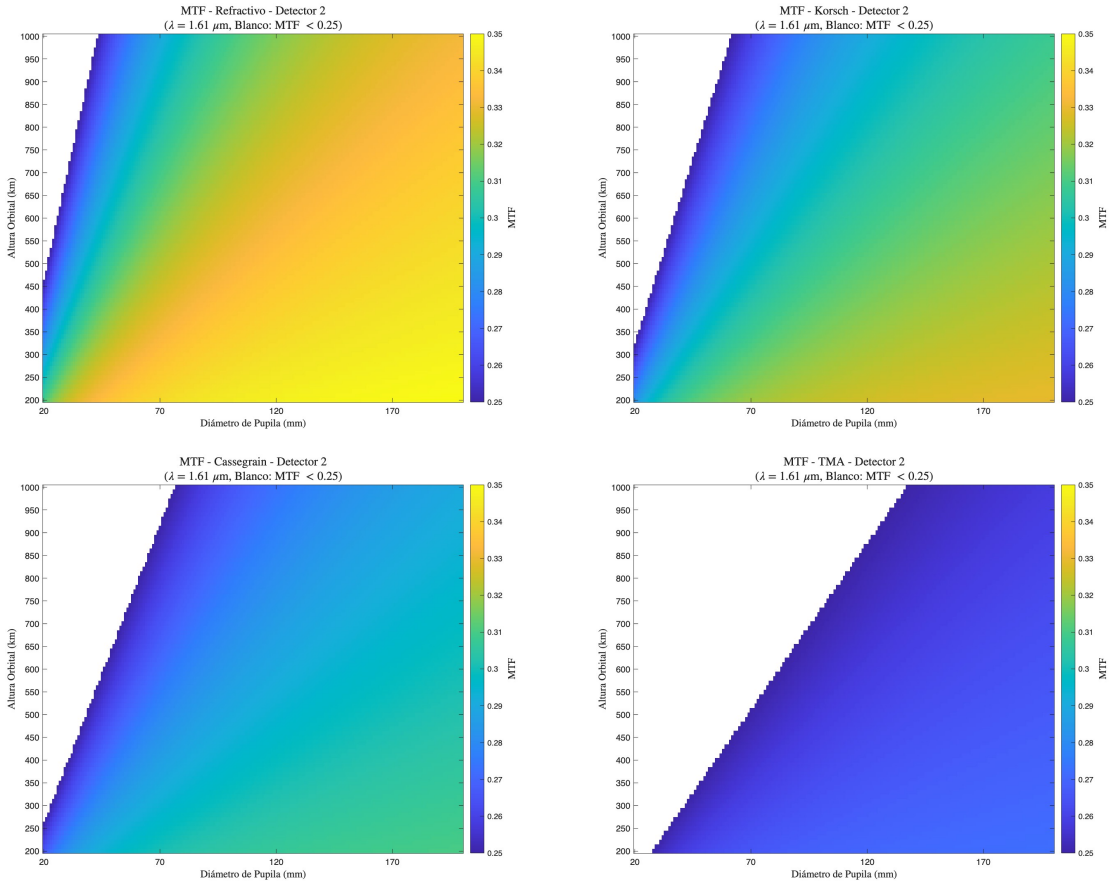


Figura A.5: Mapas de calor resultantes del cálculo de MTF: Banda 1,61 µm; Detector 2

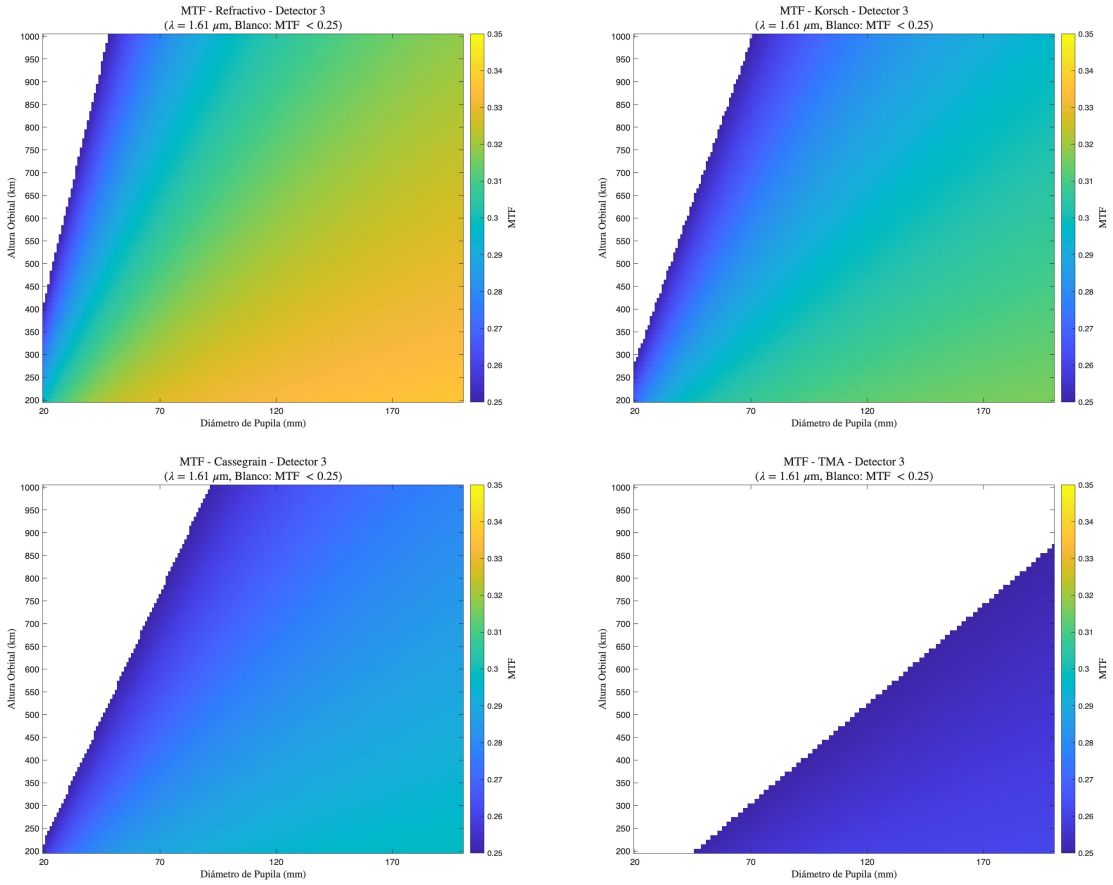


Figura A.6: Mapas de calor resultantes del cálculo de MTF: Banda 1,61 µm; Detector 3

Banda $2,01\ \mu m$

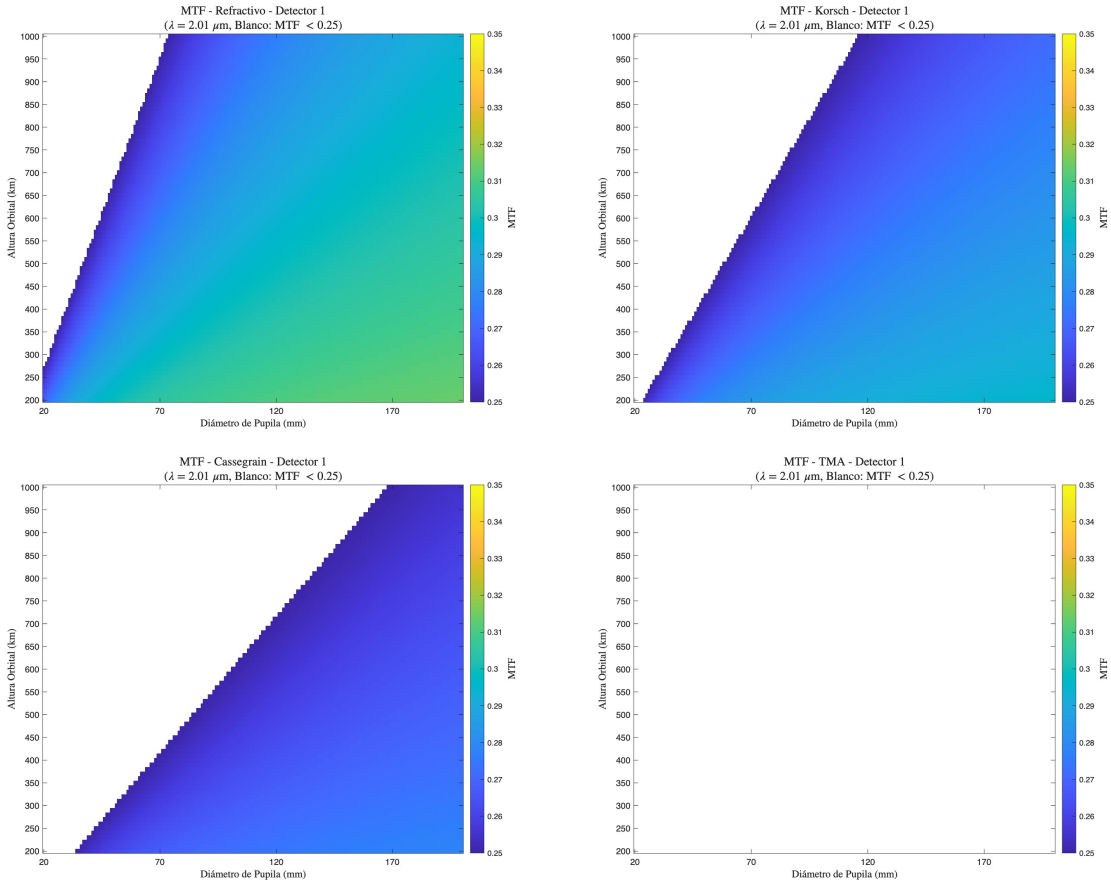


Figura A.7: Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 1

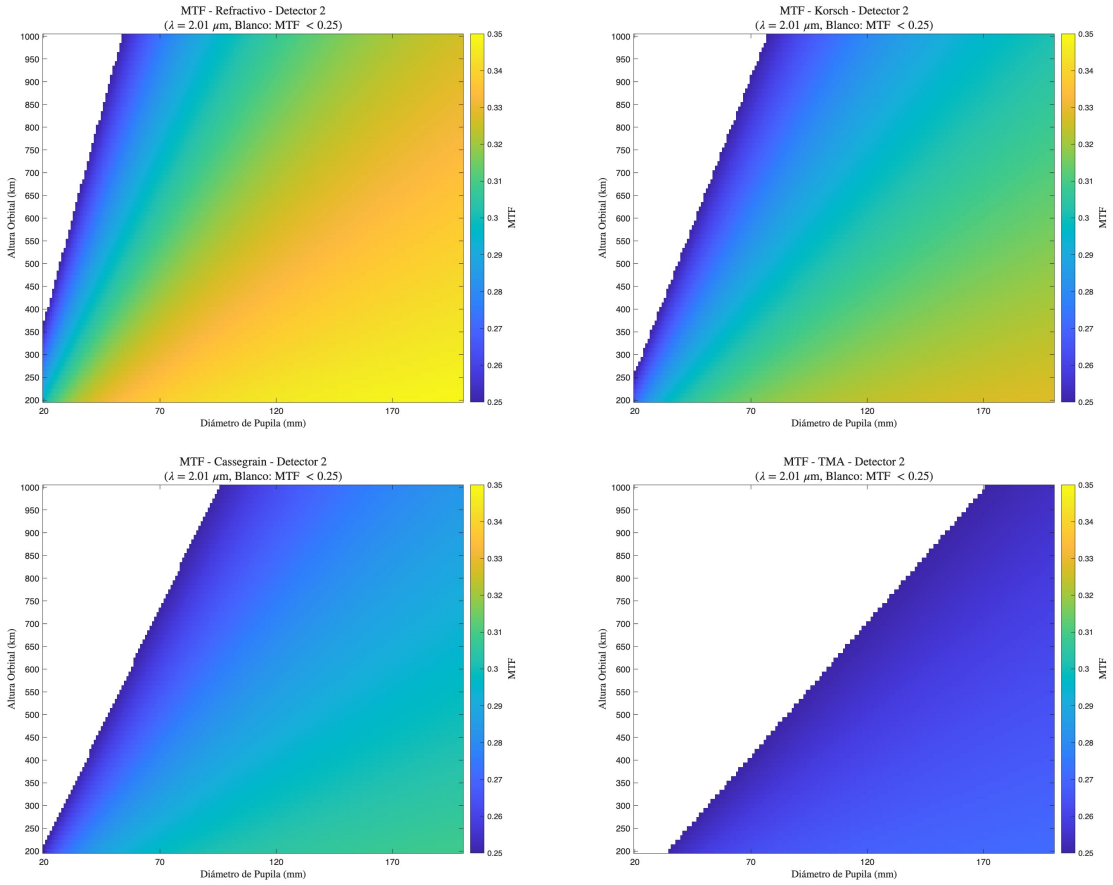


Figura A.8: Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 2

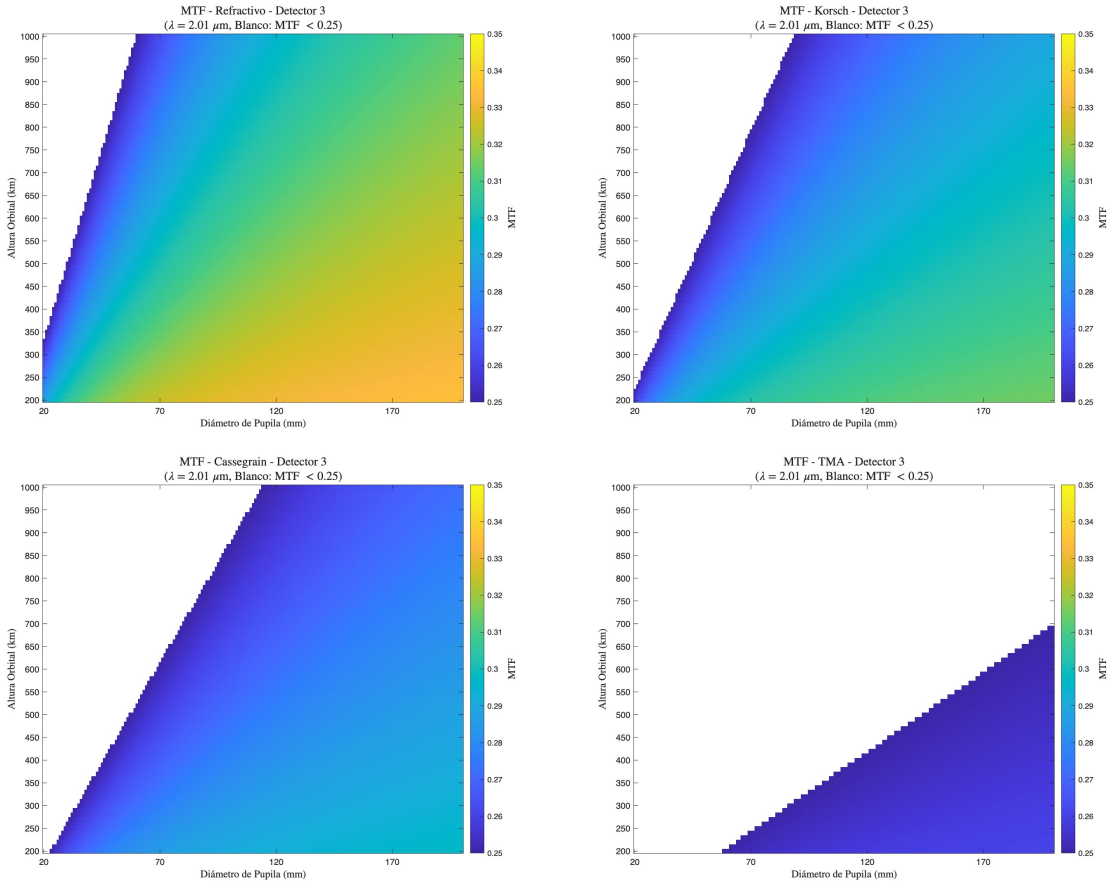


Figura A.9: Mapas de calor resultantes del cálculo de MTF: Banda 2,01 µm; Detector 3

SNR

Banda 0,76 µm

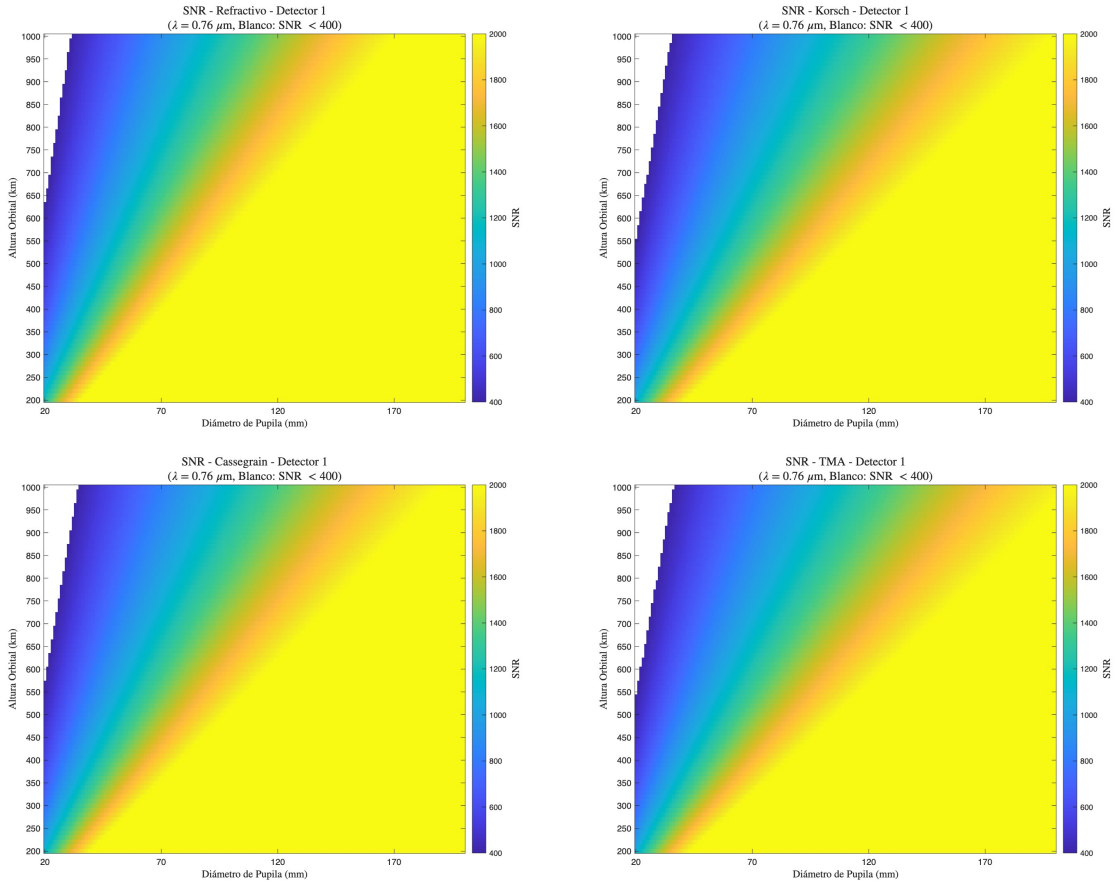


Figura A.10: Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 4

Banda 0,76 µm

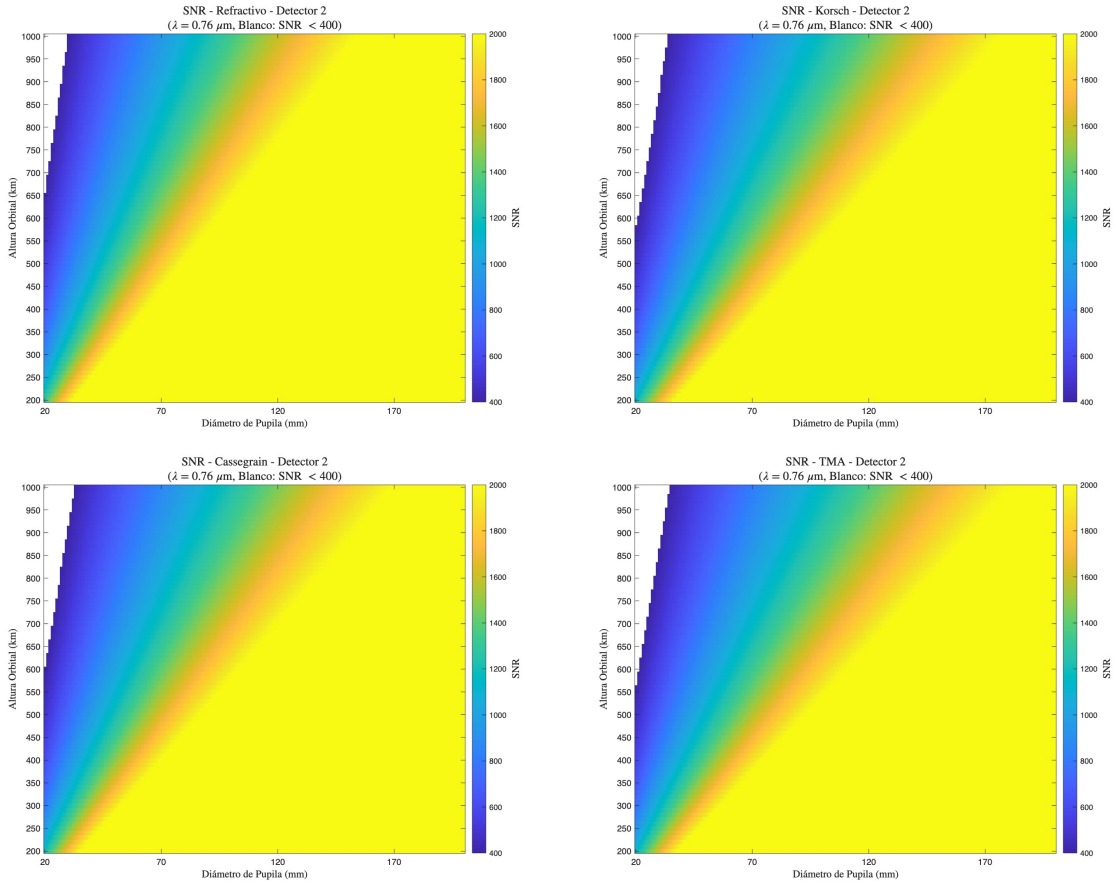


Figura A.11: Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 5

Banda 0,76 µm

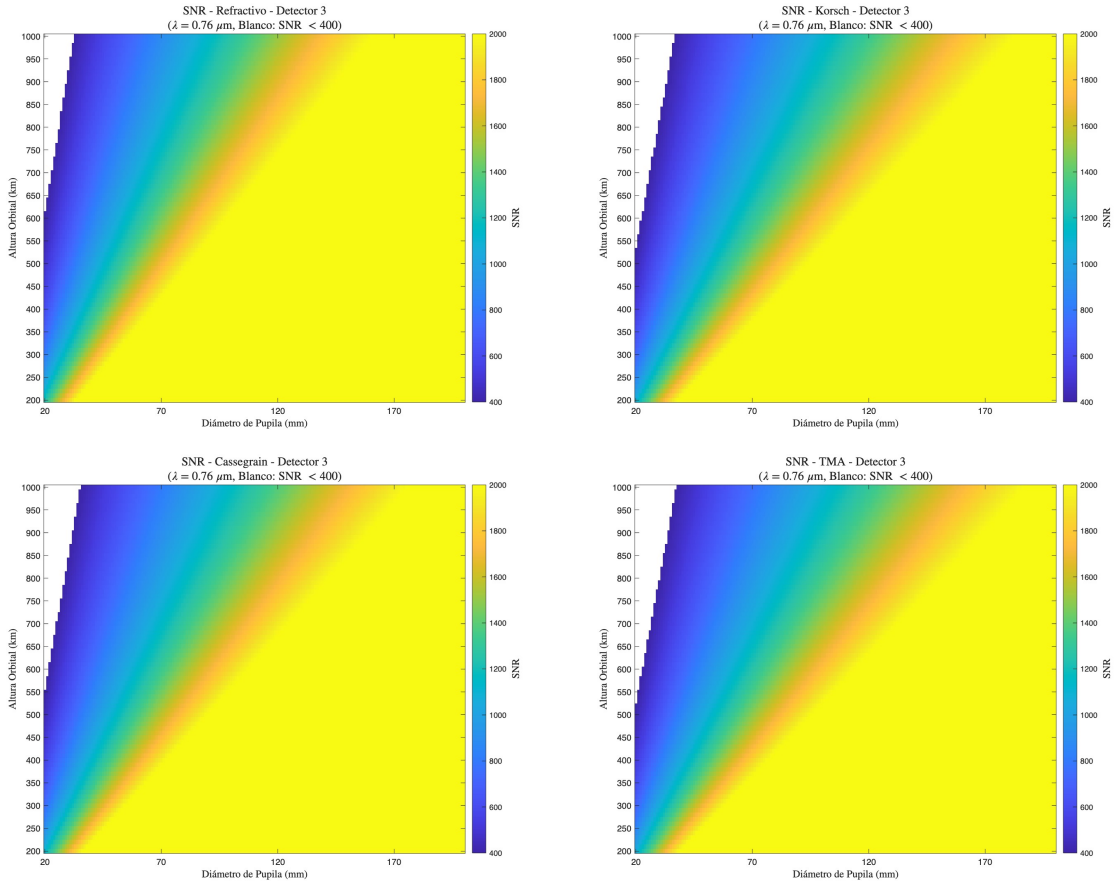


Figura A.12: Mapas de calor resultantes del cálculo de SNR: Banda 0,76 µm; Detector 6

Banda 1,61 µm

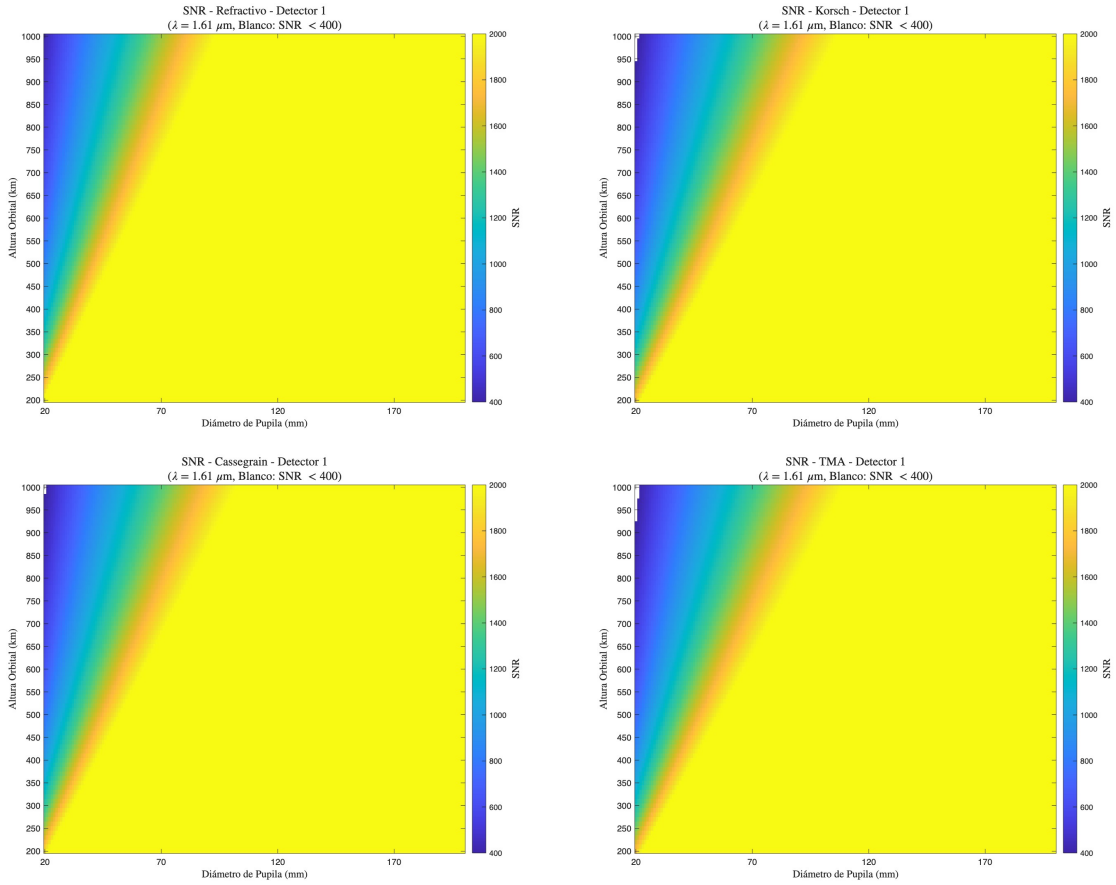


Figura A.13: Mapas de calor resultantes del cálculo de SNR: Banda 1,61 µm; Detector 1

Banda 1,61 µm

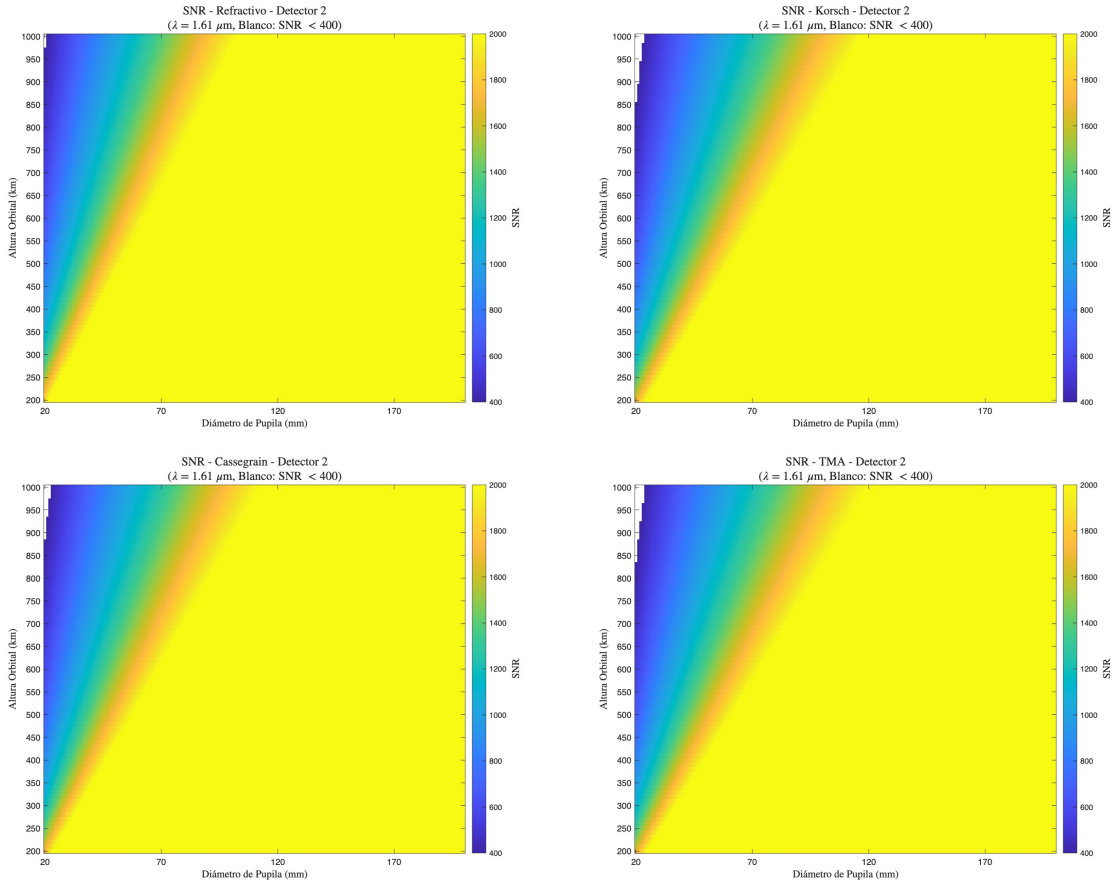


Figura A.14: Mapas de calor resultantes del cálculo de SNR: Banda 1,61 µm; Detector 2

Banda 1,61 µm

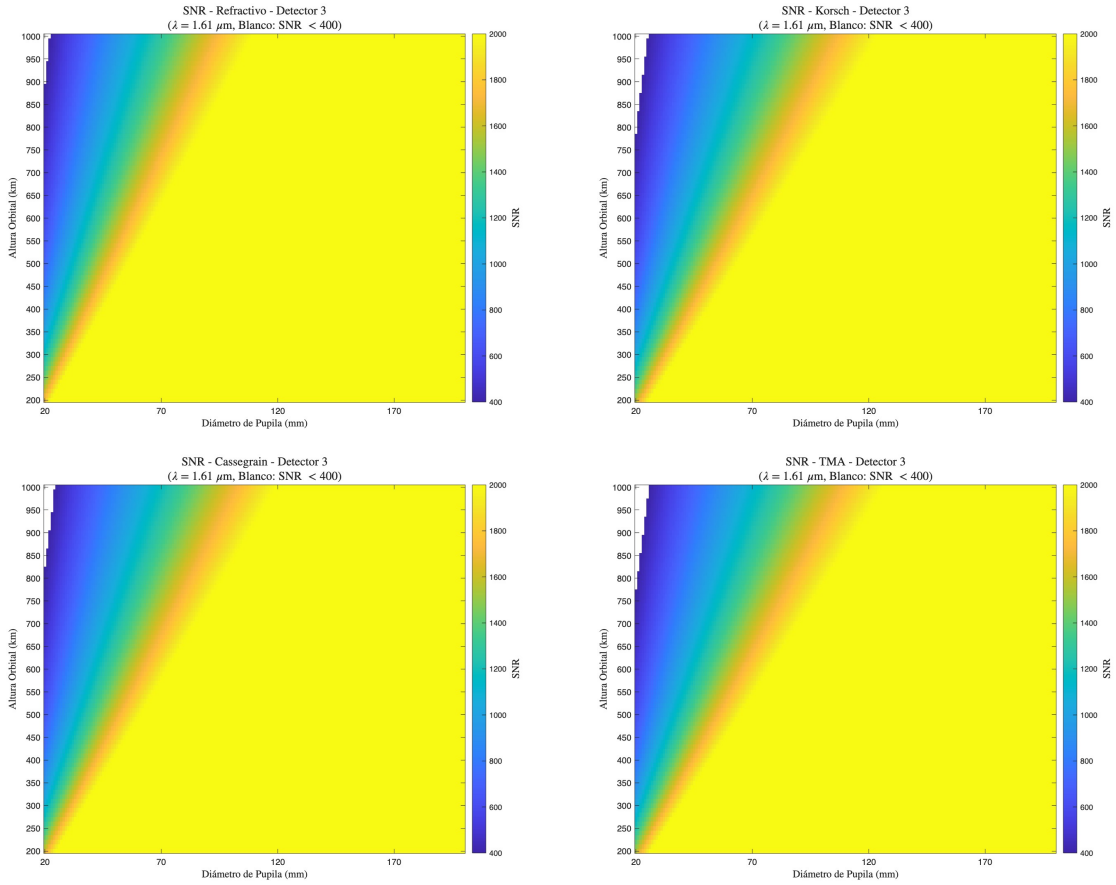


Figura A.15: Mapas de calor resultantes del cálculo de SNR: Banda 1,61 µm; Detector 3

Banda 2,01 µm

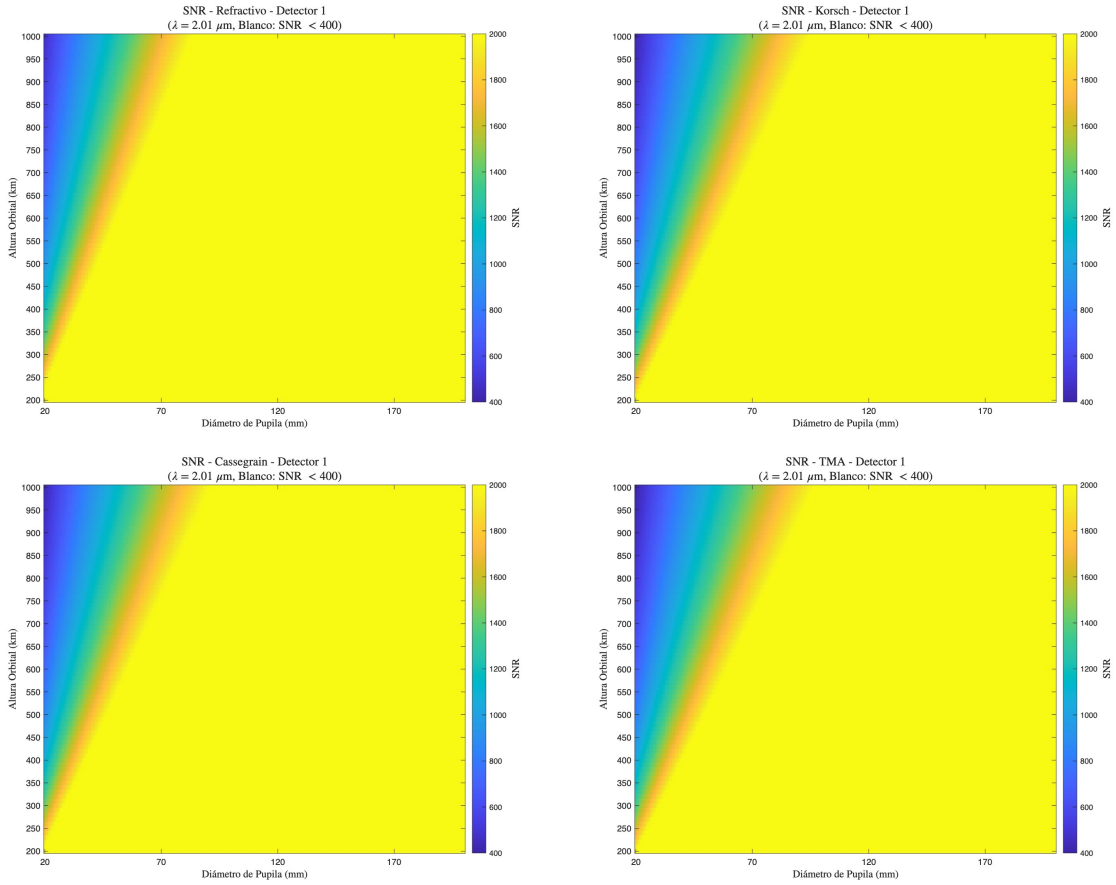


Figura A.16: Mapas de calor resultantes del cálculo de SNR: Banda 2,01 µm; Detector 1

Banda 2,01 µm

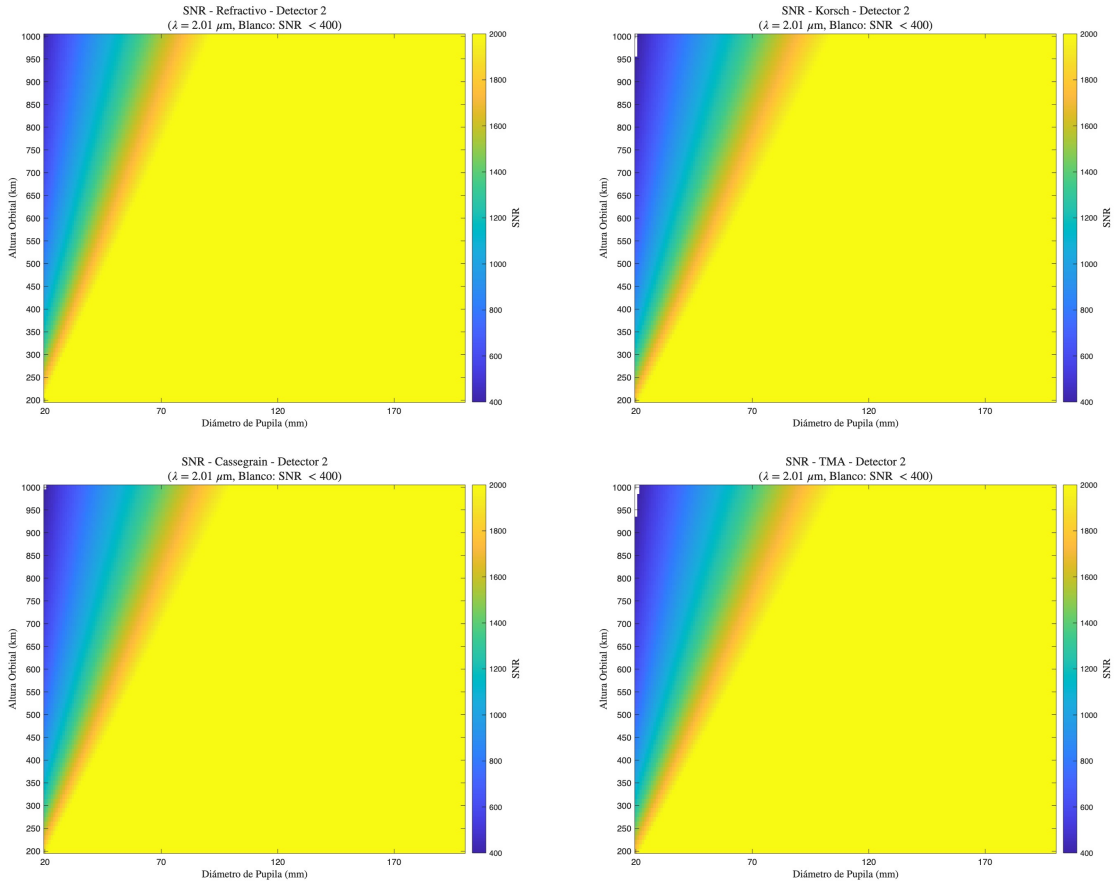


Figura A.17: Mapas de calor resultantes del cálculo de SNR: Banda 2,01 µm; Detector 2

Banda 2,01 µm

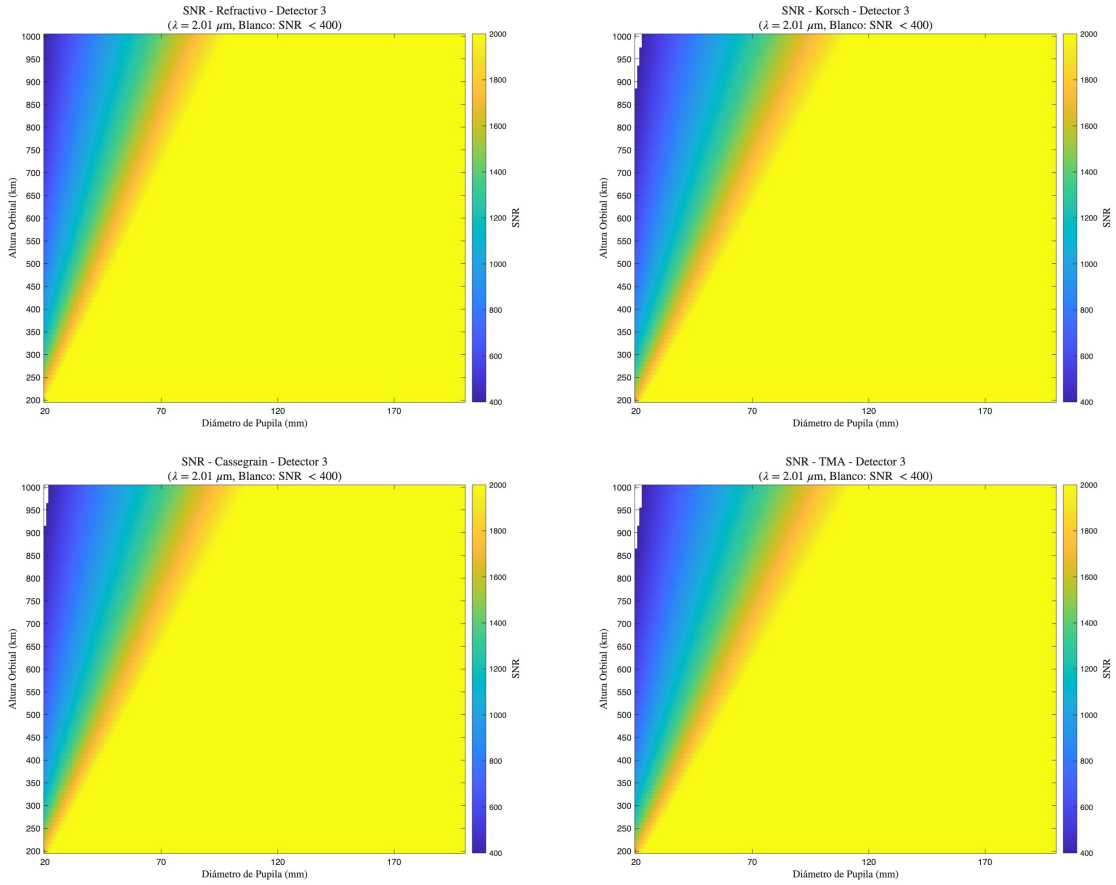


Figura A.18: Mapas de calor resultantes del cálculo de SNR: Banda 2,01 µm; Detector 3

Análisis de Configuraciones. Posibles soluciones

Tabla A.1: Soluciones viables tras el filtro regional. Masa de selección sin reserva de fin de misión.

Sat.	Tel.	Det.	Óptica	h [km]	D [mm]	T_{eff} [d]	m_{const} [kg]
2	2	2	Refractivo	690	38	6.642	6.784
2	2	2	Refractivo	750	41	6.645	8.395
2	2	3	Refractivo	690	42	6.642	9.092
2	2	2	Refractivo	820	45	5.469	11.099
2	2	3	Refractivo	750	45	6.645	11.099
2	2	2	Refractivo	860	47	6.666	12.646
2	2	2	Refractivo	920	50	6.664	15.225
2	2	2	Refractivo	930	50	5.373	15.225
2	2	1	Refractivo	690	51	6.642	16.278
2	2	2	Refractivo	960	52	6.002	17.126
2	2	3	Refractivo	860	52	6.666	17.126
2	2	2	Refractivo	990	54	6.636	19.179
2	2	2	Refractivo	1000	54	6.006	19.179
2	2	1	Refractivo	750	56	6.645	21.390
2	2	1	Refractivo	820	61	5.469	27.646
2	2	1	Refractivo	860	64	6.666	31.929
2	2	1	Refractivo	920	68	6.664	38.298
2	2	1	Refractivo	960	71	6.002	43.594
2	2	2	TMA	860	147	6.666	264.922
2	2	2	TMA	920	158	6.664	328.956

Resultado de simulación de impulsos necesarios

Para la solución de 690 km se obtienen dos maniobras de mantenimiento durante ocho años, con $\Delta V_{\text{mant}} = 29{,}325$ m/s y $m_{\text{prop,mant}} = 0{,}0503$ kg. La maniobra final requiere 84,067 m/s y una reserva adicional de 0,1461 kg, incluido el margen del 10 %. Los datos numéricos se conservan en `Compiled data/Mission` del repositorio.

Apéndice B

Código MATLAB

Archivo maestro. Parámetros entrada y llamada a funciones.

```
%% Earth observation study -- editable inputs
% Edit this file to change the study. runAnalysis coordinates the calculations.
% Wavelength roles remain CO2 band 1, CO2 band 2, and O2 A-band, respectively.
config = struct();

%% Execution and exports
config.writeOutputs = true;
config.outputRoot = fullfile(fileparts(fileparts(mfilename('fullpath'))),'Compiled data','Pipeline');

%% Mission requirements and design axes
config.groundSampleDistance_m = 80;
config.wavelengths_m = [1.61e-6, 2.01e-6, 0.76e-6];
config.altitudes_km = 200:10:1000;
config.pupilDiameters_mm = 20:1:200;
config.swaths_km = 20:10:500;
config.snrRequirement = 400;
config.mtfRequirement = 0.25;
config.revisitRequirement_days = 7;
config.maxDetectorCount = 3;
config.overlapFraction = 0.05;
config.cloudFraction = 1/6;
config.localTimeAscendingNode_h = 6;
config.maintenanceLowerFactor = 0.98;
config.maintenanceUpperFactor = 1.02;

% Each row is [satellites, telescopes per satellite].
config.satelliteConfigurations = [1 1; 2 1; 2 2; 1 2; 3 1];

%% CO2 detectors -- shared by the first two wavelength bands
% Property vectors follow the detector names in the same order.
config.co2Detectors.names = {'CAPYORK', 'H2RG', 'SATURN VISIR'};
config.co2Detectors.quantumEfficiency = [0.85, 0.70, 0.60];
config.co2Detectors.detectorMtf = [0.45, 0.50, 0.48];
config.co2Detectors.pixelCount = [1200, 2048, 1000];
config.co2Detectors.pixelSize_m = [15, 18, 30] * 1e-6;

%% O2 detectors -- third wavelength band
config.o2Detectors.names = {'CCD98-50', 'H2RG', 'SATURN VISIR'};
config.o2Detectors.quantumEfficiency = [0.65, 0.70, 0.60];
config.o2Detectors.detectorMtf = [0.50, 0.50, 0.45];
config.o2Detectors.pixelCount = [6000, 2048, 1000];
config.o2Detectors.pixelSize_m = [13, 18, 30] * 1e-6;

%% Telescopes
% The existing selection model treats the first entry as refractive (200 mm cap).
config.telescopeNames = {'Refractivo', 'Korsch', 'Cassegrain', 'TMA'};
config.telescopeAlignmentMtf = [0.90, 0.85, 0.80, 0.70];
config.telescopeTransmittance = [0.80, 0.65, 0.70, 0.60];
config.telescopeFov_deg = [10, 3, 3, 8];
config.obstructionRatios = [0, 0.2, 0.2, 0];

%% Optional command-line overrides
% Unset these environment variables when using only the inputs above.
gsdOverride = getenv('TFG_GSD');
if ~isempty(gsdOverride)
    config.groundSampleDistance_m = str2double(gsdOverride);
    fprintf('TFG_GSD override: %g m\n', config.groundSampleDistance_m);
```

```
end
outputOverride = getenv('TFG_OUTPUT_ROOT');
if ~isempty(outputOverride), config.outputRoot = outputOverride; end

%% Run the analytical study
% An unsuccessful run must not leave previous results looking current.
results = [];
results = runAnalysis(config);
```

Listing B.1: runAnalysis.m

```
function results = runAnalysis(config, revisitSolver)
%RUNANALYSIS Execute the analytical study using explicit, validated inputs.
% master.m owns the editable inputs. This function does not read environment
% overrides, clear the caller workspace, or close the caller's figures.
% The optional solver handle supports deterministic coverage integration tests.
analysisDirectory = fileparts(mfilename('fullpath'));
previousPath = path;
pathCleanup = onCleanup(@() path(previousPath));
addpath(fullfile(analysisDirectory,'lib'));
validateStudyConfig(config);
if nargin < 2, revisitSolver = @RevisitCalc; end
validateattributes(revisitSolver,{'function_handle'},{'scalar'});

% Relative paths are confined to compatibility exporters. Restore the caller's
% directory on return or error; compute-only runs never change directory.
if config.writeOutputs && ~isempty(config.outputRoot)
    if ~exist(config.outputRoot,'dir'), mkdir(config.outputRoot); end
    previousDirectory = pwd;
    directoryCleanup = onCleanup(@() cd(previousDirectory));
    cd(config.outputRoot);
end
if config.writeOutputs
    outputDirectories = {'MTF','SNR','coverage','HvsDmin','Masa_seca','Masa_total','OptimumConfigs'};
    for directoryIndex = 1:numel(outputDirectories)
        if ~exist(outputDirectories{directoryIndex},'dir'), mkdir(outputDirectories{directoryIndex}); end
    end
end

% Optical cells: {band, detector, telescope}; coverage cells: {configuration,
% detector, telescope}. Each cell holds one grid with explicit physical axes.
results.config = config;
results.executionDirectory = pwd;
results.revisitSolver = func2str(revisitSolver);
results.axes.wavelengths_m = config.wavelengths_m(:).';
results.axes.altitudes_km = config.altitudes_km(:);
results.axes.pupilDiameters_mm = config.pupilDiameters_mm(:).';
results.axes.swaths_km = config.swaths_km(:).';
results.axes.telescopeNames = config.telescopeNames;
results.axes.satelliteConfigurations = config.satelliteConfigurations;
telescopeCount = numel(config.telescopeNames);
configurationCount = size(config.satelliteConfigurations,1);
results.optics.mtf = cell(3,3,telescopeCount);
results.optics.snr = cell(3,3,telescopeCount);
results.coverage.days = cell(configurationCount,3,telescopeCount);
results.coverage.diagnostics = cell(size(results.coverage.days));
results.coverage.stageErrors = cell(size(results.coverage.days));

% Flatten hardware combinations; each iteration owns one altitude/diameter grid.
opticalCases = analysisCases(numel(config.wavelengths_m),3,telescopeCount);
for caseIndex = 1:size(opticalCases,1)
    bandIndex = opticalCases(caseIndex,1);
    detectorIndex = opticalCases(caseIndex,2);
    telescopeIndex = opticalCases(caseIndex,3);
    if bandIndex <= 2
        detectors = config.co2Detectors;
        detectorExportOffset = 0;
    else
        detectors = config.o2Detectors;
        detectorExportOffset = 3;
    end
    wavelength_m = config.wavelengths_m(bandIndex);
    mtfPrefix = sprintf('MTF/MTF_Lambda%d_Detector%d_Telescopio%d', ...
        bandIndex,detectorIndex+detectorExportOffset,telescopeIndex);
    snrPrefix = sprintf('SNR/SNR_Lambda%d_Detector%d_Telescopio%d', ...
        bandIndex,detectorIndex+detectorExportOffset,telescopeIndex);
    results.optics.mtf{bandIndex,detectorIndex,telescopeIndex} = MTFfunction( ...
        wavelength_m,detectors.pixelSize_m(detectorIndex), ...
        detectors.detectorMtf(detectorIndex),config.telescopeAlignmentMtf(telescopeIndex), ...
        config.groundSampleDistance_m,config.obstructionRatios(telescopeIndex), ...
        config.altitudes_km,config.pupilDiameters_mm,mtfPrefix, ...
        config.telescopeNames{telescopeIndex},detectorIndex, ...
        config.mtfRequirement,config.writeOutputs);
    results.optics.snr{bandIndex,detectorIndex,telescopeIndex} = SNRfunction( ...
        wavelength_m,detectors.pixelSize_m(detectorIndex), ...
        detectors.quantumEfficiency(detectorIndex),config.telescopeTransmittance(telescopeIndex), ...
        config.groundSampleDistance_m,config.obstructionRatios(telescopeIndex), ...
        config.altitudes_km,config.pupilDiameters_mm,snrPrefix, ...
        config.telescopeNames{telescopeIndex},detectorIndex, ...
```

```
        config.snrRequirement,config.writeOutputs);
end

coverageCache = containers.Map('KeyType','char','ValueType','double');
coverageSolver = @(varargin) cachedRevisit(coverageCache,revisitSolver,varargin{:});
coverageCases = analysisCases(configurationCount,3,telescopeCount);
for caseIndex = 1:size(coverageCases,1)
    configurationIndex = coverageCases(caseIndex,1);
    detectorIndex = coverageCases(caseIndex,2);
    telescopeIndex = coverageCases(caseIndex,3);
    satelliteCount = config.satelliteConfigurations(configurationIndex,1);
    telescopesPerSatellite = config.satelliteConfigurations(configurationIndex,2);
    [coverageGrid_days,diagnostics] = computeCoverageGrid( ...
        config.groundSampleDistance_m,config.altitudes_km,config.swaths_km, ...
        config.revisitRequirement_days,config.co2Detectors.pixelCount(detectorIndex), ...
        config.telescopeFov_deg(telescopeIndex),satelliteCount,telescopesPerSatellite, ...
        config.overlapFraction,config.cloudFraction,config.maxDetectorCount,coverageSolver);
    results.coverage.days{configurationIndex,detectorIndex,telescopeIndex} = coverageGrid_days;
    results.coverage.diagnostics{configurationIndex,detectorIndex,telescopeIndex} = diagnostics;
    if config.writeOutputs
        configurationName = sprintf('%d Satelite(s); %d Telescopio(s): %s; Detector %d', ...
            satelliteCount,telescopesPerSatellite, ...
            strrep(config.telescopeNames{telescopeIndex},' ',''),detectorIndex);
        writeCoverageResult(coverageGrid_days,config.altitudes_km,config.swaths_km, ...
            config.revisitRequirement_days,diagnostics,configurationName);
    end
end
coverageSolverFailed = cellfun(@(entry) ...
    any(~cellfun(@isempty,entry.solverErrors(:))) || any(entry.nonfiniteSolverResult(:)), ...
    results.coverage.diagnostics(:));
results.coverage.executionSucceeded = ~any(coverageSolverFailed);

results.selection = CrossDataFunction(config.altitudes_km,config.swaths_km, ...
    config.pupilDiameters_mm,config.telescopeNames,config.telescopeFov_deg, ...
    config.satelliteConfigurations,config.mtfRequirement,config.snrRequirement, ...
    config.revisitRequirement_days,results,config.writeOutputs);
results.configuration = ConfigAnalysis(legacyAnalysisParameters(config,coverageSolver),results,config.writeOutputs);
results.coverage.uniqueSolverCalls = coverageCache.Count;
if ~results.coverage.executionSucceeded
    warning('eo:IncompleteAnalysis', ...
        'Coverage solver failures occurred. Inspect results.coverage.diagnostics.');
end
end

function params = legacyAnalysisParameters(config,revisitSolver)
% Keep the older mass/selection interface behind one explicit naming adapter.
params.satellite_configs = config.satelliteConfigurations;
params.telescope_names = config.telescopeNames;
params.alturas_orbitales = config.altitudes_km;
params.swaths_km = config.swaths_km;
params.diametros_pupila = config.pupilDiameters_mm;
params.Cov_Requirement = config.revisitRequirement_days;
params.MTF_Requirement = config.mtfRequirement;
params.SNR_Requirement = config.snrRequirement;
params.GSD = config.groundSampleDistance_m;
params.Npix_detectors = config.co2Detectors.pixelCount;
params.fov_limit = config.telescopeFov_deg;
params.LTAN_hour = config.localTimeAscendingNode_h;
params.Pixel_size = config.co2Detectors.pixelSize_m;
params.maintenance_lower_factor = config.maintenanceLowerFactor;
params.maintenance_upper_factor = config.maintenanceUpperFactor;
params.solapamiento = config.overlapFraction;
params.cobertura_nubes = config.cloudFraction;
params.regionalRevisitSolver = revisitSolver;
end

function revisit = cachedRevisit(cache,solver,varargin)
% Revisit depends on the orbit and viewing geometry, not detector identity.
key = sprintf('%.17g_', [varargin{:}]);
if isKey(cache,key)
    revisit = cache(key);
else
    revisit = solver(varargin{:});
    cache(key) = revisit;
end
end
```

Calculo de la MTF

```
function mtfGrid = MTFfunction(wavelength_m, pixelSize_m, detectorMtf, ...
    alignmentMtf, groundSampleDistance_m, obstructionRatio, ...
    altitudes_km, pupilDiameters_mm, outputPrefix, ...
    telescopeName, detectorIndex, mtfRequirement, writeOutputs)
%MTFFUNCTION Preserve positional calls and output filenames used by master.m.
% Set writeOutputs=false to obtain the numerical grid without exports.
if nargin < 13, writeOutputs = true; end
mtfGrid = computeMtfGrid(wavelength_m, pixelSize_m, detectorMtf, ...
    alignmentMtf, groundSampleDistance_m, obstructionRatio, ...
    altitudes_km, pupilDiameters_mm, mtfRequirement);
if writeOutputs
    writeMtfResult(mtfGrid, wavelength_m, pixelSize_m, detectorMtf, ...
        alignmentMtf, groundSampleDistance_m, obstructionRatio, ...
        altitudes_km, pupilDiameters_mm, outputPrefix, ...
        telescopeName, detectorIndex, mtfRequirement);
end
end
```

Listing B.2: computeMtfGrid.m

```
function mtfGrid = computeMtfGrid(wavelength_m, pixelSize_m, detectorMtf, ...
    alignmentMtf, groundSampleDistance_m, obstructionRatio, ...
    altitudes_km, pupilDiameters_mm, mtfRequirement)
%COMPUTEMTFGRID Evaluate the historical MTF model without figures or files.
% Rows correspond to altitudes_km; columns correspond to pupilDiameters_mm.
% Values below mtfRequirement become NaN. The threshold itself is accepted.
validateOpticalInputs(wavelength_m, pixelSize_m, groundSampleDistance_m, ...
    obstructionRatio, altitudes_km, pupilDiameters_mm);
validateattributes(detectorMtf, {'numeric'}, {'scalar','real','finite','>=',0,'<=',1});
validateattributes(alignmentMtf, {'numeric'}, {'scalar','real','finite','>=',0,'<=',1});
validateattributes(mtfRequirement, {'numeric'}, {'scalar','real','finite','nonnegative'});

% Altitude-dependent quantities are columns; aperture-dependent ones are rows.
altitudes_m = altitudes_km(:) * 1e3;
pupilDiameters_m = pupilDiameters_mm(:).' * 1e-3;
focalLengths_m = (altitudes_m * pixelSize_m) / groundSampleDistance_m;
cutoffFrequencies = pupilDiameters_m / wavelength_m;
nyquistFrequency_per_m = 1 / (2 * pixelSize_m);
sampledFrequencies = nyquistFrequency_per_m * focalLengths_m;

% Preserve the original saturation at the diffraction cutoff.
normalizedFrequencies = min(sampledFrequencies, cutoffFrequencies) ./ cutoffFrequencies;
diffractionMtf = computeDiffractionMtf(normalizedFrequencies, obstructionRatio);

% Keep individual budget factors visible and preserve their original order.
aberrationMtf = 0.95;
manufacturingMtf = 0.98;
vibrationMtf = 0.99;
thermoelasticMtf = 0.95;
marginMtf = 0.9;
otherMtf = marginMtf * vibrationMtf * manufacturingMtf * thermoelasticMtf * ...
    aberrationMtf * detectorMtf;
mtfGrid = diffractionMtf * alignmentMtf * otherMtf;
mtfGrid(mtfGrid < mtfRequirement) = NaN;
end

function diffractionMtf = computeDiffractionMtf(normalizedFrequencies, obstructionRatio)
% Terms A, B and C follow the original annular-aperture equations 6-10 to 6-15.
% Evaluate each expression only on its valid domain, avoiding invalid roots.
apertureTermA = zeros(size(normalizedFrequencies));
obstructionTermB = zeros(size(normalizedFrequencies));
overlapTermC = zeros(size(normalizedFrequencies));

apertureMask = normalizedFrequencies >= 0 & normalizedFrequencies <= 1;
frequencies = normalizedFrequencies(apertureMask);
apertureTermA(apertureMask) = (2/pi) * ...
    (acos(frequencies) - frequencies .* sqrt(1 - frequencies.^2));

% An unobscured aperture has B = C = 0. Skip divisions by obstructionRatio.
if obstructionRatio > 0
    scaledFrequencies = normalizedFrequencies / obstructionRatio;
    angleCosines = (1 + obstructionRatio^2 - 4 * normalizedFrequencies.^2) / ...
        (2 * obstructionRatio);
    overlapAngles_rad = zeros(size(normalizedFrequencies));
    validAngleMask = angleCosines >= -1 & angleCosines <= 1;
    overlapAngles_rad(validAngleMask) = acos(angleCosines(validAngleMask));

    obstructionMask = scaledFrequencies >= 0 & scaledFrequencies <= 1;
    frequencies = scaledFrequencies(obstructionMask);
    obstructionTermB(obstructionMask) = (2 * obstructionRatio^2/pi) * ...
        (acos(frequencies) - frequencies .* sqrt(1 - frequencies.^2));

    fullOverlapMask = normalizedFrequencies > 0 & ...
        normalizedFrequencies <= (1 - obstructionRatio)/2;
    overlapTermC(fullOverlapMask) = -2 * obstructionRatio^2;
```

```
    partialOverlapMask = normalizedFrequencies > (1 - obstructionRatio)/2 & ...
        normalizedFrequencies < (1 + obstructionRatio)/2;
    angles_rad = overlapAngles_rad(partialOverlapMask);
    overlapTermC(partialOverlapMask) = (2 * obstructionRatio/pi) * sin(angles_rad) + ...
        ((1 + obstructionRatio^2)/pi) * angles_rad - ...
        ((2 * (1 - obstructionRatio^2))/pi) * ...
        atan((1 + obstructionRatio)/(1 - obstructionRatio) * tan(angles_rad/2)) - ...
        2 * obstructionRatio^2;
end

diffractionMtf = (apertureTermA + obstructionTermB + overlapTermC) / ...
    (1 - obstructionRatio^2);
diffractionMtf(isnan(diffractionMtf) | diffractionMtf < 0) = 0;
end
```

Calculo de la SNR

```
function snrGrid = SNRfunction(wavelength_m, pixelSize_m, quantumEfficiency, ...
    transmittance, groundSampleDistance_m, obstructionRatio, ...
    altitudes_km, pupilDiameters_mm, outputPrefix, ...
    telescopeName, detectorIndex, snrRequirement, writeOutputs)
%SNRFUNCTION Preserve positional calls and output filenames used by master.m.
% Set writeOutputs=false to obtain the numerical grid without exports.
if nargin < 13, writeOutputs = true; end
[snrGrid, model] = computeSnrGrid(wavelength_m, pixelSize_m, quantumEfficiency, ...
    transmittance, groundSampleDistance_m, obstructionRatio, ...
    altitudes_km, pupilDiameters_mm, snrRequirement);
if writeOutputs
    writeSnrResult(snrGrid, wavelength_m, pixelSize_m, quantumEfficiency, ...
        transmittance, groundSampleDistance_m, obstructionRatio, ...
        altitudes_km, pupilDiameters_mm, outputPrefix, ...
        telescopeName, detectorIndex, snrRequirement, model);
end
end
```

Listing B.3: computeSnrGrid.m

```
function [snrGrid, model] = computeSnrGrid(wavelength_m, pixelSize_m, quantumEfficiency, ...
    transmittance, groundSampleDistance_m, obstructionRatio, ...
    altitudes_km, pupilDiameters_mm, snrRequirement)
%COMPUTESNRGRID Evaluate the historical SNR model without figures or files.
% Rows correspond to altitudes_km; columns correspond to pupilDiameters_mm.
% Values below snrRequirement become NaN. The threshold itself is accepted.
validateOpticalInputs(wavelength_m, pixelSize_m, groundSampleDistance_m, ...
    obstructionRatio, altitudes_km, pupilDiameters_mm);
validateattributes(quantumEfficiency, {'numeric'}, {'scalar','real','finite','>=',0,'<=',1});
validateattributes(transmittance, {'numeric'}, {'scalar','real','finite','>=',0,'<=',1});
validateattributes(snrRequirement, {'numeric'}, {'scalar','real','finite','nonnegative'});

% Keep axis-dependent quantities as a column or row until they are combined.
altitudes_m = altitudes_km(:) * 1e3;
pupilDiameters_m = pupilDiameters_mm(:).' * 1e-3;
focalLengths_m = pixelSize_m * altitudes_m / groundSampleDistance_m;
orbitalSpeeds_m_s = sqrt(3.986e14 ./ (6371e3 + altitudes_m));
integrationTimes_s = groundSampleDistance_m ./ orbitalSpeeds_m_s;

% Preserve the original model constants. Radiance is specified per micrometre,
% so the historical bandwidth value 20e-3 is expressed here in micrometres.
model.spectralBandwidth_um = 20e-3;
model.referenceRadiance_W_m2_sr_um = 100;
model.tdiStages = 1;
model.snrMargin = 0.9;
model.noiseRms_e = struct('dark',50, 'read',100, 'preamplifier',5, ...
    'video',10, 'jitter',5, 'electromagnetic',5, 'quantization',2, 'nonlinearity',2);
planckConstant_J_s = 6.626e-34;
speedOfLight_m_s = 3e8;

% Implicit expansion combines an altitude column with a diameter row.
effectiveDiameters_m = sqrt(pupilDiameters_m.^2 * (1 - obstructionRatio^2));
focalRatios = focalLengths_m ./ effectiveDiameters_m;
irradiance_W_m2 = (pi * transmittance * model.spectralBandwidth_um * ...
    model.referenceRadiance_W_m2_sr_um) ./ (1 + 4 * focalRatios.^2);
pixelArea_m2 = pixelSize_m^2;
photoelectrons = (irradiance_W_m2 * pixelArea_m2 * quantumEfficiency * ...
    wavelength_m * model.tdiStages .* integrationTimes_s) / ...
    (planckConstant_J_s * speedOfLight_m_s);

% Independent noise variances add; photoelectrons supply shot-noise variance.
noise = model.noiseRms_e;
fixedNoiseVariance_e2 = noise.dark^2 + noise.read^2 + noise.preamplifier^2 + ...
    noise.video^2 + noise.jitter^2 + noise.electromagnetic^2 + ...
    noise.quantization^2 + noise.nonlinearity^2;
totalNoiseRms_e = sqrt(fixedNoiseVariance_e2 + photoelectrons);
snrGrid = model.snrMargin * photoelectrons ./ totalNoiseRms_e;
snrGrid(snrGrid < snrRequirement) = NaN;
end
```

Wrapper para el cálculo del tiempo de revisita

```
function [coverageGrid_days, diagnostics] = CoverageRevisitCalc( ...
    groundSampleDistance_m, altitudes_km, swaths_km, ...
    revisitRequirement_days, detectorPixelCount, telescopeFov_deg, ...
    satelliteCount, telescopeCount, detectorIndex, telescopeName, ...
    overlapFraction, cloudFraction, maxDetectorCount, writeOutputs)
%COVERAGEREVISITCALC Preserve positional calls and filenames used by master.m.
% Set writeOutputs=false to obtain numerical results without creating files.
% Solver failures are returned in diagnostics and reported without muting warnings.
if nargin < 14, writeOutputs = true; end
[coverageGrid_days, diagnostics] = computeCoverageGrid( ...
    groundSampleDistance_m, altitudes_km, swaths_km, ...
    revisitRequirement_days, detectorPixelCount, telescopeFov_deg, ...
    satelliteCount, telescopeCount, overlapFraction, cloudFraction, maxDetectorCount);
solverErrorCount = nnz(~cellfun(@isempty, diagnostics.solverErrors));
if solverErrorCount > 0
    warning('eo:CoverageSolverFailure', ...
        '%d coverage cells failed. Inspect diagnostics.solverErrors for details.', solverErrorCount);
end
if writeOutputs
    configurationName = sprintf('%d Satelite(s); %d Telescopio(s): %s; Detector %d', ...
        satelliteCount, telescopeCount, strrep(telescopeName, ' ', ''), detectorIndex);
    writeCoverageResult(coverageGrid_days, altitudes_km, swaths_km, ...
        revisitRequirement_days, diagnostics, configurationName);
end
end
```

Listing B.4: computeCoverageGrid.m

```
function [coverageGrid_days, diagnostics] = computeCoverageGrid( ...
    groundSampleDistance_m, altitudes_km, swaths_km, ...
    revisitRequirement_days, detectorPixelCount, telescopeFov_deg, ...
    satelliteCount, telescopeCount, overlapFraction, ...
    cloudFraction, maxDetectorCount, revisitSolver)
%COMPUTECOVERAGEGRID Evaluate analytical revisit coverage without file exports.
% Rows correspond to altitudes_km; columns correspond to swaths_km.
% Geometry is vectorized. The external revisit solver is called per valid cell.
% Rejected cells and solver errors remain NaN; diagnostics distinguish them.
% An optional solver handle permits tests without running the external toolkit.
if nargin < 12, revisitSolver = @RevisitCalc; end
validateCoverageInputs(groundSampleDistance_m, altitudes_km, swaths_km, ...
    revisitRequirement_days, detectorPixelCount, telescopeFov_deg, ...
    satelliteCount, telescopeCount, overlapFraction, cloudFraction, maxDetectorCount);
validateattributes(revisitSolver, {'function_handle'}, {'scalar'});

% J2 nodal precession equals the apparent solar motion.
earthRadius_m = 6378.137e3;
earthGravitationalParameter_m3_s2 = 3.986004418e14;
earthJ2 = 1.082629989052e-3;
sunSynchronousNodalRate_rad_s = 360/(365.2421897*24*3600) * (pi/180);
targetLatitude_deg = 25;
solverDayLimits_days = [1, revisitRequirement_days];
% Named adapter arguments preserve the historical external-solver setup.
useFovConstraint = 1;
useElevationConstraint = 0;
orbitalPlaneCount = 1;
relativePlaneSpacing = 0;
includeDescendingPasses = 1;
useOneSidedViewing = 0;

% Orbital quantities depend only on altitude, not on the selected swath.
altitudes_km = altitudes_km(:);
swaths_km = swaths_km(:).';
semimajorAxes_m = earthRadius_m + altitudes_km * 1000;
meanMotions_rad_s = sqrt(earthGravitationalParameter_m3_s2 ./ semimajorAxes_m.^3);
inclinationCosines = -sunSynchronousNodalRate_rad_s ./ ...
    (1.5 * meanMotions_rad_s * earthJ2 .* (earthRadius_m ./ semimajorAxes_m).^2);
sunSynchronousMask = abs(inclinationCosines) <= 1;
inclinations_rad = nan(size(altitudes_km));
inclinations_rad(sunSynchronousMask) = acos(inclinationCosines(sunSynchronousMask));

% Keep the historical two-telescope FOV rule and detector-width constraint.
effectiveFov_deg = telescopeFov_deg;
if telescopeCount == 2, effectiveFov_deg = 2 * telescopeFov_deg; end
maxSwathByFov_km = 2 * altitudes_km * tand(effectiveFov_deg/2);
detectorSwathLimits_km = (1:maxDetectorCount) * ...
    groundSampleDistance_m * detectorPixelCount / 1000;
% Preserve the original operation order for the admissibility threshold.
maxSwathByDetector_km = maxDetectorCount * groundSampleDistance_m * detectorPixelCount / 1000;
geometryMask = swaths_km <= maxSwathByFov_km & ...
    swaths_km <= maxSwathByDetector_km & sunSynchronousMask;
effectiveSwaths_km = swaths_km * (1 - overlapFraction);
halfFovAngles_deg = atand(effectiveSwaths_km ./ (2 * altitudes_km));

coverageGrid_days = nan(size(geometryMask));
```

```
diagnostics.geometryFeasible = geometryMask;
diagnostics.sunSynchronousFeasible = sunSynchronousMask;
diagnostics.maxSwathByFov_km = maxSwathByFov_km;
diagnostics.detectorSwathLimits_km = detectorSwathLimits_km;
diagnostics.requirementExceeded = false(size(geometryMask));
diagnostics.nonfiniteSolverResult = false(size(geometryMask));
diagnostics.solverErrors = cell(size(geometryMask));
diagnostics.solverErrorIdentifiers = cell(size(geometryMask));

% Transpose before find to preserve the original altitude-then-swath order.
% Each iteration adapts one valid pair to the scalar external solver API.
[swathIndices, altitudeIndices] = find(geometryMask.');
for caseIndex = 1:numel(altitudeIndices)
    altitudeIndex = altitudeIndices(caseIndex);
    swathIndex = swathIndices(caseIndex);
    orbitalElements = [semimajorAxes_m(altitudeIndex), 0, ...
        inclinations_rad(altitudeIndex), 0, 0]; % [a, eccentricity, i, RAAN, AoP]
    try
        revisit_days = revisitSolver(orbitalElements, targetLatitude_deg, ...
            useFovConstraint, useElevationConstraint, satelliteCount, ...
            orbitalPlaneCount, relativePlaneSpacing, includeDescendingPasses, ...
            useOneSidedViewing, solverDayLimits_days, ...
            halfFovAngles_deg(altitudeIndex, swathIndex));
        validateattributes(revisit_days, {'numeric'}, {'real','scalar'});
    catch solverError
        diagnostics.solverErrors{altitudeIndex, swathIndex} = solverError.message;
        diagnostics.solverErrorIdentifiers{altitudeIndex, swathIndex} = solverError.identifier;
        continue;
    end

    % Keep our result processing outside the solver failure boundary.
    correctedRevisit_days = revisit_days / (1 - cloudFraction);
    diagnostics.nonfiniteSolverResult(altitudeIndex, swathIndex) = ~isfinite(correctedRevisit_days);
    if correctedRevisit_days > revisitRequirement_days
        diagnostics.requirementExceeded(altitudeIndex, swathIndex) = true;
    else
        coverageGrid_days(altitudeIndex, swathIndex) = correctedRevisit_days;
    end
end
end

function validateCoverageInputs(groundSampleDistance_m, altitudes_km, swaths_km, ...
    revisitRequirement_days, detectorPixelCount, telescopeFov_deg, ...
    satelliteCount, telescopeCount, overlapFraction, cloudFraction, maxDetectorCount)
% RevisitCalc advances in whole days and compares its horizon by equality.
validateattributes(revisitRequirement_days, {'numeric'}, {'scalar','real','positive','finite'});
if revisitRequirement_days ~= fix(revisitRequirement_days)
    error('eo:WholeDayHorizon', 'The current revisit solver requires a whole-day horizon.');
end
validateattributes(groundSampleDistance_m, {'numeric'}, {'scalar','real','positive','finite'});
validateattributes(altitudes_km, {'numeric'}, {'vector','nonempty','real','positive','finite'});
validateattributes(swaths_km, {'numeric'}, {'vector','nonempty','real','positive','finite'});
validateattributes(detectorPixelCount, {'numeric'}, {'scalar','integer','positive','finite'});
validateattributes(telescopeFov_deg, {'numeric'}, {'scalar','real','positive','finite','<',90});
validateattributes(satelliteCount, {'numeric'}, {'scalar','integer','positive','finite'});
validateattributes(telescopeCount, {'numeric'}, {'scalar','integer','>=',1,'<=',2});
validateattributes(overlapFraction, {'numeric'}, {'scalar','real','finite','>=',0,'<',1});
validateattributes(cloudFraction, {'numeric'}, {'scalar','real','finite','>=',0,'<',1});
validateattributes(maxDetectorCount, {'numeric'}, {'scalar','integer','positive','finite'});
end
```

Listing B.5: selectRegionalSwath.m

```
function [width_km,worstRevisit_days] = selectRegionalSwath(altitude_km,satelliteCount,telescopeCount,widths_km,params,cache)
%SELECTREGIONALSWATH Apply the sampled CONUS requirement before mass ranking.
width_km = NaN; worstRevisit_days = NaN;
for candidate_km = widths_km(:).'
    acquired_km = telescopeCount*ceil(candidate_km*1000/(params.GSD*telescopeCount))*params.GSD/1000;
    key = sprintf('%.17g_', [altitude_km,satelliteCount,acquired_km, ...
        params.Cov_Requirement,params.solapamiento,params.cobertura_nubes]);
    if isKey(cache,key)
        worst = cache(key);
    else
        worst = regionalRevisit(altitude_km,satelliteCount,acquired_km,params);
        cache(key) = worst;
    end
    if worst <= params.Cov_Requirement
        width_km = candidate_km; worstRevisit_days = worst; return;
    end
end
end

function worst = regionalRevisit(altitude_km,satelliteCount,acquired_km,params)
a = 6378.137e3+altitude_km*1000;
n = sqrt(3.986004418e14/a^3);
sunRate = 2*pi/(365.2421897*86400);
inc = acos(-sunRate/(1.5*n*1.082629989052e-3*(6378.137e3/a)^2));
psi = atand((1-params.solapamiento)*acquired_km/(2*altitude_km));
worst = 0;
```

```
for latitude = 25:2:49
    revisit = params.regionalRevisitSolver([a,0,inc,0,0],latitude,1,0, ...
        satelliteCount,1,0,1,0,[1,params.Cov_Requirement],psi)/(1-params.cobertura_nubes);
    assert(isfinite(revisit) && revisit > 0,'eo:RegionalRevisit','Invalid regional revisit.');
    worst = max(worst,revisit);
    if worst > params.Cov_Requirement, return; end
end
end
```

Determinación de mínimos. h vs D_{min}

```
function selection = CrossDataFunction(altitudes_km, swaths_km, pupilDiameters_mm, ...
    telescopeNames, telescopeFov_deg, satelliteConfigurations, mtfRequirement, ...
    snrRequirement, revisitRequirement_days, analysisResults, writeOutputs)
%CROSSDATAFUNCTION Compatibility adapter for selection, import and presentation.
if nargin < 10, analysisResults = []; end
if nargin < 11, writeOutputs = true; end
if isempty(analysisResults)
    analysisResults = importAnalysisResults(altitudes_km,pupilDiameters_mm,swaths_km,telescopeNames,satelliteConfigurations);
else
    validateAnalysisAxes(analysisResults,altitudes_km,pupilDiameters_mm,swaths_km,telescopeNames,satelliteConfigurations);
end
selection = computeSelectionResults(analysisResults,telescopeFov_deg,mtfRequirement,snrRequirement,revisitRequirement_days);
if writeOutputs
    requirements = struct('mtfRequirement',mtfRequirement,'snrRequirement',snrRequirement, ...
        'revisitRequirement_days',revisitRequirement_days);
    writeSelectionResults(selection,altitudes_km,telescopeNames,satelliteConfigurations,requirements);
end
end
```

Listing B.6: computeSelectionResults.m

```
function selection = computeSelectionResults(data, telescopeFov_deg, mtfRequirement, snrRequirement, revisitRequirement_days)
%COMPUTESELECTIONRESULTS Select designs from explicit matrices without file access.
studyAxes = data.axes;
configurationCount = size(studyAxes.satelliteConfigurations,1);
telescopeCount = numel(studyAxes.telescopeNames);
selection = cell(configurationCount,1);
for configurationIndex = 1:configurationCount
    selection{configurationIndex}.viableDiameters_mm = nan(numel(studyAxes.altitudes_km),telescopeCount,3);
    selection{configurationIndex}.theoreticalDiameters_mm = selection{configurationIndex}.viableDiameters_mm;
end
cases = analysisCases(configurationCount,3,telescopeCount);
for caseIndex = 1:size(cases,1)
    configurationIndex = cases(caseIndex,1); detectorIndex = cases(caseIndex,2); telescopeIndex = cases(caseIndex,3);
    [selection{configurationIndex}.viableDiameters_mm(:,telescopeIndex,detectorIndex), ...
        selection{configurationIndex}.theoreticalDiameters_mm(:,telescopeIndex,detectorIndex)] = ...
        selectMinimumDiameters(data.optics.mtf{2,detectorIndex,telescopeIndex}, ...
        data.optics.snr{3,detectorIndex,telescopeIndex},data.coverage.days{configurationIndex,detectorIndex,telescopeIndex}, ...
        studyAxes.altitudes_km,studyAxes.pupilDiameters_mm,studyAxes.swaths_km,telescopeFov_deg(telescopeIndex), ...
        studyAxes.satelliteConfigurations(configurationIndex,2),telescopeIndex,mtfRequirement,snrRequirement,revisitRequirement_days);
end
end
```

Listing B.7: selectMinimumDiameters.m

```
function [viableDiameters_mm, theoreticalDiameters_mm] = selectMinimumDiameters( ...
    mtfGrid, snrGrid, coverageGrid_days, altitudes_km, pupilDiameters_mm, ...
    swaths_km, telescopeFov_deg, telescopeCount, telescopeIndex, ...
    mtfRequirement, snrRequirement, revisitRequirement_days)
%SELECTMINIMUMDIAMETERS Preserve the original first-feasible input-order policy.
validateAnalysisMatrices(mtfGrid,snrGrid,coverageGrid_days,altitudes_km,pupilDiameters_mm,swaths_km);
[hasDiameter,diameterIndices] = max(mtfGrid >= mtfRequirement & snrGrid >= snrRequirement,[],2);
pupilDiameters_mm = pupilDiameters_mm(:);
theoreticalDiameters_mm = pupilDiameters_mm(diameterIndices);
theoreticalDiameters_mm(~hasDiameter) = NaN;
[hasSwath,swathIndices] = max(coverageGrid_days <= revisitRequirement_days & ~isnan(coverageGrid_days),[],2);
swaths_km = swaths_km(:);
selectedSwaths_km = swaths_km(swathIndices);
effectiveFov_deg = telescopeFov_deg * (1 + (telescopeCount == 2));
actualFov_deg = 2 * atand(selectedSwaths_km ./ (2 * altitudes_km(:)));
viableDiameters_mm = theoreticalDiameters_mm;
viableDiameters_mm(~hasSwath | actualFov_deg > effectiveFov_deg) = NaN;
if telescopeIndex == 1, viableDiameters_mm(viableDiameters_mm > 200) = NaN; end
end
```

Cálculo de masa seca

```
function [dryMass_kg,thematicMapperDryMass_kg,seosatDryMass_kg, ...
    instrumentLength_m,instrumentArea_m2,instrumentVolume_m3, ...
    instrumentMass_kg,instrumentPower_W,satelliteVolume_m3, ...
    satelliteLength_m,satelliteArea_m2,cubesatUnits] = calcularMasaSeca(pupilDiameters_mm,telescopeCount)
%CALCULARMASASECA Preserve the historical positional output interface.
mass = computeDryMass(pupilDiameters_mm,telescopeCount);
dryMass_kg = mass.dryMass_kg;
thematicMapperDryMass_kg = mass.thematicMapperDryMass_kg;
seosatDryMass_kg = mass.seosatDryMass_kg;
instrumentLength_m = mass.instrumentLength_m;
instrumentArea_m2 = mass.instrumentArea_m2;
instrumentVolume_m3 = mass.instrumentVolume_m3;
instrumentMass_kg = mass.instrumentMass_kg;
instrumentPower_W = mass.instrumentPower_W;
satelliteVolume_m3 = mass.satelliteVolume_m3;
satelliteLength_m = mass.satelliteLength_m;
satelliteArea_m2 = mass.satelliteArea_m2;
cubesatUnits = mass.cubesatUnits;
end
```

Listing B.8: computeDryMass.m

```
function mass = computeDryMass(pupilDiameters_mm,telescopeCount)
%COMPUTEDRYMASS Evaluate the original instrument-scaling and dry-mass laws.
% Output arrays preserve the diameter shape. Two telescopes increase instrument
% mass by 1.5; reference estimates and remaining dimensions retain their formulas.
validateattributes(pupilDiameters_mm,{'numeric'},{'vector','nonempty','real','finite','positive'});
validateattributes(telescopeCount,{'numeric'},{'scalar','integer','>=',1,'<=',2});
thematicMapperAperture_mm = 400;
thematicMapperLength_m = 1;
thematicMapperMass_kg = 240;
thematicMapperPower_W = 280;
seosatAperture_mm = 250;
seosatLength_m = 1;
seosatMass_kg = 100;
seosatPower_W = 100;
[tmLength_m, tmArea_m2, tmVolume_m3, tmMass_kg, tmPower_W] = scaleInstrument(pupilDiameters_mm, thematicMapperAperture_mm, thematicMapperLength_m,
      thematicMapperMass_kg, thematicMapperPower_W);
[seosatLengthScaled_m, seosatArea_m2, seosatVolume_m3, seosatMassScaled_kg, seosatPowerScaled_W] = scaleInstrument(pupilDiameters_mm,
      seosatAperture_mm, seosatLength_m, seosatMass_kg, seosatPower_W);
instrumentLength_m = (tmLength_m + seosatLengthScaled_m) / 2;
instrumentArea_m2 = (tmArea_m2 + seosatArea_m2) / 2;
instrumentVolume_m3 = (tmVolume_m3 + seosatVolume_m3) / 2;
instrumentMass_kg = (tmMass_kg + seosatMassScaled_kg) / 2;
instrumentPower_W = (tmPower_W + seosatPowerScaled_W) / 2;
if telescopeCount == 2
    instrumentMass_kg = instrumentMass_kg * 1.5;
end
thematicMapperDryMass_kg = 4 * tmMass_kg;
seosatDryMass_kg = 4 * seosatMassScaled_kg;
dryMass_kg = 4 * instrumentMass_kg;
satelliteDensity_kg_m3 = 79;
satelliteVolume_m3 = dryMass_kg ./ satelliteDensity_kg_m3;
satelliteLength_m = satelliteVolume_m3.^(1/3);
satelliteArea_m2 = satelliteLength_m.^2;
cubesatUnits = satelliteVolume_m3 / (0.1^3);
mass = struct('dryMass_kg',dryMass_kg,'thematicMapperDryMass_kg',thematicMapperDryMass_kg, ...
    'seosatDryMass_kg',seosatDryMass_kg,'instrumentLength_m',instrumentLength_m, ...
    'instrumentArea_m2',instrumentArea_m2,'instrumentVolume_m3',instrumentVolume_m3, ...
    'instrumentMass_kg',instrumentMass_kg,'instrumentPower_W',instrumentPower_W, ...
    'satelliteVolume_m3',satelliteVolume_m3,'satelliteLength_m',satelliteLength_m, ...
    'satelliteArea_m2',satelliteArea_m2,'cubesatUnits',cubesatUnits);
end
function [length_m, area_m2, volume_m3, mass_kg, power_W] = scaleInstrument(aperture_mm, referenceAperture_mm, referenceLength_m, referenceMass_kg,
      referencePower_W)
    apertureRatio = aperture_mm / referenceAperture_mm;
    smallApertureFactor = ones(size(apertureRatio));
    smallApertureFactor(apertureRatio <= 0.5) = 2;
    length_m = apertureRatio .* referenceLength_m;
    area_m2 = length_m.^2;
    volume_m3 = length_m.^3;
    mass_kg = (apertureRatio.^3) .* referenceMass_kg .* smallApertureFactor;
    power_W = (apertureRatio.^3) .* referencePower_W .* smallApertureFactor;
end
```

Calculo de masa total

```
function [totalMass_kg,fuelMass_kg,impulseCounts,totalDeltaV_m_s, ...
    cycleTimes_s,cycleDeltaV_m_s,unmarginedFuelMass_kg] = calcularMasaTotal( ...
    altitudes_km,dryMass_kg,dragAreas_m2,maintenanceLowerFactor,maintenanceUpperFactor)
%CALCULARMASATOTAL Preserve the historical positional output interface.
if nargin < 4, maintenanceLowerFactor = 0.98; end
if nargin < 5, maintenanceUpperFactor = 1.02; end
mass = computeTotalMass(altitudes_km,dryMass_kg,dragAreas_m2,maintenanceLowerFactor,maintenanceUpperFactor);
totalMass_kg = mass.totalMass_kg; fuelMass_kg = mass.fuelMass_kg;
impulseCounts = mass.impulseCounts; totalDeltaV_m_s = mass.totalDeltaV_m_s;
cycleTimes_s = mass.cycleTimes_s; cycleDeltaV_m_s = mass.cycleDeltaV_m_s;
unmarginedFuelMass_kg = mass.unmarginedFuelMass_kg;
end
```

Listing B.9: computeTotalMass.m

```
function mass = computeTotalMass(altitudes_km, dryMass_kg, dragAreas_m2, ...
    maintenanceLowerFactor, maintenanceUpperFactor)
%COMPUTETOTALMASS Preserve the eight-year drag/maintenance/propellant model.
% Each counted cycle is a complete two-burn Hohmann manoeuvre.
% Arrays follow the altitude input shape. Scalar mass/area inputs are reused.
% Atmospheric constants, integration tolerances, margins and cutoff are unchanged.
    earthGravitationalParameter_m3_s2 = 3.986004418e14;
    earthRadius_m = 6378e3;
    dragCoefficient = 2.5;
    specificImpulse_s = 220;
    standardGravity_m_s2 = 9.80665;
    missionDuration_s = 8 * 365.25 * 24 * 3600;
    if nargin < 4, maintenanceLowerFactor = 0.98; end
    if nargin < 5, maintenanceUpperFactor = 1.02; end
    validateattributes(altitudes_km,{'numeric'},{'vector','nonempty','real','finite','positive'});
    validateattributes(dryMass_kg,{'numeric'},{'vector','real','finite','positive'});
    validateattributes(dragAreas_m2,{'numeric'},{'vector','real','finite','nonnegative'});
    assert((isscalar(dryMass_kg) || numel(dryMass_kg)==numel(altitudes_km)) && ...
        (isscalar(dragAreas_m2) || numel(dragAreas_m2)==numel(altitudes_km)), ...
        'eo:MassInputSize','Mass and drag area must be scalar or match the altitude count.');
    validateattributes(maintenanceLowerFactor,{'numeric'},{'scalar','real','finite','positive'});
    validateattributes(maintenanceUpperFactor,{'numeric'},{'scalar','real','finite','positive'});
    impulseCounts = zeros(size(altitudes_km));
    totalDeltaV_m_s = zeros(size(altitudes_km));
    fuelMass_kg = zeros(size(altitudes_km));
    totalMass_kg = zeros(size(altitudes_km));
    cycleTimes_s = zeros(size(altitudes_km));
    cycleDeltaV_m_s = zeros(size(altitudes_km));
    unmarginedFuelMass_kg = zeros(size(altitudes_km));
    % Each case requires separate piecewise numerical quadrature.
    for altitudeIndex = 1:numel(altitudes_km)
        altitude_km = altitudes_km(altitudeIndex);
        nominalAltitude_m = altitude_km * 1e3;
        lowerAltitude_m = maintenanceLowerFactor * nominalAltitude_m;
        upperAltitude_m = maintenanceUpperFactor * nominalAltitude_m;
        caseDryMass_kg = getIndexedValue(dryMass_kg, altitudeIndex);
        dragAreaMargin = 1.2;
        dragArea_m2 = getIndexedValue(dragAreas_m2, altitudeIndex) * dragAreaMargin;
        cycleTime_s = decayCycleTime(lowerAltitude_m, upperAltitude_m, earthRadius_m, earthGravitationalParameter_m3_s2, dragCoefficient,
      dragArea_m2, caseDryMass_kg);
        if isfinite(cycleTime_s) && cycleTime_s > 0
            impulseCount = floor((missionDuration_s - eps(missionDuration_s)) / cycleTime_s);
        else
            impulseCount = 0;
        end
        lowerRadius_m = earthRadius_m + lowerAltitude_m;
        upperRadius_m = earthRadius_m + upperAltitude_m;
        deltaV_m_s = hohmannDeltaV(lowerRadius_m, upperRadius_m, earthGravitationalParameter_m3_s2);
        caseTotalDeltaV_m_s = impulseCount * deltaV_m_s;
        impulseCounts(altitudeIndex) = impulseCount;
        totalDeltaV_m_s(altitudeIndex) = caseTotalDeltaV_m_s;
        unmarginedFuel_kg = caseDryMass_kg * (exp(caseTotalDeltaV_m_s / (specificImpulse_s * standardGravity_m_s2)) - 1);
        unmarginedFuelMass_kg(altitudeIndex) = unmarginedFuel_kg;
        fuelMass_kg(altitudeIndex) = unmarginedFuel_kg * 1.1;
        totalMass_kg(altitudeIndex) = fuelMass_kg(altitudeIndex) + caseDryMass_kg;
        cycleTimes_s(altitudeIndex) = cycleTime_s;
        cycleDeltaV_m_s(altitudeIndex) = deltaV_m_s;
        if totalMass_kg(altitudeIndex) > 100000
            totalMass_kg(altitudeIndex) = NaN;
            fuelMass_kg(altitudeIndex) = NaN;
            unmarginedFuelMass_kg(altitudeIndex) = NaN;
        end
    end
    mass = struct('totalMass_kg',totalMass_kg,'fuelMass_kg',fuelMass_kg, ...
        'impulseCounts',impulseCounts,'totalDeltaV_m_s',totalDeltaV_m_s, ...
        'cycleTimes_s',cycleTimes_s,'cycleDeltaV_m_s',cycleDeltaV_m_s, ...
        'unmarginedFuelMass_kg',unmarginedFuelMass_kg);

end
    function value = getIndexedValue(values, valueIndex)
```

```
    if isscalar(values)
        value = values;
    else
        value = values(valueIndex);
    end
end
function decayTime_s = decayCycleTime(lowerAltitude_m, upperAltitude_m, earthRadius_m, earthGravitationalParameter_m3_s2, dragCoefficient,
  dragArea_m2, satelliteMass_kg)
    if upperAltitude_m <= lowerAltitude_m || dragCoefficient <= 0 || dragArea_m2 <= 0 || satelliteMass_kg <= 0
        decayTime_s = Inf;
        return;
    end
    [baseAltitudes_km, ~, ~] = atmosphereTable();
    lowerAltitude_km = lowerAltitude_m / 1e3;
    upperAltitude_km = upperAltitude_m / 1e3;
    splitAltitudes_km = baseAltitudes_km(baseAltitudes_km > lowerAltitude_km & baseAltitudes_km < upperAltitude_km);
    intervalBounds_m = [lowerAltitude_m, splitAltitudes_km * 1e3, upperAltitude_m];
    decayTime_s = 0;
    % Preserve separate integrations at atmospheric discontinuities.
    for intervalIndex = 1:(numel(intervalBounds_m) - 1)
        intervalStart_m = intervalBounds_m(intervalIndex);
        intervalEnd_m = intervalBounds_m(intervalIndex + 1);
        if intervalEnd_m <= intervalStart_m
            continue;
        end
        integrand = @(altitude_m) satelliteMass_kg ./ (dragCoefficient .* dragArea_m2 .* getAtmosphericDensity(altitude_m) .* sqrt(
  earthGravitationalParameter_m3_s2 .* (earthRadius_m + altitude_m)));
        decayTime_s = decayTime_s + integral(integrand, intervalStart_m, intervalEnd_m, ...
            'RelTol', 1e-8, 'AbsTol', 1e-3, 'ArrayValued', true);
    end
end
function deltaV_m_s = hohmannDeltaV(lowerRadius_m, upperRadius_m, earthGravitationalParameter_m3_s2)
    transferSemimajorAxis_m = (lowerRadius_m + upperRadius_m) / 2;
    lowerCircularSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 / lowerRadius_m);
    transferPerigeeSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 * (2/lowerRadius_m - 1/transferSemimajorAxis_m));
    firstDeltaV_m_s = transferPerigeeSpeed_m_s - lowerCircularSpeed_m_s;
    upperCircularSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 / upperRadius_m);
    transferApogeeSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 * (2/upperRadius_m - 1/transferSemimajorAxis_m));
    secondDeltaV_m_s = upperCircularSpeed_m_s - transferApogeeSpeed_m_s;
    deltaV_m_s = firstDeltaV_m_s + secondDeltaV_m_s;
end
function density_kg_m3 = getAtmosphericDensity(altitude_m)
    altitude_km = max(altitude_m / 1e3, 0);
    [baseAltitudes_km, baseDensities_kg_m3, scaleHeights_km] = atmosphereTable();
    % Count lower breakpoints to reproduce find(...,1,'last') for every sample.
    % Column indexing avoids accidental row/column expansion of table values.
    originalShape = size(altitude_km);
    samples_km = altitude_km(:);
    layerIndices = max(sum(samples_km >= baseAltitudes_km,2),1);
    baseDensities_kg_m3 = baseDensities_kg_m3(:);
    baseAltitudes_km = baseAltitudes_km(:);
    scaleHeights_km = scaleHeights_km(:);
    density_kg_m3 = baseDensities_kg_m3(layerIndices) .* ...
        exp(-(samples_km-baseAltitudes_km(layerIndices)) ./ scaleHeights_km(layerIndices));
    density_kg_m3 = reshape(density_kg_m3,originalShape);
end
function [baseAltitudes_km, baseDensities_kg_m3, scaleHeights_km] = atmosphereTable()
    % Original Vallado/Curtis piecewise exponential table, consistent with USSA76.
    baseAltitudes_km = [0, 25, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, ...
                 130, 140, 150, 180, 200, 250, 300, 350, 400, 450, ...
                 500, 600, 700, 800, 900, 1000];
    baseDensities_kg_m3 = [1.225, 3.899e-2, 1.774e-2, 3.972e-3, 1.057e-3, ...
                3.206e-4, 8.770e-5, 1.905e-5, 3.396e-6, 5.297e-7, ...
                9.661e-8, 2.438e-8, 8.484e-9, 3.845e-9, 2.070e-9, ...
                5.464e-10, 2.789e-10, 7.248e-11, 2.418e-11, ...
                9.518e-12, 3.725e-12, 1.585e-12, 6.967e-13, ...
                1.454e-13, 3.614e-14, 1.170e-14, 5.245e-15, 3.019e-15];
    scaleHeights_km = [7.249, 6.349, 6.682, 7.554, 8.382, 7.714, 6.549, ...
            5.799, 5.382, 5.877, 7.263, 9.473, 12.636, 16.149, ...
            22.523, 29.740, 37.105, 45.546, 53.628, 53.298, ...
            58.515, 60.828, 63.822, 71.835, 88.667, 124.64, ...
            181.05, 268.00];
end
```

```
function summary = ConfigAnalysis(params, analysisResults, writeOutputs)
%CONFIGANALYSIS Compatibility adapter for input, mass calculations and reports.
if nargin < 2, analysisResults = []; end
if nargin < 3, writeOutputs = true; end
if isempty(analysisResults)
    analysisResults = importAnalysisResults(params.alturas_orbitales,params.diametros_pupila, ...
        params.swaths_km,params.telescope_names,params.satellite_configs);
    analysisResults.selection = computeSelectionResults(analysisResults,params.fov_limit, ...
        params.MTF_Requirement,params.SNR_Requirement,params.Cov_Requirement);
end
validateAnalysisAxes(analysisResults,params.alturas_orbitales,params.diametros_pupila, ...
    params.swaths_km,params.telescope_names,params.satellite_configs);
cases = analysisCases(size(params.satellite_configs,1),3,numel(params.telescope_names));
for caseIndex = 1:size(cases,1)
    c = cases(caseIndex,1); d = cases(caseIndex,2); t = cases(caseIndex,3);
    validateAnalysisMatrices(analysisResults.optics.mtf{2,d,t},analysisResults.optics.snr{3,d,t}, ...
        analysisResults.coverage.days{c,d,t},params.alturas_orbitales,params.diametros_pupila,params.swaths_km);
end
summary = computeConfigurationSummary(params,analysisResults);
if writeOutputs, writeConfigurationSummary(summary); end
end
```

Listing B.10: computeConfigurationSummary.m

```
function summary = computeConfigurationSummary(parameters, analysisResults)
%COMPUTECONFIGURATIONSUMMARY Preserve candidate equations, row order and mass ties.
% Case enumeration removes nested metadata loops; cached integrations are reused.
if ~isfield(parameters,'maintenance_lower_factor'), parameters.maintenance_lower_factor = 0.98; end
if ~isfield(parameters,'maintenance_upper_factor'), parameters.maintenance_upper_factor = 1.02; end
if ~isfield(parameters,'solapamiento'), parameters.solapamiento = 0.05; end
if ~isfield(parameters,'cobertura_nubes'), parameters.cobertura_nubes = 1/6; end
if ~isfield(parameters,'regionalRevisitSolver'), parameters.regionalRevisitSolver = @RevisitCalc; end
regionalCache = containers.Map('KeyType','char','ValueType','double');
cases = analysisCases(size(parameters.satellite_configs,1),3,numel(parameters.telescope_names),numel(parameters.alturas_orbitales));
rows = cell(2*size(cases,1),numel(configurationHeaders()));
rowCount = 0; viableCases = false(size(cases,1),1);
viableKeys = containers.Map('KeyType','char','ValueType','logical');
massCache = containers.Map('KeyType','char','ValueType','any');
for caseIndex = 1:size(cases,1)
    indices = cases(caseIndex,:);
    diameter_mm = analysisResults.selection{indices(1)}.viableDiameters_mm(indices(4),indices(3),indices(2));
    if isnan(diameter_mm), continue; end
    row = viableRow(parameters,analysisResults,indices,diameter_mm,massCache,regionalCache);
    if isempty(row), continue; end
    rowCount = rowCount+1; rows(rowCount,:) = row; viableCases(caseIndex) = true;
    viableKeys(candidateKey(parameters,indices)) = true;
end
for caseIndex = 1:size(cases,1)
    if viableCases(caseIndex), continue; end
    indices = cases(caseIndex,:);
    if isKey(viableKeys,candidateKey(parameters,indices)), continue; end
    diameter_mm = analysisResults.selection{indices(1)}.theoreticalDiameters_mm(indices(4),indices(3),indices(2));
    if isnan(diameter_mm), continue; end
    row = theoreticalRow(parameters,indices,diameter_mm,massCache);
    if isempty(row), continue; end
    rowCount = rowCount+1; rows(rowCount,:) = row;
end
summary.rows = rows(1:rowCount,:);
summary.best = struct('mass',Inf);
if any(viableCases)
    [summary.best.mass,bestIndex] = min(cell2mat(summary.rows(1:sum(viableCases),29)));
    summary.best.data = summary.rows(bestIndex,:);
end
summary.table = cell2table(summary.rows,'VariableNames',configurationHeaders());
summary.maintenance = computeMaintenanceComparison(summary.rows);
end
function row = viableRow(parameters, analysisResults, indices, pupilDiameter_mm, massCache, regionalCache)
configurationIndex = indices(1); detectorIndex = indices(2); telescopeIndex = indices(3); altitudeIndex = indices(4);
satelliteCount = parameters.satellite_configs(configurationIndex,1); telescopeCount = parameters.satellite_configs(configurationIndex,2);
altitude_km = parameters.alturas_orbitales(altitudeIndex);
coverageRow_days = analysisResults.coverage.days{configurationIndex,detectorIndex,telescopeIndex}(altitudeIndex,:);
validSwathIndices = find(coverageRow_days <= parameters.Cov_Requirement & ~isnan(coverageRow_days));
if isempty(validSwathIndices), row = {}; return; end
groundSampleDistance_m = parameters.GSD; pixelSizes_m = parameters.Pixel_size; localTimeAscendingNode_h = parameters.LTAN_hour;
swaths_km = parameters.swaths_km; telescopeNames = parameters.telescope_names;
maintenanceLowerFactor = parameters.maintenance_lower_factor; maintenanceUpperFactor = parameters.maintenance_upper_factor;
earthRadius_m = 6378.137 * 1000; earthGravitationalParameter_m3_s2 = 3.986004418e14; earthJ2 = 1.082629989052e-3;
[selectedSwath_km,revisit_days] = selectRegionalSwath( ...
    altitude_km,satelliteCount,telescopeCount,swaths_km(validSwathIndices), ...
    parameters,regionalCache);
if isnan(selectedSwath_km), row = {}; return; end
detectorPixelCount = parameters.Npix_detectors(detectorIndex);
requiredDetectorCount = max(telescopeCount,ceil(selectedSwath_km / ...
    (groundSampleDistance_m * detectorPixelCount / 1000)));
```

```
pixelsPerTelescope = ceil(selectedSwath_km*1000/(groundSampleDistance_m*telescopeCount));
acquiredSwath_km = telescopeCount*pixelsPerTelescope*groundSampleDistance_m/1000;
actualSwath_km = min(requiredDetectorCount*groundSampleDistance_m*detectorPixelCount/1000, ...
    2*altitude_km*tand(telescopeCount*parameters.fov_limit(telescopeIndex)/2));
if acquiredSwath_km > actualSwath_km, row = {}; return; end
excessSwath_km = actualSwath_km-selectedSwath_km;
excessSwath_percent = 100*excessSwath_km/selectedSwath_km;
diameterIndex = find(parameters.diametros_pupila == pupilDiameter_mm,1);
mtfValue = analysisResults.optics.mtf{2,detectorIndex,telescopeIndex}(altitudeIndex,diameterIndex);
snrValue = analysisResults.optics.snr{3,detectorIndex,telescopeIndex}(altitudeIndex,diameterIndex);
altitude_m = altitude_km * 1000;
pixelSize_m = pixelSizes_m(detectorIndex);
pupilDiameter_m = pupilDiameter_mm / 1000;
focalLength_m = (pixelSize_m * altitude_m) / groundSampleDistance_m;
focalRatio = focalLength_m / pupilDiameter_m;
massProperties = computeDryMass(pupilDiameter_mm,telescopeCount);
[totalMass_kg, fuelMass_kg, impulseCount, totalDeltaV_m_s] = cachedTotalMass(massCache, altitude_km, massProperties.dryMass_kg, massProperties.
      satelliteArea_m2, maintenanceLowerFactor, maintenanceUpperFactor);
if isnan(totalMass_kg), row = {}; return; end
constellationMass_kg = totalMass_kg * satelliteCount;
semimajorAxis_m = (altitude_km * 1000) + earthRadius_m;
sunSynchronousRate_rad_s = 2 * pi / (365.2421897 * 86400);
inclinationCosine = - (2 * sunSynchronousRate_rad_s * semimajorAxis_m^(3.5)) / (3 * earthJ2 * earthRadius_m^2 * sqrt(
      earthGravitationalParameter_m3_s2));
inclinationCosine = min(max(inclinationCosine, -1), 1);
inclination_deg = acosd(inclinationCosine);
relativeAnomaly_deg = 360 / satelliteCount;
row = {true, ...
    satelliteCount, telescopeCount, requiredDetectorCount, detectorIndex, telescopeNames{telescopeIndex}, altitude_km, pupilDiameter_mm, ...
    selectedSwath_km, revisit_days, ...
    actualSwath_km, excessSwath_km, excessSwath_percent, ...
    inclination_deg, localTimeAscendingNode_h, relativeAnomaly_deg, ...
    mtfValue, snrValue, ...
    focalLength_m, focalRatio, ...
    massProperties.instrumentMass_kg, massProperties.instrumentLength_m, massProperties.instrumentArea_m2, massProperties.instrumentVolume_m3,
      massProperties.instrumentPower_W, ...
    massProperties.dryMass_kg, fuelMass_kg, totalMass_kg, constellationMass_kg, ...
    impulseCount, totalDeltaV_m_s, ...
    massProperties.thematicMapperDryMass_kg, massProperties.seosatDryMass_kg, ...
    massProperties.satelliteLength_m, massProperties.satelliteArea_m2, massProperties.satelliteVolume_m3, massProperties.cubesatUnits,
      acquiredSwath_km
};
end
function row = theoreticalRow(parameters, indices, pupilDiameter_mm, massCache)
configurationIndex = indices(1); detectorIndex = indices(2); telescopeIndex = indices(3);
satelliteCount = parameters.satellite_configs(configurationIndex,1); telescopeCount = parameters.satellite_configs(configurationIndex,2);
altitude_km = parameters.alturas_orbitales(indices(4));
groundSampleDistance_m = parameters.GSD; pixelSizes_m = parameters.Pixel_size; localTimeAscendingNode_h = parameters.LTAN_hour;
swaths_km = parameters.swaths_km; telescopeNames = parameters.telescope_names;
maintenanceLowerFactor = parameters.maintenance_lower_factor; maintenanceUpperFactor = parameters.maintenance_upper_factor;
earthRadius_m = 6378.137 * 1000; earthGravitationalParameter_m3_s2 = 3.986004418e14; earthJ2 = 1.082629989052e-3;
massProperties = computeDryMass(pupilDiameter_mm,telescopeCount);
[totalMass_kg, fuelMass_kg, impulseCount, totalDeltaV_m_s] = cachedTotalMass(massCache, altitude_km, massProperties.dryMass_kg, massProperties.
      satelliteArea_m2, maintenanceLowerFactor, maintenanceUpperFactor);
if isnan(totalMass_kg), row = {}; return; end
relativeAnomaly_deg = 360 / satelliteCount;
row = {false, satelliteCount, telescopeCount, NaN, detectorIndex, telescopeNames{telescopeIndex}, altitude_km, pupilDiameter_mm, ...
    NaN, NaN, NaN, NaN, NaN, ...
    NaN, localTimeAscendingNode_h, relativeAnomaly_deg, ...
    NaN, NaN, NaN, NaN, ...
    NaN, NaN, NaN, NaN, NaN, ...
    massProperties.dryMass_kg, fuelMass_kg, totalMass_kg, totalMass_kg * satelliteCount, ...
    impulseCount, totalDeltaV_m_s, massProperties.thematicMapperDryMass_kg, massProperties.seosatDryMass_kg, ...
    massProperties.satelliteLength_m, massProperties.satelliteArea_m2, massProperties.satelliteVolume_m3, massProperties.cubesatUnits, NaN};
end
function [totalMass_kg, fuelMass_kg, impulseCount, totalDeltaV_m_s] = cachedTotalMass(cache, altitude_km, dryMass_kg, dragArea_m2,
      maintenanceLowerFactor, maintenanceUpperFactor)
    key = sprintf('%.6f_%.9f_%.9f_%.6f_%.6f', altitude_km, dryMass_kg, dragArea_m2, maintenanceLowerFactor, maintenanceUpperFactor);
    if isKey(cache, key)
        values = cache(key);
    else
        mass = computeTotalMass(altitude_km,dryMass_kg,dragArea_m2,maintenanceLowerFactor,maintenanceUpperFactor);
        values = [mass.totalMass_kg,mass.fuelMass_kg,mass.impulseCounts,mass.totalDeltaV_m_s];
        cache(key) = values;
    end
    totalMass_kg = values(1);
    fuelMass_kg = values(2);
    impulseCount = values(3);
    totalDeltaV_m_s = values(4);
end

function key = candidateKey(parameters,indices)
% Preserve the historical key and duplicate-configuration handling.
key = sprintf('%d_%s_%d_%d_%d',indices(2),parameters.telescope_names{indices(3)}, ...
    parameters.alturas_orbitales(indices(4)),parameters.satellite_configs(indices(1),1), ...
    parameters.satellite_configs(indices(1),2));
end
```

Generación de gráficas de masa

```
function GenerateMassPlots(resultsTable)
 %GENERATEMASSPLOTS Compatibility entry point for the existing report schema.
writeMassPlots(resultsTable);
end
```

```
%% Inputs from the selected design; edit config below for an auxiliary study.
studies = missionStudyInputs();
config = studies.contact;
results = runContactStudy(config);
```

Listing B.11: runContactStudy.m

```
function results = runContactStudy(config)
%RUNCONTACTSTUDY Size raw acquisition, SGP4 contacts and onboard memory.
% The station equipment rate is documented; service availability is assumed.
previousPath=path; pathCleanup=onCleanup(@() path(previousPath));
addpath(fullfile(fileparts(mfilename('fullpath')),'lib'));
if exist('satelliteScenario','file')==0 && exist('satelliteScenario','class')==0
    error('eo:MissingSatelliteToolbox','This study requires MATLAB satelliteScenario support.');
end
startTime=config.startTime; stopTime=startTime+days(config.duration_days); sampleTime_s=config.sampleTime_s;
groundStationName=config.groundStationName; groundStationLatitude_deg=config.groundStationLatitude_deg; groundStationLongitude_deg=config.
      groundStationLongitude_deg;
minimumElevation_deg=config.minimumElevation_deg; satelliteCount=config.satelliteCount; altitude_km=config.altitude_km;
earthRadius_km=config.earthRadius_km; semimajorAxis_m=(earthRadius_km+altitude_km)*1000;
eccentricity=config.eccentricity; inclination_deg=config.inclination_deg; argumentOfPerigee_deg=config.argumentOfPerigee_deg;
earthGravitationalParameter_m3_s2=config.earthGravitationalParameter_m3_s2; orbitalPeriod_s=2*pi*sqrt(semimajorAxis_m^3/
      earthGravitationalParameter_m3_s2);
localTimeAscendingNode_h=config.localTimeAscendingNode_h; groundSampleDistance_m=config.groundSampleDistance_m;
nominalSwath_km=config.nominalSwath_km; bandCount=config.bandCount; bitsPerBand=config.bitsPerBand;
memoryCapacity_GB=config.memoryCapacity_GB; regionBounds=config.regionBounds;
referenceRate_Mbps=config.referenceRate_Mbps;
bitsPerPixel = bandCount * bitsPerBand;
activePixelsPerTelescope = ceil(nominalSwath_km*1000 / ...
    (groundSampleDistance_m*config.telescopesPerSatellite));
pixelsPerLine = config.telescopesPerSatellite*activePixelsPerTelescope;
acquiredSwath_km = pixelsPerLine*groundSampleDistance_m/1000;
scenario = satelliteScenario(startTime, stopTime, sampleTime_s);
groundStationObject = groundStation(scenario, 'Name', groundStationName, 'Latitude', groundStationLatitude_deg, ...
    'Longitude', groundStationLongitude_deg, 'MinElevationAngle', minimumElevation_deg);
fprintf('Calculando RAAN para LTAN = %d:00...\n', localTimeAscendingNode_h);
try
    posSun = planetEphemeris(juliandate(startTime), 'Earth', 'Sun');
    sunRightAscension_deg = atan2d(posSun(2), posSun(1));
catch
    sunRightAscension_deg = approxSunRightAscensionDeg(startTime);
end
ascendingNode_deg = sunRightAscension_deg + (localTimeAscendingNode_h - 12) * 15;
satellites = [];
anomalySeparation_deg = 360 / satelliteCount;
colors=lines(max(2,satelliteCount)); colors(1:2,:)=[[0.8500,0.3250,0.0980];[0.0,0.4470,0.7410]];
for satelliteIndex = 1:satelliteCount
    trueAnomaly_deg = (satelliteIndex - 1) * anomalySeparation_deg;
    satellites = [satellites, satellite(scenario, semimajorAxis_m, eccentricity, inclination_deg, ...
        ascendingNode_deg, argumentOfPerigee_deg, trueAnomaly_deg, "Name", sprintf('Sat %d', satelliteIndex), ...
        "OrbitPropagator", config.propagator)];
end
timeVector = startTime:seconds(sampleTime_s):stopTime;
accessIntervalsAllSats = cell(satelliteCount, 1);
contactTime_s = zeros(satelliteCount, 1);
dataVolume_GB = zeros(satelliteCount, 1);
trackLength_km = zeros(satelliteCount, 1);
segmentDataBits = cell(satelliteCount, 1);
accessMask = false(satelliteCount, numel(timeVector));
trackLatitudes_deg = cell(satelliteCount, 1);
trackLongitudes_deg = cell(satelliteCount, 1);
fprintf('Calculando contactos y trazas dentro de CONUS...\n');
for satelliteIndex = 1:satelliteCount
    accessObject = access(satellites(satelliteIndex), groundStationObject);
    accessIntervalsAllSats{satelliteIndex} = accessIntervals(accessObject);
    if ~isempty(accessIntervalsAllSats{satelliteIndex}) && height(accessIntervalsAllSats{satelliteIndex}) > 0
        durations = accessIntervalsAllSats{satelliteIndex}.EndTime - accessIntervalsAllSats{satelliteIndex}.StartTime;
        contactTime_s(satelliteIndex) = sum(seconds(durations));
        accessMask(satelliteIndex, :) = accessMaskFromIntervals(accessIntervalsAllSats{satelliteIndex}, timeVector);
    end
    [trackLength_km(satelliteIndex), segmentLengths_km, trackLatitudes_deg{satelliteIndex}, trackLongitudes_deg{satelliteIndex}] = ...
        groundTrackLengthInMask(satellites(satelliteIndex), timeVector, regionBounds);
    segmentDataBits{satelliteIndex} = segmentLengths_km * 1000 / groundSampleDistance_m * pixelsPerLine * bitsPerPixel;
    dataVolume_GB(satelliteIndex) = sum(segmentDataBits{satelliteIndex}) / 8e9;
end
totalData_GB = sum(dataVolume_GB);
minimumRate_Mbps = dataVolume_GB * 8e3 ./ contactTime_s;
missionMinimumRate_Mbps = max(minimumRate_Mbps);
fprintf('\n--- Resultados por satélite ---\n');
for satelliteIndex = 1:satelliteCount
    fprintf('RESULT Sat %d: trace_km=%.3f, data_GB=%.6f, contact_s=%.3f, Rmin_Mbps=%.6f\n', ...
        satelliteIndex, trackLength_km(satelliteIndex), dataVolume_GB(satelliteIndex), contactTime_s(satelliteIndex), minimumRate_Mbps(
      satelliteIndex));
end
```

```
fprintf('RESULT Constellation: data_GB=%.6f, contact_s=%.3f\n', ...
    totalData_GB, sum(contactTime_s));
fprintf('RESULT Link: Rmin_Mbps=%.6f, S_reference_Mbps=%.3f, ratio=%.3f\n', ...
    missionMinimumRate_Mbps, referenceRate_Mbps, ...
    referenceRate_Mbps / missionMinimumRate_Mbps);
if referenceRate_Mbps >= missionMinimumRate_Mbps
    fprintf('RESULT Link viability: VIABLE in S-band.\n');
else
    fprintf('RESULT Link viability: NOT VIABLE in S-band reference case.\n');
end
downloadPerSample_GB = referenceRate_Mbps * 1e6 * sampleTime_s / 8e9;
memory=computeOnboardMemory(segmentDataBits,accessMask,memoryCapacity_GB,downloadPerSample_GB);
memoryState=memory.bounded_GB; unboundedMemoryState=memory.required_GB; overflow_GB=memory.overflow_GB;
for satelliteIndex=1:satelliteCount
    fprintf('RESULT Memory Sat %d: peak_required_GB=%.6f\n',satelliteIndex,max(unboundedMemoryState(satelliteIndex,:)));
end
if any(overflow_GB > 0)
    fprintf('RESULT Memory viability: NOT VIABLE.\n');
else
    fprintf('RESULT Memory viability: VIABLE.\n');
end
results.summary = table((1:satelliteCount).',trackLength_km,dataVolume_GB,contactTime_s, ...
    minimumRate_Mbps,max(memory.required_GB,[],2), ...
    'VariableNames',{'Satellite','Trace_km','Data_GB','Contact_s','MinimumRate_Mbps','PeakMemory_GB'});
results.summary.NominalAltitude_km = repmat(altitude_km,satelliteCount,1);
results.summary.AcquiredSwath_km = repmat(acquiredSwath_km,satelliteCount,1);
results.summary.ActivePixelsPerTelescope = repmat(activePixelsPerTelescope,satelliteCount,1);
results.summary.Propagator = repmat(string(config.propagator),satelliteCount,1);
assert(all(isfinite(results.summary{:,2:6}),'all'),'eo:ContactResult','Non-finite contact result.');
results.config=config; results.time=timeVector; results.contactTime_s=contactTime_s;
results.dataVolume_GB=dataVolume_GB; results.minimumRate_Mbps=minimumRate_Mbps;
results.trackLength_km=trackLength_km; results.memory=memory; results.accessMask=accessMask;
results.trackLatitudes_deg=trackLatitudes_deg; results.trackLongitudes_deg=trackLongitudes_deg;
if ~config.writeOutputs, return; end
if ~exist(config.outputDirectory,'dir'), mkdir(config.outputDirectory); end
writetable(results.summary,fullfile(config.outputDirectory,['contacts_' char(config.propagator) '.csv']));
scriptDirectory = fileparts(mfilename('fullpath'));
outputDirectory = config.outputDirectory;
elapsedDays = days(timeVector - startTime);
figure('Name', 'Estado de memoria de satélites', 'Visible', 'off');
hold on;
yline(memoryCapacity_GB, '--k', 'LineWidth', 1.2, 'DisplayName', 'Límite de memoria');
for satelliteIndex = 1:satelliteCount
    plot(elapsedDays, memoryState(satelliteIndex, :), 'Color', colors(satelliteIndex, :), ...
        'LineWidth', 1.5, 'DisplayName', sprintf('Satélite %d', satelliteIndex));
end
hold off; grid on;
xlabel('Tiempo [días]');
ylabel('Memoria ocupada [GB]', 'Interpreter', 'latex');
title('Estado de memoria a bordo durante una semana');
legend('show', 'Interpreter', 'latex', 'Location', 'northeast');
exportgraphics(gcf, fullfile(outputDirectory, 'memoria.jpg'), 'Resolution', 300);
close(gcf);
if exist('worldmap', 'file') == 2
    try
        load coastlines;
    catch
        coastlat = NaN; coastlon = NaN;
    end
    figureHandle = figure('Name', 'Traza semanal', 'Visible', 'off');
    worldmap('north america');
    geoshow(coastlat, coastlon, 'Color', 'black');
    hold on;
    for satelliteIndex = 1:satelliteCount
        geoshow(trackLatitudes_deg{satelliteIndex}, trackLongitudes_deg{satelliteIndex}, 'DisplayType', 'line', ...
            'Color', colors(satelliteIndex, :), 'LineWidth', 1.2);
    end
    geoshow(groundStationObject.Latitude, groundStationObject.Longitude, 'DisplayType', 'point', 'Marker', 'o', ...
        'MarkerEdgeColor', 'r', 'MarkerFaceColor', 'r', 'MarkerSize', 7);
    title('Trazas orbitales durante una semana');
    gridm on; mlabel on; plabel on;
    exportgraphics(figureHandle, fullfile(outputDirectory, 'Traza 2D 1 Semana.jpg'), 'Resolution', 300);
    close(figureHandle);
end
end

function [trackLength_km, segmentLengths_km, latitudes_deg, longitudes_deg] = groundTrackLengthInMask(satelliteObject, timeVector, bounds)
    position = states(satelliteObject,'CoordinateFrame','geographic');
    assert(size(position,2)==numel(timeVector),'eo:ContactTimeGrid','Unexpected state time grid.');
    latitudes_deg = position(1, :);
    longitudes_deg = position(2, :);
    segmentLengths_km = zeros(1, numel(timeVector) - 1);
    for sampleIndex = 1:numel(segmentLengths_km)
        segmentLengths_km(sampleIndex) = clippedSegmentLength(latitudes_deg(sampleIndex), longitudes_deg(sampleIndex), ...
            latitudes_deg(sampleIndex + 1), longitudes_deg(sampleIndex + 1), bounds);
    end
    trackLength_km = sum(segmentLengths_km);
end
function trackLength_km = clippedSegmentLength(lat1, lon1, lat2, lon2, bounds)
    if any(~isfinite([lat1, lon1, lat2, lon2])) || abs(lon2-lon1)>180
        trackLength_km = 0;
        return;
```

```
    end
    dx = lon2 - lon1;
    dy = lat2 - lat1;
    tEnter = 0;
    tExit = 1;
    for sampleIndex = 1:2
        if sampleIndex == 1
            d = dx; lower = bounds.lonMin; upper = bounds.lonMax; x = lon1;
        else
            d = dy; lower = bounds.latMin; upper = bounds.latMax; x = lat1;
        end
        if d == 0
            if x < lower || x > upper
                trackLength_km = 0;
                return;
            end
        else
            t1 = (lower - x) / d;
            t2 = (upper - x) / d;
            if t1 > t2
                tmp = t1; t1 = t2; t2 = tmp;
            end
            tEnter = max(tEnter, t1);
            tExit = min(tExit, t2);
        end
    end
    if tEnter > tExit
        trackLength_km = 0;
        return;
    end
    latA = lat1 + tEnter * dy; lonA = lon1 + tEnter * dx;
    latB = lat1 + tExit * dy; lonB = lon1 + tExit * dx;
    trackLength_km = haversineKm(latA, lonA, latB, lonB);
end
function distance_km = haversineKm(lat1, lon1, lat2, lon2)
    radius_km = 6371;
    dLat = deg2rad(lat2 - lat1);
    dLon = deg2rad(lon2 - lon1);
    a = sin(dLat / 2).^2 + cosd(lat1) .* cosd(lat2) .* sin(dLon / 2).^2;
    distance_km = 2 * radius_km * atan2(sqrt(a), sqrt(max(0, 1 - a)));
end
function mask = accessMaskFromIntervals(intervals, timeVector)
    mask = false(size(timeVector));
    for sampleIndex = 1:height(intervals)
        mask = mask | (timeVector >= intervals.StartTime(sampleIndex) & ...
            timeVector <= intervals.EndTime(sampleIndex));
    end
end
function sunRightAscension_deg = approxSunRightAscensionDeg(timeIndex)
    jd = juliandate(timeIndex);
    n = jd - 2451545.0;
    L = mod(280.460 + 0.9856474 * n, 360);
    g = mod(357.528 + 0.9856003 * n, 360);
    lambda = L + 1.915 * sind(g) + 0.020 * sind(2 * g);
    epsilon = 23.439 - 0.0000004 * n;
    sunRightAscension_deg = mod(atan2d(cosd(epsilon) * sind(lambda), cosd(lambda)), 360);
end
```

Listing B.12: computeOnboardMemory.m

```
function memory = computeOnboardMemory(segmentDataBits, accessMask, capacity_GB, downloadPerSample_GB)
%COMPUTEONBOARDMEMORY Preserve the original backlog and overflow accumulation.
% Time remains sequential; satellites are updated together at each time step.
% Required backlog uses its unbounded previous state, as in the original model.
validateattributes(accessMask,{'logical'},{'2d','nonempty'});
validateattributes(segmentDataBits,{'cell'},{'vector'});
validateattributes(capacity_GB,{'numeric'},{'scalar','real','finite','nonnegative'});
validateattributes(downloadPerSample_GB,{'numeric'},{'scalar','real','finite','nonnegative'});
satelliteCount = size(accessMask,1); sampleCount = size(accessMask,2);
assert(numel(segmentDataBits)==satelliteCount,'eo:MemoryAxes','One data series is required per satellite.');
if sampleCount == 1
    assert(all(cellfun(@isempty,segmentDataBits)),'eo:MemoryAxes','A single sample has no data segments.');
    generated_GB = zeros(satelliteCount,0);
else
    generated_GB = cell2mat(cellfun(@(bits) bits(:).'/8e9,segmentDataBits(:),'UniformOutput',false));
end
assert(isequal(size(generated_GB),[satelliteCount,sampleCount-1]),'eo:MemoryAxes','Data segments do not match time samples.');
memory.bounded_GB = zeros(satelliteCount,sampleCount);
memory.required_GB = zeros(satelliteCount,sampleCount);
memory.overflow_GB = zeros(satelliteCount,1);
for sampleIndex = 2:sampleCount
    requested_GB = max(0,memory.required_GB(:,sampleIndex-1) + generated_GB(:,sampleIndex-1) - ...
        accessMask(:,sampleIndex)*downloadPerSample_GB);
    memory.required_GB(:,sampleIndex) = requested_GB;
    memory.overflow_GB = memory.overflow_GB + max(0,requested_GB-capacity_GB);
    memory.bounded_GB(:,sampleIndex) = min(capacity_GB,requested_GB);
end
end
```

Días en orbita vs altitud

```
%% Editable inputs -- historical approximate orbital lifetimes
% Preserve the empirical anchors; these values are not propagation results.
config = struct();
config.altitudes_km = 400:50:2000;
config.anchorAltitudes_km = [400 500 700 900 2000];
config.anchorLifetimes_days = [365 3650 36500 365000 3650000];
config.writeOutputs = true;
results = runLifetimeStudy(config);
```

Listing B.13: runLifetimeStudy.m

```
function results = runLifetimeStudy(config)
%RUNLIFETIMESTUDY Compute the historical approximation and optionally plot it.
previousPath = path;
pathCleanup = onCleanup(@() path(previousPath));
addpath(fullfile(fileparts(mfilename('fullpath')),'lib'));
results.config = config;
results.altitudes_km = config.altitudes_km;
results.lifetime_days = computeLifetimeApproximation(config.altitudes_km, ...
    config.anchorAltitudes_km,config.anchorLifetimes_days);
if ~config.writeOutputs, return; end
figure('Position',[100 100 800 480]);
plot(results.altitudes_km,results.lifetime_days,'-','LineWidth',2);
set(gca,'YScale','log'); xlabel('Altura orbital (km)');
ylabel('Dias de vida media en orbita'); grid on;
set(gca,'GridLineStyle','-','GridAlpha',0.2,'XMinorGrid','off','YMinorGrid','off');
end
```

Listing B.14: computeLifetimeApproximation.m

```
function lifetime_days = computeLifetimeApproximation(altitudes_km, anchorAltitudes_km, anchorLifetimes_days)
%COMPUTELIFETIMEAPPROXIMATION Preserve the original empirical log-log segments.
% This is an approximate table fit, not an orbital propagator. Below the first
% anchor the historical result is zero; above the final anchor it remains NaN.
validateattributes(altitudes_km,{'numeric'},{'real','finite'});
validateattributes(anchorAltitudes_km,{'numeric'},{'vector','real','positive','finite'});
validateattributes(anchorLifetimes_days,{'numeric'},{'vector','positive','finite','numel',numel(anchorAltitudes_km)});
assert(all(diff(anchorAltitudes_km)>0),'eo:LifetimeAnchors','Lifetime anchor altitudes must be strictly increasing.');
lifetime_days = zeros(size(altitudes_km));
mask = altitudes_km >= anchorAltitudes_km(1);
lifetime_days(mask) = exp(interp1(log(anchorAltitudes_km),log(anchorLifetimes_days),log(altitudes_km(mask))));
end
```

Modelo deorbitacion. Fin de misión

```
%% Inputs from the selected design; edit config below for an auxiliary study.
studies = missionStudyInputs();
config = studies.deorbit;
results = runDeorbitStudy(config);
```

Listing B.15: runDeorbitStudy.m

```
function results = runDeorbitStudy(config)
%RUNDEORBITSTUDY Integrate maintenance cycles and circular-equivalent end-of-life decay.
validateattributes(config.missionDuration_years,{'numeric'},{'scalar','real','finite','positive'});
validateattributes(config.dryMass_kg,{'numeric'},{'scalar','real','finite','positive'});
validateattributes(config.dragArea_m2,{'numeric'},{'scalar','real','finite','positive'});
validateattributes(config.maintenanceDrop_percent,{'numeric'},{'scalar','real','finite','>',0,'<',100});
assert(config.nominalAltitude_km>config.reentryAltitude_km,'eo:DeorbitBounds','Nominal altitude must exceed reentry altitude.');
results.config=config;
nominalAltitude_km=config.nominalAltitude_km; missionDuration_years=config.missionDuration_years;
maintenanceDrop_percent=config.maintenanceDrop_percent; reentryAltitude_km=config.reentryAltitude_km;
deorbitRequirement_days=config.deorbitRequirement_days; dryMass_kg=config.dryMass_kg;
dragArea_m2=1.2*config.dragArea_m2; dragCoefficient=config.dragCoefficient; specificImpulse_s=config.specificImpulse_s;
earthGravitationalParameter_m3_s2 = 3.986004418e14; earthRadius_m = 6378e3; standardGravity_m_s2 = 9.80665;
nominalAltitude_m = nominalAltitude_km * 1e3;
resetAltitude_m = 1.02*nominalAltitude_m;
postMissionHorizon_s = 100*365.25*86400;
maintenanceThreshold_m = nominalAltitude_km * (1 - maintenanceDrop_percent / 100) * 1e3;
missionDuration_s = missionDuration_years * 365.25 * 24 * 3600;
timeSegments_s = {0}; altitudeSegments_m = {resetAltitude_m};
elapsedTime_s = 0; currentAltitude_m = resetAltitude_m; currentMass_kg = dryMass_kg;
totalDeltaV_m_s = 0; impulseCount = 0;
previousPath=path; pathCleanup=onCleanup(@() path(previousPath));
addpath(fullfile(fileparts(mfilename('fullpath')),'lib'));
if isfield(config,'maintenanceFuel_kg')
    maintenanceFuel_kg = config.maintenanceFuel_kg;
else
    mass = computeTotalMass(nominalAltitude_km,dryMass_kg,config.dragArea_m2,0.98,1.02);
    maintenanceFuel_kg = mass.fuelMass_kg;
end
initialMass_kg = dryMass_kg + maintenanceFuel_kg;
currentMass_kg = initialMass_kg;
fprintf('Calculo preliminar:\n  - Masa total inicial: %.2f kg\n\n', initialMass_kg);
fprintf('Iniciando simulacion de la mision de %d a~nos...\n', missionDuration_years);
maintenanceTargetAxis_m = earthRadius_m + maintenanceThreshold_m;
while elapsedTime_s < missionDuration_s
    initialSemimajorAxis_m = earthRadius_m + currentAltitude_m;
    options = odeset('Events', @(t,semimajorAxis_m) decayEvent(t, semimajorAxis_m, maintenanceTargetAxis_m), 'RelTol', 1e-7);
    [integrationTime_s, integratedAxes_m] = ode45(@(t,semimajorAxis_m) decayODE(t, semimajorAxis_m, earthRadius_m, earthGravitationalParameter_m3_s2
      , dragCoefficient, dragArea_m2, currentMass_kg), ...
                        [0, missionDuration_s - elapsedTime_s], initialSemimajorAxis_m, options);
    timeSegments_s{end+1,1} = elapsedTime_s + integrationTime_s(2:end);
    altitudeSegments_m{end+1,1} = integratedAxes_m(2:end) - earthRadius_m;
    elapsedTime_s = elapsedTime_s + integrationTime_s(end); currentAltitude_m = integratedAxes_m(end) - earthRadius_m;
    if elapsedTime_s >= missionDuration_s; break; end
    impulseCount = impulseCount + 1;
    fprintf('Impulso #%d en t = %.2f a~nos. Altitud: %.1f km\n', impulseCount, elapsedTime_s / (365.25*24*3600), currentAltitude_m/1000);
    initialRadius_m = earthRadius_m + currentAltitude_m; targetRadius_m = earthRadius_m + resetAltitude_m;
    impulseDeltaV_m_s = hohmannDeltaV(initialRadius_m, targetRadius_m, earthGravitationalParameter_m3_s2);
    currentMass_kg = currentMass_kg / exp(impulseDeltaV_m_s / (specificImpulse_s * standardGravity_m_s2));
    currentAltitude_m = resetAltitude_m;
    timeSegments_s{end+1,1} = elapsedTime_s; altitudeSegments_m{end+1,1} = currentAltitude_m;
end
missionTime_s = vertcat(timeSegments_s{:});
missionAltitudes_m = vertcat(altitudeSegments_m{:});
fprintf('Simulacion de mision completada.\n\n');
massBeforeDeorbit_kg = currentMass_kg;
reentryTargetAxis_m = earthRadius_m + reentryAltitude_km * 1e3;
reentryOptions = odeset('Events', @(t,semimajorAxis_m) decayEvent(t, semimajorAxis_m, reentryTargetAxis_m), 'RelTol', 1e-7);
fprintf('Simulando decaimiento natural (sin impulso)...\n');
naturalInitialAxis_m = earthRadius_m + missionAltitudes_m(end);
[naturalTime_s, naturalAxes_m] = ode45(@(t,semimajorAxis_m) decayODE(t, semimajorAxis_m, earthRadius_m, earthGravitationalParameter_m3_s2,
      dragCoefficient, dragArea_m2, massBeforeDeorbit_kg), ...
                            [0, postMissionHorizon_s], naturalInitialAxis_m, reentryOptions);
assert(naturalAxes_m(end)-earthRadius_m <= reentryAltitude_km*1000+1, ...
    'eo:NaturalDecayHorizon','Natural decay exceeds the integration horizon.');
naturalReentryTime_days = naturalTime_s(end) / (24 * 3600);
fprintf('  - Reentrada natural estimada en %.2f dias (%.2f a~nos).\n\n', naturalReentryTime_days, naturalReentryTime_days/365.25);
fprintf('Buscando impulso minimo de deorbitacion para %.0f dias...\n', deorbitRequirement_days);
deorbitApogeeRadius_m = earthRadius_m + missionAltitudes_m(end);
lowerPerigee_m = reentryAltitude_km * 1e3;
upperPerigee_m = missionAltitudes_m(end);
for iterationIndex = 1:50
    trialPerigee_m = 0.5 * (lowerPerigee_m + upperPerigee_m);
    trialPerigeeRadius_m = earthRadius_m + trialPerigee_m;
    trialTransferAxis_m = 0.5 * (deorbitApogeeRadius_m + trialPerigeeRadius_m);
    [trialTime_s, ~] = ode45(@(t,semimajorAxis_m) decayODE(t, semimajorAxis_m, earthRadius_m, earthGravitationalParameter_m3_s2, dragCoefficient,
      dragArea_m2, dryMass_kg), ...
                        [0, postMissionHorizon_s], trialTransferAxis_m, reentryOptions);
```

```
    trialReentryTime_days = trialTime_s(end) / (24 * 3600);
    if trialReentryTime_days <= deorbitRequirement_days
        lowerPerigee_m = trialPerigee_m;
    else
        upperPerigee_m = trialPerigee_m;
    end
end
deorbitPerigee_m = lowerPerigee_m;
deorbitPerigeeRadius_m = earthRadius_m + deorbitPerigee_m;
finalCircularSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 / deorbitApogeeRadius_m);
deorbitTransferAxis_m = 0.5 * (deorbitApogeeRadius_m + deorbitPerigeeRadius_m);
transferApogeeSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 * (2/deorbitApogeeRadius_m - 1/deorbitTransferAxis_m));
deorbitDeltaV_m_s = finalCircularSpeed_m_s - transferApogeeSpeed_m_s;
deorbitFuel_kg = dryMass_kg * (exp(deorbitDeltaV_m_s / (specificImpulse_s * standardGravity_m_s2)) - 1) * 1.1;
fprintf('Maniobra de Deorbitacion:\n');
fprintf('  - Perigeo objetivo: %.1f km\n', deorbitPerigee_m / 1e3);
fprintf('  - Delta-V minimo: %.2f m/s\n', deorbitDeltaV_m_s);
fprintf('  - Combustible reservado (10%% margen): %.3f kg\n', deorbitFuel_kg);
fprintf('Simulando reentrada con impulso...\n');
[activeTime_s, activeAxes_m] = ode45(@(t,semimajorAxis_m) decayODE(t, semimajorAxis_m, earthRadius_m, earthGravitationalParameter_m3_s2,
      dragCoefficient, dragArea_m2, dryMass_kg), ...
                            [0, postMissionHorizon_s], deorbitTransferAxis_m, reentryOptions);
activeReentryTime_days = activeTime_s(end) / (24 * 3600);
fprintf('  - Reentrada estimada en %.2f dias.\n\n', activeReentryTime_days);
totalTime_s = [missionTime_s; elapsedTime_s + activeTime_s(2:end)];
totalAltitudes_m = [missionAltitudes_m; activeAxes_m(2:end) - earthRadius_m];
time_years = totalTime_s / (365.25 * 24 * 3600);
altitudes_km = totalAltitudes_m / 1000;
results.time_s=totalTime_s; results.altitudes_m=totalAltitudes_m;
results.maintenanceFuel_kg=maintenanceFuel_kg; results.deorbitFuel_kg=deorbitFuel_kg;
results.deorbitDeltaV_m_s=deorbitDeltaV_m_s; results.deorbitPerigee_m=deorbitPerigee_m;
results.naturalReentryTime_days=naturalReentryTime_days; results.activeReentryTime_days=activeReentryTime_days;
results.impulseCount=impulseCount;
results.summary = table(nominalAltitude_km,missionAltitudes_m(end)/1000,impulseCount, ...
    naturalReentryTime_days,deorbitPerigee_m/1000,deorbitDeltaV_m_s,maintenanceFuel_kg, ...
    deorbitFuel_kg,dryMass_kg+maintenanceFuel_kg+deorbitFuel_kg, ...
    config.satelliteCount*(dryMass_kg+maintenanceFuel_kg+deorbitFuel_kg), ...
    'VariableNames',{'NominalAltitude_km','EndAltitude_km','SimulatedMaintenanceManoeuvres', ...
    'NaturalDecay_days','DeorbitPerigee_km','DeorbitDeltaV_ms','MaintenanceFuel_kg', ...
    'DeorbitFuel_kg','TotalSatelliteMass_kg','TotalConstellationMass_kg'});
assert(activeReentryTime_days <= deorbitRequirement_days+0.1, ...
    'eo:DeorbitRequirement','Deorbit time exceeds the model limit.');
if ~config.writeOutputs, return; end
if ~exist(config.outputDirectory,'dir'), mkdir(config.outputDirectory); end
writetable(results.summary,fullfile(config.outputDirectory,'end_of_life.csv'));
figure('Name', 'Evolucion de la Altitud con Reentrada');
hold on; grid on;
plot(time_years, altitudes_km, 'b-', 'LineWidth', 2, ...
     'DisplayName', sprintf('Trayectoria ($M_{seca}$=%dkg, A=%.1fm$^2$)', dryMass_kg, dragArea_m2));
yline(nominalAltitude_km, '--g', 'LineWidth', 1.5, 'DisplayName', 'Altitud Inicial');
yline(maintenanceThreshold_m/1e3, '--r', 'LineWidth', 1.5, 'DisplayName', 'Altitud Umbral');
xline(missionDuration_years, '--m', 'LineWidth', 1.5, 'DisplayName', 'Fin de Mision');
plot(missionDuration_years, altitudes_km(find(time_years >= missionDuration_years, 1)), ...
     'r*', 'MarkerSize', 10, 'LineWidth', 2, ...
     'DisplayName', sprintf('Impulso Deorbit ($\\Delta V$=%.1f m/s)', deorbitDeltaV_m_s));
title(sprintf('Evolucion de la Altitud durante %d a~nos y Reentrada', missionDuration_years), 'FontSize', 16, 'Interpreter', 'latex');
xlabel('Tiempo (a~nos)', 'FontSize', 14, 'Interpreter', 'latex');
ylabel('Altitud (km)', 'FontSize', 14, 'Interpreter', 'latex');
xlim([0, time_years(end) * 1.05]); ylim([0, nominalAltitude_km * 1.05]);
legend('Location', 'best', 'Interpreter', 'latex', 'FontSize', 12);
hold off;
deorbitFigure = figure('Name', 'Decaimiento Post-Mision');
hold on; grid on;
semilogx(naturalTime_s / (24*3600), (naturalAxes_m - earthRadius_m) / 1000, ...
    'k--', 'LineWidth', 2, 'DisplayName', 'Decaimiento Natural (sin impulso)');
semilogx(activeTime_s / (24*3600), (activeAxes_m - earthRadius_m) / 1000, ...
    'r-', 'LineWidth', 2, 'DisplayName', sprintf('Con impulso deorbitado (%.1f m/s)', deorbitDeltaV_m_s));
xline(deorbitRequirement_days, 'b:', 'LineWidth', 1.5, 'DisplayName', 'Limite 365 dias');
title('Decaimiento Post-Mision', 'FontSize', 16, 'Interpreter', 'latex');
xlabel('Tiempo tras fin de mision (dias)', 'FontSize', 14, 'Interpreter', 'latex');
ylabel('Altitud (km)', 'FontSize', 14, 'Interpreter', 'latex');
legend('Location', 'best', 'Interpreter', 'latex', 'FontSize', 12);
hold off;
exportgraphics(deorbitFigure,fullfile(config.outputDirectory,'deorbit.jpg'),'Resolution',300);
end

function axisRate_m_s = decayODE(~, semimajorAxis_m, earthRadius_m, earthGravitationalParameter_m3_s2, dragCoefficient, dragArea_m2,
      satelliteMass_kg)
    altitude_m = semimajorAxis_m - earthRadius_m; if altitude_m < 0; altitude_m = 0; end
    density_kg_m3 = getAtmosphericDensity(altitude_m);
    axisRate_m_s = -dragCoefficient * dragArea_m2 * density_kg_m3 * sqrt(earthGravitationalParameter_m3_s2 * semimajorAxis_m) / satelliteMass_kg;
end
function [value, isterminal, direction] = decayEvent(~, semimajorAxis_m, targetAxis_m)
    value = semimajorAxis_m - targetAxis_m; isterminal = 1; direction = -1;
end
function deltaV_m_s = hohmannDeltaV(initialRadius_m, targetRadius_m, earthGravitationalParameter_m3_s2)
    initialSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 / initialRadius_m); targetSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 /
      targetRadius_m); transferAxis_m = (initialRadius_m + targetRadius_m) / 2;
    firstTransferSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 * (2/initialRadius_m - 1/transferAxis_m));
    secondTransferSpeed_m_s = sqrt(earthGravitationalParameter_m3_s2 * (2/targetRadius_m - 1/transferAxis_m));
    deltaV_m_s = abs(firstTransferSpeed_m_s - initialSpeed_m_s) + abs(targetSpeed_m_s - secondTransferSpeed_m_s);
end
```

```
function density_kg_m3 = getAtmosphericDensity(altitude_m)
    % Preserve this script's original exponential-atmosphere table and lookup.
    altitude_km = max(altitude_m / 1e3, 0);
    baseAltitudes_km = [0, 25, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, ...
                 130, 140, 150, 180, 200, 250, 300, 350, 400, 450, ...
                 500, 600, 700, 800, 900, 1000];
    baseDensities_kg_m3 = [1.225, 3.899e-2, 1.774e-2, 3.972e-3, 1.057e-3, ...
                3.206e-4, 8.770e-5, 1.905e-5, 3.396e-6, 5.297e-7, ...
                9.661e-8, 2.438e-8, 8.484e-9, 3.845e-9, 2.070e-9, ...
                5.464e-10, 2.789e-10, 7.248e-11, 2.418e-11, ...
                9.518e-12, 3.725e-12, 1.585e-12, 6.967e-13, ...
                1.454e-13, 3.614e-14, 1.170e-14, 5.245e-15, 3.019e-15];
    scaleHeights_km = [7.249, 6.349, 6.682, 7.554, 8.382, 7.714, 6.549, ...
            5.799, 5.382, 5.877, 7.263, 9.473, 12.636, 16.149, ...
            22.523, 29.740, 37.105, 45.546, 53.628, 53.298, ...
            58.515, 60.828, 63.822, 71.835, 88.667, 124.64, ...
            181.05, 268.00];
    layerIndex = find(altitude_km >= baseAltitudes_km, 1, 'last');
    if isempty(layerIndex)
        layerIndex = 1;
    elseif layerIndex == numel(baseAltitudes_km)
        layerIndex = numel(baseAltitudes_km);
    end
    density_kg_m3 = baseDensities_kg_m3(layerIndex) * exp(-(altitude_km - baseAltitudes_km(layerIndex)) / scaleHeights_km(layerIndex));
end
```

Azimut de lanzamiento

```
%% Editable study inputs

config = struct();
config.launchLatitude_deg = 34.7;
config.launchLongitude_deg = -120.6;
config.minimumAzimuth_deg = 170;
config.maximumAzimuth_deg = 300;
config.boundaryRadius_deg = 8;
config.arcRadius_deg = 5.5;
config.arcSampleCount = 80;
config.launchAzimuth_deg = 190;
config.distances_km = linspace(0,900,90);
config.outputDirectory = fullfile(fileparts(mfilename('fullpath')),'..','Latex_Code','6.Lanzadores');
config.writeOutputs = true;
results = runAzimuthStudy(config);
```

Listing B.16: runAzimuthStudy.m

```
function results = runAzimuthStudy(config)
% Explicit study inputs; keep the caller workspace and MATLAB path intact.
previousPath = path;
pathCleanup = onCleanup(@() path(previousPath));
addpath(fullfile(fileparts(mfilename('fullpath')),'lib'));
results.config = config;
% Preserve the original local latitude/longitude launch-corridor approximation.
launchLatitude_deg=config.launchLatitude_deg; launchLongitude_deg=config.launchLongitude_deg;
minimumAzimuth_deg=config.minimumAzimuth_deg; maximumAzimuth_deg=config.maximumAzimuth_deg; boundaryRadius_deg=config.boundaryRadius_deg;
minimumAzimuth_rad = deg2rad(minimumAzimuth_deg);
minimumBoundaryLatitude_deg = launchLatitude_deg + boundaryRadius_deg * cos(minimumAzimuth_rad);
minimumBoundaryLongitude_deg = launchLongitude_deg + boundaryRadius_deg * sin(minimumAzimuth_rad) / cosd(launchLatitude_deg);
maximumAzimuth_rad = deg2rad(maximumAzimuth_deg);
maximumBoundaryLatitude_deg = launchLatitude_deg + boundaryRadius_deg * cos(maximumAzimuth_rad);
maximumBoundaryLongitude_deg = launchLongitude_deg + boundaryRadius_deg * sin(maximumAzimuth_rad) / cosd(launchLatitude_deg);
arcAzimuths_deg = linspace(minimumAzimuth_deg, maximumAzimuth_deg, config.arcSampleCount);
arcRadius_deg = config.arcRadius_deg;
arcLatitudes_deg = launchLatitude_deg + arcRadius_deg * cosd(arcAzimuths_deg);
arcLongitudes_deg = launchLongitude_deg + arcRadius_deg * sind(arcAzimuths_deg) ./ cosd(launchLatitude_deg);
launchAzimuth_deg = config.launchAzimuth_deg;
distances_km = config.distances_km;
trackLatitudes_deg = launchLatitude_deg + (distances_km .* cosd(launchAzimuth_deg)) / 111;
trackLongitudes_deg = launchLongitude_deg + (distances_km .* sind(launchAzimuth_deg)) ./ (111 * cosd(launchLatitude_deg));
results.trackLatitudes_deg = trackLatitudes_deg;
results.trackLongitudes_deg = trackLongitudes_deg;
results.arcLatitudes_deg = arcLatitudes_deg;
results.arcLongitudes_deg = arcLongitudes_deg;
if ~config.writeOutputs, return; end
if ~exist(config.outputDirectory,'dir'), mkdir(config.outputDirectory); end
figureHandle = figure('Color', 'white', 'Position', [100 100 900 650]);
mapAxes = geoaxes(figureHandle);
hold(mapAxes, 'on');
geolimits(mapAxes, [27 42], [-128 -115]);
try
    geobasemap(mapAxes, 'grayland');
catch
    geobasemap(mapAxes, 'streets-light');
end
mapAxes.Grid = 'on';
mapAxes.FontSize = 11;
geoplot(mapAxes, [launchLatitude_deg minimumBoundaryLatitude_deg], [launchLongitude_deg minimumBoundaryLongitude_deg], ...
    'k--', 'LineWidth', 1.8, 'DisplayName', 'Limites WTR');
geoplot(mapAxes, [launchLatitude_deg maximumBoundaryLatitude_deg], [launchLongitude_deg maximumBoundaryLongitude_deg], ...
    'k--', 'LineWidth', 1.8, 'HandleVisibility', 'off');
geoplot(mapAxes, arcLatitudes_deg, arcLongitudes_deg, 'k-', 'LineWidth', 1.6, ...
    'DisplayName', 'Rango permitido');
geoplot(mapAxes, trackLatitudes_deg, trackLongitudes_deg, 'r-', 'LineWidth', 3.0, ...
    'DisplayName',sprintf('Trayectoria %g deg',config.launchAzimuth_deg));
geoscatter(mapAxes, launchLatitude_deg, launchLongitude_deg, 120, [0 0.65 0.15], 'filled', ...
    'MarkerEdgeColor', 'white', 'LineWidth', 1.2, 'DisplayName', 'Vandenberg SFB');
text(mapAxes, launchLatitude_deg + 0.35, launchLongitude_deg + 0.2, 'Vandenberg SFB', ...
    'FontSize', 12, 'FontWeight', 'bold', 'Color', 'black', 'Interpreter', 'latex');
text(mapAxes, minimumBoundaryLatitude_deg - 0.15, minimumBoundaryLongitude_deg + 0.15, sprintf('$%g^\circ$',config.minimumAzimuth_deg), ...
    'FontSize', 11, 'Color', 'black', 'Interpreter', 'latex');
text(mapAxes, maximumBoundaryLatitude_deg + 0.2, maximumBoundaryLongitude_deg - 0.85, sprintf('$%g^\circ$',config.maximumAzimuth_deg), ...
    'FontSize', 11, 'Color', 'black', 'Interpreter', 'latex');
text(mapAxes, trackLatitudes_deg(min(42,numel(trackLatitudes_deg))), trackLongitudes_deg(min(42,numel(trackLongitudes_deg))) + 0.15, sprintf('$%g^\
      circ$',config.launchAzimuth_deg), ...
    'FontSize', 12, 'Color', 'red', 'Interpreter', 'latex', 'FontWeight', 'bold');
title(mapAxes, 'Azimut de lanzamiento desde Vandenberg SFB', ...
    'Interpreter', 'latex', 'FontSize', 14, 'Color', 'black');
legendHandle = legend(mapAxes, 'Location', 'northeast', 'Interpreter', 'latex');
legendHandle.Color = 'white';
legendHandle.TextColor = 'black';
exportgraphics(figureHandle,fullfile(config.outputDirectory,'azimuth.jpg'),'Resolution',300);
close(figureHandle);
end
```

Simulación de puesta en orbita de transferencia

El siguiente código conserva el ejemplo auxiliar de faseo a 630 km. No valida la secuencia de despliegue de la solución nominal a 690 km.

```
%% Editable phasing-manoeuvre inputs
config = struct();
config.earthGravitationalParameter_m3_s2 = 3.986004418e14;
config.earthRadius_m = 6378.137e3;
config.finalAltitude_m = 630e3;
config.phaseApogeeAltitude_m = 830e3;
config.timeStep_s = 120;
config.outputDirectory = fullfile(fileparts(mfilename('fullpath')),'..','Latex_Code','6.Lanzadores');
config.writeOutputs = true;
results = runPhasingStudy(config);
```

Listing B.17: runPhasingStudy.m

```
function results = runPhasingStudy(config)
%RUNPHASINGSTUDY Preserve the original mean-motion drift and polar animation.
% This remains the source's mean-anomaly approximation, not a Kepler solver.
mu_m3_s2 = config.earthGravitationalParameter_m3_s2;
earthRadius_m = config.earthRadius_m;
finalRadius_m = earthRadius_m + config.finalAltitude_m;
phaseApogeeRadius_m = earthRadius_m + config.phaseApogeeAltitude_m;
phaseSemimajorAxis_m = (finalRadius_m+phaseApogeeRadius_m)/2;
phaseEccentricity = (phaseApogeeRadius_m-finalRadius_m)/(phaseApogeeRadius_m+finalRadius_m);
finalPeriod_s = 2*pi*sqrt(finalRadius_m^3/mu_m3_s2);
phasePeriod_s = 2*pi*sqrt(phaseSemimajorAxis_m^3/mu_m3_s2);
finalMeanMotion_rad_s = 2*pi/finalPeriod_s;
phaseMeanMotion_rad_s = 2*pi/phasePeriod_s;
driftRate_rad_s = finalMeanMotion_rad_s-phaseMeanMotion_rad_s;
assert(driftRate_rad_s>0,'eo:PhasingDirection','The original positive-drift model requires a higher phasing apogee.');
validateattributes(config.timeStep_s,{'numeric'},{'scalar','positive','finite'});
phasingDuration_s = pi/abs(driftRate_rad_s);
% The source loop samples the first step at/after pi, then increments its clock.
time_s = (0:ceil(phasingDuration_s/config.timeStep_s))*config.timeStep_s;
finalAngles_rad = finalMeanMotion_rad_s*time_s;
phaseAngles_rad = phaseMeanMotion_rad_s*time_s;
phaseRadii_m = (phaseSemimajorAxis_m*(1-phaseEccentricity^2)) ./ ...
    (1+phaseEccentricity*cos(phaseAngles_rad));
results.config=config; results.time_s=time_s;
results.phasingDuration_s=phasingDuration_s; results.finalPeriod_s=finalPeriod_s;
results.phasePeriod_s=phasePeriod_s; results.phaseOrbitCount=phasingDuration_s/phasePeriod_s;
results.finalPositions_m=[finalRadius_m*cos(finalAngles_rad);finalRadius_m*sin(finalAngles_rad)];
results.phasePositions_m=[phaseRadii_m.*cos(phaseAngles_rad);phaseRadii_m.*sin(phaseAngles_rad)];
results.accumulatedPhase_rad=driftRate_rad_s*time_s;
results.displayedDeploymentTime_s=time_s(end)+config.timeStep_s;
if ~config.writeOutputs, return; end
if ~exist(config.outputDirectory,'dir'), mkdir(config.outputDirectory); end
figure('Name','Simulación de Desfase Orbital','NumberTitle','off'); hold on; axis equal; grid on;
[x,y,z]=sphere;
surf(x*earthRadius_m/1e3,y*earthRadius_m/1e3,z*earthRadius_m/1e3, ...
    'FaceColor','#0077be','EdgeColor','none','DisplayName','Tierra');
angles_rad=linspace(0,2*pi,300);
plot(finalRadius_m/1e3*cos(angles_rad),finalRadius_m/1e3*sin(angles_rad),'g--', ...
    'LineWidth',1.5,'DisplayName','Orbita final');
phaseCurve_m=phaseSemimajorAxis_m*(1-phaseEccentricity^2)./(1+phaseEccentricity*cos(angles_rad));
plot(phaseCurve_m/1e3.*cos(angles_rad),phaseCurve_m/1e3.*sin(angles_rad),'b', ...
    'LineWidth',1.5,'DisplayName','Orbita de fase');
firstSatellite=plot(NaN,NaN,'go','MarkerFaceColor','g','DisplayName','Satelite 1');
phaseVehicle=plot(NaN,NaN,'ro','MarkerFaceColor','r','DisplayName','Etapa/Satelite 2');
secondSatellite=plot(NaN,NaN,'mo','MarkerFaceColor','m','DisplayName','Satelite 2 (Desplegado)');
titleHandle=title('Inicio de Mision','Interpreter','none','FontSize',14);
legend('Location','northeastoutside','AutoUpdate','off','Interpreter','latex');
xlim([-1.2 1.2]*phaseApogeeRadius_m/1e3); ylim([-1.2 1.2]*phaseApogeeRadius_m/1e3);
xlabel('X (km)'); ylabel('Y (km)');
% Rendering is sequential; the trajectories themselves were computed as arrays.
for sampleIndex=1:numel(time_s)
    set(firstSatellite,'XData',results.finalPositions_m(1,sampleIndex)/1e3, ...
        'YData',results.finalPositions_m(2,sampleIndex)/1e3);
    set(phaseVehicle,'XData',results.phasePositions_m(1,sampleIndex)/1e3, ...
        'YData',results.phasePositions_m(2,sampleIndex)/1e3);
    titleHandle.String=sprintf('Fase de Deriva | Tiempo: %.1f h | Desfase: %.1f deg', ...
        time_s(sampleIndex)/3600,rad2deg(results.accumulatedPhase_rad(sampleIndex)));
    drawnow limitrate;
end
titleHandle.String=sprintf('Desfase de 180 deg alcanzado en %.1f horas.',results.displayedDeploymentTime_s/3600);
set(secondSatellite,'XData',results.phasePositions_m(1,end)/1e3, ...
    'YData',results.phasePositions_m(2,end)/1e3,'Visible','on');
set(phaseVehicle,'Visible','off'); hold off;
exportgraphics(gcf,fullfile(config.outputDirectory,'phasing.jpg'),'Resolution',300);
end
```

Cálculo de órbita RGT con J2

```
%% Inputs from the selected design; edit config below for an auxiliary study.
studies = missionStudyInputs();
config = studies.rgt;
results = runRgtStudy(config);
```

Listing B.18: runRgtStudy.m

```
function results = runRgtStudy(config)
%RUNRGTSTUDY Preserve the original J2/J3 SSO rates and RGT bisection criterion.
% Independent sampled rates and repeat-day residuals are array operations.
% Root refinement remains sequential and retains the original tolerances.
validateattributes(config.altitudes_km,{'numeric'},{'vector','nonempty','real','finite','positive'});
validateattributes(config.maxRepeatDays,{'numeric'},{'scalar','integer','positive','finite'});
validateattributes(config.maxIterations,{'numeric'},{'scalar','integer','positive','finite'});
validateattributes(config.rootTolerance,{'numeric'},{'scalar','real','positive','finite'});
validateattributes(config.rootBounds_m,{'numeric'},{'vector','numel',2,'real','positive','finite'});
assert(config.rootBounds_m(1)<config.rootBounds_m(2),'eo:RgtBounds','Root bounds must be increasing.');
altitudes_km=config.altitudes_km(:).';
[inclinations_deg,nodalRates_rad_s,precessionRates_rad_s]=rgtRates(altitudes_km*1e3,config);
nodalPeriods_s=2*pi./nodalRates_rad_s;
ratios=nodalRates_rad_s./(config.earthRotation_rad_s-precessionRates_rad_s);
repeatDays=1:config.maxRepeatDays;
% Enumerate only integer orbit-count/day-count candidates in the original order.
candidates=cell(config.maxRepeatDays,1);
for dayCount=repeatDays
    orbitCounts=ceil(dayCount*min(ratios)):floor(dayCount*max(ratios));
    candidates{dayCount}=[orbitCounts(:),repmat(dayCount,numel(orbitCounts),1)];
end
candidates=vertcat(candidates{:});
solutions=zeros(size(candidates,1),6); solutionCount=0;
for caseIndex=1:size(candidates,1)
    orbitCount=candidates(caseIndex,1); dayCount=candidates(caseIndex,2);
    trialAltitude_m=solveRepeatAltitude(orbitCount,dayCount,config);
    if isnan(trialAltitude_m), continue; end
    [inclination_deg,nodalRate_rad_s]=rgtRates(trialAltitude_m,config);
    solutionCount=solutionCount+1;
    solutions(solutionCount,:)=[trialAltitude_m/1e3,inclination_deg,orbitCount,dayCount, ...
        2*pi/nodalRate_rad_s,orbitCount/dayCount];
end
solutions=sortrows(solutions(1:solutionCount,:),1);
% One altitude column against one repeat-day row replaces the nested residual sweep.
orbitCounts=round(ratios(:).*repeatDays);
errors_rad_s=abs(nodalRates_rad_s(:) - ...
    (config.earthRotation_rad_s-precessionRates_rad_s(:)).*orbitCounts./repeatDays);
residualDrift_km_day=min(errors_rad_s,[],2).' * config.earthRadius_m * ...
    (2*pi/config.earthRotation_rad_s)/1e3;
results.config=config; results.altitudes_km=altitudes_km;
results.inclinations_deg=inclinations_deg; results.nodalPeriods_s=nodalPeriods_s;
results.solutions=solutions; results.residualDrift_km_day=residualDrift_km_day;
if isempty(solutions)
    results.nearestSolution=[];
else
    [~,nearestIndex]=min(abs(solutions(:,1)-config.referenceAltitude_km));
    results.nearestSolution=solutions(nearestIndex,:);
end
if ~config.writeOutputs, return; end
if ~exist(config.outputDirectory,'dir'), mkdir(config.outputDirectory); end
if ~isempty(results.nearestSolution)
    nearest = array2table(results.nearestSolution,'VariableNames', ...
        {'Altitude_km','Inclination_deg','Orbits','Days','NodalPeriod_s','Ratio'});
    writetable(nearest,fullfile(config.outputDirectory,'nearest_rgt.csv'));
end

figure('Position',[100 100 700 420]);
semilogy(altitudes_km,residualDrift_km_day,'b-','LineWidth',1.2); hold on;
for solutionIndex=1:size(solutions,1)
    [~,altitudeIndex]=min(abs(altitudes_km-solutions(solutionIndex,1)));
    plot(solutions(solutionIndex,1),residualDrift_km_day(altitudeIndex),'g.','MarkerSize',18);
end
xline(config.referenceAltitude_km,'r--','LineWidth',1.5); hold off;
xlabel('Altura [km]'); ylabel('Deriva longitudinal por día sideral [km/día sideral]');
title('Condición RGT para órbita SSO con J_2'); grid on; set(gca,'FontSize',11);
exportgraphics(gcf,fullfile(config.outputDirectory,'RGT_residual.pdf'),'ContentType','vector');
end

function [inclination_deg,nodalRate_rad_s,precessionRate_rad_s,frozenEccentricity] = rgtRates(altitudes_m,config)
semimajorAxes_m=config.earthRadius_m+altitudes_m;
meanMotions_rad_s=sqrt(config.earthGravitationalParameter_m3_s2./semimajorAxes_m.^3);
inclinationCosines=-(2*semimajorAxes_m.^(7/2)*config.sunNodalRate_rad_s) / ...
    (3*config.earthJ2*config.earthRadius_m^2*sqrt(config.earthGravitationalParameter_m3_s2));
inclination_deg=acosd(max(-1,min(1,inclinationCosines)));
frozenEccentricity=-config.earthJ3/(2*config.earthJ2) * ...
    (config.earthRadius_m./semimajorAxes_m).*sind(inclination_deg);
semilatusRecta_m=semimajorAxes_m.*(1-frozenEccentricity.^2);
precessionRate_rad_s=-(3*config.earthJ2*meanMotions_rad_s*config.earthRadius_m^2 ./ ...
```

```
    (2*semilatusRecta_m.^2)).*cosd(inclination_deg);
perigeeRates_rad_s=(3*config.earthJ2*meanMotions_rad_s*config.earthRadius_m^2 ./ ...
    (4*semilatusRecta_m.^2)).*(4-5*sind(inclination_deg).^2);
anomalyCorrections_rad_s=(3*config.earthJ2*meanMotions_rad_s*config.earthRadius_m^2 .* ...
    sqrt(1-frozenEccentricity.^2)./(4*semilatusRecta_m.^2)).*(2-3*sind(inclination_deg).^2);
nodalRate_rad_s=meanMotions_rad_s+anomalyCorrections_rad_s+perigeeRates_rad_s;
end

function residual = rgtResidual(altitude_m,orbitCount,dayCount,config)
[~,nodalRate_rad_s,precessionRate_rad_s]=rgtRates(altitude_m,config);
residual=nodalRate_rad_s-(config.earthRotation_rad_s-precessionRate_rad_s)*orbitCount/dayCount;
end

function trialAltitude_m=solveRepeatAltitude(orbitCount,dayCount,config)
    lowerAltitude_m=config.rootBounds_m(1); upperAltitude_m=config.rootBounds_m(2);
    lowerResidual=rgtResidual(lowerAltitude_m,orbitCount,dayCount,config);
    upperResidual=rgtResidual(upperAltitude_m,orbitCount,dayCount,config);
    if lowerResidual*upperResidual>0, trialAltitude_m=NaN; return; end
    for iterationIndex=1:config.maxIterations
        trialAltitude_m=(lowerAltitude_m+upperAltitude_m)/2;
        trialResidual=rgtResidual(trialAltitude_m,orbitCount,dayCount,config);
        if abs(trialResidual)<config.rootTolerance, break; end
        if lowerResidual*trialResidual<0
            upperAltitude_m=trialAltitude_m; upperResidual=trialResidual;
        else
            lowerAltitude_m=trialAltitude_m; lowerResidual=trialResidual;
        end
    end
end
```

Cálculo de órbita frozen

```
%% Inputs from the selected design; edit config below for an auxiliary study.
studies = missionStudyInputs();
config = studies.frozen;
results = runFrozenOrbitStudy(config);
```

Listing B.19: runFrozenOrbitStudy.m

```
function results = runFrozenOrbitStudy(config)
%RUNFROZENORBITSTUDY Compare continuous latitude arcs in the J2/J3 mean model.
% Ascending/descending excursions and their joint envelope are separate metrics.
a = config.earthRadius_m+config.selectedAltitude_km*1000;
mu = config.earthGravitationalParameter_m3_s2;
J2 = config.earthJ2; J3 = config.earthJ3;
n = sqrt(mu/a^3);
inc = acos(-2*a^(7/2)*config.sunNodalRate_rad_s / ...
    (3*J2*config.earthRadius_m^2*sqrt(mu)));
eFrozen = -J3/(2*J2)*(config.earthRadius_m/a)*sin(inc);
time_days = (0:config.missionDuration_days).';
p = a*(1-eFrozen^2);
alpha = -3*n*J2*(config.earthRadius_m/p)^2*(5/4*sin(inc)^2-1)*time_days*86400;
latitudes_deg = linspace(config.latitudeBounds_deg(1),config.latitudeBounds_deg(2),241);
caseNames = {'frozen','circular'};
summaryRows = zeros(2,9);
results.config = config;
for caseIndex = 1:2
    if caseIndex == 1
        ex = zeros(size(alpha)); ey = eFrozen*ones(size(alpha));
    else
        ex = eFrozen*sin(alpha); ey = eFrozen*(1-cos(alpha));
    end
    eccentricity = hypot(ex,ey); omega = atan2(ey,ex);
    [minimum_km,maximum_km,perArc_km] = computeAcquisitionHeightEnvelope( ...
        eccentricity,omega,latitudes_deg,rad2deg(inc),a,config.earthRadius_m);
    envelope_km = maximum_km-minimum_km;
    deviation_km = max(abs([minimum_km,maximum_km]-config.selectedAltitude_km),[],2);
    assert(all(isfinite([minimum_km;maximum_km;perArc_km(:)])), ...
        'eo:FrozenResult','Non-finite height result.');
    assert(all(perArc_km <= envelope_km+1e-10,'all'), ...
        'eo:FrozenEnvelope','An arc cannot exceed the joint envelope.');
    evolution = table(time_days,eccentricity,rad2deg(omega),minimum_km,maximum_km, ...
        perArc_km(:,1),perArc_km(:,2),envelope_km,deviation_km, ...
        'VariableNames',{'MissionDay','Eccentricity','ArgumentOfPerigee_deg', ...
        'MinHeight_km','MaxHeight_km','AscendingArcVariation_km', ...
        'DescendingArcVariation_km','CombinedEnvelope_km','MaxDeviationFromNominal_km'});
    results.(caseNames{caseIndex}) = evolution;
    summaryRows(caseIndex,:) = [min(eccentricity),max(eccentricity),min(minimum_km), ...
        max(maximum_km),max(perArc_km,[],1),max(envelope_km), ...
        max(deviation_km),config.selectedAltitude_km];
end
results.summary = array2table(summaryRows,'VariableNames', ...
    {'MinEccentricity','MaxEccentricity','MinHeight_km','MaxHeight_km', ...
    'MaxAscendingArcVariation_km','MaxDescendingArcVariation_km', ...
    'MaxCombinedEnvelope_km','MaxDeviationFromNominal_km','NominalAltitude_km'});
results.summary.Case = caseNames.';
assert(abs(summaryRows(1,5)-summaryRows(1,6))<1e-9, ...
    'eo:FrozenSymmetry','Frozen northbound and southbound arcs must be symmetric.');
[~,~,~,profile_km] = computeAcquisitionHeightEnvelope(eFrozen,pi/2, ...
    latitudes_deg,rad2deg(inc),a,config.earthRadius_m);
results.profile = table(latitudes_deg(:),profile_km(1,1:241).', ...
    fliplr(profile_km(1,242:end)).','VariableNames', ...
    {'Latitude_deg','AscendingHeight_km','DescendingHeight_km'});
if ~config.writeOutputs, return; end
if ~exist(config.outputDirectory,'dir'), mkdir(config.outputDirectory); end
writetable(results.profile,fullfile(config.outputDirectory,'acquisition_arcs.csv'));
writetable(results.frozen,fullfile(config.outputDirectory,'frozen_evolution.csv'));
writetable(results.circular,fullfile(config.outputDirectory,'circular_evolution.csv'));
writetable(results.summary,fullfile(config.outputDirectory,'frozen_comparison.csv'));
end
```

Listing B.20: computeAcquisitionHeightEnvelope.m

```
function [minimumHeights_km,maximumHeights_km,perArcVariation_km,heights_km] = computeAcquisitionHeightEnvelope( ...
    eccentricities,perigeeAngles_rad,arcLatitudes_deg,inclination_deg,semimajorAxis_m,earthRadius_m)
%COMPUTEACQUISITIONHEIGHTENVELOPE Evaluate both passes for every time sample.
% Time-dependent elements are columns; the two latitude passes form one row.
% Preserve the source ellipsoid and true-anomaly construction.
latitudes_deg=[arcLatitudes_deg(:).',fliplr(arcLatitudes_deg(:).')];
geocentricLatitudes_deg=atand((1-1/298.257223563)^2*tand(arcLatitudes_deg(:).'));
ascendingAngles_rad=asin(sind(geocentricLatitudes_deg)/sind(inclination_deg));
descendingAngles_rad=pi-asin(sind(fliplr(geocentricLatitudes_deg))/sind(inclination_deg));
anomalies_rad=[ascendingAngles_rad,descendingAngles_rad]-perigeeAngles_rad(:);
```

```
radii_m=semimajorAxis_m*(1-eccentricities(:).^2) ./ (1+eccentricities(:).*cos(anomalies_rad));
polarRadius_m=earthRadius_m*(1-1/298.257223563);
ellipsoidRadii_m=sqrt(((earthRadius_m^2.*cosd(latitudes_deg)).^2 + ...
    (polarRadius_m^2.*sind(latitudes_deg)).^2) ./ ...
    ((earthRadius_m.*cosd(latitudes_deg)).^2+(polarRadius_m.*sind(latitudes_deg)).^2));
heights_km=(radii_m-ellipsoidRadii_m)/1e3;
minimumHeights_km=min(heights_km,[],2); maximumHeights_km=max(heights_km,[],2);
arcSize = numel(arcLatitudes_deg);
perArcVariation_km = [max(heights_km(:,1:arcSize),[],2)-min(heights_km(:,1:arcSize),[],2), ...
    max(heights_km(:,arcSize+1:end),[],2)-min(heights_km(:,arcSize+1:end),[],2)];
end
```

Representación Gráfica de las bandas en el espectro electromagnético.

```
%% Editable study inputs

config = struct();
config.sBand_Hz = [2e9 4e9];
config.xBand_Hz = [8e9 12e9];
config.detectorWavelengths_m = [0.76e-6 2.3e-6];
config.writeOutputs = true;
results = runSpectrumStudy(config);
```

Listing B.21: runSpectrumStudy.m

```
function results = runSpectrumStudy(config)
% Explicit study inputs; keep the caller workspace and MATLAB path intact.
previousPath = path;
pathCleanup = onCleanup(@() path(previousPath));
addpath(fullfile(fileparts(mfilename('fullpath')),'lib'));
results.config = config;
% Frequencies retain the original speed-of-light conversion and band ranges.
sBandLower_Hz = config.sBand_Hz(1);
sBandUpper_Hz = config.sBand_Hz(2);
xBandLower_Hz = config.xBand_Hz(1);
xBandUpper_Hz = config.xBand_Hz(2);
speedOfLight_m_s = 3e8;
detectorLower_Hz = speedOfLight_m_s / config.detectorWavelengths_m(2);
detectorUpper_Hz = speedOfLight_m_s / config.detectorWavelengths_m(1);
detectorPosition = 1;
xBandPosition = 2;
sBandPosition = 3;
bandHeight = 0.8;
results.sBand_Hz = [sBandLower_Hz sBandUpper_Hz]; results.xBand_Hz = [xBandLower_Hz xBandUpper_Hz];
results.detectorBand_Hz = [detectorLower_Hz detectorUpper_Hz];
if ~config.writeOutputs, return; end
graphicsCleanup = studyGraphicsDefaults();
figure('Position',[100 100 1200 400]);
semilogx([1e9, 1e15], [0, 0], 'HandleVisibility', 'off');
hold on;
rectangle('Position', [sBandLower_Hz, sBandPosition, sBandUpper_Hz-sBandLower_Hz, bandHeight], ...
          'FaceColor', [0.12, 0.47, 0.71], 'EdgeColor', 'none');
rectangle('Position', [xBandLower_Hz, xBandPosition, xBandUpper_Hz-xBandLower_Hz, bandHeight], ...
          'FaceColor', [1.0, 0.5, 0.05], 'EdgeColor', 'none');
rectangle('Position', [detectorLower_Hz, detectorPosition, detectorUpper_Hz-detectorLower_Hz, bandHeight], ...
          'FaceColor', [0.17, 0.63, 0.17], 'EdgeColor', 'none');
sBandHandle = plot(NaN, NaN, 's', 'MarkerSize', 15, 'MarkerFaceColor', [0.12, 0.47, 0.71], ...
          'MarkerEdgeColor', 'none');
xBandHandle = plot(NaN, NaN, 's', 'MarkerSize', 15, 'MarkerFaceColor', [1.0, 0.5, 0.05], ...
          'MarkerEdgeColor', 'none');
detectorHandle = plot(NaN, NaN, 's', 'MarkerSize', 15, 'MarkerFaceColor', [0.17, 0.63, 0.17], ...
          'MarkerEdgeColor', 'none');
xlabel('Frecuencia (Hz, escala logaritmica)');
ylabel('Componentes');
yticks([detectorPosition + bandHeight/2, xBandPosition + bandHeight/2, sBandPosition + bandHeight/2]);
yticklabels({'Detector', 'Banda X', 'Banda S'});
title('Ubicacion de las Bandas S, X y Detector en el Espectro de Frecuencias');
xlim([1e9, 1e15]);
ylim([0.5, 4]);
grid on;
legend([sBandHandle, xBandHandle, detectorHandle], {sprintf('Banda S (%g-%g GHz)',config.sBand_Hz/1e9),sprintf('Banda X (%g-%g GHz)',config.
      xBand_Hz/1e9),sprintf('Detector (%g-%g um)',config.detectorWavelengths_m/1e-6)}, ...
       'Location', 'best');
hold off;
end
```

Simulación auxiliar del enlace en banda X

Las tasas de descarga de este ejemplo se fijan por escenario meteorológico; no proceden de un presupuesto de enlace ni se utilizan para validar el enlace en banda S de la solución seleccionada.

```
%% Editable study inputs
% Historical rate assumptions: ITU-R P.838-3 at 8.1 GHz, k=0.00425, alpha=1.32,
% with 60-degree elevation and 2 km rain height in the original study notes.
% Seasonal probabilities were attributed to NOAA Climate Normals 1991-2020,
% Fairbanks Intl Airport (USW00026411). Preserve these inputs without recalibration.
% Seasonal rows: spring, summer, autumn, winter; condition columns follow names.
config = struct();
config.dayCount = 365;
config.randomSeed = 42;
config.conditionNames = {'Cond. favorables','Lluvia moderada','Lluvia intensa','Nubes densas'};
config.rateRanges_Mbps = [130 150;120 140;90 120;125 145];
config.seasonProbabilities_percent = [76 12 3 9;52 36 7 5;64 22 5 9;82 1 0 17];
config.outputDirectory = fullfile(fileparts(mfilename('fullpath')),'..','Latex_Code','7.Segmento_Tierra');
config.writeOutputs = true;
results = runWeatherLinkStudy(config);
```

Listing B.22: runWeatherLinkStudy.m

```
function results = runWeatherLinkStudy(config)
%RUNWEATHERLINKSTUDY Preserve the historical seasonal probabilities and rate ranges.
% Two interleaved uniform draws per day reproduce the original seeded sequence.
previousPath = path; pathCleanup = onCleanup(@() path(previousPath));
addpath(fullfile(fileparts(mfilename('fullpath')),'lib'));
validateattributes(config.dayCount,{'numeric'},{'scalar','integer','finite','>=',7});
validateattributes(config.seasonProbabilities_percent,{'numeric'},{'real','finite','nonnegative','<=',100});
validateattributes(config.rateRanges_Mbps,{'numeric'},{'size',[4 2],'real','finite','nonnegative'});
assert(all(config.rateRanges_Mbps(:,2)>=config.rateRanges_Mbps(:,1)),'eo:WeatherRates','Rate ranges must have increasing endpoints.');
assert(isequal(size(config.seasonProbabilities_percent),[4 4]) && ...
    all(abs(sum(config.seasonProbabilities_percent,2)-100)<1e-12), ...
    'eo:WeatherProbabilities','Each seasonal probability row must sum to 100.');
previousRng = rng; rngCleanup = onCleanup(@() rng(previousRng));
rng(config.randomSeed);
draws = rand(2,config.dayCount);
days = (1:config.dayCount).'; seasons = 4*ones(config.dayCount,1);
seasons(days>=80 & days<=172)=1; seasons(days>=173 & days<=265)=2;
seasons(days>=266 & days<=355)=3;
probabilities = config.seasonProbabilities_percent(seasons,:);
conditionIndices = 1 + sum(draws(1,:).'*100 > cumsum(probabilities(:,1:3),2),2);
ranges = config.rateRanges_Mbps(conditionIndices,:);
dailyRates_Mbps = ranges(:,1) + draws(2,:).' .* (ranges(:,2)-ranges(:,1));
weekCount = floor(config.dayCount/7);
weeklyRates_Mbps = mean(reshape(dailyRates_Mbps(1:weekCount*7),7,weekCount),1);
results.config=config; results.conditionIndices=conditionIndices;
results.dailyConditions=reshape(config.conditionNames(conditionIndices),[],1);
results.dailyRates_Mbps=dailyRates_Mbps; results.weekIndices=1:weekCount;
results.weeklyRates_Mbps=weeklyRates_Mbps;
% Preserve the old annual statistic: mean of complete weekly means (364 days).
results.annualWeeklyMean_Mbps=mean(weeklyRates_Mbps);
results.annualDailyMean_Mbps=mean(dailyRates_Mbps);
if ~config.writeOutputs, return; end
if ~exist(config.outputDirectory,'dir'), mkdir(config.outputDirectory); end
graphicsCleanup = studyGraphicsDefaults();
weekIndices=results.weekIndices; weeklyRates_Mbps=results.weeklyRates_Mbps;
annualWeeklyMean_Mbps=results.annualWeeklyMean_Mbps; conditionNames=config.conditionNames;
dailyConditions=results.dailyConditions; dailyRates_Mbps=results.dailyRates_Mbps;
figure('Position', [100, 100, 1000, 600]);
hold on;
primaryColor = [0, 0.4470, 0.7410];
meanColor = [0.8500, 0.3250, 0.0980];
plot(weekIndices, weeklyRates_Mbps, 'o-', 'Color', primaryColor, ...
    'LineWidth', 2.5, 'MarkerSize', 8, 'MarkerFaceColor', primaryColor);
yline(annualWeeklyMean_Mbps, '--', 'Color', meanColor, 'LineWidth', 3, ...
    'DisplayName', sprintf('Media anual (%.1f Mbps)', annualWeeklyMean_Mbps));
xlim([1 max(2,numel(weekIndices))]);
xlabel('Semana del a\~no', 'FontSize', 12);
ylabel('Velocidad promedio (Mbps)', 'FontSize', 12);
title('Velocidad promedio semanal en banda X durante un a\~no', 'FontSize', 14);
grid on; grid minor;
legend({'Velocidad promedio semanal', sprintf('Media anual (%.1f Mbps)', annualWeeklyMean_Mbps)}, ...
    'Location', 'best', 'FontSize', 11);
hold off;
outputDirectory = config.outputDirectory;
exportgraphics(gcf, fullfile(outputDirectory, 'velmediasemanal.png'), 'Resolution', 150);
figure('Position', [150, 150, 1000, 600]);
hold on;
conditionColors = [
    0, 0.4470, 0.7410;
```

```
    0.8500, 0.3250, 0.0980;
    0.6350, 0.0780, 0.1840;
    0.4660, 0.6740, 0.1880
];
colorMap = containers.Map(conditionNames, ...
    {conditionColors(1,:), conditionColors(2,:), ...
     conditionColors(3,:), conditionColors(4,:)});
for conditionIndex = 1:length(conditionNames)
    conditionName = conditionNames{conditionIndex};
    conditionMask = strcmp(dailyConditions, conditionName);
    conditionDays = find(conditionMask);
    conditionRates_Mbps = dailyRates_Mbps(conditionMask);
    scatter(conditionDays, conditionRates_Mbps, 50, ...
        'MarkerFaceColor', colorMap(conditionName), ...
        'MarkerEdgeColor', 'none', 'MarkerFaceAlpha', 0.8, ...
        'DisplayName', conditionName);
end
annualDailyMean_Mbps = mean(dailyRates_Mbps);
yline(annualDailyMean_Mbps, '--', 'Color', 'k', 'LineWidth', 3, ...
    'DisplayName', sprintf('Media anual (%.1f Mbps)', annualDailyMean_Mbps));
xlim([1 max(2,config.dayCount)]);
xlabel('D\''ia del a\~no', 'FontSize', 12);
ylabel('Velocidad de descarga (Mbps)', 'FontSize', 12);
title('Velocidad diaria en banda X durante un a\~no', 'FontSize', 14);
grid on; grid minor;
legend('Location', 'best', 'FontSize', 11);
hold off;
exportgraphics(gcf, fullfile(outputDirectory, 'veldiariabandax.png'), 'Resolution', 150);
end
```

Reproducción del análisis completo

```
function report = runThesisAnalysis()
%RUNTHESISANALYSIS Reproduce the design, mission budgets and thesis figures.
scriptDirectory = fileparts(mfilename('fullpath'));
root = fileparts(scriptDirectory);
previousPath = path; pathCleanup = onCleanup(@() path(previousPath));
addpath(scriptDirectory,fullfile(scriptDirectory,'lib'));
stage = tempname; mkdir(stage);
stageCleanup = onCleanup(@() rmdir(stage,'s'));
pipelineDirectory = fullfile(stage,'Pipeline');
previousOutput = getenv('TFG_OUTPUT_ROOT');
environmentCleanup = onCleanup(@() setenv('TFG_OUTPUT_ROOT',previousOutput));
setenv('TFG_OUTPUT_ROOT',pipelineDirectory);
previousVisibility = get(groot,'defaultFigureVisible');
graphicsCleanup = onCleanup(@() set(groot,'defaultFigureVisible',previousVisibility));
set(groot,'defaultFigureVisible','off');
analysis = runPipeline(scriptDirectory);
assert(analysis.coverage.executionSucceeded,'eo:CoverageExecution','Coverage solver failures occurred.');
solutions = analysis.configuration.table;
viable = sortrows(solutions(logical(solutions.Es_Viable),:),'Masa_Total_Constelacion_kg');
assert(~isempty(viable),'eo:NoViableDesign','No design satisfies the mission requirements.');
best = viable(1,:); best.GSD = analysis.config.groundSampleDistance_m;
design = table2struct(best);
missionDirectory = fullfile(stage,'Mission'); mkdir(missionDirectory);
writetable(best,fullfile(missionDirectory,'selected_design.csv'));
writetable(viable,fullfile(missionDirectory,'viable_designs.csv'));
inputs = missionStudyInputs(design,missionDirectory);
frozen = runFrozenOrbitStudy(inputs.frozen);
contacts = runContactStudy(inputs.contact);
keplerianInputs = inputs.contact;
keplerianInputs.propagator = 'two-body-keplerian'; keplerianInputs.writeOutputs = false;
keplerian = runContactStudy(keplerianInputs);
writetable(keplerian.summary,fullfile(missionDirectory,'Contacts','contacts_two-body-keplerian.csv'));

assert(all(contacts.minimumRate_Mbps <= inputs.contact.referenceRate_Mbps), ...
    'eo:DownlinkRequirement','The assumed station rate cannot download the acquired data.');
assert(all(max(contacts.memory.required_GB,[],2) <= inputs.contact.memoryCapacity_GB), ...
    'eo:MemoryRequirement','The selected memory capacity is insufficient.');
endOfLife = runDeorbitStudy(inputs.deorbit);
rgt = runRgtStudy(inputs.rgt);
latitudes = (25:2:49).';
revisit = zeros(size(latitudes));
a = 6378.137e3+design.Altura_km*1000;
inc = deg2rad(inputs.contact.inclination_deg);
psi = atand((1-analysis.config.overlapFraction)*design.Swath_Adquirido_km/(2*design.Altura_km));
for latitudeIndex = 1:numel(latitudes)
    revisit(latitudeIndex) = RevisitCalc([a,0,inc,0,0],latitudes(latitudeIndex), ...
        1,0,design.Num_Satelites,1,0,1,0,[1,analysis.config.revisitRequirement_days],psi) / ...
        (1-analysis.config.cloudFraction);
end
assert(all(isfinite(revisit) & revisit>0 & revisit<=analysis.config.revisitRequirement_days), ...
    'eo:RegionalRequirement','The selected design fails the sampled regional requirement.');
writetable(table(latitudes,revisit,'VariableNames',{'Latitude_deg','EffectiveRevisit_days'}), ...
    fullfile(missionDirectory,'latitude_revisit.csv'));
report = struct('matlab_version',version,'selected',design, ...
    'telescope_index',find(strcmp(analysis.config.telescopeNames,design.Tipo_Telescopio),1), ...
    'revisit_requirement_days',analysis.config.revisitRequirement_days, ...
    'cloud_fraction',analysis.config.cloudFraction,'overlap_fraction',analysis.config.overlapFraction, ...
    'latitudes_deg',latitudes,'revisit_days',revisit, ...
    'selected_frozen',table2struct(frozen.summary), ...
    'keplerian',table2struct(keplerian.summary), ...
    'sgp4',table2struct(contacts.summary),'end_of_life',table2struct(endOfLife.summary), ...
    'rgt',rgt.nearestSolution);
fid = fopen(fullfile(missionDirectory,'summary.json'),'w');
assert(fid>=0,'eo:OutputFile','Cannot open mission summary.');
fprintf(fid,'%s\n',jsonencode(report)); fclose(fid);
publishThesisFigures(stage);
gsdInputs = gsdSensitivityInputs(design);
gsdInputs.outputDirectory = fullfile(root,'Compiled data');
gsdInputs.figureDirectory = fullfile(root,'Latex_Code','4.Payload');
runGsdSensitivityStudy(gsdInputs);
dataDirectory = fullfile(root,'Compiled data');
copyfile(pipelineDirectory,fullfile(dataDirectory,'Pipeline'),'f');
copyfile(missionDirectory,fullfile(dataDirectory,'Mission'),'f');
fprintf('THESIS_ANALYSIS_COMPLETE\n'); disp(best);
end

function analysis = runPipeline(scriptDirectory)
run(fullfile(scriptDirectory,'master.m'));
analysis = results;
end
```

Listing B.23: missionStudyInputs.m

```
function studies = missionStudyInputs(design,outputDirectory)
```

```
%MISSIONSTUDYINPUTS Share the selected design across the independent studies.
root = fileparts(fileparts(mfilename('fullpath')));
if nargin < 2, outputDirectory = fullfile(root,'Compiled data','Mission'); end
if nargin < 1
    design = table2struct(readtable(fullfile(outputDirectory,'selected_design.csv')));
end
mu = 3.986004418e14; radius_m = 6378.137e3;
J2 = 1.082629989052e-3; J3 = -2.5327e-6;
sunRate = 2*pi/(365.2421897*86400);
a = radius_m+design.Altura_km*1000;
inclination_deg = acosd(-2*a^(7/2)*sunRate/(3*J2*radius_m^2*sqrt(mu)));
eccentricity = -J3/(2*J2)*(radius_m/a)*sind(inclination_deg);
studies.frozen = struct('earthGravitationalParameter_m3_s2',mu,'earthRadius_m',radius_m, ...
    'earthJ2',J2,'earthJ3',J3,'sunNodalRate_rad_s',sunRate, ...
    'selectedAltitude_km',design.Altura_km,'latitudeBounds_deg',[25 49], ...
    'missionDuration_days',8*365.25,'outputDirectory',fullfile(outputDirectory,'Frozen'), ...
    'writeOutputs',true);
studies.contact = struct('startTime',datetime('28-Apr-2024 00:00:00','TimeZone','UTC'), ...
    'duration_days',7,'sampleTime_s',10,'groundStationName','Fairbanks (referencia geográfica)', ...
    'groundStationLatitude_deg',64.84,'groundStationLongitude_deg',-147.712, ...
    'minimumElevation_deg',10,'satelliteCount',design.Num_Satelites, ...
    'telescopesPerSatellite',design.Num_Telescopios,'altitude_km',design.Altura_km, ...
    'earthRadius_km',radius_m/1000,'eccentricity',eccentricity, ...
    'inclination_deg',inclination_deg,'argumentOfPerigee_deg',90,'propagator','sgp4', ...
    'earthGravitationalParameter_m3_s2',mu,'localTimeAscendingNode_h',6, ...
    'groundSampleDistance_m',design.GSD,'nominalSwath_km',design.Swath_Adquirido_km, ...
    'bandCount',3,'bitsPerBand',12,'memoryCapacity_GB',2, ...
    'regionBounds',struct('latMin',25,'latMax',49,'lonMin',-125,'lonMax',-66), ...
    'referenceRate_Mbps',10,'outputDirectory',fullfile(outputDirectory,'Contacts'), ...
    'writeOutputs',true);
studies.deorbit = struct('nominalAltitude_km',design.Altura_km, ...
    'missionDuration_years',8,'maintenanceDrop_percent',2,'reentryAltitude_km',100, ...
    'deorbitRequirement_days',365,'dryMass_kg',design.Masa_Seca_Satelite_kg, ...
    'dragArea_m2',design.Area_Satelite_m2,'maintenanceFuel_kg',design.Masa_Combustible_kg, ...
    'satelliteCount',design.Num_Satelites,'dragCoefficient',2.5,'specificImpulse_s',220, ...
    'outputDirectory',fullfile(outputDirectory,'EndOfLife'),'writeOutputs',true);
studies.rgt = struct('earthGravitationalParameter_m3_s2',mu,'earthRadius_m',radius_m, ...
    'earthJ2',J2,'earthJ3',J3,'earthRotation_rad_s',7.292115373194e-5, ...
    'sunNodalRate_rad_s',sunRate,'altitudes_km',200:2:1000, ...
    'rootBounds_m',[200e3 1000e3],'referenceAltitude_km',design.Altura_km, ...
    'maxRepeatDays',30,'rootTolerance',1e-10,'maxIterations',100, ...
    'outputDirectory',fullfile(outputDirectory,'RGT'),'writeOutputs',true);
end
```

— Fin —